

Systematic Reasoning in LLMs and Their Applications in the Biomedical Domain

by

Mihir Prafullsinh Parmar

A Dissertation Presented in Partial Fulfillment
of the Requirements for the Degree
Doctor of Philosophy

Approved August 2025 by the
Graduate Supervisory Committee

Chitta Baral, Chair
Yezhou Yang
Ben Zhou
Irbaz Bin Riaz
Swaroop Mishra

ARIZONA STATE UNIVERSITY
December 2025

ABSTRACT

Reasoning is a fundamental capability for large language models (LLMs), enabling them to tackle complex problem-solving tasks. Recent research has focused on enhancing a wide range of reasoning skills in LLMs, including mathematical, common-sense, temporal, and logical reasoning, to name a few. Recent large-scale LLMs such as GPT-family, Gemini-family, and Claude-family achieve remarkable performance on tasks that were previously thought to be exclusive to humans. However, despite their success, several limitations remain, especially in their ability to reason systematically, apply formal logic to natural language settings, and generalize across diverse reasoning depths. Moreover, the underlying causes of reasoning failures in LLMs are not yet well understood or thoroughly analyzed. With the goal of improving the reasoning capabilities of LLMs and enabling their use in domain-specific (i.e., biomedical) applications, this dissertation explores the following central question: how can we systematically evaluate, analyze, and improve the reasoning performance of LLMs, particularly in logic-intensive and biomedical tasks?

To this end, I present three major contributions. First, I propose multiple reasoning benchmarks—*LogicBench*, *Multi-LogiEval*, and *GridPuzzle*—that enable a systematic and comprehensive evaluation of the logical reasoning capabilities of LLMs. Second, I introduce methods to improve reasoning in LLMs, including *PlanGEN*, a multi-agent inference-time natural planning and reasoning framework, and PLAN-TUNING, a post-training approach that improves model reasoning capabilities using synthetic planning trajectories. Third, I address the challenge of data quality and coverage by analyzing *instruction bias* in crowdsourced datasets and proposing *Tar-GEN*, a targeted data generation pipeline with self-correction. These contributions are further extended to biomedical and clinical domains. I introduce *In-BoXBART*, the first instruction-tuned language model developed for biomedical tasks. Addition-

ally, I present a method to simulate a two-reviewer setting using LLMs for improving the automation of cancer-specific clinical systematic reviews. At the end, I also explore strategies for improving LLM-agent performance in clinical settings through prompt reformulation techniques. Overall, this dissertation offers a unified framework for evaluating and improving systematic reasoning in LLMs, with demonstrated impact across both general-purpose and domain-specific applications.

DEDICATION

*Dedicated to, my loving parents, amazing partner, family, and friends for the love,
patience, and faith in this short journey and in much more to come...*

ACKNOWLEDGMENTS

Writing this thesis is a remarkable journey of my graduate life in terms of learning from experts, exploring new arenas, improving my research abilities, and much more. I take this opportunity to convey my sincere gratitude to all those who helped me directly and indirectly in making this PhD thesis possible.

First and foremost, I would like to express my deepest gratitude to my supervisor, Dr. Chitta Baral, for his invaluable guidance, motivation, and support throughout every phase of this journey. His mentorship has inspired confidence and enthusiasm in my work and has been very important in the successful completion of this thesis. I am also thankful to him for fostering an excellent research environment and providing access to high-quality facilities and resources in the lab.

I am grateful to my dissertation committee members—Dr. Yezhou Yang, Dr. Irbaz Bin Riaz, Dr. Ben Zhou, and Dr. Swaroop Mishra—for their insightful suggestions and constructive feedback, which have greatly strengthened the structure and quality of this work. I extend special thanks to the Mayo Clinic team, particularly Dr. Irbaz Bin Riaz (MD) and Dr. Syed Arsalan Ahmed Naqvi, for providing high-quality research data and continued support.

My heartfelt thanks go to Dr. Swaroop Mishra for his invaluable mentorship, collaboration, and encouragement toward high-quality publications, which significantly enriched my early PhD years. I am also thankful to Dr. Man Luo for our fruitful collaborations during my doctoral studies. I would like to express my sincere appreciation to my outstanding external collaborators—Mor Geva (Tel Aviv University), Daniel Khashabi (Johns Hopkins University), Arindam Mitra (Microsoft Research), M. Hassan Murad (MD, Mayo Clinic), Aakanksha Naik (Allen Institute for AI), and Franck Deroncourt (Adobe Research)—for their intellectual contributions and support. I am equally grateful to my PhD internship mentors—Hamid Palangi (Google),

Hanieh Deilamsalehy (Adobe Research), and Murthy Devarakonda (Novartis). I am thankful for all my industry collaborations with Google, Microsoft, the Allen Institute for AI (AI2), Cisco Research, Adobe Research, and the Mayo Clinic, which provided invaluable learning experiences and resources.

I extend my heartfelt appreciation to my colleagues and labmates—Neeraj Varshney, Tejas Gokhale, Kuntal Pal, Pulkit Verma, Pratyay Banerjee, Shailaja Sampat, Maitreya Patel, Himanshu Gupta, Ujjwala Anantheswaran, Ravsehaj Singh Puri, Kirby Kuznia, Pruthvi Patel, Pavel Dolin, Mutsumi Nakamura, Santosh Mashetty, Nisarg Patel, Mohith Kulkarni, Aswin RRV, Md Nayem Uddin, Divij Handa, Venkatesh Mishra, Amir Saeidi, Shrinidhi Kumbhar, Bimsara Pathiraja, and Nemika Tyagi—for their collaboration, friendship, and constant encouragement throughout this journey.

Finally, I owe my deepest gratitude to my parents, Prafullsinh Parmar and Siddhi Chaudhari, for their unwavering trust, love, and sacrifices that have shaped my path. My heartfelt thanks go to my partner Mirali Purohit, whose constant support—both in research and in life—has been my greatest strength throughout this PhD journey. I am also thankful to my family, especially my brothers and sisters. I am also lucky to have amazing friends from school and college (Rabari, Patel, Swapnil, Krunal, Dhaval, Vasava, Yash, Nisarg, Jenish, Kaushik, Savan, Modi), whose love and encouragement have been a source of endless motivation.

Thank you all for your faith, and support—you all have been the foundation that sustained me through this incredible journey.

TABLE OF CONTENTS

	Page
LIST OF TABLES	xiii
LIST OF FIGURES	xxiii
CHAPTER	
1 INTRODUCTION	1
1.1 Overview	1
1.2 Main Contributions	4
1.3 Chapter Summaries	8
2 BACKGROUND AND RELATED WORKS	11
2.1 Logical Reasoning with LLMs	11
2.1.1 Datasets and Benchmarks	11
2.1.2 Puzzle-Solving Task	13
2.1.3 Automatic Evaluation of Reasoning Chains	14
2.2 LLM Agents for Reasoning and Planning	15
2.2.1 Inference-Time Algorithms	15
2.2.2 Post-Training Methods for LLMs	17
2.3 Instruction-Tuning	18
2.4 Biases in NLU Benchmarks	19
2.5 Data Synthesis using LLMs	20
2.6 Application: Automation of Systematic Review	21
3 LOGICBENCH: BENCHMARKING LOGICAL REASONING	23
3.1 Introduction	23
3.2 LogicBench	26
3.2.1 Logics Types	26
3.2.2 Data Creation	28

3.2.2.1	Sentence Generation	29
3.2.2.2	NL Conversion	32
3.2.2.3	Task Instance Generation	39
3.2.3	Statistics and Qualitative Analysis	41
3.2.4	Examples of Data Instances	43
3.3	Results and Analysis	45
3.3.1	Experimental Setup	45
3.3.2	Prompts for Experiments	50
3.3.3	Main Results	53
3.3.4	Few-shot Results	54
3.3.5	Analysis and Discussion	58
3.4	Conclusions and Future Works	65
4	MULTI-LOGIEVAL: TOWARDS EVALUATING MULTI-STEP LOGICAL REASONING	67
4.1	Introduction	67
4.2	Multi-LogiEval	70
4.2.1	Data Generation for Monotonic Logic	71
4.2.1.1	Monotonic Logic Description	71
4.2.1.2	Generation of Rule Combination	73
4.2.1.3	Generation of Data Instances	76
4.2.2	Non-Monotonic Reasoning	81
4.2.3	Qualitative Analysis	94
4.3	Results and Analysis	96
4.3.1	Experimental Setup	96

4.3.2	Main Results	98
4.3.3	Analysis and Discussion	100
4.4	Conclusions and Future Works	105
5	GRID PUZZLES: DIAGNOSING FAILURES IN LLM REASONING	107
5.1	Introduction	107
5.2	Evaluation of Reasoning Chains	109
5.2.1	<i>GridPuzzle</i>	109
5.2.2	Manual Evaluation	111
5.2.3	Proposed Error Taxonomy	113
5.2.4	Automated Evaluation	115
5.2.5	Example Annotated Reasoning Chains	118
5.3	Results and Analysis	118
5.3.1	Experiments	118
5.3.2	Metrics	121
5.3.3	Objective Evaluation	121
5.3.4	Reasoning Chain Evaluation	122
5.3.5	Mitigation Strategies	126
5.4	Conclusion	133
6	PLANGEN: A MULTI-AGENT FRAMEWORK FOR COMPLEX REASONING TASKS	135
6.1	Introduction	135
6.2	PlanGEN	139
6.2.1	Proposed LLM Agents	139
6.2.1.1	Constraint Agent	139

6.2.1.2	Verification Agent	141
6.2.1.3	Selection Agent	147
6.2.2	Proposed Frameworks	153
6.2.2.1	PlanGEN (Best of $\mathcal{N}$)	153
6.2.2.2	PlanGEN (ToT)	153
6.2.2.3	PlanGEN (REBASE)	154
6.2.2.4	PlanGEN (Mixture of Algorithms)	158
6.3	Experiments and Results	158
6.3.1	Experimental Setup	159
6.3.2	Main Results	161
6.4	Analysis and Discussion	167
6.5	Conclusions	175
7	PLAN-TUNING: POST-TRAINING FOR IMPROVING REASONING CAPABILITIES OF LLMs	181
7.1	Introduction	181
7.2	Proposed Method	188
7.2.1	Task Formulation	188
7.2.2	Data Generation	190
7.2.3	PLAN-TUNING	193
7.2.3.1	SFT Training	193
7.2.3.2	GRPO Training	194
7.3	Results and Analysis	197
7.3.1	Experimental Setup	197
7.3.2	Main Results	199

7.3.3	Analysis	201
7.4	Conclusions	204
8	DON'T BLAME ANNOTATORS	205
8.1	Introduction	205
8.2	Instruction Bias in NLU Benchmarks	207
8.2.1	Patterns in Crowdsourcing Instructions	208
8.2.2	Instruction Bias Propagation to Datasets	210
8.3	Effect on Model Learning	215
8.3.1	Experimental Setting	215
8.3.2	Results	216
8.4	Conclusions and Discussion	219
9	TARGEN: TARGETED DATA GENERATION	221
9.1	Introduction	221
9.2	TarGEN	223
9.2.1	Problem Formulation	224
9.2.2	Task-specific prompting strategies	225
9.2.3	Self-correction	228
9.2.4	Task-specific Instance Generation Pipelines	231
9.2.5	Prompts for Instruction Tuning	247
9.3	Experiments and Results	257
9.3.1	Results	258
9.4	Analysis	258
9.5	Conclusion	266
10	IN-BOXBART: INSTRUCTION-TUNING FOR BIOMEDICAL TASKS	268

10.1	Introduction	268
10.2	BoX	270
10.2.1	Tasks	271
10.2.2	Biomedical Instructions.....	275
10.2.2.1	Unified Schema.....	275
10.2.2.2	Construction of BI	276
10.3	Problem Setup and Models	278
10.3.1	Problem setup	278
10.3.2	Models	279
10.3.2.1	Baselines	280
10.3.2.2	Proposed Model	280
10.4	Experiments and Analysis	281
10.4.1	Experimental Setup	281
10.4.2	Results and Findings	282
10.4.3	Analysis	285
10.4.4	Discussion	287
10.5	Summary and Conclusions.....	290
11	LLMS FOR CLINICAL SYSTEMATIC REVIEWS	291
11.1	Introduction	291
11.2	Data and Methods	293
11.2.1	Data Sources	293
11.2.2	Task Formulation and Models.....	295
11.2.3	Baseline Method	295
11.2.4	Proposed Methods: Collaborative LLMs	296

11.2.5	Prompt Development	299
11.2.6	Performance Evaluation	302
11.3	Results and Discussion	304
11.3.1	Objective Results	304
11.3.2	Manual Work Saved	306
11.3.3	Conversational LLM-LLM Interaction	307
11.4	Discussion	312
12	PROMPT REFORMULATION IN CLINICAL TASKS	316
12.1	Introduction	316
12.2	Prompt Reformulation and Results	318
12.2.1	Data Source	318
12.2.2	Manual Error Annotation	320
12.2.3	Prompt Reformulation	321
12.2.4	Main Results	326
12.3	Future Works	330
13	CONCLUSIONS AND FUTURE WORKS	331
	REFERENCES	337
	APPENDIX	
A	RELATED PUBLICATION DETAILS	369

LIST OF TABLES

Table		Page
2.1	Comparison of existing reasoning logical reasoning datasets with proposed benchmarks.	13
3.1	Inference rules and (two) axioms that establish the relationship between premises and conclusions. MP: Modus Ponens, MT: Modus Tollens, HS: Hypothetical Syllogism, DS: Disjunctive Syllogism, CD: Constructive Dilemma, DD: Destructive Dilemma, BD: Bidirectional Dilemma, CT: Commutation, MI: Material Implication, EG: Existential Generalization, UI: Universal Instantiation	27
3.2	Illustrative examples of non-monotonic reasoning adapted from [120]. ..	29
3.3	Illustrative examples of logical context and questions created using sentences that are generated in the first stage (§3.2.2.1) for both tasks, i.e., BQA and MCQA.....	36
3.4	Illustrative examples of NL logical context and questions created using sentences that are generated in §3.2.2.1 for rules covered in PL and FOL for BQA.	37
3.5	Illustrative examples of NL logical context and questions created using sentences that are generated in §3.2.2.1 for NM logic for BQA.	38
3.6	Statistics of <i>LogicBench(Eval)</i>	42
3.7	Examples of context and question-answer pairs for each rule of <i>Proportional logic</i> from the <i>LogicBench(Eval)</i>	46
3.8	Examples of context and question-answer pairs for each rule of <i>First order logic</i> from the <i>LogicBench(Eval)</i>	47
3.9	Examples of context and question-answer pairs for each rule of <i>Non-monotonic logic</i> from the <i>LogicBench(Eval)</i>	48

3.10	Evaluation of LLMs in terms of label-wise accuracy on $LogicBench(Eval)_{BQA}$, where $A(Yes)$ and $A(No)$ denote the accuracy for the <i>Yes</i> and <i>No</i> labels, respectively. DRI: Default Reasoning with Irrelevant Information, DRS: Default Reasoning with Several Defaults, DRD: Default Reasoning with a Disabled Default, DRO: Default Reasoning in an Open Domain, RE1: Reasoning about Unknown Expectations I, RE2: Reasoning about Unknown Expectations II, RE3: Reasoning about Unknown Expectations III, RAP: Reasoning about Priorities	55
3.11	Evaluation of LLMs in terms of accuracy on $LogicBench(Eval)_{MCQA}$. . .	56
3.12	Performance of LLMs in few-shot setting in terms of label-wise accuracy on $LogicBench(Eval)$, where $A(Yes)$ and $A(No)$ denote the accuracy for the <i>Yes</i> and <i>No</i> labels, respectively.	57
3.13	Human performance on three logic types averaged across three annotators for both tasks.	59
3.14	Case study of reasoning chains for PL(MT) generated from ChatGPT, and FOL(DD) generated from GPT-4.	61
3.15	Evaluation of Yi-34B in terms of accuracy on $LogicBench(Eval)_{BQA}$ and $LogicBench(Eval)_{MCQA}$	64
3.16	Performance comparison between LogicT5 and baseline T5-large in terms of accuracy.	66

4.1	Inference rules that establish the relationship between premises and their corresponding conclusions. A subset of these inference rules is adapted from Parmar <i>et al.</i> [174]. MP: Modus Ponens, MT: Modus Tollens, HS: Hypothetical Syllogism, DS: Disjunctive Syllogism, CD: Constructive Dilemma, DD: Destructive Dilemma, BD: Bidirectional Dilemma, CT: Commutation, DMT: De Morgan’s Theorem, CO: Composition, IM: Importation, MI: Material Implication, EG: Existential Generalization, UI: Universal Instantiation	72
4.2	FOL inference rules that establish the relationship between multiple variables	74
4.3	2-step reasoning rule combinations for PL and FOL.	75
4.4	3-step reasoning rule combinations for PL and FOL.	77
4.5	4-step reasoning rule combinations for PL and FOL.	78
4.6	5-step reasoning rule combinations for PL and FOL.	79
4.7	Natural language examples of rule combinations of each depth for PL. .	82
4.8	Natural language examples of rule combinations of each depth for FOL.	83
4.9	Illustrative examples of non-monotonic reasoning adapted from [120]. . .	84
4.10	Natural language examples of rule combinations of depth-2 for NM. . .	86
4.11	Natural language examples of rule combinations of depth-3 for NM. . .	87
4.12	Natural language examples of rule combinations of depth-4 for NM. . .	88
4.13	Natural language examples of rule combinations of depth-5 for NM. . .	89
4.14	Statistics of <i>Multi-LogiEval</i>	95
4.15	Evaluation of LLMs in terms of accuracy on <i>Multi-LogiEval</i>	99
4.16	Few-shot Evaluation of LLMs in terms of accuracy on <i>Multi-LogiEval</i> . .	100

4.17	Performance Metrics by Reasoning Depths	105
5.1	Proposed error taxonomy for broad categories based on manual analysis. If a statement starts with “so, therefore, hence, this means, this implies, etc.” and/or is not followed by any premise, consider the previous statement’s conclusion or the previous NC as the premise.	114
5.2	Proposed error taxonomy for sub-categories based on manual analysis. These sub-categories are defined for cases where either the conclusion or premise is incorrect (“RW” or “WR”) or both are incorrect (“WW”). For “WW”, the error sub-categories might appear in any combinations between (1-6) and (a-c) such as ‘1a’, ‘4b’, or ‘6c’.	115
5.3	Examples of reasoning chain evaluated by Auto-evaluator (GPT-4o). . .	117
5.4	The results for accuracy using GPT-4-Turbo, with and without mitigation strategies for the 60 samples of 3×4 grid-size.	130
6.1	Example of constraint generation for the calendar scheduling benchmark from NATURAL PLAN.	142
6.2	Example of constraint generation for the meeting planning benchmark from NATURAL PLAN.	143
6.3	Examples of constraints generated by the constraint agent given reasoning question. The example illustrates the constraint generation for solving physics question from GPQA	144
6.4	Examples of constraints generated by the constraint agent given mathematical reasoning problem. The example illustrates the constraint generation for solving MATH question from OlympiadBench	145

6.5	Examples of verification generated by the verification agent given a generated plan and constraints. This example illustrates the verification process for calendar scheduling from NATURAL PLAN	148
6.6	Performance comparison for model-agnostic nature of PlanGEN . We utilize Gemini-1.5-Pro, Gemini-2.0-Flash, GPT-4o, and Gemini-2.5-Pro as baseline and underlying models in PlanGEN frameworks. Comparing methods that use the same base and underlying model for a fair assessment. MoA: Mixture of Algorithms.	171
6.7	Performance impact of hyper-parameters on inference-time algorithms in OlympiadBench	172
6.8	Algorithm Selection Frequency by Dataset	172
6.9	Comparison between a constraints-based CoT baseline and best-performing PlanGEN framework.	173
6.10	Accuracy on blocksworld v1 and v3 across different methods for plan generation task.	174
6.11	Example of generated plan for the calendar scheduling category from NATURAL PLAN.	176
6.12	Example of generated plan for the meeting planning category from NATURAL PLAN.	177
6.13	Examples of generated plans by PlanGEN (Best of $\mathcal{N}$) given physics problem. The example illustrates the plan generation and its execution for physics problem from GPQA.	178

6.14	Examples of generated plans by PlanGEN (Best of $\mathcal{N}$) given financial reasoning problem. The example illustrates the plan generation and its execution for a problem from DocFinQA.	179
6.15	Examples of generated plans by PlanGEN (Best of $\mathcal{N}$) given maths problem. The example illustrates the plan generation and its execution for a problem from OlympiadBench (MATH).	180
7.1	Statistics of the datasets used in our experiments. Training set sizes are shown for in-domain benchmarks, and evaluation set sizes for both in-domain and out-of-domain benchmarks.	198
7.2	This table reports the accuracy (%) of the base LLM and its two plan-tuned variants (custom SFT and GRPO) on four mathematical reasoning benchmarks. Columns 1–2 show in-domain performance on GSM8K and MATH. Columns 3–4 present out-of-domain generalization on OlympiadBench and AIME 2024. These results demonstrate that leveraging synthetic planning trajectories via both SFT and RL objectives improves reasoning accuracy in smaller LLMs. $\mathcal{M}$ (GSM8k): Model trained using GSM8k dataset, $\mathcal{M}$ (MATH): Similar for MATH. .	199
8.1	Statistics of number of train and test examples with and without instruction patterns. $\mathcal{S}_{\text{train}}$: set of examples in train set, $\mathcal{S}_{\text{train}}^p$: set of examples in train set with instruction pattern, $\mathcal{S}_{\text{train}}^{-p}$: set of examples in train set without instruction pattern, $\mathcal{S}_{\text{test}}$: set of examples in test set, $\mathcal{S}_{\text{test}}^p$: set of examples in test set with instruction pattern, $\mathcal{S}_{\text{test}}^{-p}$: set of examples in test set without instruction pattern.	208

8.2	Tasks of each dataset and number of examples in crowdsourcing instruction of each dataset. RC: Reading Comprehension, QA: Question Answering.	209
8.3	Portion of patterns in instruction examples (Ins.) and in the corresponding train ($\mathcal{S}_{\text{train}}$) and test ($\mathcal{S}_{\text{test}}$) sets of NLU datasets. $\text{AUX} \in \{\text{am, is, are, was, were, has, have, had, do, does, did, will, would, can, could, may, might, shall, should, must}\}$, and $_$ is an empty string. MC-TACO has 5 different data subsets corresponding to different types of temporal reasoning (see Tab. 8.2), hence, the sum of percentages for this dataset exceeds 100%.	211
8.4	Examples of data instances from original dataset that contain instruction patterns. $\text{AUX} \in \{\text{am, is, are, was, were, has, have, had, do, does, did, will, would, can, could, may, might, shall, should, must}\}$. $_ :$ $\langle \text{blank} \rangle$	213
8.5	Performance of T5 models on $\mathcal{S}_{\text{test}}^p$ vs. $\mathcal{S}_{\text{test}}^{-p}$ of models trained on data instances containing instruction patterns ($\mathcal{S}_{\text{train}}^p$).	216
8.6	Performance of T5 models on $\mathcal{S}_{\text{test}}^p$ vs. $\mathcal{S}_{\text{test}}^{-p}$ of models trained on $\mathcal{S}_{\text{train}}$	217
8.7	Performance of BART models on $\mathcal{S}_{\text{test}}^p$ vs. $\mathcal{S}_{\text{test}}^{-p}$ of models trained on data instances containing instruction patterns ($\mathcal{S}_{\text{train}}^p$).	218
8.8	Performance of BART on $\mathcal{S}_{\text{test}}^p$ vs. $\mathcal{S}_{\text{test}}^{-p}$ of models trained on $\mathcal{S}_{\text{train}}$	218
9.1	Statistics of the SuperGLUE dataset. The following abbreviations are used Bin. Class: Binary Classification, Ent: Entailment, Cont: Contradiction, NLI: Natural Language inference	226

9.2	Performance of various models over SuperGLUE tasks. For each dataset, we compare the performance of these models (Cerebras, Pythia, T5, FLAN, RoBERTa) when trained on 4 distinct variants: Og (original train data), Syn (synthetic train data), Og-I (inst. tuning original train data), Syn-I (inst. tuning synthetic train data). For each task, we denote the highest performing model trained on original train data in green, and the highest performing model trained on synthetic train data in blue. All results are presented in %.....	259
9.3	Avg. performance across tasks, models, and data variants. Og, Syn, and I represent Original, Synthetic, and Instruction tuning, respectively.	260
9.4	Results using a T5-3B model trained in a multi-task fashion. Original: Using a combined set of original datasets to train the model. Synthetic: Using synthetic versions to train the model. All numbers are in %. We measure performance in accuracy, except in the case of ReCoRD, where we use the Rouge-L score. Higher score is highlighted. We find that model trained on synthetic data is significantly better than the one trained on original data.	260
9.5	Lexical diversity of the dataset. Figures in red correspond to where the diversity of the synthetic dataset is limited. Blue corresponds to where the synthetic dataset has more diversity than the original.	261
9.6	Comparison of Synthetic SuperGLUE and Self-Instruct Dataset. Llama 2 (7B) is pre-finetuned on both datasets individually and evaluated on the OpenLLM leaderboard.	267

10.1	Statistics of training (i.e., Train), validation (i.e, Dev) and evaluation (i.e., Test) data for all instruction tasks from the BoX. RFHD: Risk Factor for Heart Disease.	272
10.2	Examples of one instruction tasks converted into text-to-text format for each category	279
10.3	Results comparison between single-task baseline, Vanilla-BoXBART and In-BoXBART in terms of Rouge-L and F_1 -Score. All the results for F_1 -Score are presented in %. V-BB: Vanilla-BoXBART, I-BB: In-BoXBART, RFHD: Risk Factor for Heart Disease.	283
10.4	Comparison of few-shot learning results in terms of Rouge-L between single-task models and In-BoXBART for 32/100/1000 training samples per instruction tasks. All results are presented in %. S: Single-task model, I-BB: In-BoxBART, RFHD: Risk Factor for Heart Disease.....	286
10.5	The state-of-the-art (SOTA) results for each task compared with Vanilla-BoXBART and In-BoXBART. All results are in %. F: F_1 -score, V-BB: Vanilla-BoXBART, I-BB: In-BoXBART, RFHD: Risk Factor for Heart Disease.	288
11.1	Characteristics of the included systematic review datasets. N – number; mHSPC : metastatic hormone-sensitive prostate cancer; mCRPC : metastatic castration resistant prostate cancer; RCC : renal cell carcinoma; mHCC : metastatic hepatocellular carcinoma; PARPi : poly (ADP-ribose) polymerase inhibitors	294
11.2	Performance of individual LLMs and conflict resolution approaches. Ex: Exclusion, In: Inclusion	304

11.3	Work saved and work saved over samples lost across different proposed approaches.....	306
12.1	Sample tasks from MedAgentBench [93] with corresponding context....	319
12.2	Error Categories Identified in MedAgentBench LLM Outputs	322
12.3	Example for Output Format Violation	326
12.4	Example for Wrong Input Argument to POST API.....	327
12.5	Example for Wrong Input Argument to GET API	328
12.6	Example for Incorrect Calculation of Average	329
12.7	Performance Comparison on MedAgentBench with GPT-4o	329

LIST OF FIGURES

Figure		Page
1.1	Overview of the dissertation contributions grouped by theme. The figure highlights three major thrusts of this work, Natural Language Reasoning, Data Analysis, and Biomedical Applications, along with corresponding publications that are covered in this dissertation. Each box represents a core contribution.	2
3.1	Comprehensive representation of different inference rules and reasoning patterns covered by propositional, first-order, and non-monotonic logics. <i>Exp.</i> indicates Expectation.	24
3.2	Schematic representation of three-stage procedure for data creation. NL: Natural Language	30
3.3	Schematic representation of prompt.	30
3.4	Example prompt for <i>Modus Tollens</i> inference rule from PL.	31
3.5	Word cloud of context present in the <i>LogicBench(Eval)</i>	43
4.1	Performance (avg. accuracy across each depth for PL & FOL) of various LLMs on <i>Multi-LogiEval</i>	68
4.2	Process for combining multiple logical inference rules for PL and FOL: <i>Premise 1</i> is the set of premises for the first inference rule, leading to <i>Conclusion 1</i> . <i>Conclusion 1</i> and <i>Premise 2</i> derive <i>Conclusion 2</i> , and so on. $\vdash$: Entails.	74
4.3	Data generation prompt for PL and FOL	76
4.4	An example prompt for the 3-step combination of inference rules CD, DS, and MP from propositional logic.	80

5.1	Schematic representation of proposed pipeline. Begins with the data collection of <i>GridPuzzle</i> dataset (top left) and evaluating various LLMs in zero-shot CoT setting (bottom left), then analyzing reasoning chains of LLMs manually to find various error types (top right) and automate this analysis process using LLM to check the correctness of reasoning chain by finding errors (bottom right).....	108
5.2	The prompt structure of a 4 x 4 grid size puzzle from <i>GridPuzzle</i> . Every Zero-shot-CoT prompt from the dataset consists of Puzzle, its corresponding Clues, the Instruction for solving the puzzle, along with the Gold solution of the Puzzle.	110
5.3	The 12-step guideline provided to the annotators to conduct manual evaluation of the reasoning chains.	112
5.4	The prompt structure of auto-evaluator which is followed by a user prompt with the puzzle, its gold solution, and the reasoning chain to be evaluated.	116

- 5.5 The **top left** section of the figure consists of a 3x4 sample puzzle from the *GridPuzzle* dataset along with the Zero-shot-CoT prompt. Right below the prompt, we have the Gold solution for the corresponding puzzle. In the **top right** section of the figure, we have the Model-generated Reasoning chain to solve this puzzle along with the Final Answer. In this particular instance, the reasoning chain was generated by the GPT-4 model. In the **bottom half** of the figure, we have the manually evaluated reasoning chain, split into individual sentences. The corresponding error categories found in each step is given the **bottom right** column called ‘Error Categories.’ 119
- 5.6 The **top left** section of the figure consists of a 3x4 sample puzzle from the *GridPuzzle* dataset along with the Zero-shot-CoT prompt. Right below the prompt, we have the Gold solution for the corresponding puzzle. In the **top right** section of the figure, we have the Model-generated Reasoning chain to solve this puzzle along with the Final Answer. In this instance, the reasoning chain was generated by the Llama2-13b model. In the **bottom half** of the figure, we have the GPT-4o Auto-Evaluated Reasoning chain. The auto-evaluation is done sentence-wise and the output is in a JSON-structured format consisting of 5 components: the *Sentence*, the *Premise*, the *Conclusion*, the *Error category* and the *Sub-category*. The corresponding error categories found in each sentence are given in the **bottom right** columns called ‘Error Categories’ and ‘Sub Categories.’ 120

5.7	Performance of five different LLMs in terms of accuracy on the <i>Grid-Puzzle</i> dataset.	122
5.8	The percentage distribution of the broad error categories across the combined reasoning steps of all five LLMs. The total number of steps generated by each model is provided inside the round brackets below the model names.	123
5.9	The first five sub-figures in the above section show the error Sub-category distribution over five LLMs. The last sub-figure denotes the top 10 error sub-category distribution across all model reasoning steps.	123
5.10	The prompt structure for the Plan-and-Solve strategy which is split into two prompts one for planning and the other for solving the puzzle.	127
5.11	The prompt structure for the Feedback-learning strategy. The system prompts consist of instructions regarding the major errors as well as ways to rectify those errors.....	128
5.12	The prompt structure for the Program of Thought prompting technique. In the first part, we asked the LLM to generate a code to solve the given puzzle. In the second part, an LLM is prompted to implement the code produced in the first part to get the Final Answer table.	129
5.13	The prompt structure for the Self-Correct strategy is split into 3 parts. The first prompt solves the puzzle, the second prompt verifies the solution and gives suggestions to improve the solution, and the third prompt uses these suggestions along with error taxonomy-based instructions to refine the final solution.	131

5.14	The prompt structure for the Self-Discover strategy. In the first part of this prompt the model is prompted to assess the problem and select the appropriate reasoning module to solve it. Then the module is modified to give a structured plan to solve the puzzle. In the second part, the model uses this structured plan along with instructions from our error taxonomy to solve the puzzle.	132
6.1	Schematic representation of PlanGEN (Mixture of Algorithms). An initial plan and constraints guide iterative plan refinement. The verification agent provides reward scores for plan quality, and the selection agent chooses inference algorithms until the highest-reward plan is found and used for downstream reasoning (if needed). UCB: Upper Confidence Bound, BoN: Best of $\mathcal{N}$, ToT: Tree-of-Thought, RS: REBASE.	137
6.2	Ablation Study of UCB Modifications on Selection Agent and its impact on PlanGEN (Mixture of Algorithms). div.: diversity bonus, rec: recovery term.	150
6.3	Schematic representation of PlanGEN (BoN).	154
6.4	Schematic representation of PlanGEN (ToT). Highest-reward steps are highlighted in green.	155
6.5	Schematic representation of PlanGEN (REBASE). Green shading indicates step reward (darker = higher). Darker steps are prioritized for exploration.	156

6.6	Performance comparison of the proposed multi-agent frameworks against baselines across four benchmarks. All experiments are conducted using Gemini-1.5-Pro. Algo: Algorithms, PHY: Physics.	162
6.7	Performance comparison of inference-time algorithms across different complexity levels for calendar scheduling from NATURAL PLAN.	163
6.8	Performance comparison of inference-time algorithms across different complexity levels for meeting and trip planning from NATURAL PLAN	164
6.9	Comparison of baselines and our frameworks, showing the trade-off between LLM calls and performance (%) for OlympiadBench (MATH).	166
6.10	Comparison of baselines and our frameworks, showing the trade-off be- tween time (seconds) and performance (%) for OlympiadBench (MATH).	167
6.11	Logistic regression plot showing verification agent’s positive perfor- mance impact. $P(\text{Successful Outcome}) = \text{probability of prediction be-}$ ing correct.	168
6.12	KDE plot illustrating the relationship between reward value and out- come (success/failure).	169
6.13	Case study on NATURAL PLAN, showing the impact of selection agent. Ver.: Verification	170
7.1	An example planning trajectory for a problem adapted from the MATH [78].	185

7.2	Left: Large-scale LLM generates multiple planning candidates (green = high-quality, red = low-quality). A <i>Plan Verifier</i> scores plans, and an <i>Answer Verifier</i> confirms the final answer; only trajectories passing both become the training corpus. Right: These trajectories train the policy model via SFT and RL objectives. P and A denote gold synthetic planning and answer trajectories, while R_p and R_a are their respective rewards.	186
7.3	Overview of the PLAN-TUNING pipeline. First, a large LLM generates multiple candidate natural-language planning trajectories for each problem. Next, a Plan Verifier scores and filters these trajectories, and a Math Verifier executes and validates only those above a quality threshold. Finally, the curated plan-answer pairs are used to train the target model via both SFT and RL (GRPO) objectives.	189
7.4	Comparison of plan-tuning accuracy on GSM8K and MATH when trained on each dataset individually <i>vs.</i> their combined corpus. $\mathcal{M}_i$ (Single Dataset) indicates that the respective method is trained on a given dataset (green), while other indicates that the respective method is trained on both datasets (blue).....	202
7.5	Qualitative Example: Comparison of SFT <i>vs.</i> PLAN-TUNING on a sample problem from GSM8k.	203

9.1	An overview of using TarGEN to generate instances for the WiC task. We first create a set of prompts (boxes 1, and 2 in the Figure) to generate instance seeds, or linguistic components unique to each task instance. Next, we create label-specific prompts (box 3) that generate instances based on instance seeds and the relationship implied by the label for this task. We use zero-shot LLM inference to generate an initial set of synthetic instances. The instances are then passed to our <i>self-correction</i> module, consisting of a single meta-prompt which allows us to re-label mislabeled data instances, helping us reduce noise. Hence, based on the task description, we obtain high-quality synthetic instances to evaluate a task.	222
9.2	Matrices showing the effect of the <i>self-correction</i> step across various datasets of SuperGLUE. The row values show the number of labels that were originally assigned to that label (ent, non: entailment, non-entailment; neutr, contr: neutral, contradiction). The number in a cell (i, j) reflects the number of labels originally assigned to label i which were re-labeled to label j after self-correction. While the majority of the instances had their labels reaffirmed by self-correction, a significant number of instances were re-labeled as a result of this step.	229
9.3	Comparison of semantic diversity across datasets among the original and the synthetically generated dataset. It can be seen that the original datasets' cosine similarity is higher for most tasks as compared to the synthetic datasets, which have a consistently lower cosine similarity, indicating higher semantic diversity.	261

9.4	Comparison of PVI ($\mathcal{V}$ -usable information) across datasets for the original dataset and the synthetically generated dataset. Synthetic data seems to have better quality as Original datasets PVI is concentrated around -0.1 to 0.1 whereas the synthetic data generated has a diverse mix of difficulty level among the samples.	262
9.5	Comparison of dataset bias for the BoolQ dataset and the synthetically generated BoolQ dataset. This represents the distribution for the GPE (Geo Political Entity) named entity tag. The distribution for other entities can be found in the appendix section.	263
9.6	Comparison of dataset bias for the BoolQ dataset and the synthetically generated BoolQ dataset.	264
9.7	Comparison of dataset bias for the CB dataset and the synthetically generated CB dataset.	264
9.8	Comparison of dataset bias for the Record dataset and the synthetically generated Record dataset.	265
9.9	Comparison of dataset bias for the RTE dataset and the synthetically generated RTE dataset.	265
9.10	Comparison of dataset bias for the WIC dataset and the synthetically generated WIC dataset.	266
10.1	Schematic representation of multi-tasking in the biomedical domain using instructional prompts. In this approach, a model is allowed to utilize tasks to get familiar with instructions and use them to map a given input to its corresponding output.	269

10.2	Schematic representation of 9 categories of tasks: each block represents one category with various tasks equipped with instruction.	271
10.3	Unified schema used to create a Biomedical Instruction (BI).	276
10.4	Example of Biomedical Instruction (BI) and task instances from BioNLP11ID (NER) dataset.	277
10.5	Comparison of on average Rouge-L across all instruction tasks between single-task and In-BoXBART based on the average number of training instances per task.	285
11.1	Illustration of Zero-Shot Evaluation Pipeline.	296
11.2	A schematic representation of collaborative LLMs approaches for conflict resolution.	297
11.3	Illustration of cross-LLM interaction for conflict resolution.	298
11.4	(a) Average overall accuracy for independent LLMs as well as our proposed approaches. (b) Evaluation of LLMs in terms of overall accuracy across the five systematic review datasets for independent LLMs as well as our proposed approaches.	305

12.1	Schematic overview of the MedAgentBench clinical agent pipeline (top - adapted from Jiang <i>et al.</i> [93]) and the prompt reformulation loop (bottom). Top: A clinician’s instruction is sent via the Agent Orches- trator to an LLM agent, which issues FHIR GET/POST requests to the electronic medical record environment and returns structured feed- back. Bottom: The original prompt and its outputs are annotated for error types, then fed to an LLM-based reformulator that generates an updated prompt; this loop repeats until the desired performance is achieved.	317
------	---	-----

Chapter 1

INTRODUCTION

1.1 Overview

Reasoning is a fundamental aspect of human intelligence, and equipping large language models (LLMs) with strong reasoning capabilities is crucial for tackling complex, logic-intensive tasks. Recent advances in LLMs—exemplified by models such as GPT-3 and GPT-4 [20], Google’s Gemini [236] and Anthropic’s Claude [5]—have led to remarkable performance on a wide range of natural language tasks previously thought to be exclusive to humans. Past attempts have been made in enhancing various reasoning skills in LLMs [102], including mathematical problem solving, commonsense reasoning, temporal reasoning, and formal logical inference. These efforts indicate that advanced LLMs can appear to reason in sophisticated ways, solving arithmetic problems or answering commonsense queries by leveraging vast amounts of data and learned patterns.

Despite this progress, significant limitations remain in current LLMs’ reasoning capabilities when it comes to complex problem solving. In particular, models struggle with systematic, formal, and multi-step reasoning tasks that require strict logic or longer inference chains. For example, when exposed with a logical puzzle or a deductive reasoning problems that require applying several rules in sequence, a state-of-the-art LLMs might produce a confident-sounding answer that is logically incorrect or inconsistent with the premises. Such failures suggest that, while LLMs are good at pattern matching and fluent language generation, they often lack the ability to apply formal logic or to carry out step-by-step reasoning/planning beyond a few hops of

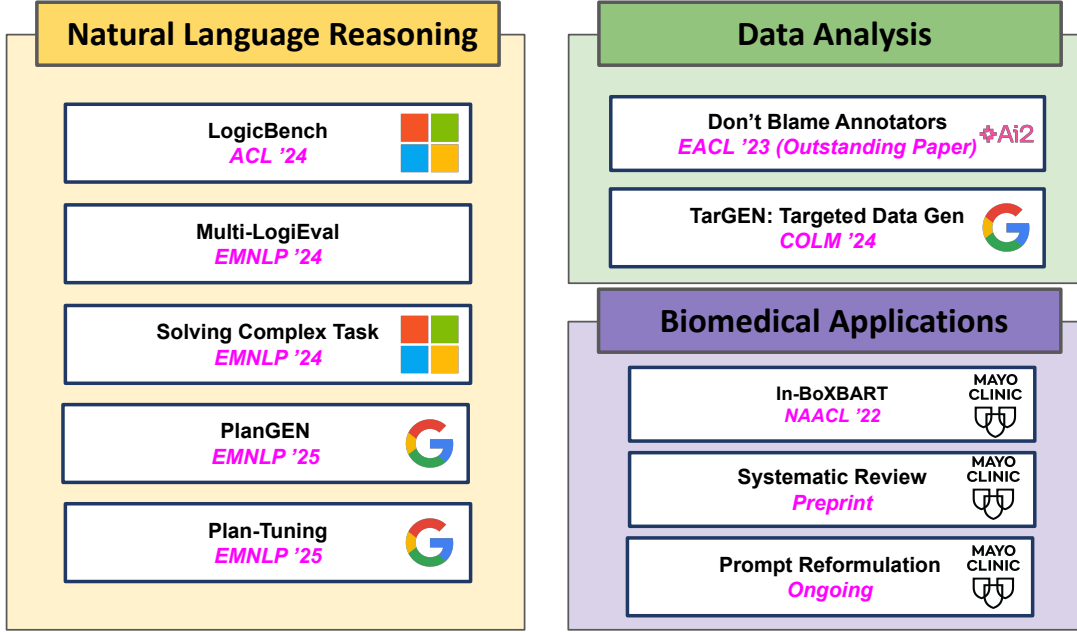


Figure 1.1: Overview of the dissertation contributions grouped by theme. The figure highlights three major thrusts of this work, Natural Language Reasoning, Data Analysis, and Biomedical Applications, along with corresponding publications that are covered in this dissertation. Each box represents a core contribution.

inference. These shortcomings are especially more prominent in tasks such as deduction, complex puzzle-solving, handling negation and quantifiers – areas where a single misstep can lead to a fundamentally wrong conclusion, and complex scientific natural language (NL) reasoning or planning problems. Moreover, the underlying causes of these reasoning failures are not yet well understood. It remains an open challenge to determine why LLMs have problem in performing systematic NL reasoning.

LLM reasoning is important in domain-specific applications, particularly in the biomedical field, where not only factual knowledge but also reasoning and interpretability are necessary. For instance, a clinical decision-support system powered by an LLM must be able to logically combine medical facts, follow clinical guidelines, and draw correct conclusions—a failure to do so could have serious real-world consequences. Similarly, conducting a biomedical literature review or planning a treatment guideline involves reasoning through complex, structured information. Current LLMs

often fall short in these scenarios: they might overlook critical details (e.g., applying inclusion/exclusion criteria in a systematic review), or produce answers that are not reliable. All these challenges show a need for improving the reasoning performance of LLMs, especially for logic-intensive tasks and domains such as clinical applications. In summary, there is a motivation to enhance both our evaluation of the natural language reasoning and planning capabilities.

This dissertation is driven by the following central research question: **How can we systematically evaluate, analyze, and improve the reasoning performance of LLMs, particularly in logic-intensive tasks and in the biomedical domain?** By addressing this question, we aim to push LLMs beyond pattern recognition toward systematic reasoning, enabling their deployment in complex problem-solving and real-world settings. This dissertation is based on research works divided in three different groups, as shown in Figure 1.1.

To answer the above question, we first identify key challenges that must be overcome. First, we need ways to evaluate logical and multi-step reasoning in LLMs. Traditional NLP benchmarks only implicitly assess reasoning, and many existing logical reasoning datasets either entangle logic with language understanding or cover a narrow subset of reasoning skills. A more systematic evaluation is required—one that can isolate different reasoning types (e.g., propositional, and first-order logic) and vary the depth of reasoning (how many inferential steps are needed). Second, we must analyze the failure modes of LLM reasoning. This includes identifying patterns in the mistakes models make when reasoning and understanding how factors such as training data or prompt design contribute to these errors. Third, we face the challenge of improving NL reasoning in LLMs in a generalizable way. Potential approaches include better inference algorithms (e.g., natural planning or self-refinement during generation), as well as training-time interventions (fine-tuning or data augmentation).

Another important challenge is data quality and coverage for training and evaluating reasoning. Many benchmarks and training datasets for LLMs come from crowdsourced annotations and instruction-following data. These datasets often have biases or artifacts – for instance, annotators might follow hidden patterns or prefer certain phrasings that models then latch onto, rather than truly learning the underlying reasoning task. Such instruction biases can give models spurious cues that degrade their performance on unbiased, real-world inputs. A thorough examination of these biases is needed, along with strategies to create or generate higher-quality data that cover reasoning cases more comprehensively. By curating or synthesizing datasets that target specific reasoning skills, we can hope to train models that are both more robust and more better at reasoning.

With these challenges in mind, the goals of this dissertation are threefold: (1) to develop systematic evaluation frameworks and benchmarks that can evaluate LLMs’ reasoning abilities across different logic-based tasks and depths of reasoning; (2) to devise and inference-time agentic and post-training methods for improving reasoning; and (3) to apply these advances to clinical applications, demonstrating that improved reasoning leads to tangible gains in domain-specific tasks.

1.2 Main Contributions

This dissertation makes several major contributions (Figure 1.1), organized into three broad thrusts. We present details about these contributions in this section.

Logical Reasoning We create and release multiple benchmarks that enable systematic and comprehensive evaluation of LLMs’ logical reasoning capabilities. The first is LogicBench [174], a broad collection of natural language inference tasks grounded in formal logic. LogicBench covers a wide spectrum of logic types – including propo-

sitional logic, first-order logic, and non-monotonic reasoning – and isolates individual inference rules in a question-answering format. By evaluating LLMs on single-step logical inferences, LogicBench provides fine-grained insight into which logical patterns a model can handle and which it struggles with. Building on this, I introduce Multi-LogiEval [176], a depth-aware evaluation framework for multi-step logic. Multi-LogiEval combines multiple inference rules to require reasoning chains of varying lengths; in doing so, it assesses an LLM’s ability to perform iterative, multi-hop logical deductions and to generalize to deeper reasoning than seen in training. In addition, we present GridPuzzle [246], a novel dataset consisting of complex logic puzzles (inspired by grid-based logic games). GridPuzzle is designed to diagnose reasoning errors in LLMs: it not only poses challenging puzzle problems but also provides an error taxonomy that categorizes the types of reasoning mistakes (e.g. contradiction, omission of a constraint, misapplication of a rule) that models make on each step. Together, these benchmarks form a foundation for evaluating logical reasoning systematically, under controlled conditions and across increasing levels of difficulty.

Multi-Agent System w/ Inference Strategies and Post-Training Methods for Reasoning The dissertation proposes new methods to improve the reasoning performance of LLMs, focusing on two complementary approaches. First, I develop PlanGEN [171], an inference-time framework that transforms the LLM into a multi-agent reasoner. In PlanGEN, the model’s reasoning process is guided by multiple specialized agents (for example, a constraint-checking agent, a verification agent, and a solution selection agent) that work together to plan and validate the reasoning steps before producing a final answer. I experiment with different reasoning strategies within this framework—such as a Best-of-N sampling, a Tree-of-Thought search, and the REBASE self-refinement technique—and show that combining planning with

these strategies can significantly improve the accuracy and consistency of LLM outputs on reasoning tasks. Second, I introduce Plan-Tuning [170], a post-training fine-tuning strategy to imbue LLMs with better reasoning skills. Plan-Tuning works by fine-tuning models on synthetic “planning” trajectories: essentially, we generate step-by-step reasoning examples (plans) for various tasks and use them to train the model to internalize a planning procedure. We leverage both supervised fine-tuning (learning from model-generated plans deemed successful) and reinforcement learning (rewarding the model for correct reasoning outcomes) in this approach. Our experiments demonstrate that Plan-Tuning yields models with notably higher reasoning accuracy, and importantly, that these models can generalize their improved reasoning to out-of-distribution problems – indicating an improvement to their reasoning ability beyond the fine-tuning data distributions.

Data Quality, Bias Analysis, and Targeted Data Generation Another major contribution of this work is in addressing the quality and coverage of training data for reasoning. We conduct an in-depth analysis of instruction biases in crowdsourced datasets – essentially asking how the way humans phrase instructions or construct training examples might systematically bias an LLM’s behavior. This analysis uncovers that many natural language understanding (NLU) benchmarks and instruction-following datasets exhibit implicit biases (for instance, certain answer choices being more likely regardless of context, or extraneous cues that shortcuts genuine reasoning). We provide a taxonomy of these instruction patterns and demonstrate how they propagate into model predictions, thereby highlighting an often overlooked obstacle to fair and robust reasoning [172]. To combat these issues and to improve the coverage of reasoning patterns in training data, we propose TarGEN [67], a targeted data generation pipeline. TarGEN uses LLMs themselves to synthesize new task data in a

controlled way: we prompt models to generate examples focusing on specific reasoning skills or corner cases that are under-represented in existing data. A key feature of TarGEN is an integrated self-correction loop, where the LLM not only generates candidate examples but also critiques and refines them to ensure quality and correctness. The outcome is a diverse set of high-quality synthetic data that can be used to augment training or evaluation. We show that training LLMs on TarGEN-generated data leads to better generalization on downstream tasks, and that the injection of instructional diversity (different ways of phrasing or structuring problems) makes models more robust to variations in how questions are asked.

Extensions to Biomedical and Clinical Domains Finally, this dissertation extends the NLP applications to specialized domains, to demonstrate the impact of improved systematic reasoning. I introduce In-BoXBART [173], which, to our knowledge, is the first instruction-tuned language model tailored for biomedical tasks. In-BoXBART is built by unifying a wide range of biomedical NLP tasks under a common instruction schema called BoX (Biomedical Task Ontology with XML-like tags), allowing the model to handle diverse tasks – such as named entity recognition, question answering, summarization of medical texts, and more – within a single, instruction-driven framework. By fine-tuning a base language model on a curated collection of 32 biomedical tasks expressed as instructions, we achieve a model that outperforms task-specific models and exhibits strong generalization across various biomedical benchmarks. Furthermore, we tackle a crucial use-case in evidence-based medicine: automating systematic literature reviews for clinical domains (specifically oncology). We develop an agentic methodology to simulate a two-reviewer process using LLMs, inspired by the standard practice in which two independent human reviewers screen and verify findings in a systematic review. In our simulation, two LLM

agents work in parallel to assess clinical study documents, cross-verify each other’s judgments, and resolve conflicts through interaction—effectively mirroring a human two-reviewer protocol. Our results show that this approach significantly reduces the manual workload required for literature screening while maintaining accuracy comparable to human reviewers. Finally, we explore prompt reformulation strategies to improve the reliability of LLMs when used as agents in clinical workflows. Using a benchmark we developed called MedAgentBench [93]—which evaluates clinical task automation scenarios—we identify common failure modes of an LLM-driven agent (such as misunderstanding a physician’s query or misinterpreting database retrieval results). We then propose an iterative prompt refinement loop: the original prompt and the agent’s outputs are analyzed (either by another LLM or simple heuristics) to detect error types, and the prompt is automatically rewritten to clarify or adjust the request. This reformulation process can be repeated until the LLM agent’s performance meets the desired criteria. We demonstrate that such prompt reformulation can substantially mitigate failures and improve the consistency of LLM agents in a clinical setting.

In summary, the contributions of this dissertation span the creation of new evaluation benchmarks, the development of novel reasoning algorithms, the curation and generation of improved data, and the application of these to challenging real-world domains. Collectively, these contributions advance a unified framework for evaluating and enhancing systematic reasoning in LLMs, with demonstrated benefits for both general-purpose reasoning tasks and specialized biomedical applications.

1.3 Chapter Summaries

A summary of the main contributions of this dissertation is presented across the following chapters:

- In Chapter 2, we present surveys of foundational work in instruction-tuning, logical reasoning with LLMs, bias in natural language understanding (NLU) benchmarks, LLM agents, inference-time planning, and post-training methods. It gives an overview of prior research and outlines gaps in existing evaluation and training approaches that this dissertation addresses.
- Chapter 3 introduces LogicBench, a comprehensive benchmark for evaluating LLMs’ performance on natural language formulations of formal logic. It covers propositional logic, first-order logic, and non-monotonic reasoning, providing examples, evaluation setups, and results across multiple LLMs.
- In Chapter 4, we extend LogicBench to Multi-LogiEval, a depth-aware evaluation framework that evaluates multi-step logical reasoning by combining inference rules. It includes controlled generation of rule combinations and reasoning depths to systematically test LLMs’ reasoning ability.
- The work in Chapter 5 presents GridPuzzle, a dataset of logic puzzles designed to expose and diagnose reasoning errors in LLMs. It includes a novel error taxonomy for step-wise reasoning failures and demonstrates both manual and LLM-based evaluation of reasoning chains.
- Chapter 6 introduces PlanGEN, a multi-agent framework for improving natural planning and reasoning capabilities of LLMs during inference. PlanGEN consists of constraint, verification, and selection agents and evaluates several inference-time strategies (Best-of- $\mathcal{N}$, Tree-of-Thought, and REBASE) on various reasoning benchmarks.
- PLAN-TUNING is introduced in Chapter 7. PLAN-TUNING is a post-training strategy where models are fine-tuned on synthetic natural planning trajectories.

It utilizes supervised fine-tuning (SFT) and reinforcement learning to improve model reasoning accuracy, including generalization to out-of-distribution tasks.

- In Chapter 8, we investigate how biases in instruction–originating from crowd-sourced datasets–affect the generalization and behavior of LLMs. It presents a taxonomy of instruction patterns, analyzes their propagation into training data, and conducts experiments showing their impact on model performance.
- Chapter 9 presents TarGEN, a method for generating high-quality, task-specific synthetic data with self-correction and instruction diversity. It includes prompting strategies, data quality analysis, and evaluation across multiple NLU tasks.
- In Chapter 10, we introduce In-BoXBART, the first instruction-tuned model in the biomedical domain. It introduces the BoX schema for unifying biomedical tasks and evaluates performance across NER, QA, summarization, and other biomedical benchmarks, demonstrating gains from instruction tuning.
- The work in Chapter 11 explores how LLMs can automate evidence synthesis in oncology. It simulates a two-reviewer setup for clinical systematic reviews, evaluates different collaboration strategies, and demonstrates significant reductions in manual workload while maintaining good accuracy.
- In Chapter 12, we investigate the effect of prompt variation on the reliability of LLM agents in clinical workflows. Using MedAgentBench, it categorizes common errors and proposes a reformulation approach to develop better prompts to mitigate failure cases.
- Finally, Chapter 13 summarizes the key contributions of the dissertation and outlines directions for future work in reasoning with LLMs, evaluation, planning, and domain-specific applications.

BACKGROUND AND RELATED WORKS

2.1 Logical Reasoning with LLMs

2.1.1 *Datasets and Benchmarks*

As LLMs continue to evolve rapidly, it becomes increasingly crucial to evaluate their diverse reasoning capabilities, as well as those of forthcoming LLMs. LogiQA [128] and ReClor [300] have made notable contributions by compiling multiple-choice questions from standardized examinations that demand diverse forms of logical reasoning. In contrast to *LogicBench*, these datasets involve mixed forms of reasoning and do not focus on assessing logical reasoning in isolation.

A few past attempts have been made to evaluate only logical reasoning while excluding other forms of reasoning. For example, CLUTTER [221] covers inductive reasoning, [69] covers temporal logic, and Rulemaker [39] evaluates whether a transformer-based model emulates deductive reasoning over synthetically generated statements in a limited setting. LogicNLI [239] introduced a diagnostic benchmark for FOL reasoning, with the dataset constructed by automatically generating logic expressions and replacing the entity and attribute placeholders.

Our proposed dataset is similar (in terms of task formulation) to ProofWriter [233], FOLIO [71], and ProntoQA [211], which are QA datasets designed to test reasoning ability. ProofWriter provides multi-hop proofs for each example, while FOLIO gives diverse and complex logical expressions, however, it is only limited to FOL. ProntoQA [211] provides explanation and reasoning steps but is limited to modus ponens in FOL. Nevertheless, several crucial attributes motivated us to create *LogicBench* (see Table

2.1 for comparison). Additional datasets for evaluating logical reasoning also exist, such as SimpleLogic [305] provides a class of logical reasoning problems, TaxiNLI [97] introduces logical taxonomy in the NLI task, and RuleBert [204] covers only soft logical rules. In summary, *LogicBench* evaluates logical reasoning in isolation and provides diverse inference rules and logic types compared to existing datasets.

FraCas [12] offers a unique approach to temporal semantics by converting syntax trees into logical formulas tailored for inference, emphasizing temporal elements such as references, adverbs, aspectual classes, and progressives. The Monotonicity Entailment Dataset (MED) [285] dives deep into monotonicity reasoning within NLI, probing the synergy between lexical and syntactic structures and spotlighting inherent challenges in both upward and downward monotonic reasoning trajectories. The SICK [141] dataset, with its foundation in 10,000 English sentence pairs, is designed to rigorously evaluate semantic relatedness and entailment, leveraging crowdsourced annotations for precision. HANS, or Heuristic Analysis for NLI Systems [144], stands out by rigorously scrutinizing the dependability of NLI models, putting the spotlight on potential pitfalls tied to syntactic heuristics such as lexical overlap. Lastly, CAD [252] introduces a meticulously crafted dataset from Reddit entries, targeting the detection of online abuse. Nakamura *et al.* [160] introduced LogicAttack, a method for performing adversarial attacks on NLI models using PL inference rules and proposed a dataset of $\sim 9k$ attack samples derived from the SNLI dataset. In contrast to these works, *LogicBench* evaluates the logical reasoning capabilities of LLMs beyond the NLI task, focusing on the application of individual inference rules.

Although LogicBench [174] covers various inference rules and reasoning patterns comprehensively, it only contains single-step logical reasoning (see Table 2.1 for comparison). In contrast, *Multi-LogiEval* evaluates logical reasoning independently beyond modus ponens and in a multi-step manner by utilizing various depths. In ad-

Dataset	Logic Covered			Inference Rules/ Axioms Provided	Generation Code Available	Multi-Step Reasoning
	PL	FOL	NM			
RuleTaker	✗	✓	✗	✗	Human-annotated	✓
LogicNLI	✗	✓	✗	✗	Semi-automated	✗
ProofWriter	✓	✓	✗	✗	✗	✓
FOLIO	✗	✓	✗	✗	Human-annotated	✓
SimpleLogic	✓	✗	✗	✗	✓	✓
ProntoQA	✗	✓	✗	✓	✓	✗
LogicBench	✓	✓	✓	✓	✓	✗
<i>Multi-LogiEval</i>	✓	✓	✓	✓	✓	✓

Table 2.1: Comparison of existing reasoning logical reasoning datasets with proposed benchmarks.

dition, FLD (Formal Logic Deduction) [158] has formal logic theory-based inference rules, and their combinations to create multi-step reasoning, but is limited to PL and FOL. However, *Multi-LogiEval* offers a broader set of inference rules for PL and FOL, along with their combinations for multi-step reasoning, in addition to NM reasoning.

2.1.2 Puzzle-Solving Task

Puzzle-solving task provides insights into LLMs’ logical reasoning. Giadikiaroglou *et al.* [62] categorizes puzzles into (1) rule-based and (2) rule-less puzzles. Rule-less puzzles include riddles [121], MCQs [310], programming puzzles [217], and common-sense reasoning puzzles [65]; however, in our work we focus on rule-based puzzles. In rule-based puzzles, past attempts have explored Sudoku [163], Rubik’s Cube, 8-puzzle, Game of 24 [289], crosswords [289], chess puzzles [55], card games [66], BoardgameQA [101], and Lateral Thinking Puzzles [86]. However, grid-based puzzle solving is underexplored. Mitra and Baral [156] proposed a grid-based puzzle dataset and Dziri *et al.* [50] studied compositionality in LLMs using Grid Puzzle, but these works do not provide any insights into the performance of recent LLMs. Motivated by this, we

propose a systematically curated grid-based puzzle dataset, *GridPuzzle*, and provide an evaluation of various LLMs in puzzle-solving.

2.1.3 Automatic Evaluation of Reasoning Chains

There are two types of automated evaluations for reasoning chains: (i) reference-based and (ii) reference-free. Reference-based evaluations is tedious as it require gold reasoning chains (typically human-written explanations) to calculate the similarity with LLM-generated reasoning chains. Conversely, reference-free evaluation does not require any human-annotated reasoning chains. Previous works [43, 210, 71] have focused on reference-free evaluation, which is not reliant on gold-reasoning chains. Recently, ROSCOE [64] proposed a suite of metrics to measure the semantic consistency, logicity, informativeness, fluency, and factuality of reasoning chains, while the ReCEval framework [185] evaluates reasoning chains based on two key properties: correctness and informativeness. Recent evaluation methods such as *LLM evaluation* [34] and G-Eval [131] leverage LLMs to measure the quality of reasoning chains. *LLM evaluation* involves presenting task instructions and a text sample to LLMs, asking them to rate the sample’s quality on a 5-point Likert scale, whereas the latter incorporates an automatic chain-of-thought generated by the LLM describing the detailed evaluation steps. Additionally, Tyen *et al.* [247]’s attempt to use GPT-4 as an evaluator in a few-shot setting shows that evaluating reasoning chains remains a challenge. Furthermore, AutoRace [72] proposed a fully automated approach for evaluating reasoning chains that adapt to different tasks without human effort. However, these methods do not evaluate reasoning chains at the level of fine-grained error types and do not provide detailed task-specific insights. To address this, we propose LLM-based reference-free evaluation methods that identify fine-grained errors and assess the correctness of generated reasoning chains.

2.2 LLM Agents for Reasoning and Planning

Agent-based frameworks for planning have gained interest, focusing on enhancing how LLMs decompose tasks and refine their outputs. The Sibyl framework [269] effectively decomposes tasks into smaller subtasks, assigning each to specialized agents that iteratively collaborate until a solution is reached. OS-Copilot [281] introduces a generalist computer agent that employs self-improvement through modularization and feedback loops. Another approach is KnowAgent [320], which integrates knowledge-augmented planning to enhance the decision-making capabilities of LLM agents. Similarly, Tool-Planner [133] proposed grouping tools based on similar functionalities into toolkits, allowing LLMs to select the best tool for a given task. Many agent-based works focusing on planning have been developed [27, 283, 264]. Despite the progress, these methods generally (i) focus on domain-specific tasks or limited benchmarks, reducing generalizability, and (ii) lack or under-explore mechanisms for verifying and refining plans iteratively. While some works explore natural language planning [16, 116], they either use single-agent frameworks or evaluate the proposed framework on domain-specific benchmarks.

2.2.1 Inference-Time Algorithms

Inference-time algorithms have recently shown a significant improvement in LLMs' performance during inference. For instance, Best of $\mathcal{N}$ sampling [18] selects the most promising output from multiple generations performed using temperature sampling, while Tree-of-Thought (ToT) [289] models reasoning as an iterative tree search. REBASE [280] optimizes search-space pruning using reward balancing. One very popular approach is Monte Carlo Tree Search (MCTS) [304], which iteratively explores solution paths during inference. Applied to models such as LLaMa-3-8B, it enables

self-refinement by revisiting and improving initial solutions. Test-time optimization [222] focuses on dynamically adjusting computational resources during inference [280]. Furthermore, Wang *et al.* [257] uses the inference time algorithms to improve LLMs’ planning capabilities to solve code synthesis problems. In inference-time algorithms, verification is the key component. In contrast to these past works, here, we enhance the performance of inference-time algorithms utilizing constraint-guided verification and multi-agent collaboration for natural language planning, and its applications in downstream complex reasoning tasks.

Inference-time algorithms have recently shown a powerful way to optimize LLM output during inference, providing significant improvements in accuracy without scaling the model. Chain-of-thought (CoT) prompting and its variants [272, 107] showed that adding intermediate reasoning steps during inference-time greatly boosts the performance of LLMs. New methods have been proposed, such as self-consistency [265], which generates multiple reasoning chains from LLM and then selects the final answer based on majority voting. One very popular approach is the use of Monte Carlo Tree Search (MCTS) [304], which iteratively explores multiple solution paths during inference. This technique has been successfully applied to models like LLaMa-3-8B, which integrates a self-refinement mechanism that allows the model to revisit and improve its initial solutions over time. Another method, test-time optimization [222], focuses on dynamically adjusting computational resources during inference. By optimizing compute resources based on the complexity of a task, this approach strikes a balance between efficiency and accuracy, ensuring that difficult tasks receive more attention while simpler tasks are processed with fewer resources. Additionally, compute-optimal inference [280] highlights the importance of effectively distributing computational power during problem-solving tasks. Finally, repeated sampling [18] is a technique that uses multiple inference attempts to improve solution quality. Wang

et al. [256] uses the inference time algorithms to improve LLMs’ planning capabilities to solve code synthesis problems. Recent works such as Parmar *et al.* [171] show that better natural language planning improves downstream reasoning capabilities of underlying LLMs. In contrast to all these approaches focused on increasing compute at inference-time, our work focuses on inference-aware post-training with planning trajectories.

2.2.2 Post-Training Methods for LLMs

Past attempts have been made to improve the performance of smaller LMs using various types of training trajectories [166, 187, 205]. Our work is closer to Jiao *et al.* [95] where authors propose a two-stage approach: first, creating human-like, step-by-step planning trajectories, then automatically synthesizing detailed “process rewards” that score how accurately each trajectory follows its plan. In a similar direction, Song *et al.* [224] pioneers an exploration-driven paradigm, generating multiple candidate trajectories and iteratively refining them via performance feedback to improve long-horizon reasoning. Zhang *et al.* [306] proposes a pipeline that unifies demonstration data, self-play rollouts, and human preferences, demonstrating that diverse training signals yield more robust agent policies. Wang *et al.* [261] shows that incorporating explicitly negative trajectories during fine-tuning helps the model learn to avoid erroneous or unsafe actions. Other work investigates curriculum design and data curation strategies, revealing that carefully scheduled exposure to increasingly complex tasks significantly boosts final policy quality [32]. Meanwhile, Song *et al.* [223] demonstrates that scaling up trajectory volume and diversity is key to robust generalization. Unlike prior work that applies planning only at inference or via large-scale behavior cloning, PLAN-TUNING incorporates synthetic planning trajectories into small LLMs’ parameters through supervised and RL post-training objectives.

2.3 Instruction-Tuning

The introduction of LLMs has significantly shifted the research trend in NLP to prompt-based learning methodologies [129]. Many studies have been conducted to investigate the efficacy of prompt-based learning in various applications, including Text classification [296], Natural Language Inference (NLI) [213], and Question Answering (QA) [94], Information Extraction (IE) [31, 42], to name a few. In a recent development, the T0 model employs prompts to achieve zero-shot generalization across various NLP tasks [209]. Le Scao and Rush [114] suggested that the use of prompts could be as valuable as hundreds of data points on average.

Owing to the problems associated with single-task learning in terms of their space and time requirements, several multi-task learning approaches have been proposed over the years. DecaNLP [143] built a multi-tasking model by converting format of each tasks to question answering format. Several other works have followed similar approach, for example, by converting tasks to reading comprehension [155] and textual entailment format [262]. The multitasking model T5 [190] was built with the help of a unified framework that converts all text-based language problems into a text-to-text format. SCIFIVE [179] involved building a text-to-text model for the biomedical literature. Aghajanyan *et al.* [2] introduced pre-finetuning, an additional large-scale learning stage between language model pre-training and fine-tuning to improve multitask learning performance. Models empowered by multi-task learning have achieved SOTA in many different tasks, e.g., Question Answering (QA) [105], commonsense reasoning [134] and structured knowledge grounding tasks [284].

The turking test [51] was proposed to measure the efficacy of models to follow instructions. Studies have been made to investigate the effect of natural language instructions on model performance [74, 294, 315, 275]. Moreover, Mishra *et al.* [153]

proposed Natural Instructions which break down each task to multiple sub-tasks that help models in following instructions and subsequently generalize to unseen tasks (i.e., cross-task generalization). FLAN [271] and T0 [209] models were built by leveraging instruction/prompt-tuning on diverse range of tasks and achieving zero-shot generalization on target unseen tasks. Task reframing [152] proposed several guidelines to reframe task instructions to improve model response to follow instructions. Analysis introduced to understand in-context learning better on a large set of training tasks [147, 148]. InstructGPT model [166] is proposed, which is fine-tuned with human feedback to follow natural instructions. Furthermore, many works focused on investigating whether LMs understands meaning of natural language and prompts [270, 311]. Weller *et al.* [275] and Ye and Ren [293] use task descriptions to achieve generalization to new tasks. Puri *et al.* [186] introduced instruction augmentation to improve model performance and sample complexity. Wang *et al.* [258] has developed instruction-based multi-task framework for the few-shot Named Entity Recognition (NER) task. Prasad *et al.* [184] introduced Gradient-free Instructional Prompt Search (GrIPS) for improving task instructions for large LMs. Recently, many approaches have been proposed to improve model performance using instructions [279, 278, 125, 111].

2.4 Biases in NLU Benchmarks

Crowdsourcing has been a widely adapted approach to create large-scale datasets such as SQUAD 1.1[193, 192], DROP[49], QUOREF[44], and many more [159, 22, 112, 313, 24]. Many past works investigate different types of bias in crowdsourcing datasets, such as cognitive bias [52], annotator bias [68, 60], sampling bias [82], demographic bias [191], and others [80]. Many works on bias in NLU benchmarks focus on biases resulting from the crowdsourcing annotations, and how annotator-specific patterns create biases in data [60].

To mitigate the bias, prior works have focused on priming crowdsourcing annotators with minimal information to increase their imagination [61, 37] to avoid recurring patterns. Arunkumar *et al.* [6] develops a real time feedback and metric-in-the loop [151] workflow to educate crowdworkers in controlling dataset biases. Nangia *et al.* [161] provides an iterative protocol with expert assessments for crowdsourcing data collection to increase the difficulty of instances. [231] introduces the dataset map as a model-based tool to characterize and diagnose datasets. Also, Karimi Mahabadi *et al.* [99], Mahabadi *et al.* [140] propose learning strategies to train neural models, which are more robust to such biases and transfer better to out-of-domain datasets.

2.5 Data Synthesis using LLMs

Recent research has witnessed the emergence of various methods harnessing LLMs as synthetic data generators. Specifically, in the context of few-shot classification tasks where labeled data is scarce, several approaches have been introduced. SuperGEN [214] and ZeroGEN [145] leverage LLMs to produce synthetic data. For zero-shot tasks, SunGen [56] employs noise-filtering techniques, while ProGEN [292] utilizes model feedback to ensure generated data quality. Similarly, [33] introduces structured prompts for tasks like relation triplet extraction. Moreover, [127] and [276] propose synthetic data generation methods for natural language entailment (NLI) tasks and free-text explanations in a human-AI collaborative setting. There have also been approaches proposed to generate tabular data [17] and instruction data [177, 229].

The prevailing research direction in synthetic data generation predominantly focuses on zero/few-shot classification or entails fine-tuning [25] or iterative fine-tuning of open-source LLMs [301]. In contrast, our method is simple, lightweight, and adaptable even to closed-source LLMs like ChatGPT. The aforementioned methods often

rely on seed samples where expert annotators are employed to curate and label a considerable number of samples for various downstream tasks. These seed samples are used to generate additional synthetic samples for the downstream tasks. However, our approach does not rely on labeled examples or seed tasks. We exploit the generative capabilities of LLMs to completely generate seed instances that drive the diverse synthetic dataset generation. Furthermore, our approach uses a multi-step prompting strategy alongside *self-correction* to do targeted data generation and uphold data generation quality in terms of diversity, bias, noise, and mislabelling. Finally, existing dataset generation methods are often limited by reliance on seed tasks from the original dataset [266], whereas our seedless pipeline leverages high-level dataset characteristics for the generation process.

2.6 Application: Automation of Systematic Review

With the advent of NLP and ML, many past attempts have been made to automate or semi-automate abstract screening in SRs [243, 244, 196, 57, 58, 91, 202, 195]. SRs of many clinical problems are implemented using ML approaches, which include drug class efficacy assessment, public health, toxicology, genetic associations, etc. [10]. There are several tools available for automating SRs, which can be grouped into three categories [146]. The first category is text visualization, such as Covidence [1], Early Review Organizing Software [63], PICO portal (<https://picoportal.org/>), etc. The second category is term frequency and inverse document frequency weighting methods, such as SWIFT-Active Screener [81]. The third category is semi-automated or automated screening and selection tools (i.e., text classification) methods such as Support Vector Machine (SVM)-based models [299, 59, 167].

Previous efforts to automate the title and abstract screening process have typically relied on labor- or computationally intensive methods, such as supervised labeling,

pretraining, or vectorization. For example, Rayyan [167] and Abstrackr [59] are two free web tools that offer a semi-automated approach to article filtering. Beyond these standard methods, past attempts have also been made to utilize pre-trained generative models. For instance, Syriani *et al.* [232] used ChatGPT for document screening and found that its effectiveness is limited when the document set is imbalanced — a common issue in systematic reviews. Bio-SIEVE [199], a model fine-tuned from the Guanaco checkpoint (built on LLaMa architecture), demonstrated higher classification accuracy than ChatGPT for article screening. Sakai and Lam [208] also assessed GPT3.5/4 to screen titles and abstracts in clinical reviews by utilizing plain language prompts in a zero-shot setting, but the performance was suboptimal across all datasets. Wang *et al.* [263] evaluated the effectiveness of eight LLMs by exploring a calibration technique that applies a predefined recall threshold to determine eligibility of a given article. However, these efforts have all relied on the capabilities of a single LLM. Conversely, the approach in this study is the first to simulate the real-world two-reviewer process by employing two independent LLMs. This innovative method not only mirrors the collaborative nature of human reviewers but also outperforms strategies that use a single LLM, setting a new benchmark in evaluating the performance of these models for screening articles.

LOGICBENCH: BENCHMARKING LOGICAL REASONING

3.1 Introduction

Large language models such as GPT-4, ChatGPT, Google Gemini, Llama-2 [241], and Mistral [92] have made remarkable progress in NLP research enabling machines to perform a variety of language tasks that were previously thought to be exclusive to humans [165, 19, 312]. However, the ability of these LLMs to reason “logically” over natural language text remains under-explored, even though logical reasoning is a fundamental aspect of intelligence and a crucial requirement for many practical applications, such as question-answering systems [103] and conversational agents [13]. Although several datasets have been proposed [39, 239, 97, 204] to evaluate the logical reasoning capabilities of LLMs, these datasets are limited in their scope by (1) not evaluating logical reasoning independently of other forms of reasoning, such as LogiQA [128] and ReClor [300]; and (2) evaluating only a single type of logic and covering only a few logical inference rules, as done in FOLIO [71] and ProntoQA [211]. Thus, our aim in this work is to address the lacuna of having a more comprehensive set of inference rules for evaluating the logical reasoning ability of LLMs.

To this end, we introduce a systematically created question-answering dataset for the evaluation of logical reasoning ability using a single inference rule, called *LogicBench*. Besides evaluating the logical reasoning ability of LLMs, by evaluating models on single inference rules, we can also gain insights into the frequency of text sequences corresponding to these rules in the pre-training data and their impact on model performance.

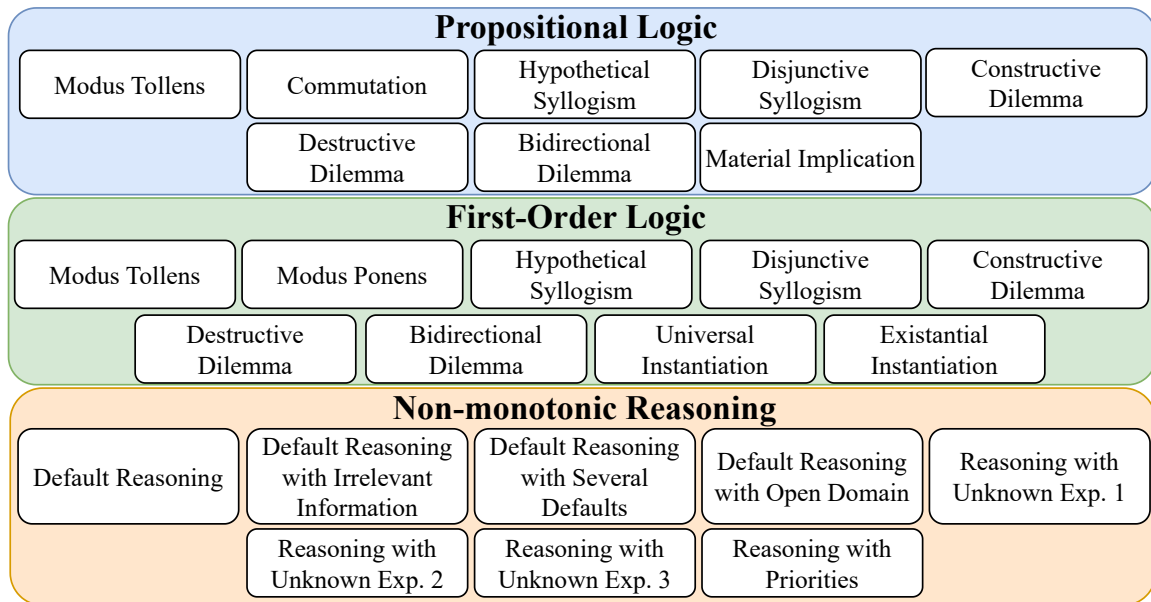


Figure 3.1: Comprehensive representation of different inference rules and reasoning patterns covered by propositional, first-order, and non-monotonic logics. *Exp.* indicates Expectation

As illustrated in Figure 3.1, *LogicBench* includes a total of 25 reasoning patterns across ‘propositional, first-order, and non-monotonic’ logics. To the best of the authors’ knowledge, this is the first work to study non-monotonic reasoning, as well as various inference rules in propositional and first-order logics, including hypothetical and disjunctive syllogism; and bidirectional, constructive, and destructive dilemmas in the NLP domain. To evaluate LLMs using *LogicBench*, we formulate two different tasks: (i) a Binary Question-Answering (BQA) task in which the context comprises logical statements and the models have to determine whether a conclusion given in the question is logically entailed by the context, and (ii) a Multiple-Choice Questions-Answering (MCQA) task where models must select the most appropriate logical conclusion from four distinct options, based on the provided context. The rationale behind having BQA and MCQA tasks is that it provides systematic standard metric-based evaluation (i.e., direct comparison of LLMs’ performance in terms

of accuracy), which could be more challenging with open-ended question-answer formats. Example instances of various reasoning patterns are presented in Table 3.3 and §3.2.4.

To construct *LogicBench*, we use a three-stage procedure (refer to §3.2). In the first stage, we prompt GPT-3.5 to generate a variety of coherent natural language sentences having different ‘ontologies’ (i.e., a collection of concepts as car, person, and animals) and their corresponding negations (refer to §3.2.2.1). In the second stage, we generate *(context, question)* pairs where the context represents a natural language narrative consisting of logical statements, and the question is formulated to exhibit the logical conclusion derived from the context. In the third stage, we generate task-specific data instances (i.e., *(context, question, answer)* triplets).

We conduct a comprehensive evaluation with a range of LLMs on *LogicBench* including GPT-4, ChatGPT (GPT-3.5-Turbo), Gemini-Pro, Llama-2-7B-Chat, and Mistral-7B-Instruct using chain-of-thought [272]. In particular, we measure the accuracy of LLMs’ predictions on both BQA and MCQA tasks. Our experiments result in several interesting findings, such as LLMs often struggle to reason over complex logical contexts that involve only a single inference rule and encounter difficulties with inference rules involving negations. Experimental results reveal that these models struggle with respect to many of the inference rules and patterns, suggesting significant room for improvement in their logical reasoning abilities.

To further demonstrate the use of *LogicBench*, we synthetically augment it and fine-tune T5-large. Our preliminary results (§3.3.5) show that this improves the logical reasoning ability of existing models, leading to performance improvement on other logic datasets, LogicNLI, and FOLIO ($\sim 2\%$ on average), and shows competitive performance on LogiQA and ReClor.

3.2 LogicBench

In this section, we discuss the logic types, inference rules, and patterns that are explored in this research. We also outline the methods for generating the data and statistics of *LogicBench*.

3.2.1 Logics Types

LogicBench consists of three distinct types of logic: propositional logic, first-order logic, and non-monotonic logic. To create this benchmark, we utilize a set of 25 carefully selected axioms as shown in Figure 3.1. From Figure 3.1, it is evident that our benchmark incorporates 8 axioms for propositional logic, 9 axioms for first-order logic, and 8 axioms for non-monotonic logic.

Propositional Logic (PL) Propositional logic employs a collection of statements or propositions (denoted as $\mathcal{P} = p_1, p_2, \dots, p_n$, where p_i represents a proposition) and builds upon them using logical connectives such as ‘ $\wedge$ ’, ‘ $\vee$ ’, ‘ $\rightarrow$ ’, ‘ $\leftrightarrow$ ’, and ‘ $\neg$ ’. Several inference rules for propositional logic have been defined using which given a set of premises, one can derive a sound conclusion. For example, let’s consider two propositions: p_1 , which states "It is raining," and p_2 , which states "It is cloudy." From these propositions, we can construct a context/knowledge base (KB) with two premises: (1) $p_1 \rightarrow p_2$ and (2) p_1 . With this KB, we can conclude p_2 . This inference rule is written as $((p_1 \rightarrow p_2) \wedge p_1) \vdash p_2$ and is known as ‘Modus Ponens’. In our study, we explore nine distinct inference rules of propositional logic, extensions of seven of them with one-variable and a universal quantifier, and two axioms of first-order logic as shown in Table 3.1. These inference rules provide a proper framework for deriving valid conclusions.

Names	Propositional Logic	Extension to a (restricted) First-order Logic
MP	$((p \rightarrow q) \wedge p) \vdash q$	$(\forall x(p(x) \rightarrow q(x)) \wedge p(a)) \vdash q(a)$
MT	$((p \rightarrow q) \wedge \neg q) \vdash \neg p$	$(\forall x(p(x) \rightarrow q(x)) \wedge \neg q(a)) \vdash \neg p(a)$
HS	$((p \rightarrow q) \wedge (q \rightarrow r)) \vdash (p \rightarrow r)$	$(\forall x((p(x) \rightarrow q(x)) \wedge (q(x) \rightarrow r(x)))) \vdash (p(a) \rightarrow r(a))$
DS	$((p \vee q) \wedge \neg p) \vdash q$	$(\forall x(p(x) \vee q(x)) \wedge \neg p(a)) \vdash q(a)$
CD	$((p \rightarrow q) \wedge (r \rightarrow s) \wedge (p \vee r)) \vdash (q \vee s)$	$(\forall x((p(x) \rightarrow q(x)) \wedge (r(x) \rightarrow s(x)) \wedge (p(a) \vee r(a)))) \vdash (q(a) \vee s(a))$
DD	$((p \rightarrow q) \wedge (r \rightarrow s) \wedge (\neg q \vee \neg s)) \vdash (\neg p \vee \neg r)$	$(\forall x((p(x) \rightarrow q(x)) \wedge (r(x) \rightarrow s(x)) \wedge (\neg q(a) \vee \neg s(a)))) \vdash (\neg p(a) \vee \neg r(a))$
BD	$((p \rightarrow q) \wedge (r \rightarrow s) \wedge (p \vee \neg s)) \vdash (q \vee \neg r)$	$(\forall x((p(x) \rightarrow q(x)) \wedge (r(x) \rightarrow s(x)) \wedge (p(a) \vee \neg s(a)))) \vdash (q(a) \vee \neg r(a))$
CT	$(p \vee q) \vdash (q \vee p)$	-
MI	$(p \rightarrow q) \vdash (\neg p \vee q)$	-
EG	-	$P(a) \Rightarrow \exists x P(x)$
UI	-	$\forall x A \Rightarrow A\{x \mapsto a\}$

Table 3.1: Inference rules and (two) axioms that establish the relationship between premises and conclusions. MP: Modus Ponens, MT: Modus Tollens, HS: Hypothetical Syllogism, DS: Disjunctive Syllogism, CD: Constructive Dilemma, DD: Destructive Dilemma, BD: Bidirectional Dilemma, CT: Commutation, MI: Material Implication, EG: Existential Generalization, UI: Universal Instantiation

First-order Logic (FOL) In this work, we consider a restricted set of logical axioms for FOL that utilize quantifiers, $\forall$ (universal quantifier) and $\exists$ (existential quantifier). The universal quantifier ($\forall$) denotes that a statement holds true for all instances within a specific category. In contrast, the existential quantifier ($\exists$) indicates that a statement is true for at least one instance within its scope. For instance, a simple extension of propositional ‘Modus Ponens’ is an inference rule where given the premises $\forall(p(x) \rightarrow q(x))$ and $p(a)$, we conclude $q(a)$ (e.g., given “All kings are greedy” and “Sam is a king”, we can conclude “Sam is greedy”). Here, we explore two axioms (EG and UI) and various inference rules that incorporate the quantifiers (shown in Table 3.1). *Existential Generalization (EG)*, formally expressed as $P(a) \Rightarrow \exists x P(x)$ indicates that there is an element a in the domain for which $P(a)$ is true, then we know that $\exists x P(x)$ is true. *Universal Instantiation* formally expressed as $\forall x A \Rightarrow A\{x \mapsto a\}$

indicates that a statement holds true for all instances (x) within a specific category A , hence it is also true for specific instance a . First-order logic can be arbitrarily more complex with alternate use of $\forall$ and $\exists$ and an arbitrary number of variables. Here we restrict it to one variable, and in the first seven inference rules, we use the quantifier $\forall$ only once.

Non-monotonic (NM) Reasoning In this work, we analyze a range of logical reasoning templates in NM logics involving “Default Reasoning,” “Reasoning about Unknown Expectations,” and “Reasoning about Priorities.” These templates are inspired by the compilation [120] made in 1989 to evaluate the abilities of various non-monotonic logics that were being developed at that time. To illustrate the concept of default reasoning, let’s consider an example. Default reasoning involves inferring plausible conclusions based on general rules that may have exceptions. Below Table 3.2 shows examples of NM reasoning. A key aspect of NM logics is to formalize notions such as “normally,” “typically,” and “usually” that are not directly formalizable using classical quantifiers in the first-order setting. The general rule “Heavy blocks are normally located on the table” does not imply that “All heavy blocks are always located on the table”. Rather, this rule allows for exceptions. Our work explores various NM reasoning patterns, as depicted in Figure 3.1, to delve deeper into the nuances of this type of reasoning. Additional examples are given in §3.2.4.

3.2.2 Data Creation

Here, we leverage the formal language of various logical axioms and transform them into natural language to construct the LogicBench. Our data creation procedure (Figure 3.2) consists of three stages. Examples of generated data corresponding to each logic type and reasoning patterns are presented in §3.2.4.

Basic Default Reasoning	Default Reasoning with Irrelevant Information
Context: Block A and block B are both heavy objects that are typically found on the table. However, there is a possibility that block A might not follow this usual convention. It is important to note this exception.	Context: In a room filled with various objects, two heavy blocks, block A and block B, stand out. Normally, heavy blocks like these are placed on the table, but surprisingly, block A is not found on the table. On the other hand, block B grabs attention with its vibrant red color.
Conclusion: B is on the table.	Conclusion: B is on the table.
Reasoning about Unknown Expectations	Reasoning about Priorities
Context: In this situation, there are three heavy blocks: A, B, and C. Typically, heavy blocks are found on the table. However, it is known that at least one of the blocks, either A or B, is not currently on the table.	Context: John confidently states that the vehicle is situated in the driveway, while Sara adamantly counters, asserting that it is not parked inside the garage.
Conclusion: C is on the table. Exactly one of A, B is not on the table.	Conclusion: If John's evidence is more reliable than Sara's then the car is parked in the driveway.

Table 3.2: Illustrative examples of non-monotonic reasoning adapted from [120].

1. Sentence Generation: Starting with a given prompt, we generate coherent sentences and their negations that incorporate different ontologies.

2. NL Conversion: Pairs of (*context*, *question*) are generated using pre-defined templates from which context is then converted to a natural language narrative using the prompt.

3. Task Instance Generation: Task-specific (*context*, *question*, *answer*) triplets are generated. BQA requires answers in the form of “yes” or “no”, whereas MCQA involves selecting one correct option from a set of four. We generate semantically preserving and inverting variations of these triplets to add more diversity for BQA.

3.2.2.1 Sentence Generation

Here, the first step is to generate sentences with diverse *ontologies*. An ontology represents a collection of concepts (e.g. car, person, animals, etc.) along with their

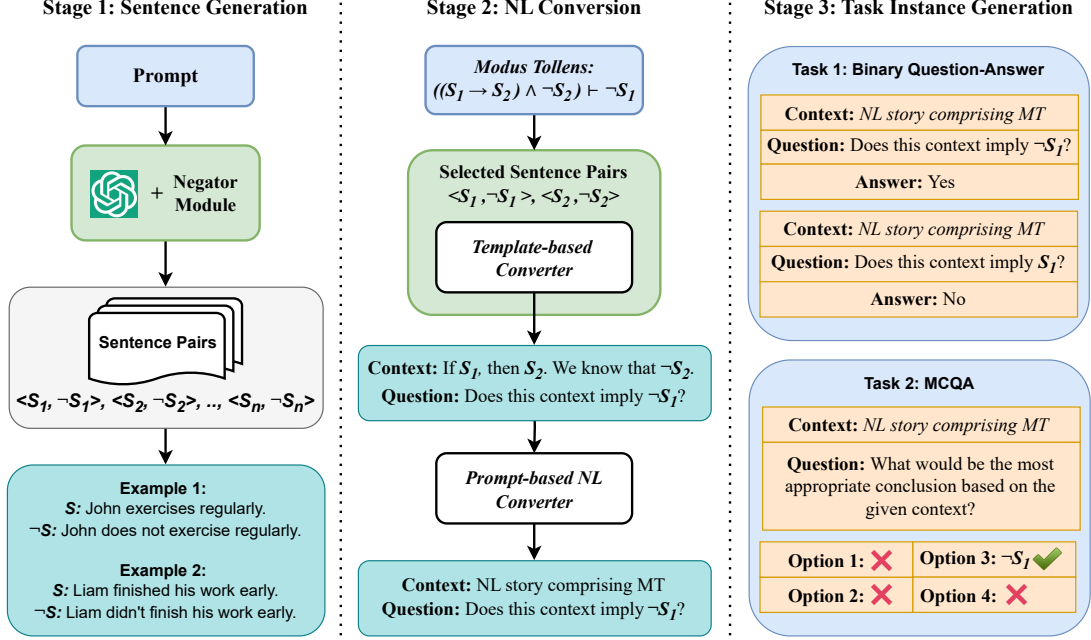


Figure 3.2: Schematic representation of three-stage procedure for data creation. NL: Natural Language

corresponding associated properties. To generate these sentences, we prompt the GPT-3.5 model with instructions tailored for each inference rule. The prompt schema, as depicted in Figure 3.3, comprise three crucial components:

Definition provides a detailed explanation of the task and offers a natural language representation of the reasoning pattern for which we are generating sentences.

Examples provide sample sentences that need to be generated. We also illustrate how these sentences will be utilized in later stages, emphasizing the importance of coherence and the inclusion of relevant ontological concepts.

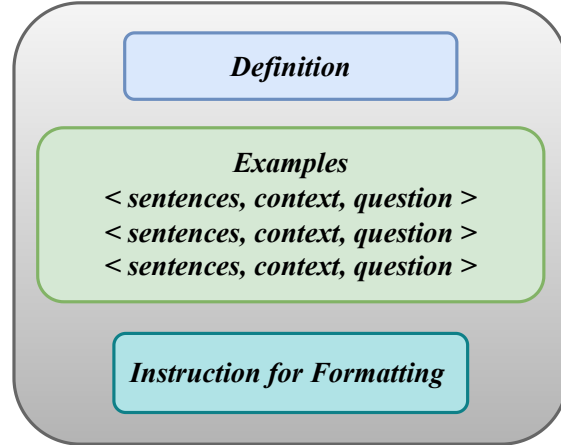


Figure 3.3: Schematic representation of prompt.

Format We provide specific formatting instructions to guide the generation of sentences.

An example of a prompt corresponding to the ‘Modus Tollens’ from PL is presented in Figure 3.4 for better illustration. Modus tollens is formally represented as $((p \rightarrow q) \wedge \neg q) \vdash \neg p$, which can be understood in natural language as “If p implies q , and we know $\neg q$, then we can conclude $\neg p$.” In this prompt, the definition provides a comprehensive description of the inference rule in natural language. To encourage the generation of more relevant and coherent sentences, the prompt includes an examples section that demonstrates how the generated sentences will be utilized in a later stage. This serves as an illustration to guide GPT-3 in produc-

ing suitable outputs. In Figure 3.4, we present three examples involving sentences p and q , along with their respective contexts and questions. The prompt also includes instructions on how the generated sentences should be formatted.

Definition of rule:
If p then q ; not q , therefore not p
Instructions: Understand the below examples to learn the connection between sentences and how they have been used. Use that understanding to generate coherent sentences.

Examples:

p: Liam finished his work early.
q: He will order pizza for dinner.
Context: If Liam finished his work early, then he will order pizza for dinner. He won't order pizza for dinner
Question: Does this imply that Liam didn't finish his work early?
Answer: yes

p: Lisa gets an A in the class.
q: She will reward herself with a trip.
Context: If Lisa gets an A in the class, then she will reward herself with a trip. She won't reward herself with a trip
Question: Does this imply that Lisa gets A in the class?
Answer: no

p: Benjamin is going to the beach.
q: He will go swimming.
Context: If Benjamin is going to the beach, then he will go swimming. He won't go swimming
Question: Does this imply that Benjamin isn't going to the beach?
Answer: yes

Instruction for Formatting

1. Generate only one pair of p , and q based on the above understanding.
2. Generate each sentence in a new line.
3. Do not generate p , and q prefixes.

Figure 3.4: Example prompt for *Modus Tollens* inference rule from PL.

Note that our objective at this stage is not to generate logical sentences but rather to generate a diverse and coherent set of sentences that encompass various concepts. We also create a negation sentence corresponding to each generated sentence¹. In this work, the scope of generating negations is simple, as simple as adding “not” to a sentence; however, negations can be more complicated in the case of logic. These generated sentences will be combined with logical connectives in a later stage to form context and questions.

3.2.2.2 NL Conversion

Here, the NL conversion is accomplished using two steps. First, we leverage the formal expressions of reasoning patterns to create templates that establish the desired NL formulation for each logical connective (i.e., templatized context). Second, we prompt GPT-3.5 to transform the templatized context into a story/narrative-based context, enhancing its naturalness. For instance, implication: “ $p \rightarrow q$ ” is expressed as “If p , then q ”, conjunction: “ $p \wedge q$ ” as “ p and q .”, and disjunction: “ $p \vee q$ ” as “At least one of the following is true: (1) p and (2) q . Note that we do not know which of (1) and (2) is true. It is possible that only (1) is true, or only (2) is true, or both are true.” since understanding the logical implication of ‘or’ when integrated into logical formulations posed challenges to both humans and models. With these established formulations, we proceed to utilize the sentences generated in §3.2.2.1 to create the templatized context and questions corresponding to reasoning patterns.

Then, the templatized context is converted into a narrative-based context using a prompt-based converter, enhancing its naturalness. The prompt-based converter (essentially, prompting GPT-3.5) ensures that the context is no longer templatized, yet follows the logical connection between sentences as mentioned in the logical rule.

¹We use <https://github.com/dmlls/negate> to generate negated sentences

To ensure the quality of the narrative in the *LogicBench(Eval)* for task 1 and task 2, we have created category-specific prompts to convert the templated context to convert more human-like narrative. In total, we have created three different prompts, each for PL [13.2.2.2], FOL [23.2.2.2], and NM [33.2.2.2]. The prompts are designed to ensure that the logical connection is established in the narrative. Each rule of PL, FOL, and NM has a unique logical progression that should be followed in the narrative is the motivation for us to go with three different instruction-based prompts. In the prompt, used for PL rules, the logical rule is also mentioned in two parts, “Condition” and “Situation”. Consider the example of logical rule MT - “If p then q; not q; therefore not p”, the templated context will have “If p then q; not q;” so condition will be the first part “If p then q” while the situation will be the second part “not q”. In other terms, the specific conditions are the rules to be followed, while the situation is the case present in the logical rule. In contrast, for FOL, we do not need to focus more on the specific condition; rather, we have to make sure a generalized case is present in the narrative with 1-2 specific sentences related to the rule. For NM, we do not have such a rule; instead, we have a logical connection between sentences, and hence we go with the instruction-based prompting.

Prompt for templated to narrative conversion for PL(MT)

Rule: Condition: If p then q; **Situation:** not q

Context: Templated context

Improve the context in human-like language and make story with rephrased sentences.

Instructions for generating a good story:

1. While generating a story, Use the rephrased sentences from the context of the story.

2. Make sure to include sentences corresponding to the condition and situation from the rule in the story.
3. Do not add any other extra information.
4. To generate a story, DO NOT change the name of the main character from the context, if there's any. If the main character within the context is 'I', please generate a story with 'I' as the main character.
5. Generate only one paragraph with rephrased sentences.

Prompt for templatized to narrative conversion for FOL

Context: Templatized context

Improve the context in human-like language and make a rephrase sentences around it.

Instructions for generating a good story:

1. Make sure to include only rephrased sentences in the story.
2. Do not add any other extra information and do not reach any conclusion based on context.
3. To generate a story, use only generic sentences without mentioning any main character.
4. Extract a specific case sentence from the context, rephrase it, and append it at the end of the context only.
5. Generate only one paragraph with rephrased sentences.
6. DO NOT add any prefixes.

Prompt for templatized to narrative conversion for NM

Context: Templatized context

Improve the context in human-like language and rephrase sentences around it.

Instructions for generating a good story:

1. Make sure to include only rephrased sentences in the story.
2. Do not add any other extra information, but add some story-like context.
3. To generate a story, DO NOT change the name of the main character from the context, if there’s any. Use the same names/words for main character. If the main character within the context is 'I', please generate a story with 'I' as the main character.
4. Generate only one paragraph with rephrased sentences.

For instance, let’s consider the “Modus Tollens” from PL ($((p \rightarrow q) \wedge \neg q) \vdash \neg p$), and the “Bidirectional Dilemma” from FOL ($(\forall x((p(x) \rightarrow q(x)) \wedge (r(x) \rightarrow s(x))) \wedge (p(a) \vee \neg s(a))) \vdash (q(a) \vee \neg r(a))$). For these rules, Table 3.3 presents examples of logical templated and narrative-based context, question, and task instances for both BQA and MCQA. Furthermore, Table 3.4 shows examples of how context and question are generated from sentences corresponding to each inference rule for PL and FOL. Similarly, Table 3.5 shows examples of NM reasoning. From Table 3.4, we can see an example of sentence pairs (p, q) and their corresponding negation pairs $(\neg p, \neg q)$ for the ‘modus tollens’ inference rule from PL. These pairs are utilized to generate templated logical context, which is converted to narrative using a specific prompt[13.2.2.2] designed for PL. In addition to the narrative, the question and its variations are also created using these sentence pairs. Similarly, in the second row, we have four generic rules with variable x ($p(x), q(x), r(x), s(x)$) and their specific cases (i.e., $x = a$), along with their respective negative sentence pairs $[(p(a), \neg p(a)), (q(a), \neg q(a)), (r(a), \neg r(a)), (s(a), \neg s(a))]$. In the same manner as PL, these examples demonstrate the generation of narrative and questions for the FOL inference rule called ‘Bidirectional Dilemma (BD)’, as shown in Table 3.4 using a specific prompt[23.2.2.2] designed for FOL. From Table 3.5, the first row presents an example

Generated Sentences in Stage 1	Context	Binary QA	MCQA
Inference rule: MT p: Liam finished his work early. ¬p: Liam did not finish his work early. q: he will order pizza for dinner. ¬q: he will not order pizza for dinner.	Templatized Context: If Liam finishes his work early, then he will order pizza for dinner. He won't order pizza for dinner. NL Context: Liam knows that if he finishes his work early for the day, he will order pizza for dinner. However, on this particular day, he decided against ordering pizza.	Question 1: Does this imply that Liam didn't finish his work early? (Yes) Question 2: Does this imply that Liam finishes his work early? (No)	Question: Based on the context, what conclusion would be deemed most suitable? Option 1: Liam didn't finish his work early. (Yes) Option 2: Sarah had already ordered Chinese takeout. (No) Option 3: Rebecca finished her work early. (No) Option 4: Liam decided to order sushi instead. (No)
Inference rule: BD p(x): someone drinks lots of water q(x): they will feel hydrated r(x): they eat too much sugar s(x): they will experience a sugar crash p(a): Jane drinks lots of water ¬p(a): Jane does not drink lots of water q(a): she will feel hydrated ¬q(a): she will not feel hydrated r(a): she eats too much sugar ¬r(a): she does not eat too much sugar s(a): she will experience a sugar crash ¬s(a): she will not experience a sugar crash	Templatized Context: If someone drinks lots of water, then they will feel hydrated. If they eat too much sugar, then they will experience a sugar crash. We know that at least one of the following is true (1) Jane drinks lots of water and (2) she won't experience a sugar crash. Note that we do not know which ones of (1) and (2) are true. It might be the case that only (1) is true, or only (2) is true or both are true. NL Context: If someone consumes a significant amount of water, they will experience a state of hydration. conversely, if excessive amounts of sugar are ingested by them, a sugar crash will ensue. it is known that at least one of the following statements is true: either the Jane consumes ample water or she will not experience a sugar crash. however, the actual veracity of either statement remains ambiguous, as it could be the case that only the first statement is true, only the second statement is true, or both statements are true.	Question 1: Can we say at least one of the following must always be true? (a) she will feel hydrated and (b) she doesn't eat too much sugar (Yes) Question 2: Can we say at least one of the following must always be true? (a) she won't feel hydrated and (b) she eats too much sugar (No) Question 3: Can we say at least one of the following must always be true? (a) she will feel hydrated and (b) she eats too much sugar (No) Question 4: Can we say at least one of the following must always be true? (a) she won't feel hydrated and (b) she doesn't eat too much sugar (No)	Question: Taking into account the context provided, what conclusion would be the most appropriate? Option 1: If Jane consumes ample water, she will experience a sugar crash (No) Option 2: John will feel hydrated or he won't experience a sugar crash (No) Option 3: Jane will feel hydrated or she doesn't eat too much sugar (Yes) Option 4: Jane won't feel hydrated or she will eat too much sugar (No)

Table 3.3: Illustrative examples of logical context and questions created using sentences that are generated in the first stage (§3.2.2.1) for both tasks, i.e., BQA and MCQA.

of narrative and questions generated from a sentence pair for the ‘Default Reasoning with Irrelevant Information (DRI)’ from NM reasoning. In this specific instance, the generated sentences are (p, q, r, s, t) , and the negation is only required for the sentence t . Therefore, there is a single negation pair $(t, \neg t)$, which is used to generate questions specific to the ‘DRI’. Same as PL and FOL, templatized context is also converted to narrative using a prompt[33.2.2.2] created for NM. Each prompt is shown below for PL, FOL, and NM, illustrating the templatized context-to-narrative conversion.

Rule	Generate Sentences in Step 1	NL logical expressions
MT	p: Liam finished his work early. $\sim$p: Liam did not finish his work early. q: He will order pizza for dinner. $\sim$q: He will not order pizza for dinner.	Context: Liam had finished his work early for the day, which meant that he would typically have ordered pizza for dinner. However, on this particular day, he decided against ordering pizza and opted for something else instead. Question: Does this imply that liam didn't finish his work early?
BD	p(x): someone drinks lots of water q(x): they will feel hydrated r(x): they eat too much sugar s(x): they will experience a sugar crash p(a): Jane drinks lots of water $\sim$p(a): Jane does not drink lots of water q(a): she will feel hydrated $\sim$q(a): she will not feel hydrated r(a): she eats too much sugar $\sim$r(a): she does not eat too much sugar s(a): she will experience a sugar crash $\sim$s(a): she will not experience a sugar crash	Context: If an individual consumes a significant amount of water, they will experience a state of hydration. Conversely, if excessive amounts of sugar are ingested, a sugar crash will ensue. It is known that at least one of the following statements is true: either the Jane consumes ample water or she will not experience a sugar crash. However, the actual veracity of either statement remains ambiguous, as it could be the case that only the first statement is true, only the second statement is true, or both statements are true. Question: Can we say at least one of the following must always be true? (a) she will feel hydrated and (b) she doesn't eat too much sugar
MP	p(x): someone is exhausted. q(x): they will take a rest. p(a): Jack is exhausted. $\sim$p(a): Jack is not exhausted. q(a): he will take a rest. $\sim$q(a): he will not take a rest.	Context: If someone is extremely tired, then they will seek some rest and relaxation. Today, Jack finds himself utterly exhausted. Question: Does this entail that he will take rest?
DS	p: Levi is not studying for his exams $\sim$p: Levi is studying for his exams q: Maya is writing a book $\sim$q: Maya is not writing a book	Context: Either Levi is not studying for his exams or Maya is writing a book, or maybe both. It was unclear which choice to make as he didn't know if either of the options was true. However, it turned out that Levi decided to prioritize his exams and focus on studying. He knew that in order to succeed, he needed to dedicate his time and energy to preparing for the upcoming tests. Question: Does this mean that Maya is writing a book?
HS	p(x): someone buys all the necessary supplies. q(x): they can start the project. r(x): they will finish it on time. p(a): Lily bought all the necessary supplies. $\sim$p(a): Lily did not buy all the necessary supplies. q(a): She can start the project. $\sim$q(a): She can not start the project. s(a): She will finish it on time. $\sim$s(a): She will not finish it on time.	Context: If all the necessary supplies have been purchased by someone, then they can initiate the project. Once the project is started by someone, they will complete it within the expected timeframe. Question: If lily bought all the necessary supplies, does this mean that she will finish it on time?
CD	p: Harry goes to the park. $\sim$p: Harry does not go to the park. q: he will have a picnic with his family $\sim$q: he will not have a picnic with his family r: he goes to the beach $\sim$r: he does not go to the beach s: he will swim in the ocean $\sim$s: he will not swim in the ocean	Context: Harry had two options for his day off: going to the park or going to the beach. If he decided to go to the park, it meant that he would have a lovely picnic with his family. On the other hand, if he chose the beach, he would be able to enjoy a refreshing swim in the ocean. The only thing that was certain was that either Harry would go to the park or he would go to the beach. It was unclear which option he would ultimately choose, as it could be that he would go to the park alone, go to the beach alone, or even do both activities. Question: Can we say at least one of the following must always be true? (a) he will have a picnic with his family and (b) he will swim in the ocean
DD	p: I order takeout $\sim$p: I did not order takeout q: I will save time $\sim$q: I will not save time r: I cook a meal $\sim$r: I did not cook a meal s: I will save money $\sim$s: I will not save money	Context: If I decide to order takeout, it means I will save time. On the other hand, if I choose to cook a meal, it means I will save money. The interesting thing is that I am in a situation where I won't be able to save time or money. It is uncertain whether I won't save time or I won't save money, or it could even be both scenarios. The only thing that is clear is that at least one of these possibilities is true. Question: Can we say at least one of the following must always be true? (a) i don't order takeout and (b) i don't cook a meal
CT	p: Tom is an avid reader $\sim$p: Tom is not an avid reader q: he devours books of all genres $\sim$q: he does not devour books of all genres	Context: At least one of two things is true about Tom - he is either an avid reader or he devours books of all genres. We are unsure which one of these statements is true or if both are true. It could be that only the first statement is true, or only the second statement is true, or even that both are true. Question: Can we say at least one of the following must always be true? (a) he devours books of all genres or (b) tom is an avid reader
MI	p: he is not eating healthy $\sim$p: he is eating healthy q: he will not gain weight $\sim$q: he will gain weight	Context: Once upon a time, there was a guy named John who was very conscious about his weight. He knew that if he didn't eat healthy, he wouldn't be able to maintain his desired weight. John understood the condition that if he didn't eat healthy, he wouldn't be able to gain weight. Question: Based on context, can we say, at least one of the following must always be true? (a) he is eating healthy and (b) he will not gain weight
EG	p(x): someone has coding skills a: Sheila p(a): Sheila is a proficient programmer	Context: Sheila possesses exceptional coding skills and is highly proficient in programming. She has mastered various programming languages and is known for her ability to write efficient and error-free code. Sheila's expertise in programming allows her to solve complex problems with ease. Her colleagues often seek her assistance when facing technical challenges as she consistently delivers high-quality solutions. Question: does this mean that someone has coding skills?
UI	p(x): students need to take an exam to complete their degree a: Reema p(a): Reema is a student.	Context: All students are required to take an examination in order to fulfill the requirements for their degree. Reema, being a student, is also expected to fulfill the requirements. Question: Does reema need to take an exam to complete her degree?

Table 3.4: Illustrative examples of NL logical context and questions created using sentences that are generated in §3.2.2.1 for rules covered in PL and FOL for BQA.

Rule	Generate Sentences in Step 1	NL logical expressions
DRI	p: Cats and dogs are mammals. q: Mammals typically have fur. r: Cats don't have fur. s: Dogs are loyal animals. t: Dogs have fur. ~t: Dogs don't have fur.	Context: Once upon a time, in a land filled with animals, there were two popular mammalian creatures, cats and dogs. Mammals typically possessed a coat of fur, which kept them warm and protected. However, cats were an exception to this rule, as their bodies lacked fur. Nonetheless, both cats and dogs were beloved by many for their unique traits. Dogs, known for their loyalty, were particularly cherished by humans. Question: Does this imply that dogs have fur?
DRS	p: John and Mary are parents. q: Parents are usually loving and supportive. r: Parents are normally responsible. s: Mary isn't loving and supportive. t: John is not responsible. u: Mary is responsible. ~u: Mary isn't responsible. v: John is loving and supportive. ~v: John isn't loving and supportive.	Context: John and Mary were expecting their first child, filled with the anticipation and excitement that all parents feel. Parents are usually loving and supportive. Parents are normally responsible. However, something seemed amiss in their relationship. Mary, usually affectionate and caring, seemed distant and uninvolved. On the other hand, John, known for his responsible nature, started neglecting his duties and became unreliable. Question: Does this imply that Mary is responsible and John is loving and supportive?
DRD	p: Jenny and Anna are tall. q: Tall people usually play basketball. r: Anna is possibly an exception to this rule. s: Jenny plays basketball. ~s: Jenny doesn't play basketball.	Context: Jenny and Anna are known for their tall stature, which is often associated with playing basketball. However, Anna might be an exception to this norm. Question: Does this entail that Jenny plays basketball?
DRO	p: Hummingbirds are birds. q: Birds migrate south for the winter. r: Hummingbirds do not migrate south for the winter. s: All other birds than hummingbirds migrate south for the winter. ~s: All other birds than hummingbirds don't migrate south for the winter.	Context: In the bird kingdom, there are many different species that possess unique characteristics. One such species is the hummingbird, known for its ability to hover in mid-air and its vibrant colors. While most birds engage in the annual migration south for the winter, the hummingbird chooses to stay put and brave the cold weather. This decision sets the hummingbird apart from its fellow avian companions, as it relies on its resilience and resourcefulness to survive the harsh conditions. Question: Does this mean that all other birds than hummingbirds migrate south for the winter?
RE1	p: Cats, dogs, and horses are animals. q: Animals are usually considered to be intelligent creatures. r: At least one of the cats or dogs is not considered intelligent. s: Horses are considered to be intelligent creatures. ~s: Horses aren't considered to be intelligent creatures. t: Exactly one of the cats or dogs is not considered intelligent. ~t: Exactly one of the cats or dogs is considered intelligent.	Context: In a world where animals are often regarded as intelligent creatures, there is a captivating tale that revolves around cats, dogs, and horses. It is commonly believed that most animals possess a level of intellect. However, there is an intriguing twist to this belief as it is known that either cats or dogs are not considered particularly intelligent. As the story unfolds, we delve into the lives of these remarkable creatures, their interactions, and the unique qualities that each of them possesses. Question: Does this entail that horses are considered to be intelligent creatures and exactly one of the cats or dogs is not considered intelligent?
RE2	p: cats normally meow. q: At least one species of cat doesn't meow. r: Exactly one species of cat doesn't meow. ~r: Exactly one species of cat meows.	Context: In the realm of cat communication, meowing serves as a fundamental aspect of their vocal repertoire. However, intriguingly enough, there exists a distinct species of cat that deviates from this conventional norm. This peculiar feline defies the expectations associated with its kind by refraining from emitting any meows whatsoever. Question: Does this entail that exactly one species of cat doesn't meow?
RE3	p: Cars have four wheels. q: wheels normally have spokes. r: at least one wheel does not have spokes. s: Cars have four wheels with spokes. ~s: Cars don't have four wheels with spokes.	Context: In a world where cars were known for having four wheels, it was considered a common fact that wheels typically came equipped with spokes. However, amidst this widespread understanding, there was an exception. At least one wheel defied this norm and stood out from the rest by not having any spokes at all. Question: Does this imply that cars have four wheels with spokes?
RAP	p: John asserts that Sally was in the store. q: Jane asserts that Sally was not in the store. r: John's evidence is more reliable than Jane's. ~r: John's evidence is less reliable than Jane's. s: Sally was in the store. ~s: Sally wasn't in the store.	Context: In the midst of a heated argument, John adamantly claims that Sally was present at the store. However, Jane strongly opposes John's assertion, insisting that Sally was indeed absent from the store. Question: If John's evidence is more reliable than Jane's, does this mean that Sally was in the store?

Table 3.5: Illustrative examples of NL logical context and questions created using sentences that are generated in §3.2.2.1 for NM logic for BQA.

3.2.2.3 Task Instance Generation

After generating the context and questions in §3.2.2.2, we generate $(context, question, answer)$ triplets for both tasks: (i) BQA, and (ii) MCQA. Here, the narrative-based context is similar for both BQA and MCQA tasks, only the format of $(question, answer)$ pairs is different.

BQA We generate semantically preserving and inverting variations of questions. Let’s consider the example of “Modus Tollens” from Table 3.3, having the question as: “Does this imply that Liam didn’t finish his work early?” In this question, we observe one proposition: s_1 , representing the statement “Liam didn’t finish his work early,” can be used to create another question that did not follow the logical rule MT. We can create two possible tuples: $\langle \neg s_1, yes \rangle, \langle s_1, no \rangle$. Each tuple has a question-answer combination using proposition s_1 . Moreover, we do not generate variations for the context since it offers no substantial diversity in the dataset. For question variations, we replace the variation of proposition $\langle s_1 \rangle$ in the original question with the corresponding tuples to add diversity to *LogicBench*. For example, HS can be defined as “If p then q; if q then r; therefore, if p then r” and the narrative will have “If p then q; if q then r” and the question is asked as “Can we conclude if p then r?” having answer as “yes”. Now, other variations in the question are asked in the following ways by negating the sentence (p and r) comb: Variation 1: “Can we conclude if $\neg p$ then r?”, Variation 2: “Can we conclude if p then $\neg r$?”, Variation 3: “Can we conclude if $\neg p$ then $\neg r$?”. If there’s only one proposition (p) present in the question, then there are only 2 variations that can be made asking about (p) and ($\neg p$). The process allows us to create more variations of the question for BQA tasks, as illustrated in Figure 3.2 (Step 3 - Task 1).

MCQA A prompt-based approach is used to create different incorrect options. For rule MT, as shown in Table 3.3, option 1 is a correct option that follows the rule MT logically ($\langle \neg s_1 \rangle$), while the other three options are generated using prompting in a way that does not follow the rule being incorrect options. In addition to the options, the question is also replaced by a randomly selected question from the set of five questions. As seen in Figure 3.2 (Step 3 - Task 2), there is only one correct option out of four given options. The question is a bit different from the BQA question formation, as we have MCQA and have to identify which conclusion can be derived from the narrative. The question is randomly selected from the pre-defined set of questions. Here, the five different questions are as follows:

1. What would be the most appropriate conclusion based on the given context?
2. Considering the provided context, what conclusion would be deemed most suitable?
3. In light of the context provided, what conclusion can be considered the most appropriate?
4. Based on the context, what conclusion would be deemed most suitable?
5. Taking into account the context provided, what conclusion would be most appropriate?

The narrative for MCQA is the same as the BQA’s narrative, and there’s no change. Comparing BQA methodology with MCQA, MCQA’s correct option is the question asked in the BQA, which can be concluded from the logical rule present in the narrative. For example, HS can be defined as “If p then q; if q then r; therefore, if p then r” and the narrative will have “If p then q; if q then r” and the question is randomly chosen from the set of questions as mentioned above and the correct option from the

different multiple choice will be “if p then r”. Based on the information present, we have generated an incorrect option using prompting[3.2.2.3].

Prompt for incorrect option generation

Context: Narrative

Question: Randomly chosen question

Correct option: Conclusion from narrative

Create three more irrelevant, incorrect options to create multiple choice. Sometimes use different persons’ names as well in creating incorrect options.

Instruction to generate options:

1. Generate only three unique choices each on a new line.
2. Do not use any prefix while generating.

3.2.3 Statistics and Qualitative Analysis

Statistics We introduce two versions of our proposed dataset: $LogicBench(Eval)_{BQA}$ and $LogicBench(Eval)_{MCQA}$. Statistics of both versions are presented in Table 3.6. For $LogicBench(Eval)_{BQA}$, out of 1520, 520 samples are for ‘yes’ and 1000 samples are for ‘no’ labels. For $LogicBench(Eval)_{MCQA}$, there are 20 unique samples present for each rule, thus in total 500 unique samples. Furthermore, we synthetically augmented $LogicBench(Eval)$ for training purposes (i.e., $LogicBench(Aug)$), which consists of 150 unique data samples for each rule for BQA, resulting in a total of 12908 data samples including variations.

Data Validation Throughout the data generation phase of $LogicBench(Eval)$, the authors conduct a review of the logical formations to ensure they follow the intended logical structure. We examine each narrative for any potential discrepancies, ensur-

Dataset	# of Instances per Rule	Total # of Instances	Total # of Instances (Including Variations)
<i>LogicBench(Eval)</i> _{BQA}	20	500	1520
<i>LogicBench(Eval)</i> _{MCQA}	20	500	500

Table 3.6: Statistics of *LogicBench(Eval)*

ing that they are logically sound and correctly represent the intended relationships between propositions. In addition to the logical formation, we also dedicated considerable effort to eliminating typos and validating the grammar. We also analyze the diversity in terms of different ontologies and the logical nature of the *LogicBench(Eval)*. Figure 3.5 provides a word cloud derived from the *LogicBench(Eval)*. This word cloud highlights the logical nature and diversity of our evaluation dataset. Words such as ‘if’, ‘normally’, ‘usually’, and ‘then’ are prominently featured, suggesting their frequent use in the dataset, and suggesting the logical nature of the dataset. Moreover, we can also observe several words consisting of different ontologies such as ‘cat’, ‘car’, ‘garden’, and many more, suggesting diversity in the dataset.

Mitigation of Errors in *LogicBench* While validating *LogicBench(Eval)*, we encountered errors within the synthetically generated narrative-based context. We mitigate these errors manually, categorizing them into two groups: (i) eliminating leakage of logical conclusions and (ii) ensuring the inclusion of intended logical premises. In the first category, we found $\sim 15\%$ of the narrative-based contexts (out of 500 total instances) were found to explicitly present the logical conclusion as a response to the question, bypassing the logical derivation process. This enables the model to extract the final logical conclusion from the context rather than derive it logically. In the second category, we found $\sim 8\%$ of the narrative-based contexts (out of 500 total in-



Figure 3.5: Word cloud of context present in the *LogicBench(Eval)*.

stances) where narration lacked some necessary premise sentences crucial for reaching a logical conclusion. To address this issue, we manually incorporated those sentences to ensure the quality of the data. For the MCQA task, during the generation of three incorrect options, we found instances where the model produces two semantically similar options, resulting in the creation of ambiguous choices in $\sim 11\%$ of the cases (out of a total of 500 instances). We manually mitigated all these errors from the data instances, ensuring the high quality of our validation data. We believe that these two versions aim to accommodate different evaluations to explore the logical reasoning capabilities of LLMs.

3.2.4 Examples of Data Instances

This section provides examples of $(context, question, answer)$ tuples corresponding to each inference rule and reasoning pattern. Additionally, it highlights the diverse

range of question variations within the dataset associated with each inference rule and reasoning pattern.

Propositional Logic (PL) Here, we discuss examples of each inference rule present in the PL of the *LogicBench* as shown in Table 3.7. Table 3.7 has context related to the inference rule and different variations of the question according to the rule. For instance, the first row of Table 3.7 shows the example for inference rule, *Hypothetical Syllogism (HS)*, formally expressed as $((p \rightarrow q) \wedge (q \rightarrow r)) \vdash (p \rightarrow r)$. The context represents the premise, i.e., $((p \rightarrow q) \wedge (q \rightarrow r))$, and the first question (Q1) represents the conclusion, i.e., $p \rightarrow r$. Hence, Q1 is labeled as "Yes" since it supports the conclusion given the logical context. Furthermore, Q2 to Q4 represent different variations of the question by utilizing the variables $(p, \neg p, r, \neg r)$. For the HS, given the provided context, Q2 to Q4 contain the variations $\neg p \rightarrow r$, $p \rightarrow \neg r$, and $\neg p \rightarrow \neg r$, respectively, and are labeled as "No" since they do not support the conclusion.

First-Order Logic (FOL) Here, we discuss examples of each inference rule and two axioms (i.e., Existential Instantiation and Universal Instantiation) present in the FOL from the *LogicBench* as shown in Table 3.8. *Existential Generalization (EG)*, formally expressed as $P(a) \Rightarrow \exists x P(x)$ indicates that there is an element a in the domain for which $P(a)$ is true, then we know that $\exists x P(x)$ is true. *Universal Instantiation* formally expressed as $\forall x A \Rightarrow A\{x \mapsto a\}$ indicates that a statement holds true for all instances (x) within a specific category A , hence it is also true for specific instance a . Table 3.8 represents context related to the inference rule and variations of the question. The process of generating data instances for FOL follows a similar approach to that of PL. For example, the first row of Table 3.8 shows the example for axiom, *Existential Instantiation (EI)*, formally expressed as

$\exists xP(x) \Rightarrow P(a)$. The context represents the initial premise $\exists xP(x)$ and the first question (Q1) represents the conclusion, i.e., $P(a)$. Hence, Q1 is labeled as "Yes" since it supports the conclusion given the logical context. Furthermore, we generate the only variant of the question based on $\neg P(a)$ and label it as *No* since it does not support the conclusion.

Non-Monotonic (NM) Reasoning Here, we discuss examples of each reasoning pattern present in the NM reasoning from the *LogicBench* as shown in Table 3.9. Table 3.9 has context related to the reasoning pattern and different variants of the question. For example, the first row of Table 3.9 shows the example for *Default Reasoning with Irrelevant Information (DRI)*. For this reasoning, based on the given context, there are also two possible variations of the question, where one has a correct conclusion labeled as *Yes* and another with an incorrect conclusion labeled as *No*.

3.3 Results and Analysis

In this section, we begin by outlining the experimental framework in full detail, including model architectures, hyperparameter settings, and evaluation protocols. We then present the main quantitative results, performance across all key metrics. Furthermore, we conduct a thorough analysis and provide a detailed discussion.

3.3.1 Experimental Setup

Task Formulation For BQA, let us consider a set of data instances $\mathcal{I}_{r,L}$ corresponding to the inference rule r and logic type L . In this set, i^{th} instance is represented as $\mathcal{I}_{r,L}^i = \{(c_i, Q_i)\}$ where c_i represents narrative context and $Q_i = \{q_1, q_2, \dots, q_n\}$ represents set of question and its variations corresponding to i^{th} instance. As discussed in §3.2, each context (c) represents logical rules (e.g., All cats have fur. Tom is a cat.)

Rule	Context	Question
HS	If Jim cleans his room, he will receive a reward and if he receives a reward, he will use it to buy a new toy. So, Jim decided to tidy up his room, hoping to earn a reward. He diligently gathered his clothes, organized his toys, and dusted every surface. After a few hours of hard work, Jim's room was spotless.	Q1: If Jim cleaned his room, does this imply that he will buy a new toy? (Yes) Q2: If Jim didn't clean his room, does this entail that he won't buy a new toy? (No) Q3: If Jim cleaned his room, does this imply that he won't buy a new toy? (No) Q4: If Jim didn't clean his room, does this imply that he will buy a new toy? (No)
DS	Either Chloe is studying for her exams, or Mila is going on vacation, or both scenarios are unfolding. It is unclear which of the two options is true at this point. However, one thing is certain - Chloe is not studying for her exams.	Q1: Does this entail that Mila is going on vacation? (Yes) Q2: Does this mean that Mila isn't going on vacation? (No)
CD	If I decide to go for a walk, I will be able to breathe in some fresh air and revitalize myself. On the other hand, if I choose to stay home, I will have the opportunity to enjoy a movie. One thing is certain, either I go for a walk or I stay home. It remains uncertain which of the two options I will ultimately choose. It is entirely possible that I might opt for the walk, or perhaps I will find myself drawn to staying home, or even both possibilities might come to fruition.	Can we say at least one of the following must always be true? Q1: (a) I will get some fresh air or (b) I will watch a movie (Yes) Q2: (a) I won't get some fresh air and (b) I will watch a movie (No) Q3: (a) I will get some fresh air and (b) I won't watch a movie (No) Q4: (a) I won't get some fresh air and (b) I won't watch a movie (No)
DD	If I decide to order takeout, it means I will save time. On the other hand, if I choose to cook a meal, it means I will save money. The interesting thing is that I am in a situation where I won't be able to save time or money. It is uncertain whether I won't save time or I won't save money, or it could even be both scenarios. The only thing that is clear is that at least one of these possibilities is true.	Can we say at least one of the following must always be true? Q1: (a) I don't order takeout or (b) I don't cook a meal (Yes) Q2: (a) I order takeout and (b) I cook a meal (No) Q3: (a) I don't order takeout and (b) I cook a meal (No) Q4: (a) I order takeout and (b) I don't cook a meal (No)
BD	If it is sunny outside, then I will go for a walk. However, if it rains, I will stay inside. Currently, it is uncertain whether it is raining or not, but I do know that at least one of the following is true: either it is raining or I will not go for a walk. It is possible that only one of these statements is true, or perhaps both are true.	Can we say at least one of the following must always be true? Q1: (a) we will stay inside or (b) it is not sunny (Yes) Q2: (a) we will not stay inside and (b) it is sunny (No) Q3: (a) we will stay inside and (b) it is sunny (No) Q4: (a) we will not stay inside and (b) it is not sunny (No)
MT	If Mason decides to leave his job, he will not receive any salary. However, against all odds, Mason still receives his salary. He finds himself receiving his regular paycheck.	Q1: Does this infer that Mason didn't leave his job? (Yes) Q2: Does this infer that Mason left his job? (No)
MI	Rohan woke up in the morning and realized that he had forgotten his lunch. Knowing that if he forgets his lunch, he will not eat at school, he felt disappointed.	Based on context, can we say, at least one of the following must always be true? Q1: (a) Rohan didn't forget his lunch and (b) he will not eat at school (Yes) Q2: (a) Rohan forgot his lunch and (b) he will eat at school (No) Q3: (a) Rohan forgot his lunch and (b) he will not eat at school (No) Q4: (a) Rohan didn't forget his lunch and (b) he will eat at school (No)
CT	At least one of two things is true about Tom - he is either an avid reader or he devours books of all genres. We are unsure which one of these statements is true or if both are true. It could be that only the first statement is true, or only the second statement is true, or even that both are true.	Can we say at least one of the following must always be true? Q1: (a) he devours books of all genres or (b) Tom is an avid reader (Yes) Q2: (a) he doesn't devour books of all genres and (b) Tom is an avid reader (No) Q3: (a) he devours books of all genres and (b) Tom isn't an avid reader (No) Q4: (a) he doesn't devour books of all genres and (b) Tom isn't an avid reader (No)

Table 3.7: Examples of context and question-answer pairs for each rule of *Proportional logic* from the *LogicBench(Eval)*.

Rule	Context	Question
UI	All students are required to take an examination in order to fulfill the requirements for their degree. Reema, being a student, is also expected to fulfill the requirements.	Q1: Does Reema need to take an exam to complete her degree? (Yes) Q2: Does Reema need not to take an exam to complete her degree? (No)
EG	The marathon race was won by James, who emerged as the champion.	Q1: Does this imply that someone won the marathon race? (Yes) Q2: Does this mean that no one won the marathon race? (No)
MP	If someone is extremely tired, then they will seek some rest and relaxation. Today, Jack finds himself utterly exhausted.	Q1: Does this entail that he will take rest? (Yes) Q2: Does this entail that he won't take rest? (No)
HS	If all the necessary supplies have been purchased by someone, then they can initiate the project. Once the project is started by someone, they will complete it within the expected time-frame.	Q1: If Lily bought all the necessary supplies, does this mean that she will finish it on time? (Yes) Q2: If Lily didn't buy all the necessary supplies, does this imply that she won't finish it on time? (No) Q3: If Lily bought all the necessary supplies, does this entail that she won't finish it on time? (No) Q4: If Lily didn't buy all the necessary supplies, does this imply that she will finish it on time? (No)
DS	It is known that one of the following options is true: someone goes to a museum or someone visits a park. The specific scenario could involve only the option to go to a museum being true, or only the option to visit a park being true, or both options being true. However, it is stated that Jill is unable to go to a museum.	Q1: Does this imply that she can visit a park? (Yes) Q2: Does this entail that she can't visit a park? (No)
CD	If someone is painting a picture, then they will frame it. similarly, the natural course of action for a writer would be to publish their completed story. in this scenario, it is certain that at least one of the following statements holds true: (1) john is currently engrossed in painting a picture, or (2) john is currently immersed in the act of writing a story. it should be emphasized that we are unaware of which statement specifically applies, as there is a possibility that either (1) alone is true, or (2) alone is true, or even that both (1) and (2) are simultaneously true.	Can we say at least one of the following must always be true? Q1: (a) he will frame it and (b) he will publish it. (Yes) Q2: (a) he won't frame it and (b) he will publish it. (No) Q3: (a) he will frame it and (b) he won't publish it. (No) Q4: (a) he won't frame it and (b) he won't publish it. (No)
DD	If someone is taking care of their health, then they will be fit. However, indulging in unhealthy habits can make individuals susceptible to various diseases. The truth is, we can be certain about at least one of the following possibilities: either Jenny won't be fit and healthy, or she won't be prone to diseases. It is important to note that we are unaware of which statement is accurate. It could be the case that only the first statement is true, only the second statement is true, or both statements hold validity.	Can we say at least one of the following must always be true? Q1: (a) Jenny doesn't take care of her health and (b) she doesn't indulge in unhealthy habits (Yes) Q2: (a) Jenny takes care of her health and (b) she indulges in unhealthy habits (No) Q3: (a) Jenny doesn't take care of her health and (b) she indulges in unhealthy habits (No) Q4: (a) Jenny takes care of her health and (b) she doesn't indulge in unhealthy habits (No)
BD	If an individual consumes a significant amount of water, they will experience a state of hydration. Conversely, if excessive amounts of sugar are ingested, a sugar crash will ensue. It is known that at least one of the following statements is true: either the Jane consumes ample water or she will not experience a sugar crash. However, the actual veracity of either statement remains ambiguous, as it could be the case that only the first statement is true, only the second statement is true, or both statements are true.	Can we say at least one of the following must always be true? Q1: (a) she will feel hydrated and (b) she doesn't eat too much sugar (Yes) Q2: (a) she won't feel hydrated and (b) she eats too much sugar (No) Q3: (a) she will feel hydrated and (b) she eats too much sugar (No) Q4: (a) she won't feel hydrated and (b) she doesn't eat too much sugar (No)
MT	If someone decides to go to the park, it is required that they wear a mask. However, in this particular situation, John does not wear a face covering.	Q1: Does this imply that John doesn't visit the park? (Yes) Q2: Does this entail that John visits the park? (No)

Table 3.8: Examples of context and question-answer pairs for each rule of *First order logic* from the *LogicBench(Eval)*.

Rule	Context	Question
DRI	Once upon a time, in a land filled with animals, there were two popular mammalian creatures, cats and dogs. Mammals typically possessed a coat of fur, which kept them warm and protected. However, cats were an exception to this rule, as their bodies lacked fur. Nonetheless, both cats and dogs were beloved by many for their unique traits. Dogs, known for their loyalty, were particularly cherished by humans.	Q1: Does this imply that dogs have fur? (Yes) Q2: Does this entail that dogs don't have fur? (No)
DRS	John and Mary were expecting their first child, filled with the anticipation and excitement that all parents feel. Parents are usually loving and supportive. Parents are normally responsible. However, something seemed amiss in their relationship. Mary, usually affectionate and caring, seemed distant and uninvolved. On the other hand, John, known for his responsible nature, started neglecting his duties and became unreliable.	Q1: Does this imply that Mary is responsible and John is loving and supportive? (Yes) Q2: Does this entail that Mary isn't responsible and John is loving and supportive? (No) Q3: Does this imply that Mary is responsible and John isn't loving and supportive? (No) Q4: Does this entail that Mary isn't responsible and John isn't loving and supportive? (No)
DRD	Jenny and Anna are known for their tall stature, which is often associated with playing basketball. However, Anna might be an exception to this norm.	Q1: Does this entail that Jenny plays basketball? (Yes) Q2: Does this mean that Jenny doesn't play basketball? (No)
DRO	In the bird kingdom, there are many different species that possess unique characteristics. One such species is the hummingbird, known for its ability to hover in mid-air and its vibrant colors. While most birds engage in the annual migration south for the winter, the hummingbird chooses to stay put and brave the cold weather. This decision sets the hummingbird apart from its fellow avian companions, as it relies on its resilience and resourcefulness to survive the harsh conditions.	Q1: Does this mean that all other birds than hummingbirds migrate south for the winter? (Yes) Q2: Does this mean that all other birds than hummingbirds don't migrate south for the winter? (No)
RE1	In a world where animals are often regarded as intelligent creatures, there is a captivating tale that revolves around cats, dogs, and horses. It is commonly believed that most animals possess a level of intellect. However, there is an intriguing twist to this belief as it is known that either cats or dogs are not considered particularly intelligent. As the story unfolds, we delve into the lives of these remarkable creatures, their interactions, and the unique qualities that each of them possesses.	Q1: Does this entail that horses are considered to be intelligent creatures and exactly one of the cats or dogs is not considered intelligent? (Yes) Q2: Does this mean that horses aren't considered to be intelligent creatures and exactly one of cats or dogs is not considered intelligent? (No) Q3: Does this mean that horses are considered to be intelligent creatures and exactly one of cats or dogs is considered intelligent? (No) Q4: Does this implies that horses aren't considered to be intelligent creatures and exactly one of cats or dogs is considered intelligent? (No)
RE2	In the realm of cat communication, meowing serves as a fundamental aspect of their vocal repertoire. However, intriguingly enough, there exists a distinct species of cat that deviates from this conventional norm. This peculiar feline defies the expectations associated with its kind by refraining from emitting any meows whatsoever.	Q1: Does this entail that exactly one species of cat doesn't meow? (Yes) Q2: Does this imply that exactly one species of cat meows? (No)
RE3	In a world where cars were known for having four wheels, it was considered a common fact that wheels typically came equipped with spokes. However, amidst this widespread understanding, there was an exception. At least one wheel defied this norm and stood out from the rest by not having any spokes at all.	Q1: Does this imply that cars have four wheels with spokes? (Yes) Q2: Does this mean that cars don't have four wheels with spokes? (No)
RAP	In the midst of a heated argument, John adamantly claims that Sally was present at the store. However, Jane strongly opposes John's assertion, insisting that Sally was indeed absent from the store.	Q1: If John's evidence is more reliable than Jane's, does this mean that Sally was in the store? (Yes) Q2: If John's evidence is more reliable than Jane's, does this mean that Sally wasn't in the store? (No) Q3: If John's evidence is less reliable than Jane's, does this entail that Sally was in the store? (No) Q4: If John's evidence is less reliable than Jane's, does this imply that Sally wasn't in the store? (Yes)

Table 3.9: Examples of context and question-answer pairs for each rule of *Non-monotonic logic* from the *LogicBench(Eval)*.

and a question (q) represents the conclusion (e.g., Does Tom have fur?). To each context and question pair, i.e., $\langle c, q \rangle$, we assign a label from the set $\mathcal{Y} = \{Yes, No\}$. We assign a label *Yes* if the conclusion logically entails the context; otherwise, assign a label *No*. To evaluate any LLMs on this setup, we provide $\langle p, c, q \rangle$ as input to predict a label from $\mathcal{Y}$ where p is a natural language prompt. In the set $\mathcal{I}_{r,L}$ for MCQA, i^{th} instance is represented as $\mathcal{I}_{r,L}^i = \{(c_i, q_i, O_i)\}$ where c_i represents narrative context and q_i represents question and $O_i = \{o_1, o_2, o_3, o_4\}$ represents four option choices. To each context and question pair, i.e., $\langle c, q \rangle$, we assign a label from the set $\mathcal{Y} = \{o_1, o_2, o_3, o_4\}$. We assign a label o_1 if the correct conclusion is presented in the first option, and likewise for other labels. To evaluate any LLMs on this setup, we provide $\langle p, c, q, o \rangle$ as input to predict a label from $\mathcal{Y}$.

Experiments We evaluate a range of prompting models, including GPT-4, ChatGPT (GPT-3.5-Turbo), Google Gemini-Pro, Llama-2-7B-Chat, and Mistral-7B-Instruct-v0.2. Each model is evaluated in a zero-shot setting where the chain-of-thought prompt is provided to the model without any in-context examples. This approach allows us to determine LLM’s inherent ability to do logical reasoning (based on pre-training), as we can not expect that various logical inference rules/patterns will always be made part of prompts. We evaluate GPT-4 and ChatGPT by utilizing their APIs provided by OpenAI². We evaluate Google Gemini-Pro by utilizing its API provided by Google³. The evaluation is conducted on the versions of GPT-4, ChatGPT, and Gemini released in January 2024. It’s important to note that these models are regularly updated, so when reproducing the results presented in Table 3.10 and Table 3.11 (main paper), there is a possibility of variations. For Llama-2 and Mistral, we utilize the 7B-Chat and 7B-Instruct-v0.2 versions, respectively, from the huggingface

²<https://platform.openai.com/docs/guides/gpt>

³<https://ai.google.dev/>

model repository⁴. Furthermore, we also evaluate these models in a few-shot setting.

Experiments on other logic datasets In single and multi-task experiments on other logic datasets, we fine-tune the T5-large model for 10 epochs with a batch size of 16, 1024 maximum input length, an adaptive learning rate of $5e-05$, and an AdamW optimizer for each experiment. All experiments are performed using NVIDIA RTX A6000 GPUs.

Metrics Here, we evaluate performance in terms of accuracy for both tasks, BQA and MCQA. For the BQA, we measure accuracy corresponding to each label, i.e., $A(Yes)$ and $A(No)$. We evaluate each model on three different chain-of-thought prompts and report average results across these prompts.

3.3.2 Prompts for Experiments

The reported results in Table 3.10 represent the average performance across these prompts. All the prompts follow the common pattern, which includes a task description and formatting instructions. The following are the three different prompts utilized in the experiments:

Prompt 1

Given the context and question, think step-by-step logically to answer the question.

Answer the question ONLY in ‘yes’ or ‘no’. Please use the below format:

Context: [text with logical rules]

Question: [question based on context]

Reasoning steps: [generate step-by-step reasoning]

Answer: Yes/No

⁴<https://huggingface.co/models>

Prompt 2

Let's think step-by-step to answer the question given context. Answer the question ONLY in 'yes' or 'no'. Please use the below format:

Context: [text with logical rules]

Question: [question based on context]

Reasoning steps: [generate step-by-step reasoning]

Answer: Yes/No

Prompt 3

Given the context that contains rules of logical reasoning in natural language and question, perform step-by-step reasoning to answer the question. Based on context and reasoning steps answer the question ONLY in 'yes' or 'no'. Please use the below format:

Context: [text with logical rules]

Question: [question based on context]

Reasoning steps: [generate step-by-step reasoning]

Answer: Yes/No

For the MCQA task, the reported results in Table 3.11 represent the average performance across three prompts. All the prompts follow the common pattern, which includes a task description and formatting instructions. The following are the three different prompts utilized in the experiments:

Prompt 1

Given the context that contains rules of logical reasoning in natural language, question, and options, perform step-by-step reasoning to answer the question. Answer the question ONLY in 'choice_1' or 'choice_2' or 'choice_3' or 'choice_4'. DO NOT generate any text with it. Based on context and reasoning steps, choose

the correct option.

Please use the below format:

Context: [text with logical rules]

Question: [question based on context]

Options: [four different choices]

Reasoning steps: [generate step-by-step reasoning]

Answer: choice_1 / choice_2 / choice_3 / choice_4

Prompt 2

Let's think step-by-step to choose the correct option for the question given context. Answer the question ONLY in 'choice_1' or 'choice_2' or 'choice_3' or 'choice_4'. DO NOT generate any text with it. Based on context and reasoning steps, choose the correct option.

Please use the below format:

Context: [text with logical rules]

Question: [question based on context]

Options: [four different choices]

Reasoning steps: [generate step-by-step reasoning]

Answer: choice_1 / choice_2 / choice_3 / choice_4

Prompt 3

Given the context, question, and options, think step-by-step logically to answer the question by choosing the correct option. Answer the question ONLY in 'choice_1' or 'choice_2' or 'choice_3' or 'choice_4'. DO NOT generate any text with it. Based on context and reasoning steps, choose the correct option.

Please use the below format:

Context: [text with logical rules]
Question: [question based on context]
Options: [four different choices]
Reasoning steps: [generate step-by-step reasoning]
Answer: choice_1 / choice_2 / choice_3 / choice_4

3.3.3 Main Results

Table 3.10 and Table 3.11 represent inference rule-wise performance corresponding to each LLM for the BCQ and MCQA tasks, respectively. Specifically, Table 3.10 provides label-wise accuracy ($A(Yes)$ and $A(No)$) for the BQA task, and Table 3.11 provides overall accuracy for the MCQA task. Both tables provide valuable insights into the performance of different models on various logic types and lead to several interesting findings. From Table 3.10, we can observe that ChatGPT achieves 48.04%, and GPT-4 shows a performance of 63.98% $A(Yes)$ on average, which indicates the challenge of classical logical reasoning (PL) even for larger LLMs such as ChatGPT and GPT-4. Furthermore, we can observe that models struggle more with inference rules of PL compared to FOL and NM. In addition, it is noticeable that each model performs relatively better on questions with a negative response (i.e., No) compared to questions with a positive response (i.e., Yes). This observation suggests that the models struggle to fully comprehend the logical relationship between the context and the conclusion (i.e., lower $A(Yes)$). However, they demonstrate a relatively stronger understanding when the relationship is contradictory in nature (i.e., higher $A(No)$). From Table 3.11, we can observe that larger models exhibit superior performance in selecting the correct choice to arrive at a logical conclusion. Interestingly, the performance of models decreases when the inference rules are longer or include negations.

In contrast to Table 3.10, for MCQA, LLMs show superior performance for PL and FOL compared to NM. To further investigate these findings and provide a detailed analysis, we perform a thorough study of reasoning chains generated by LLMs and present our insights in the subsequent section.

3.3.4 Few-shot Results

This section discusses the performance of the different LLMs in a few-shot setting on the $LogicBench(Eval)_{BQA}$. Here, we only present a case study on the BQA task. For the fair comparison with Table 3.10, we analyze an average performance across $A(Yes)$. Table 3.12 shows the performance for each inference rule and reasoning patterns achieved by Llama-2, Mistral, Gemini, ChatGPT, and GPT-4.

As suggested in [136], prompting models are sensitive to in-context examples. Hence, we see mixed performance in Table 3.12 across all models. From Table 3.12, we can observe that in-context examples are helpful for Llama-2 since it consistently outperforms zero-shot baselines by large margins in terms of $A(Yes)$. Llama-2 is remarkably good at following the in-context exemplars and mimicking the process to reach the correct conclusions. Thus, leveraging the in-context exemplars, Llama-2 achieves high accuracy in a few-shot setting. Specifically, Mistral consistently shows degraded performance for all logic types. However, ChatGPT improves performance on NM reasoning. Improved performance in NM reasoning demonstrates that the inclusion of in-context examples enhances the ability of these models to comprehend the nuanced meanings of logical terms such as “usually” and “typically”. In particular, we see that Gemini and GPT-4 improve performance on PL and FOL, respectively, but show competitive performance on NM.

Type	Rules	Llama-2		Mistral		Gemini		ChatGPT		GPT-4	
		$A(No)$	$A(Yes)$	$A(No)$	$A(Yes)$	$A(No)$	$A(Yes)$	$A(No)$	$A(Yes)$	$A(No)$	$A(Yes)$
PL	HS	100 _{0.00}	47.35 _{0.03}	98.63 _{0.01}	64.24 _{0.07}	99.31 _{0.01}	59.60 _{0.01}	100 _{0.00}	66.82 _{0.04}	100 _{0.00}	84.75 _{0.06}
	DS	44.81 _{0.05}	56.82 _{0.02}	50.77 _{0.03}	77.66 _{0.05}	68.21 _{0.04}	91.65 _{0.04}	51.26 _{0.04}	80.71 _{0.09}	79.96 _{0.06}	100 _{0.00}
	CD	79.94 _{0.03}	25.47 _{0.01}	83.37 _{0.03}	72.30 _{0.12}	85.56 _{0.04}	28.15 _{0.01}	90.75 _{0.06}	38.63 _{0.04}	92.85 _{0.01}	66.12 _{0.05}
	DD	82.22 _{0.16}	25.22 _{0.01}	71.26 _{0.02}	14.16 _{0.03}	75.48 _{0.04}	25.22 _{0.02}	71.28 _{0.05}	23.47 _{0.03}	84.51 _{0.02}	42.40 _{0.04}
	BD	48.89 _{0.43}	25.13 _{0.01}	79.68 _{0.01}	47.36 _{0.11}	86.81 _{0.02}	29.51 _{0.01}	84.91 _{0.05}	33.47 _{0.05}	87.86 _{0.04}	59.06 _{0.13}
	MT	70.54 _{0.03}	71.46 _{0.05}	47.96 _{0.08}	44.96 _{0.10}	75.99 _{0.05}	81.47 _{0.03}	55.63 _{0.02}	66.11 _{0.07}	55.28 _{0.04}	59.05 _{0.07}
	MI	78.57 _{0.26}	25.34 _{0.01}	75.36 _{0.01}	25.51 _{0.04}	74.69 _{0.06}	24.84 _{0.04}	81.60 _{0.02}	31.79 _{0.03}	91.84 _{0.01}	39.72 _{0.03}
	CT	70.00 _{0.12}	24.98 _{0.02}	87.38 _{0.01}	71.99 _{0.03}	88.31 _{0.08}	35.11 _{0.07}	89.88 _{0.05}	43.33 _{0.03}	98.59 _{0.01}	60.71 _{0.06}
	Avg	71.87_{0.13}	37.72_{0.02}	74.30_{0.03}	52.27_{0.07}	81.79_{0.04}	46.94_{0.03}	78.16_{0.04}	48.04_{0.05}	86.36_{0.03}	63.98_{0.05}
FOL	EG	100 _{0.00}	71.67 _{0.03}	100 _{0.00}	100 _{0.00}	85.68 _{0.06}	97.97 _{0.04}	96.74 _{0.03}	96.75 _{0.03}	98.41 _{0.03}	100 _{0.00}
	UI	80.24 _{0.05}	56.04 _{0.02}	85.24 _{0.04}	85.24 _{0.04}	90.69 _{0.08}	90.29 _{0.04}	85.31 _{0.02}	96.19 _{0.03}	90.16 _{0.002}	91.58 _{0.03}
	MP	97.44 _{0.04}	73.01 _{0.04}	96.28 _{0.03}	95.24 _{0.05}	98.25 _{0.03}	96.97 _{0.05}	100 _{0.00}	91.03 _{0.04}	94.74 _{0.00}	100 _{0.00}
	HS	100 _{0.00}	36.83 _{0.01}	98.02 _{0.00}	65.00 _{0.05}	97.70 _{0.00}	52.46 _{0.04}	95.14 _{0.00}	46.74 _{0.04}	89.03 _{0.01}	61.21 _{0.01}
	DS	58.33 _{0.14}	52.08 _{0.04}	61.69 _{0.03}	92.59 _{0.13}	75.96 _{0.07}	85.82 _{0.07}	72.46 _{0.10}	94.71 _{0.05}	63.34 _{0.04}	100 _{0.00}
	CD	100 _{0.00}	26.36 _{0.01}	82.21 _{0.02}	82.02 _{0.09}	94.71 _{0.02}	33.62 _{0.02}	87.87 _{0.06}	39.37 _{0.06}	92.65 _{0.01}	74.72 _{0.05}
	DD	54.37 _{0.19}	23.36 _{0.01}	73.78 _{0.01}	6.67 _{0.11}	82.70 _{0.02}	28.96 _{0.01}	72.70 _{0.08}	26.42 _{0.11}	85.54 _{0.04}	54.55 _{0.08}
	BD	92.31 _{0.13}	25.78 _{0.01}	75.76 _{0.01}	44.44 _{0.10}	82.82 _{0.08}	28.93 _{0.05}	85.50 _{0.03}	38.82 _{0.02}	84.22 _{0.02}	66.67 _{0.10}
	MT	77.11 _{0.05}	88.41 _{0.09}	69.01 _{0.06}	91.71 _{0.01}	71.18 _{0.10}	89.58 _{0.04}	62.46 _{0.03}	92.80 _{0.06}	64.69 _{0.04}	100 _{0.00}
	Avg	84.42_{0.07}	50.39_{0.03}	82.44_{0.02}	73.66_{0.06}	86.63_{0.05}	67.17_{0.04}	84.24_{0.04}	69.20_{0.05}	84.75_{0.02}	83.19_{0.02}
NM	DRI	29.06 _{0.05}	40.98 _{0.04}	61.51 _{0.05}	72.22 _{0.05}	65.45 _{0.03}	78.50 _{0.02}	55.77 _{0.00}	88.89 _{0.09}	81.29 _{0.05}	100 _{0.00}
	DRS	66.56 _{0.03}	18.40 _{0.01}	70.13 _{0.01}	6.20 _{0.01}	68.05 _{0.01}	7.20 _{0.06}	69.06 _{0.01}	0.00 _{0.00}	77.59 _{0.02}	39.32 _{0.06}
	DRD	49.21 _{0.06}	50.47 _{0.04}	77.64 _{0.02}	97.78 _{0.04}	68.10 _{0.05}	97.43 _{0.04}	55.11 _{0.02}	100 _{0.00}	86.96 _{0.00}	100 _{0.00}
	DRO	53.25 _{0.01}	53.60 _{0.02}	68.30 _{0.04}	89.56 _{0.11}	57.85 _{0.04}	74.03 _{0.08}	50.85 _{0.01}	66.67 _{0.58}	55.56 _{0.00}	100 _{0.00}
	RE1	94.07 _{0.06}	27.55 _{0.01}	78.79 _{0.02}	44.39 _{0.03}	84.04 _{0.04}	41.98 _{0.12}	75.82 _{0.00}	31.74 _{0.03}	85.44 _{0.02}	100 _{0.00}
	RE2	54.17 _{0.47}	53.62 _{0.05}	81.31 _{0.04}	85.67 _{0.03}	58.48 _{0.03}	82.22 _{0.17}	64.67 _{0.11}	60.68 _{0.02}	59.42 _{0.00}	100 _{0.00}
	RE3	39.05 _{0.03}	38.05 _{0.10}	78.61 _{0.04}	83.59 _{0.01}	64.67 _{0.08}	79.68 _{0.02}	59.05 _{0.04}	83.57 _{0.08}	83.12 _{0.02}	89.08 _{0.01}
	RAP	76.82 _{0.05}	67.22 _{0.03}	51.73 _{0.02}	60.48 _{0.11}	75.58 _{0.04}	96.67 _{0.06}	61.44 _{0.02}	92.16 _{0.03}	66.07 _{0.05}	98.72 _{0.02}
	Avg	57.78_{0.09}	43.74_{0.04}	71.00_{0.03}	67.49_{0.05}	67.78_{0.04}	69.71_{0.07}	61.47_{0.03}	65.46_{0.10}	74.43_{0.02}	84.75_{0.02}

Table 3.10: Evaluation of LLMs in terms of label-wise accuracy on *LogicBench(Eval)_{BQA}*, where $A(Yes)$ and $A(No)$ denote the accuracy for the *Yes* and *No* labels, respectively. DRI: Default Reasoning with Irrelevant Information, DRS: Default Reasoning with Several Defaults, DRD: Default Reasoning with a Disabled Default, DRO: Default Reasoning in an Open Domain, RE1: Reasoning about Unknown Expectations I, RE2: Reasoning about Unknown Expectations II, RE3: Reasoning about Unknown Expectations III, RAP: Reasoning about Priorities

Type	Rules	Llama-2	Mistral	Gemini	ChatGPT	GPT-4
PL	HS	86.67 _{0.08}	93.33 _{0.06}	100 _{0.00}	91.67 _{0.03}	100 _{0.00}
	DS	63.33 _{0.12}	60.00 _{0.09}	86.67 _{0.10}	96.67 _{0.06}	95.00 _{0.00}
	CD	80.00 _{0.05}	70.00 _{0.13}	96.67 _{0.03}	90.00 _{0.00}	100 _{0.00}
	DD	43.33 _{0.03}	30.00 _{0.10}	90.00 _{0.05}	73.33 _{0.06}	88.33 _{0.03}
	BD	51.67 _{0.03}	53.33 _{0.03}	86.67 _{0.03}	68.33 _{0.06}	83.33 _{0.03}
	MT	31.67 _{0.03}	60.00 _{0.05}	78.33 _{0.08}	73.33 _{0.08}	76.67 _{0.03}
	MI	33.33 _{0.13}	35.00 _{0.05}	71.67 _{0.06}	63.33 _{0.08}	73.33 _{0.14}
	CT	73.33 _{0.10}	68.33 _{0.08}	100 _{0.00}	98.33 _{0.03}	100 _{0.00}
	Avg	57.92_{0.07}	58.75_{0.07}	88.75_{0.04}	81.88_{0.05}	89.58_{0.03}
FOL	EI	80.00 _{0.00}	85.00 _{0.05}	95.00 _{0.00}	93.33 _{0.03}	100 _{0.00}
	UI	63.33 _{0.03}	75.00 _{0.05}	98.33 _{0.03}	91.67 _{0.03}	98.33 _{0.03}
	MP	85.00 _{0.09}	98.33 _{0.03}	100 _{0.00}	100 _{0.00}	100 _{0.00}
	HS	61.67 _{0.06}	70.00 _{0.05}	81.67 _{0.03}	73.33 _{0.06}	76.67 _{0.03}
	DS	43.33 _{0.06}	36.67 _{0.03}	70.00 _{0.05}	78.33 _{0.10}	95.00 _{0.05}
	CD	75.00 _{0.05}	61.67 _{0.06}	93.33 _{0.03}	80.00 _{0.05}	91.67 _{0.03}
	DD	36.67 _{0.06}	46.67 _{0.03}	85.00 _{0.05}	71.67 _{0.06}	93.33 _{0.03}
	BD	35.00 _{0.05}	43.33 _{0.06}	78.33 _{0.06}	66.67 _{0.13}	91.67 _{0.06}
	Avg	57.96_{0.05}	64.81_{0.04}	87.04_{0.03}	82.41_{0.06}	91.51_{0.04}
NM	DRI	38.33 _{0.03}	28.33 _{0.06}	58.33 _{0.08}	66.67 _{0.06}	90.00 _{0.00}
	DRS	41.67 _{0.08}	16.67 _{0.10}	45.00 _{0.10}	41.67 _{0.10}	55.00 _{0.05}
	DRD	55.00 _{0.00}	50.00 _{0.05}	48.33 _{0.03}	71.67 _{0.10}	80.00 _{0.05}
	DRO	21.67 _{0.03}	21.67 _{0.03}	53.33 _{0.03}	38.33 _{0.08}	45.00 _{0.00}
	RE1	51.67 _{0.03}	31.67 _{0.08}	70.00 _{0.00}	65.00 _{0.05}	95.00 _{0.05}
	RE2	65.00 _{0.05}	75.00 _{0.00}	68.33 _{0.08}	61.67 _{0.03}	66.67 _{0.06}
	RE3	31.67 _{0.03}	33.33 _{0.03}	61.67 _{0.08}	70.00 _{0.05}	68.33 _{0.03}
	RAP	46.67 _{0.08}	35.00 _{0.09}	33.33 _{0.03}	55.00 _{0.05}	51.67 _{0.03}
	Avg	43.96_{0.04}	36.46_{0.05}	54.79_{0.05}	58.75_{0.07}	68.96_{0.03}

Table 3.11: Evaluation of LLMs in terms of accuracy on *LogicBench(Eval)_{MCQA}*.

Type	Rule	Llama-2		Mistral		Gemini		ChatGPT		GPT-4	
		$A(No)$	$A(Yes)$	$A(No)$	$A(Yes)$	$A(No)$	$A(Yes)$	$A(No)$	$A(Yes)$	$A(No)$	$A(Yes)$
PL	HS	100.0	25.3	74.3	16.7	83.3	26.9	84.2	27.9	93.8	53.1
	DS	66.7	26.9	76.6	66.7	95.2	32.8	87.5	33.3	92.2	93.8
	CD	100.0	55.6	97.1	42.2	98.1	73.1	100.0	57.1	100.0	80.0
	DD	93.8	79.2	61.1	59.1	94.7	90.0	64.3	83.3	64.3	83.3
	BD	76.9	25.4	76.4	37.5	77.8	24.6	82.4	30.4	85.1	76.9
	MT	100.0	25.6	89.3	32.7	92.9	28.1	100.0	30.3	91.5	48.5
	MI	55.6	51.6	57.9	57.1	66.7	66.7	64.0	73.3	90.5	94.7
	CT	84.7	52.6	97.4	45.2	97.4	48.7	85.7	36.8	96.7	90.0
	Avg	84.7	42.8	78.8	44.7	88.3	48.9	83.5	46.6	89.2	77.5
FOL	EG	90.9	27.5	76.8	36.4	88.9	27.6	77.4	29.6	90.4	53.6
	UI	75.0	25.8	79.7	83.3	88.0	31.5	85.7	33.3	92.1	88.2
	MP	100.0	69.0	100.0	64.5	95.2	100.0	100.0	100.0	100.0	95.2
	HS	100.0	100.0	100.0	83.3	100.0	100.0	100.0	100.0	100.0	100.0
	DS	100.0	51.3	97.3	44.2	87.5	65.0	100.0	50.0	92.9	66.7
	CD	90.5	94.7	61.9	63.2	81.0	84.2	81.0	84.2	74.1	100.0
	DD	80.0	25.3	74.7	0.0	88.0	30.9	78.3	29.4	85.1	76.9
	BD	72.7	58.6	77.8	58.1	81.8	94.1	79.2	93.8	90.5	94.7
	MT	85.7	69.2	57.7	64.3	82.4	73.9	86.4	94.4	66.7	100.0
	Avg	88.3	57.9	80.7	55.2	88.1	67.5	87.5	68.3	88.0	86.2
NM	DRI	86.2	70.6	62.5	62.5	80.4	90.9	74.0	90.0	95.2	100.0
	DRS	47.4	47.6	77.3	83.3	61.9	70.6	60.7	75.0	90.0	90.0
	DRD	100.0	66.7	100.0	62.5	78.6	68.0	63.6	55.2	70.4	92.3
	DRO	58.1	14.3	66.7	7.7	62.8	9.1	64.1	14.6	76.5	33.3
	RE1	60.9	64.7	58.6	72.7	66.7	85.7	60.6	100.0	87.0	100.0
	RE2	0.0	25.0	88.0	30.9	84.8	32.6	77.8	25.8	86.5	46.4
	RE3	52.6	52.4	68.0	80.0	82.4	71.4	62.1	81.8	61.5	71.4
	RAP	50.0	50.0	65.5	90.9	62.5	100.0	55.6	100.0	87.0	100.0
	Avg	56.9	48.9	73.3	61.3	72.5	66.0	64.8	67.8	81.8	79.2

Table 3.12: Performance of LLMs in few-shot setting in terms of label-wise accuracy on LogicBench(Eval), where $A(Yes)$ and $A(No)$ denote the accuracy for the *Yes* and *No* labels, respectively.

3.3.5 Analysis and Discussion

Human Performance We conduct a human evaluation on a subset of *LogicBench (Eval)* for both tasks, BQA and MCQA. Specifically, we selected 50 unique instances covering all 25 reasoning patterns from LogicBench(Eval). This selection resulted in total instances of 153 <context, question> pairs for BQA and 50 <context, question, choices> pairs for MCQA. We hired three graduate student volunteers to provide the evaluations. The task instructions given to all three annotators closely resemble the prompts (§3.3.2) provided to models. Each instance pair is answered/annotated by three different annotators with 0.785 inter-annotator agreement (measured with raw/observed agreement) for BQA and 0.813 for MCQA. From the results (Table 3.13) for BQA, we see that humans achieve more than $\sim 85\%$ accuracy on various logic types on *LogicBench(Eval)*, which indicates the capability of humans to comprehend single-step logical reasoning effectively. From Table 3.10, we observe that the average performance of all models is below human performance, indicating room for improvement in their reasoning capabilities. From Table 3.11, we make similar observations for MCQA. However, we can see that the performance of NM for the MCQA task remains a challenge for both humans and LLMs.

Prompt Given for BQA

Given the context that contains rules of logical reasoning in natural language and question, perform step-by-step reasoning to answer the question. Based on context and reasoning steps answer the question ONLY in “yes” or “no”.

Prompt Given for MCQA

Given the context that contains rules of logical reasoning in natural language, question, and options, perform step-by-step reasoning to answer the question. Answer the question ONLY in “choice_1” or “choice_2” or “choice_3” or “choice_4”.

Logic Type	BQA		MCQA
	A(No)	A(Yes)	Accuracy
PL	85.42%	84.17%	100%
FOL	90.74%	91.18%	97.03%
NM	72.22%	88.46%	62.05%
Avg.	82.79%	87.94%	86.36%

Table 3.13: Human performance on three logic types averaged across three annotators for both tasks.

Lower performance of LLMs on PL as compared to NM and FOL for BQA. In the development of AI, NM logic was partly developed to formalize natural language constructs, such as “normally birds fly”, that were not formalizable in a straightforward manner using classical mathematical logics. Thus, while it was difficult for researchers to come up with non-monotonic logics and formalize non-monotonic reasoning, the fact that they were usually motivated by natural language examples, suggests that many of the non-monotonic reasoning aspects are present in the NL text in the wild that is used in the pre-training of the ultra-large LLMs such as GPT4. While from human experience and complexity theory, FOL is harder than PL in general; in the LLM context, the crucial factor becomes what kind of logical sentences LLMs are pre-trained on. It seems that LLMs are pre-trained more on simple FOL sentences than on simple PL sentences. On the other hand, some PL features are perhaps less prevalent in human writing (on which LLMs are pre-trained) - such as Modes Tollens. To get an indication of this, we gave GPT-4 prompt - “Give twenty statements that have ‘if’ and ‘then’ in them”. From the results, we can observe that 13 of the 20 sentences in response to the prompt were FOL kind and only 7 were

propositional kind. This shows that LLMs’ comprehend simple FOL sentences, thus showing high overall accuracy on simpler FOL compared to PL. However, we interestingly observed that, as the depth increases, PL-based examples carrying a specific case/story become easier to understand compared to FOL-based generalized context. Table 3.10 shows that GPT-4 achieves $\sim 85\%$ accuracy ($A(Yes)$) for simple inference rules such as HS(PL). However, GPT-4 performance dropped to $\sim 59\%$ $A(Yes)$ for PL(MT).

Negations are hard to understand when embedded with logical rules. Regarding PL and FOL, it is apparent that the models struggle more with the DD and MT inference rules. A closer look at Table 3.1 reveals that all of these rules include examples where the models need to draw conclusions based on negated premises. This indicates that the models encounter difficulties when negated premises are introduced. We also analyze the effect of negations on the reasoning chain. Here, Table 3.14 presents reasoning chains corresponding to PL and FOL supporting the effect of negation on their predictions.

Longer inference rules are still challenging. Table 3.10 indicates that the models face challenges when handling longer rules, such as BD, CD, and DD, both in PL and FOL. Hence, it can be concluded that these models struggle with longer logical dependencies in the premise, particularly when a higher number of propositions are present. In the case of NM reasoning, the models exhibit lower performance in DRS, indicating that a higher number of premises often leads to more frequent mistakes.

LLMs sometimes overlook contextual information. We investigate the LLMs’ logical reasoning ability in natural language, not in artificial logical formulations. Hence, we note that LLMs sometimes hallucinate information and overlook contex-

Rule	Context/Question	Actual/Pred	Reasoning Chains
PL(MT)	Context: if elijah has a new job, then he will get a raise. however, he would not be receiving the raise he had anticipated. Question: does this infer that elijah doesn't have a new job?	Yes/No	1. The context provides a conditional statement: If Elijah has a new job, then he will get a raise. 2. The context also states that Elijah would not be receiving the raise he had anticipated. 3. The second statement does not negate the first; it only states that the raise is not as expected. 4. The lack of the anticipated raise does not necessarily mean Elijah doesn't have a new job; it could mean that the raise with the new job is different from what he expected. 5. Therefore, the information provided does not allow us to infer that Elijah doesn't have a new job based solely on the raise situation.
PL(MT)	Context: james had always been forgetful, especially when it came to wearing a sweater. his mother had repeatedly told him, 'if he forgets to wear a sweater, then he will catch a cold.' throughout the day, he went about his activities, oblivious to the dropping temperature. hours later, he returned home, but james didn't catch a cold. Question: does this infer that james didn't forget to wear a sweater?	Yes/No	1. The mother's rule establishes a conditional relationship: If James forgets to wear a sweater (condition), then he will catch a cold (result). 2. It is stated that James did not catch a cold upon returning home. 3. According to the mother's rule, if the result (catching a cold) did not occur, we cannot conclusively determine whether the condition (forgetting to wear a sweater) happened or not. 4. The absence of the result (catching a cold) does not necessarily mean the condition (forgetting to wear a sweater) did not occur, as there could be other factors at play that prevented James from catching a cold. 5. Therefore, from the information given, we cannot infer with certainty that James remembered to wear a sweater.
FOL(DD)	Context: if an individual operates a vehicle with caution, they will prevent any accidents from occurring. however, if they decide to accelerate and exceed the speed limit, they will be issued a fine. there is a certainty that at least one of the following statements is true: either john will not be able to avoid an accident or he will not face a penalty. it is unclear which of these statements is accurate, as it could be only one of them, or it is possible that both of them hold true. Question: can we say at least one of the following must always be true? (a) john doesn't drive carefully and (b) he doesn't speed up	Yes/No	1. According to the context, if an individual operates a vehicle with caution, they will prevent any accidents from occurring. 2. Also, if they decide to accelerate and exceed the speed limit, they will be issued a fine. 3. It is stated that there is a certainty that at least one of the following statements is true: either John will not be able to avoid an accident or he will not face a penalty. 4. It is unclear which of these statements is accurate, as it could be only one of them, or it is possible that both of them hold true. 5. The question asks if we can say at least one of the following must always be true: (a) John doesn't drive carefully and (b) he doesn't speed up. 6. Based on the given information, we cannot say that at least one of the statements (a) and (b) must always be true.
FOL(DD)	Context: if someone is taking care of thier health, then they will be fit. however, indulging in unhealthy habits can make individuals susceptible to various diseases. the truth is, we can be certain about at least one of the following possibilities: either jenny won't be fit and healthy, or she won't be prone to diseases. it is important to note that we are unaware of which statement is accurate. it could be the case that only the first statement is true, only the second statement is true, or both statements hold validity. Question: can we say at least one of the following must always be true? (a) jenny takes care of her health and (b) she doesn't indulge in unhealthy habits	No/Yes	1. According to the context, if someone takes care of their health, they will be fit. 2. Indulging in unhealthy habits can make individuals susceptible to various diseases. 3. The context states that we can be certain about at least one of the following possibilities: either Jenny won't be fit and healthy or she won't be prone to diseases. 4. It is important to note that we are unaware of which statement is accurate. 5. The possibilities are: a) Jenny takes care of her health and is fit. b) Jenny doesn't indulge in unhealthy habits and is not prone to diseases. c) Both statements hold validity. 6. Therefore, at least one of the following must always be true: (a) Jenny takes care of her health and (b) she doesn't indulge in unhealthy habits.

Table 3.14: Case study of reasoning chains for PL(MT) generated from ChatGPT, and FOL(DD) generated from GPT-4.

tual information, leading to incorrect conclusions. To analyze this, we manually examine all the reasoning chains generated for instances sharing the same contexts in both BQA and MCQA tasks. We observe that, although this pattern is not dominant, it affects BQA more than MCQA. In this case study, we manually examined all reasoning chains for PL and NM generated by GPT-4. The example below illustrates the observed pattern. Specifically, in step 5 of the reasoning chain, the model deviates from the provided context and introduces unrelated information, resulting in an incorrect prediction. Although these patterns are not prevalent, we found that $\sim 1\%$ of the total analyzed reasoning chains exhibited this pattern in PL and NM for BQA, and $\sim 0.8\%$ for MCQA.

Rule: Modus Tollens (PL)

Context: James had always been forgetful, especially when it came to wearing a sweater. his mother had repeatedly told him, 'If he forgets to wear a sweater, then he will catch a cold.' throughout the day, he went about his activities, oblivious to the dropping temperature. hours later, he returned home, but James didn't catch a cold.

Question: Does this infer that James didn't forget to wear a sweater?

Gold Answer: Yes

Reasoning steps:

1. James is known to be forgetful about wearing a sweater.
2. His mother stated that if he forgets to wear a sweater, he will catch a cold.
3. The temperature dropped, which would increase the likelihood of catching a cold if he were not wearing a sweater.
4. Despite the dropping temperature, James did not catch a cold.
5. The absence of a cold does not necessarily mean that James remembered to

wear a sweater, as there could be other reasons why he did not catch a cold (e.g., he may have a strong immune system, spent little time outdoors, or was in a warm environment).

6. Therefore, the fact that James did not catch a cold does not logically infer that he remembered to wear a sweater.

Answer: No

Large models are better logical reasoners. We analyze the results of both smaller (Llama-2-7B and Mistral-7B) and larger (ChatGPT, GPT-4, and Gemini) models. Table 3.10 and Table 3.11 show that larger models tend to exhibit higher performance across different types of logic. We further investigate an additional model with an intermediate size: Yi-34B-chat [297] (results are presented in Table 3.15). When compared to the Llama-2-7B, the Yi-34B model (5x larger than Llama-7B) shows improvement in average performance across three logic types. Similarly, GPT-4 outperforms Yi-34B. This suggests that increasing the model size leads to substantial gains in performance, indicating the influence of larger model capacities on carrying out better logical reasoning.

Performance of BQA *vs.* MCQA From Table 3.10 and Table 3.11, we can see that the overall performance of LLMs is higher on PL_{MCQA} compared to PL_{BQA} . Conversely, the performance is lower on NM_{MCQA} compared to NM_{BQA} . For PL, the performance gaps between the CT and DD inference rules primarily contributed to this trend, and DRO, RAP, and DRD for NM. By analyzing the reasoning chains of CT and DD for PL, we observe that LLMs often employ the elimination method in MCQA. They prioritize options to select the most appropriate conclusion, thereby improving performance. Conversely, for NM, the models typically struggle to differen-

Logic	Rule	BQA		MCQA
Type		A(No)	A(Yes)	Acc.
PL	HS	98.81 _{0.01}	92.13 _{0.02}	96.67 _{0.03}
	DS	63.61 _{0.06}	87.25 _{0.07}	90.00 _{0.05}
	CD	88.88 _{0.02}	41.04 _{0.05}	86.67 _{0.03}
	DD	73.69 _{0.01}	19.53 _{0.01}	63.33 _{0.08}
	BD	83.62 _{0.03}	43.38 _{0.04}	65.00 _{0.05}
	MT	46.88 _{0.03}	31.55 _{0.05}	58.33 _{0.06}
	MI	84.49 _{0.01}	31.57 _{0.01}	71.67 _{0.06}
	CT	95.48 _{0.04}	59.15 _{0.01}	70.00 _{0.13}
	Avg	79.43 _{0.03}	50.7 _{0.03}	75.21 _{0.06}
FOL	EG	100.0 _{0.0}	98.33 _{0.03}	88.33 _{0.03}
	UI	90.32 _{0.00}	94.64 _{0.00}	88.33 _{0.08}
	MP	84.87 _{0.02}	100.0 _{0.0}	93.33 _{0.08}
	HS	96.67 _{0.02}	94.52 _{0.00}	76.67 _{0.03}
	DS	77.42 _{0.03}	90.28 _{0.09}	55.00 _{0.05}
	CD	90.41 _{0.01}	40.39 _{0.02}	81.67 _{0.08}
	DD	72.77 _{0.02}	22.20 _{0.03}	50.00 _{0.20}
	BD	79.60 _{0.03}	30.05 _{0.06}	60.00 _{0.05}
	Avg	82.56 _{0.02}	69.61 _{0.05}	73.52 _{0.09}
NM	DRI	86.10 _{0.08}	98.25 _{0.03}	60.00 _{0.09}
	DRS	73.79 _{0.00}	10.37 _{0.10}	30.00 _{0.05}
	DRD	85.76 _{0.05}	100.0 _{0.0}	55.00 _{0.09}
	DRO	70.37 _{0.04}	100.0 _{0.0}	33.33 _{0.06}
	REI	83.80 _{0.03}	44.31 _{0.09}	73.33 _{0.08}
	REII	63.27 _{0.02}	76.03 _{0.06}	63.33 _{0.08}
	REIII	63.89 _{0.03}	88.33 _{0.13}	56.67 _{0.03}
	RAP	68.15 _{0.06}	87.12 _{0.03}	38.33 _{0.08}
	Avg	74.39 _{0.04}	75.55 _{0.05}	51.25 _{0.07}

Table 3.15: Evaluation of Yi-34B in terms of accuracy on $LogicBench(Eval)_{BQA}$ and $LogicBench(Eval)_{MCQA}$.

tiate between two options and tend to hallucinate by assuming information, resulting in lower performance. We present reasoning chains corresponding to PL and NM supporting their behavior on both BQA and MCQA tasks at <https://github.com/Mihir3009/LogicBench>.

Effect on other logic datasets We trained the T5-large model on the *LogicBench(Aug)*, resulting in a model named LogicT5. Furthermore, we performed fine-tuning on four other logical reasoning datasets: LogiQA, Reclor, LogicNLI, and FOLIO. Our experiments were carried out in two settings: single-task (fine-tuning and evaluation on one dataset) and multi-task (fine-tuning on all four datasets combined, with separate evaluations for each dataset). Table 3.16 represents the accuracy comparison between LogicT5 and baseline T5-large in both single-task and multi-task settings. The results indicate that training LLMs on *LogicBench(Aug)* has a greater impact on logic datasets that primarily focus on logical reasoning, such as FOLIO and LogicNLI. Hence, we can observe that LogicT5 consistently outperforms the baseline for LogicT5 and FOLIO. However, LogiQA and ReClor encompass other forms of reasoning in addition to logical reasoning, hence, LogicT5 demonstrates competitive performance on them.

3.4 Conclusions and Future Works

In this work, we evaluated the logical reasoning ability of LLMs on 25 distinct inference rules and reasoning patterns covering PL, FOL, and NM logics. To this end, we introduced *LogicBench*, a natural language question-answering dataset focusing on evaluating a single inference rule. We devised two tasks using *LogicBench*: (i) BQA, and (ii) MCQA. We evaluated a range of LLMs including GPT-4, ChatGPT, Gemini-Pro, Llama-2, and Mistral, on both tasks. Experimental results showed that

Methods	Models	LogiQA	FOLIO	LogicNLI	ReClor
Single-Task	T5-large	16.8	69.6	82.3	35.4
	LogicT5	16.9	71.2	84.4	36.8
Multi-Task	T5-large	21.8	83.8	68.2	42.8
	LogicT5	19.7	85.6	69.8	40.0

Table 3.16: Performance comparison between LogicT5 and baseline T5-large in terms of accuracy.

LLMs do not perform well on *LogicBench*, even though they require the application of only a single inference rule. Furthermore, we also augmented *LogicBench* to *LogicBench(Aug)*, which can be utilized for training purposes. Using *LogicBench(Aug)*, we demonstrated that LLMs trained using it showcase an improved understanding of logical reasoning, resulting in a better performance on existing logic datasets.

While *LogicBench* encompasses 25 distinct inference rules spanning three logic types (significantly more than any previous study) to comprehensively evaluate the logical reasoning capabilities of LLMs, it can be further extended by incorporating additional inference rules and logic types. However, with respect to first-order logic and logics with quantified variables, there can be an infinite number of such rules. In this study, we focused solely on evaluating model performance using a single inference rule; however, an interesting future direction can be enhancing the depth of reasoning complexity (i.e., multi-step reasoning) by incorporating combinations of inference rules to derive conclusions. We also note that this research is limited to the English language and can be extended to multilingual scenarios for evaluating the logical reasoning ability of LLMs.

MULTI-LOGIEVAL: TOWARDS EVALUATING MULTI-STEP LOGICAL REASONING

4.1 Introduction

The ability to perform multi-step reasoning – drawing conclusions from multiple premises – is a hallmark of human intelligence. Recently, Large Language Models (LLMs) such as GPT-4, ChatGPT, Gemini, and Mistral [92] have achieved impressive performance on a variety of language tasks that were previously thought to be exclusive to humans [165, 19, 312]. However, the ability of these LLMs to perform multi-step logical reasoning over natural language remains under-explored, despite its various real-world applications [103, 13]. Although several datasets have been proposed [138] to evaluate the logical reasoning capabilities of LLMs, these datasets are limited in their scope by (1) evaluating simplistic single-step logical reasoning such as ProntoQA [211] and (2) evaluating multi-step logical reasoning, but only on a single type of logic and covering only a few logical inference rules as done in FOLIO [71] and ProofWriter [233]. Furthermore, there are only a few benchmarks, such as LogicBench [174] and BoardgameQA [100], that cover reasoning such as non-monotonic, which is closer to human-like reasoning. Motivated by this, our work aims to bridge these gaps by creating a more comprehensive and logically complex evaluation dataset by incorporating varying numbers of reasoning depths (i.e., multi-steps) to reach conclusions. In addition, past attempts have been made to evaluate the multi-hop reasoning of language models [142]. In contrast, our work systematically evaluates multi-hop logical reasoning over various inference rules and their combinations.

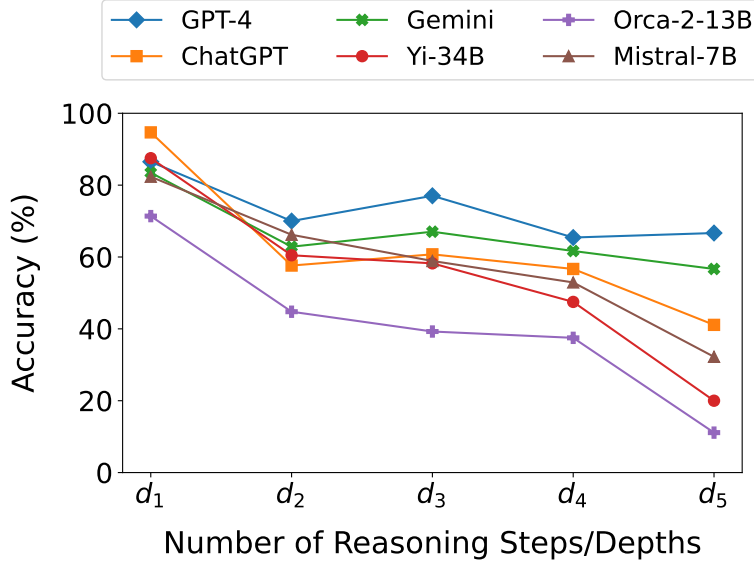


Figure 4.1: Performance (avg. accuracy across each depth for PL & FOL) of various LLMs on *Multi-LogiEval*.

To this end, we propose *Multi-LogiEval*, a systematically created Question-Answering (QA) dataset covering multi-step logical reasoning across three different logic types: Propositional Logic (PL), First-Order Logic (FOL), and Non-Monotonic (NM) reasoning. Our objective is to present a preliminary analysis of the LLMs’ ability to perform multi-step logical reasoning and demonstrate their failures even when performing simple reasoning. We believe that, regardless of whether such reasoning is available in some existing natural data (e.g., examinations), LLMs should do proper logical reasoning. Thus, we systematically compiled data using various inference rules and varying numbers of reasoning depths. In particular, our proposed dataset provides $\sim 1.6k$ high-quality instances that cover 33 inference rules and reasoning patterns and more than 60 complex combinations of these inference rules with a different number of reasoning steps ($1 \sim 5$). Our choice of inference rules is further explained in section 4.2.1.1. To evaluate LLMs, we formulate a binary classification task in *Multi-LogiEval* where the context represents a natural language story consisting of logical statements, and the models have to determine whether the story

logically entails a conclusion given in the question. Examples of instances are presented in §4.2. To develop *Multi-LogiEval*, we propose a two-stage procedure: (i) creating meaningful combinations of inference rules to generate data instances with different reasoning depths, and (ii) prompt LLMs to generate $\langle context, question, answer \rangle$ triplets consisting of different ‘ontologies’ (i.e., a collection of concepts such as car, person, and animals). In the end, we perform human validation of each generated instance to ensure the quality.

We evaluate a range of LLMs, including GPT-4, ChatGPT, Gemini-Pro, Yi-34B [298], Orca-2-13B [157], and Mistral-7B [271] on *Multi-LogiEval* using Zero-shot Chain-of-Thought (Zero-shot-CoT) prompting [272]. The zero-shot CoT approach allows us to determine LLM’s ability to do logical reasoning based on parametric knowledge (acquired during pre-training) since we can not expect in-context examples of inference rules for various reasoning depths will always be available in prompts. We measure the accuracy of LLMs’ predictions on the binary classification task. As illustrated in Figure 4.1, our experimental results indicate that LLMs performance decreases as the depth of reasoning increases, indicating mistakes in the initial reasoning step propagate further in the reasoning chain. The rationale behind the choice of a binary classification task is that it provides systematic standard metric-based evaluation (i.e., direct comparison of LLMs’ performance in terms of accuracy), which could be more challenging with open-ended question-answer formats. However, we also provide a manual and thorough analysis of the reasoning chain generated by LLMs, revealing several findings such as the importance of contextual information, the lack of correlation between longer reasoning chains and better outcomes, and the lower performance of larger-scale open-source LLMs compared to smaller ones. Thus, we believe that *Multi-LogiEval* facilitates future research for evaluating the multi-step logical reasoning ability of existing and upcoming LLMs.

4.2 Multi-LogiEval

The goal of *Multi-LogiEval* is to provide multi-step logical reasoning instances for evaluating both existing and upcoming LLMs. The selection of inference rules and reasoning patterns for our dataset is motivated by the LogicBench collection [174]. In developing *Multi-LogiEval*, we leverage the capabilities of LLMs while employing different methods to generate data for NM compared to PL and FOL, since the formulations for PL and FOL differ from NM. In particular, our data creation process consists of two major stages: (i) Generation of rule combination and (ii) Generation of data instances.

Generation of rule combination We create a meaningful combination of inference rules to achieve reasoning depths and define the complex question for each combination that will require multiple reasoning steps to answer. Here, each step generally corresponds to one inference rule.

Generation of data instances Using the combinations of inference rules generated in the above step, we prompt the LLM to generate a more human-like natural language story embedded with logical rules as a context, and then the following complex reasoning question. In this way, we generate data in the form of $\langle context, question \rangle$ pairs for each combination of inference rules at each depth.

We have created the data for three different branches of logic: Propositional logic, First-order logic, and Non-monotonic logic. We use a similar approach for propositional logic and first-order logic, which are considered classical logic, and a different approach for non-monotonic logic, which is considered non-classical. In the following sections, we explain the data creation process for both types of logic.

4.2.1 Data Generation for Monotonic Logic

We provide details of the data generation process for PL and FOL. Specifically, we delve into 14 distinct inference rules of PL and FOL, detailed in Table 4.1.

4.2.1.1 Monotonic Logic Description

Propositional Logic (PL) PL serves as a foundational framework for reasoning about truth values of statements, represented as propositions denoted by symbols like p, q, r , etc. Employing logical connectives such as ‘ $\wedge$ ’ (conjunction), ‘ $\vee$ ’ (disjunction), and ‘ $\rightarrow$ ’ (implication), it establishes relationships between these propositions. PL incorporates various inference rules, guiding the derivation of conclusions from given propositions. For instance, *Modus Ponens* is an example of such inference rules where if presented with the premises $((p \rightarrow q) \wedge p)$ —interpreted as “if p , then q , and p is true”—we can deduce the truth of q , denoted as $((p \rightarrow q) \wedge p) \vdash q$.

First-order Logic (FOL) FOL builds upon the foundations of PL by introducing predicates and quantifiers. Predicates allow us to express relationships involving variables, and quantifiers such as the universal ($\forall$) and existential ($\exists$) quantifiers enable us to make statements about all or some elements in a domain. For instance, instead of stating “John is a student,” we can express it in FOL as “There exists x such that x is John and x is a student.” This logic extends the rules of PL, such as the *Modus Ponens* rule, which lets us infer conclusions for specific instances from general premises.

Choice of inference rules Since entailment (concluding a formula in logic from another formula in that logic) in PL is Co-NP Complete, and entailment in FOL is undecidable. Even though we are interested in multi-step reasoning, our aim is

Rule	Propositional Logic	First-order Logic
MP	$((p \rightarrow q) \wedge p) \vdash q$	$(\forall x(p(x) \rightarrow q(x)) \wedge p(a)) \vdash q(a)$
MT	$((p \rightarrow q) \wedge \neg q) \vdash \neg p$	$(\forall x(p(x) \rightarrow q(x)) \wedge \neg q(a)) \vdash \neg p(a)$
HS	$((p \rightarrow q) \wedge (q \rightarrow r)) \vdash (p \rightarrow r)$	$(\forall x((p(x) \rightarrow q(x)) \wedge (q(x) \rightarrow r(x)))) \vdash (p(a) \rightarrow r(a))$
DS	$((p \vee q) \wedge \neg p) \vdash q$	$(\forall x(p(x) \vee q(x)) \wedge \neg p(a)) \vdash q(a)$
CD	$((p \rightarrow q) \wedge (r \rightarrow s) \wedge (p \vee r)) \vdash (q \vee s)$	$(\forall x((p(x) \rightarrow q(x)) \wedge (r(x) \rightarrow s(x)) \wedge (p(a) \vee r(a)))) \vdash (q(a) \vee s(a))$
DD	$((p \rightarrow q) \wedge (r \rightarrow s) \wedge (\neg q \vee \neg s)) \vdash (\neg p \vee \neg r)$	$(\forall x((p(x) \rightarrow q(x)) \wedge (r(x) \rightarrow s(x)) \wedge (\neg q(a) \vee \neg s(a)))) \vdash (\neg p(a) \vee \neg r(a))$
BD	$((p \rightarrow q) \wedge (r \rightarrow s) \wedge (p \vee \neg s)) \vdash (q \vee \neg r)$	$(\forall x((p(x) \rightarrow q(x)) \wedge (r(x) \rightarrow s(x)) \wedge (p(a) \vee \neg s(a)))) \vdash (q(a) \vee \neg r(a))$
CT	$(p \vee q) \dashv\vdash (q \vee p)$	$\forall x(p(x) \vee q(x)) \dashv\vdash \forall x(q(x) \vee p(x))$
DMT	$\neg(p \wedge q) \dashv\vdash \neg p \vee \neg q$	$\neg \forall x(p(x) \wedge q(x)) \dashv\vdash \exists x(\neg p(x) \vee \neg q(x))$
CO	$((p \rightarrow q) \wedge (p \rightarrow r)) \vdash (p \rightarrow (q \wedge r))$	$\forall x((p(x) \rightarrow q(x)) \wedge (p(x) \rightarrow r(x))) \vdash \forall x(p(x) \rightarrow (q(x) \wedge r(x)))$
IM	$(p \rightarrow (q \rightarrow r)) \dashv\vdash ((p \wedge q) \rightarrow r)$	$\forall x(p(x) \rightarrow (q(x) \rightarrow r(x))) \dashv\vdash \forall x((p(x) \wedge q(x)) \rightarrow r(x))$
MI	$(p \rightarrow q) \dashv\vdash (\neg p \vee q)$	-
EG	-	$p(a) \vdash \exists x(p(x))$
UI	-	$\forall x(p(x)) \vdash p(a)$

Table 4.1: Inference rules that establish the relationship between premises and their corresponding conclusions. A subset of these inference rules is adapted from Parmar *et al.* [174]. MP: Modus Ponens, MT: Modus Tollens, HS: Hypothetical Syllogism, DS: Disjunctive Syllogism, CD: Constructive Dilemma, DD: Destructive Dilemma, BD: Bidirectional Dilemma, CT: Commutation, DMT: De Morgan’s Theorem, CO: Composition, IM: Importation, MI: Material Implication, EG: Existential Generalization, UI: Universal Instantiation

not to build a “complete” reasoning system (the system that can make all possible entailments in that logic), rather, our goal is to make LLMs be able to at least mimic some key inference rules up to a depth of five, which itself is challenging. Thus, we start with the set of 25 inference rules used in [174] and add eight more inference rules, resulting in 33 inference rules (with zero or one variable). For a depth of five that would mean a 33^5 possible combination, which is already quite big (> 39 million).

In addition, we also consider seven FOL inference rules involving three variables and binary, ternary relations (explained in the paragraph below). In adding the new

inference rules, our main consideration was how well they match human intuition. For example, we left out $p \wedge \neg p \vdash q$ as that is not very intuitive to non-logician humans. Similarly, we left out inference rules such as simplification $((p \wedge q) \vdash p)$, conjunction $(p, q \vdash (p \wedge q))$, and addition $(p \vdash (p \vee q))$, as they would lead to infinite reasoning chains and it did not make sense to add them as an additional step of reasoning to arrive at a meaningful conclusion. Conversely, we added the DMT $(\neg(p \wedge q) \dashv\vdash \neg p \vee \neg q)$, and show its use in multi-step, as shown in Table 4.4.

Extended first-order logic with n-ary relations First-order logic often involves handling n-ary relations involving more than two variables—such as the ternary relation in “If $P(a, b, c) \wedge Q(c, d)$ then $R(a, d)$ ”. Moreover, one can alternate *for all* ($\forall$), and *there exists* ($\exists$) for any number of times in FOL, and that means there are an infinite number of such rules in first-order logic. As discussed in section 4.2.1.1, our aim is not to build a comprehensive set covering all the possible inference rules but rather to evaluate the reasoning ability of language models up to a reasoning depth of five on a systematically curated set of inference rules. However, to evaluate the ability of LLMs to reason with such complex rules, we explore 7 such inference rules for which we generated data using a similar prompt structure as depicted in Figure 4.3. We generate 10 instances for each of the inference rules, resulting in 70 instances for evaluation. The choice of inference rules can be found in Table 4.2.

4.2.1.2 Generation of Rule Combination

For introducing multi-step logical reasoning, we combine different inference rules that build upon each other, leading to a final conclusion. To achieve this, we chose combinations of rules that have a common premise or conclusion. In the combinations we choose, a rule’s conclusion will be part of the premise of another rule. An illustration

Rule	Extended First-order Logic with Multi-variable
1	$\forall x \forall y ((p(x) \wedge q(x)) \rightarrow r(x, y)) \wedge \exists u \exists v (p(u) \wedge \neg r(u, v)) \vdash \exists y \neg q(y)$
2	$\forall x \forall y ((p(x) \wedge q(x)) \rightarrow \neg s(x, y)) \wedge \forall z (r(z) \rightarrow p(z)) \wedge r(a) \wedge s(a, b) \vdash \neg q(b)$
3	$\forall x \exists y ((p(x) \rightarrow q(x, y)) \wedge \forall u \forall v ((q(u, v) \wedge r(u, v)) \rightarrow s(v)) \wedge \exists z \exists k (p(z) \wedge r(z, k)) \vdash \exists w s(w)$
4	$\forall x \forall y \forall z (p(x, y, z) \rightarrow (q(x, z) \vee r(y))) \wedge \exists u \exists v \exists w (p(u, v, w) \wedge \neg q(u, w)) \vdash \exists s r(s)$
5	$\forall x ((p(x) \rightarrow \exists y r(y, x)) \wedge p(a) \vdash \exists z r(z, a)$
6	$\forall x \forall y (p(x, y) \vee q(x, y)) \wedge \exists u \exists v \neg q(u, v) \vdash \exists z \exists w p(z, w)$
7	$\forall x \forall y (p(x, y) \rightarrow (q(x) \wedge r(y)) \wedge p(a, b) \vdash q(a) \wedge r(b))$

Table 4.2: FOL inference rules that establish the relationship between multiple variables

of this is shown in the figure 4.2.

To ensure a comprehensive approach to answering a question, we employ a method that involves leveraging contextual information and explicit details provided in the question. This process requires a logical chain of reasoning, combining knowledge from the given context with the information presented in the question. Each step in the reasoning chain corresponds to an inference rule, with combinations ensuring each step aligns with a single rule. To gener-

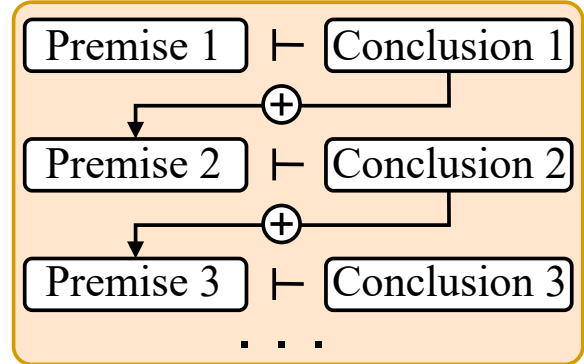


Figure 4.2: Process for combining multiple logical inference rules for PL and FOL: *Premise 1* is the set of premises for the first inference rule, leading to *Conclusion 1*. *Conclusion 1* and *Premise 2* derive *Conclusion 2*, and so on. $\vdash$: Entails.

ate the combinations, we start with the initial rule and assess whether the conclusion of this rule aligns with the premise of other rules. This iterative process results in

Rule Combinations	Premises in Story	Premise in Question	Answer
DS: $(P \vee Q) \wedge \neg P \vdash Q$ MP: $(Q \rightarrow R) \wedge Q \vdash R$	$(P \vee Q), (Q \rightarrow R)$	$\neg P$	R: ✓
MT: $(P \rightarrow Q) \wedge \neg Q \vdash \neg P$ DS: $(P \vee R) \wedge \neg P \vdash R$	$(P \rightarrow Q), (P \vee R)$	$\neg Q$	R: ✓
HS: $(P \rightarrow Q) \wedge (Q \rightarrow R) \vdash (P \rightarrow R)$ MP: $(P \rightarrow R) \wedge P \vdash R$	$(P \rightarrow Q), (Q \rightarrow R)$	P	R: ✓
CD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee R) \vdash (Q \vee S)$ DS: $(Q \vee S) \wedge \neg Q \vdash S$	$(P \rightarrow Q),$ $(R \rightarrow S), (P \vee R)$	$\neg Q$	S: ✓
DD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (\neg Q \vee \neg S) \vdash (\neg P \vee \neg R)$ DS: $(\neg P \vee \neg R) \wedge P \vdash \neg R$	$(P \rightarrow Q),$ $(R \rightarrow S), (\neg Q \vee \neg S)$	P	R: ✗
BD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee \neg S) \vdash (Q \vee \neg R)$ DS: $(Q \vee \neg R) \wedge \neg Q \vdash \neg R$	$(P \rightarrow Q),$ $(R \rightarrow S), (P \vee \neg S)$	$\neg Q$	R: ✗
HS: $(P \rightarrow Q) \wedge (Q \rightarrow R) \vdash (P \rightarrow R)$ MT: $(P \rightarrow R) \wedge \neg R \vdash \neg P$	$(P \rightarrow Q), (Q \rightarrow R)$	$\neg R$	P: ✗

Table 4.3: 2-step reasoning rule combinations for PL and FOL.

multi-step combinations/reasoning, with the conclusion of each step serving as a part of the premise for the subsequent rule.

We create 71 rule combinations, ranging from 2-step to 5-step reasoning chains. We use each single inference rule as depth-1. Let's consider a specific combination involving the *Modus Tollens* $((p \rightarrow q) \wedge \neg q) \vdash \neg p$ and *Disjunctive Syllogism* $((p \vee r) \wedge \neg p) \vdash r$ rules for creating a combination for depth-2. Given the context, including natural language statements for $(p \rightarrow q)$ and $(p \vee r)$ and information in the question as $\neg q$, we ask about the truth value of r . Applying *Modus Tollens*, we deduce $\neg p$ from the $(p \rightarrow q)$ from context and $\neg q$ in question, giving the first step. Subsequently, using $\neg p$ as the premise for *Disjunctive Syllogism*, we conclude that r is indeed true based on the $(p \vee r)$ and $\neg p$, giving the second step. Creating rule combinations at higher depths, and validating the quality of generated instances is challenging, hence,

we limit the number of rule combinations at d_5 for the scope of this study.

We use the same rule combinations for FOL for each depth. All rule combinations for 2-step, 3-step, 4-step, and 5-step reasoning for PL and FOL are presented in Tables 4.3, 4.4, 4.5, and 4.6, respectively. For each combination, we provide the inference rules to be used for reasoning, the premises present in the context and in the question, and the complex reasoning question-answer pair.

4.2.1.3 Generation of Data Instances

To generate the data, we prompt the Claude-2 model with instructions tailored to each rule combination. We used a few-shot prompting technique to generate the data. Our data creation prompt structure, as illustrated in Figure 4.3, is for both PL and FOL. The prompt schema, shown in Figure 4.3, comprise five crucial components:

Rule Definition We provide generalized rules for various combinations containing propositions represented by labels such as P and Q. For instance, Rule 1: “If P is true, then Q is true.” Utilizing these defined rules, we construct the contextual premise by combining them. Subsequently, we formulate a question that requires a step-by-step deduction using all the established rules to derive the answer.

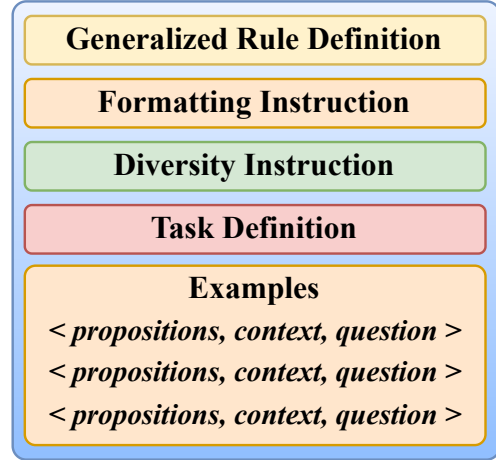


Figure 4.3: Data generation prompt for PL and FOL

Format We provide model-specific instructions for generating outputs in a designated format, simplifying the process of parsing it on a large scale.

Rule Combinations	Premises in Story	Premise in Question	Answer
HS: $(P \rightarrow Q) \wedge (Q \rightarrow R) \vdash (P \rightarrow R)$ MP: $(P \rightarrow R) \wedge P \vdash R$ MP: $(R \rightarrow S) \wedge R \vdash S$	$(P \rightarrow Q),$ $(Q \rightarrow R), (R \rightarrow S)$	P	S: ✓
CD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee R) \vdash (Q \vee S)$ DS: $(Q \vee S) \wedge \neg Q \vdash S$ MP: $(S \rightarrow T) \wedge S \vdash T$	$(P \rightarrow Q), (R \rightarrow S),$ $(P \vee R), (S \rightarrow T)$	$\neg Q$	T: ✓
BD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee \neg S) \vdash (Q \vee \neg R)$ CT: $(Q \vee \neg R) \dashv\vdash (\neg R \vee Q)$ DS: $(\neg R \vee Q) \wedge R \vdash Q$	$(P \rightarrow Q),$ $(R \rightarrow S), (P \vee \neg S)$	R	Q: ✓
BD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee \neg S) \vdash (Q \vee \neg R)$ DS: $(Q \vee \neg R) \wedge \neg Q \vdash \neg R$ MT: $(T \rightarrow R) \wedge \neg R \vdash \neg T$	$(P \rightarrow Q), (R \rightarrow S),$ $(P \vee \neg S), (T \rightarrow R)$	$\neg Q$	T: ✗
CD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee R) \vdash (Q \vee S)$ CT: $(Q \vee S) \dashv\vdash (S \vee Q)$ DS: $(S \vee Q) \wedge \neg S \vdash Q$	$(P \rightarrow Q),$ $(R \rightarrow S), (P \vee R)$	$\neg S$	Q: ✓
HS: $(P \rightarrow Q) \wedge (Q \rightarrow R) \vdash (P \rightarrow R)$ CD: $(P \rightarrow R) \wedge (S \rightarrow T) \wedge (P \vee S) \vdash (R \vee T)$ DS: $(R \vee T) \wedge \neg R \vdash T$	$(P \rightarrow Q), (Q \rightarrow R),$ $(S \rightarrow T), (P \vee S)$	$\neg R$	T: ✓
HS: $(P \rightarrow Q) \wedge (Q \rightarrow R) \vdash (P \rightarrow R)$ MT: $(P \rightarrow R) \wedge \neg R \vdash \neg P$ DS: $(P \vee S) \wedge \neg P \vdash S$	$(P \rightarrow Q),$ $(Q \rightarrow R), (P \vee S)$	$\neg R$	S: ✓
DD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (\neg Q \vee \neg S) \vdash (\neg P \vee \neg R)$ DS: $(\neg P \vee \neg R) \wedge P \vdash \neg R$ MT: $(T \rightarrow R) \wedge \neg R \vdash \neg T$	$(P \rightarrow Q), (R \rightarrow S),$ $(\neg Q \vee \neg S), (T \rightarrow R)$	P	T: ✗
DMT: $(\neg Q \vee \neg R) \dashv\vdash \neg(Q \wedge R)$ CO: $(P \rightarrow Q) \wedge (P \rightarrow R) \vdash P \rightarrow (Q \wedge R)$ MT: $(P \rightarrow (Q \wedge R)) \wedge \neg(Q \wedge R) \vdash \neg P$	$(P \rightarrow Q), (P \rightarrow R)$	$\neg Q \vee \neg R$	P: ✗

Table 4.4: 3-step reasoning rule combinations for PL and FOL.

Rule Combinations	Premises in Story	Premise in Question	Answer
CD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee R) \vdash (Q \vee S)$	$(P \rightarrow Q),$ $(R \rightarrow S), (P \vee R),$ $(S \rightarrow T), (T \rightarrow U)$	$\neg Q$	U: ✓
DS: $(Q \vee S) \wedge \neg Q \vdash S$			
MP: $(S \rightarrow T) \wedge S \vdash T$			
MP: $(T \rightarrow U) \wedge T \vdash U$			
BD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee \neg S) \vdash (Q \vee \neg R)$	$(P \rightarrow Q), (R \rightarrow S),$ $(P \vee \neg S), (Q \rightarrow T)$	R	T: ✓
CT: $(Q \vee \neg R) \dashv\vdash (\neg R \vee Q)$			
DS: $(\neg R \vee Q) \wedge R \vdash Q$			
MP: $(Q \rightarrow T) \wedge Q \vdash T$			
BD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee \neg S) \vdash (Q \vee \neg R)$	$(P \rightarrow Q),$ $(R \rightarrow S), (P \vee \neg S),$ $(T \rightarrow R), (T \vee U)$	$\neg Q$	U: ✓
DS: $(Q \vee \neg R) \wedge \neg Q \vdash \neg R$			
MT: $(T \rightarrow R) \wedge \neg R \vdash \neg T$			
DS: $(T \vee U) \wedge \neg T \vdash U$			
HS: $(P \rightarrow Q) \wedge (Q \rightarrow R) \vdash (P \rightarrow R)$	$(P \rightarrow Q),$ $(Q \rightarrow R), (S \rightarrow T),$ $(P \vee S), (T \rightarrow U)$	$\neg R$	U: ✓
CD: $(P \rightarrow R) \wedge (S \rightarrow T) \wedge (P \vee S) \vdash (R \vee T)$			
DS: $(R \vee T) \wedge \neg R \vdash T$			
MP: $(T \rightarrow U) \wedge T \vdash U$			
CD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee R) \vdash (Q \vee S)$	$(P \rightarrow Q), (R \rightarrow S),$ $(P \vee R), (Q \rightarrow T)$	$\neg S$	T: ✓
CT: $(Q \vee S) \dashv\vdash (S \vee Q)$			
DS: $(S \vee Q) \wedge \neg S \vdash Q$			
MP: $(Q \rightarrow T) \wedge Q \vdash T$			
HS: $(P \rightarrow Q) \wedge (Q \rightarrow R) \vdash (P \rightarrow R)$	$(P \rightarrow Q), (Q \rightarrow R),$ $(P \vee S), (S \rightarrow T)$	$\neg R$	T: ✓
MT: $(P \rightarrow R) \wedge \neg R \vdash \neg P$			
DS: $(P \vee S) \wedge \neg P \vdash S$			
MP: $(S \rightarrow T) \wedge S \vdash T$			
BD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee \neg S) \vdash (Q \vee \neg R)$	$(P \rightarrow Q),$ $(R \rightarrow S), (P \vee \neg S),$ $(T \rightarrow R), (U \rightarrow T)$	$\neg Q$	U: ✗
DS: $(Q \vee \neg R) \wedge \neg Q \vdash \neg R$			
MT: $(T \rightarrow R) \wedge \neg R \vdash \neg T$			
MT: $(U \rightarrow T) \wedge \neg T \vdash \neg U$			
IM: $(P \rightarrow (Q \wedge R)) \vdash (P \wedge Q) \rightarrow R$	$(P \rightarrow (Q \wedge R))$	Q, $\neg R$	P: ✗
MT: $((P \wedge Q) \rightarrow R) \wedge \neg R \vdash \neg(P \wedge Q)$			
DMT: $\neg(P \wedge Q) \vdash (\neg P \vee \neg Q)$			
DS: $(\neg P \vee \neg Q) \wedge Q \vdash \neg P$			

Table 4.5: 4-step reasoning rule combinations for PL and FOL.

Rule Combinations	Premises in Story	Premise in Question	Answer
HS: $(P \rightarrow Q) \wedge (Q \rightarrow R) \vdash (P \rightarrow R)$			
MT: $(P \rightarrow R) \wedge \neg R \vdash \neg P$	$(P \rightarrow Q),$		
DS: $(P \vee S) \wedge \neg P \vdash S$	$(Q \rightarrow R), (P \vee S),$	$\neg R$	U: ✓
MP: $(S \rightarrow T) \wedge S \vdash T$	$(S \rightarrow T), (T \rightarrow U)$		
MP: $(T \rightarrow U) \wedge T \vdash U$			
BD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee \neg S) \vdash (Q \vee \neg R)$			
CT: $(Q \vee \neg R) \dashv\vdash (\neg R \vee Q)$	$(P \rightarrow Q),$		
DS: $(\neg R \vee Q) \wedge R \vdash Q$	$(R \rightarrow S), (P \vee \neg S),$	R	U: ✓
MP: $(Q \rightarrow T) \wedge Q \vdash T$	$(Q \rightarrow T), (T \rightarrow U)$		
MP: $(T \rightarrow U) \wedge T \vdash U$			
CD: $(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee R) \vdash (Q \vee S)$			
CT: $(Q \vee S) \dashv\vdash (S \vee Q)$	$(P \rightarrow Q),$		
DS: $(S \vee Q) \wedge \neg S \vdash Q$	$(R \rightarrow S), (P \vee R),$	$\neg S$	U: ✓
MP: $(Q \rightarrow T) \wedge Q \vdash T$	$(Q \rightarrow T), (T \rightarrow U)$		
MP: $(T \rightarrow U) \wedge T \vdash U$			

Table 4.6: 5-step reasoning rule combinations for PL and FOL.

Diversity To enhance diversity, we prompt the model to generate multiple instances across various domains, such as education and finance.

Task Definitions We provide definitions to perform two tasks. First, to generate the context that serves as a human-like illustration of generalized rules. This task instructs the generation of a real-life story with sentences exemplifying the specified rules, where entity labels such as P, Q, R, S, T , and U are replaced with actual entities. To ensure clarity, entity labels are excluded from the context. Additionally, the context generation for FOL incorporates instructions specifying the use of generalized sentences with indefinite pronouns for quantification. The second task focuses on question generation, which entails formulating questions in the format: "[(...) is true/not true, then is (...) true?]" This dual-task approach ensures the generation of $\langle context, question \rangle$ pair.

Examples We present five in-context exemplars for every rule combination. Each instance comprises propositions such as P, Q, R , a contextual narrative, and an associated question.

Figure 4.4 illustrates an example prompt for the combination of rules from propositional logic, namely ‘constructive dilemma’ (CD), ‘disjunctive syllogism’ (DS), and ‘modus ponens’ (MP). CD is represented as $(p \rightarrow q) \wedge (r \rightarrow s) \wedge (p \vee r) \vdash (q \vee s)$, which can be understood in natural language as “If p implies q , and if r implies s , and either p or r or both are true, then we can conclude that either q or s or both are true.” DS is formally represented as $(p \vee q) \wedge \neg p \vdash q$, which can be understood in natural language as “If p or q are true, and we know $\neg p$, then we can conclude q .” MP is formally represented as $(p \rightarrow q) \wedge p \vdash q$, which can be understood in natural language

as “If p implies q , and we know p , then we can conclude q .” In this prompt, the

Generalized Rule Definition:

Rule 1: [if {P} is true then {Q} is true, and if {R} is true then {S} is true, and either {P} or {R} or both are true]
 Rule 2: [if {S} is true, then {T} is true]

Formatting Instruction:

Complete the following tasks, only returning text in exactly the format given in the following examples.

Diversity Instruction:

Generate 5 more examples from multiple domains

Task Definition:

Task 1: Generate a short real life story that includes sentences to illustrate the above rules, replacing the entities P, Q, R, S, T with real values. Do not include the entity labels like P, Q, R, S, T in the story.
 Task 2: Generate the following complex reasoning question using the story and the rules, by replacing the respective entities.
 Q1: [If Q is not true, then is T true?]

Examples:

{P}: Mark takes Ryan to the park.
 {Q}: Ryan gets to play on the swings.
 {R}: Jackie makes cupcakes.
 {S}: Jackie brings some cupcakes for Ryan.
 {T}: Ryan gets a sugar rush.
Context: Mark promised to take his son Ryan to the park on Saturday. If Mark takes Ryan to the park, then Ryan will get to play on the swings. Jackie said she would make cupcakes on Saturday. If Jackie makes cupcakes, then she will bring some for Ryan. Ryan was certain that either Mark would take him to the park, or Jackie would make cupcakes, or both might be true. If Jackie ends up bringing cupcakes for Ryan, then Ryan will get a sugar rush.
Question: If Ryan did not get to play on the swings, then did Ryan get a sugar rush?

{P}: Jeff meditates regularly.
 {Q}: Jeff improves his mental health.
 {R}: Jeff eats healthy meals.
 {S}: Jeff loses weight.
 {T}: Jeff feels more confident about himself.
Context: Jeff wants to improve his health and fitness. If Jeff meditates regularly, he will improve his overall mental health. Also, if Jeff eats healthy nutritious meals, he is likely to lose weight. Jeff decides to either meditate regularly, or eat healthy meals, or do both simultaneously. He also knew that if he loses weight, then he will feel more confident about himself.
Question: If Jeff did not improve his mental health, did he feel more confident about himself?

{P}: Sam studies geography.
 {Q}: Sam aces his geography test.
 {R}: Sam travels the world.
 {S}: Sam gains cultural awareness.
 {T}: Sam becomes more open-minded.
Context: Sam wants to improve his geography knowledge. If he studies geography diligently, he will ace his geography test. Also, if Sam travels the world, he will gain cultural awareness from experiencing new places. Sam decides that he will either study geography, or travel the world, or do both. If Sam gains cultural awareness, he will become more open-minded.
Question: If Sam did not ace his geography test, did he become more open-minded?

Figure 4.4: An example prompt for the 3-step combination of inference rules CD, DS, and MP from propositional logic.

generalized rule definitions provide a description of the premises given in the story in natural language. The prompt includes instructions on how the generated samples should be formatted, instructions to generate samples from diverse domains, and detailed definitions for generating propositions, and then using them to generate a context and question for each sample. To enhance the quality of samples in terms of relevance and coherence, the prompt includes an examples section that demonstrates these tasks. In Figure 3.4, we present three examples with their respective propositions, contexts, and questions.

NL Examples for PL and FOL In this section, we illustrate multi-step reasoning for PL and FOL using Natural Language (NL) examples for depths 2 through 5. Table 4.7 provides examples in natural language for PL. We provide one example of rule combinations for each depth. For each example, we provide the inference rules and propositions, as well as the respective context and complex reasoning question. Table 4.8 provides examples in natural language for FOL, with one combination for each depth. Similar to PL, we provide the inference rules, predicates, and the context-question pair for each example.

4.2.2 *Non-Monotonic Reasoning*

We utilize eight NM reasoning patterns defined in Lifschitz [120] (also presented in Table 4.9), and have generated data for depths 1 to 5. To increase reasoning depth, we integrated NM with classical logic, using only one NM rule per depth due to the 4-5 assumptions each pattern involves. Thus, combining two NM patterns with classical logic creates lengthy contexts, challenging for LLMs to generate quality instances. Our rule combinations avoid overly long contexts while requiring reasoning up to depth-5.

Depth	Rules and Propositions	Context and Question
2	Rules: CD, DS Propositions: P: There is a big snowstorm coming. Q: Schools will be closed. R: Boss tells us to work from home. S: I avoid driving in the snow	Context: If there is a big snowstorm coming, schools will be closed tomorrow. Also, if my boss tells us to work from home, I can avoid driving in the snow. It seems either there will be a snowstorm or I'll be told to work from home, maybe both. Question: If schools were not closed tomorrow, then did I avoid driving in the snow?
3	Rules: BD, DS, MT Propositions: P: The weather is nice. Q: She goes for a walk. R: Finishes chores. S: Has free time. T: It's the weekend	Context: It was a beautiful sunny day. Amy knew that if the weather is nice, she goes for a walk. Amy also had chores to complete today. If Amy finishes her chores, then she has free time. Amy is certain that either the weather is nice, or she doesn't have free time, or the weather is nice and she doesn't have free time. She also knows that if it's the weekend, then she finishes her chores. Question: If Amy didn't go for a walk, then is it the weekend?
4	Rules: HS, CD, DS, MP Propositions: P: Studied hard for the exam. Q: Feel confident. R: Score well. S: Cooked nice dinner. T: Feel relaxed. U: Sleep soundly.	Context: Jim had a big exam coming up that he needed to prepare for. If Jim studied hard for the exam, he would feel confident going into it. If Jim felt confident about the exam, he would end up scoring well on it. His wife Lucy enjoyed cooking nice dinners. If Lucy cooked a nice dinner, she felt relaxed afterwards. Last night, either Jim studied hard, or Lucy cooked a nice dinner, or they both did those things. Jim knew that if Lucy felt relaxed after dinner, she always slept soundly through the night. Question: If Jim did not score well on the exam, did Lucy sleep soundly?
5	Rules: HS, MT, DS, MP, MP Propositions: P: Train consistently. Q: Increase endurance and stamina. R: complete the 26.2 mile marathon. S: Ate nutritious food. T: More steady energy. U: Train harder staying injury free.	Context: Jessica set a goal to run a marathon. She learned that if she trained consistently, she could increase her endurance and stamina. Jessica knew that if her endurance improved, she could complete the 26.2 mile marathon. To complement her training, Jessica made sure she either trained regularly, or ate nutritious foods, or did both. Eating nutritious foods gave Jessica more steady energy for her workouts. With this extra energy, Jessica found she could train harder while staying injury-free on her road to marathon success. Question: If Jessica does not complete the marathon, then does she stay injury-free during training?

Table 4.7: Natural language examples of rule combinations of each depth for PL.

Depth	Rules and Predicates	Context and Question
2	Rules: CD, DS Predicates: P: Compose original music. Q: Work would be Unique. R: Promote music online. S: Gain following.	Context: An aspiring musician decided to try writing their own songs. They realized that if they composed original music, their work would be unique; if they promoted their music online, they would gain a following. The musician could write original songs or promote their music online. Question: Given that Maria's music was not unique, is it true that she gained a following online?
3	Rules: BD, DS, MT Predicates: P: It's Monday. Q: There is a staff meeting. R: Finish report. S: Submit the report. T: Good Employee.	Context: It was a busy morning at the office. If it was Monday, then there would be a staff meeting. If they finished the report, then they could submit it to their manager. They were certain that either it was Monday, or they did not submit the report. It is known at the office that if someone is a good employee, they finish their reports on time. Question: Sam did not have a staff meeting, is Sam a good employee?
4	Rules: BD, DS, MT, DS Predicates: P: First day of school. Q: students feel nervous and excited. R: Study Hard. S: get good grades. T: teacher is very strict. U: class textbook is very long.	Context: If it is the first day of school, then students feel nervous and excited. If someone studies hard, then they get good grades. Either it is the first day, or they do not get good grades, or it is the first day and they do not get good grades. If a teacher is very strict, then students have to study hard for that class. Either the teacher is very strict, or the class textbook is very long, or perhaps both are true. Question: Emma was not nervous on the first day, does this mean did she have a very long textbook in one of her classes?
5	Rules: HS, MT, DS, MP, MP Predicates: P: Practice drawing techniques. Q: improve artistic skills. R: sell their artwork. S: studies art history and famous artists. T: gain inspiration. U: develop creative style.	Context: Someone wanted to become an artist. They learned that if they practiced drawing techniques consistently, they would improve their artistic skills. With improved artistic skills, they could sell their artworks. Either someone practices drawing techniques consistently, or someone studies art history and famous artists, or they do both. If someone studies art history and famous artists, then they gain inspiration for their own art. If they gain inspiration, then they can develop their own creative style. Question: If Emma cannot sell her artworks yet, then has she developed her own creative style?

Table 4.8: Natural language examples of rule combinations of each depth for FOL.

Basic Default Reasoning	Default Reasoning with Irrelevant Information
Context: Blocks A and B are heavy. Heavy blocks are typically located on the table. A is not on the table. Conclusion: B is on the table.	Context: Blocks A and B are heavy. Heavy blocks are typically located on the table. A is not on the table. B is red. Conclusion: B is on the table.
Default Reasoning with a Disabled Default	Default Reasoning in an Open Domain
Context: Block A and B are heavy Heavy blocks are normally located on the table. A is possibly an exception to this rule. Conclusion: B is on the table.	Context: Block A is heavy. Heavy blocks are normally located on the table. A is not on the table. Conclusion: All heavy blocks other than A are on the table.
Reasoning about Unknown Expectations I	Reasoning about Unknown Expectations II
Context: Blocks A, B, and C are heavy. Heavy blocks are normally located on the table. At least one of A, B, is not on the table. Conclusion: C is on the table. Exactly one of A, B is not on the table.	Context: Heavy blocks are normally located on the table. At least one heavy block is not on the table. Conclusion: Exactly one heavy block is not on the table.
Reasoning about Unknown Expectations III	Reasoning about Priorities
Context: Blocks A is heavy. Heavy blocks are normally located on the table. At least one heavy block is not on the table. Conclusion: A is on the table.	Context: Jack asserts that block A is on the table. Mary asserts that block A is not on the table. When people assert something, they are normally right. Conclusion: If Mary's evidence is more reliable than Jack's. then block A is not on the table

Table 4.9: Illustrative examples of non-monotonic reasoning adapted from [120].

Generation of Rule Combination We consider reasoning patterns corresponding to default reasoning for depth-1. We generalize the rule to generate simple sentence pairs independently before combining them according to the template-based NM rule. After generating sentence pairs, we combined the sentences based on the defined rule and formulated the question-answer pair accordingly. We have manually generated 12, 2, 2, and 1 rule combinations for depth-2, depth-3, depth-4, and depth-5, provided in Tables 4.10, 4.11, 4.12, and 4.13. While formulating depthwise rule combinations, a logical relationship between the context and question is followed. The rule combinations for all depths from 2 to 5 include 6 reasoning rules from NM—BDR, PBD, DRO, PBD, REII, and REIII—and 3 inference rules from PL—MP, MT, and DS. The data for depths 2 to 5 is generated by forming a logical connection between two NM rules’ conclusions and the PL rules.

Generation of Data Instances In creating prompts for data generation, we use a four-part structure: (1) define the task, (2) explain each rule as an assumption and conclusion, (3) provide instructions for creating context and questions to ensure logical connections, and (4) establish formatting guidelines for systematic output.

Example Prompts In creating prompts for data generation, we adopt a four-part structure. First, we define the task. Second, each rule is explained in the prompt, representing a broad assumption and conclusion. Third, we give clear instructions for creating context and questions. These instructions guide the model’s adherence to specific rules, ensuring a logical connection. Finally, we establish formatting guidelines, ensuring a systematic model output. Boxes show the prompts that we used to generate data instances, respectively for depths 2, 3, 4, and 5.

Rule	Examples
BDR_MP Conclusion of BDR: X MP: $(X \rightarrow Y) \wedge X \vdash Y$	Context: Jim and Pam work at the same office. Normally, employees at that office get free lunch. Jim does not get free lunch. If Pam gets free lunch, then she gets an hour lunch break. Question: Can we conclude Pam gets an hour lunch break? (Yes)
BDR_MT Conclusion of BDR: X MT: $(X \rightarrow Y) \wedge \neg Y \vdash \neg X$	Context: Emma and Jacob are students in the same class. Usually students in that class submit homework assignments. Emma did not submit the last homework. If Jacob missed over 3 classes, that means he likely did not submit the homework. Question: Can we conclude Jacob missed over 3 classes? (No)
DRD_MP Conclusion of DRD: X MP: $(X \rightarrow Y) \wedge X \vdash Y$	Context: The Honda and Toyota are sedans. Sedans normally have four doors. The Honda might not have four doors even though it's a sedan. If the Toyota has a four doors then it has four windows. Question: Can we conclude the Toyota likely has four windows? (Yes)
DRD_MT Conclusion of DRD: X MT: $(X \rightarrow Y) \wedge \neg Y \vdash \neg X$	Context: Oaks and pines are types of trees. Typically trees grow from seeds. Oaks may not grow from seeds even though they are trees. If a pine is artificial, then it does not grow from a seed. Question: Can we conclude the pine is artificial? (No)
DRI_MP Conclusion of DRI: X MP: $(X \rightarrow Y) \wedge X \vdash Y$	Context: John and Mary are students in the same class. Usually students in their class do homework every day. John did not do his homework yesterday. Mary studied extra material last night. If Mary did her usual homework, she would have also reviewed her notes. Question: Can we conclude that Mary reviewed her notes last night? (Yes)
DRI_MT Conclusion of DRI: X MT: $(X \rightarrow Y) \wedge \neg Y \vdash \neg X$	Context: Sara and David ordered dessert at a restaurant. Usually, people who order dessert also order coffee. Sara did not order coffee. David requested extra whipped cream. If a customer asks for extra toppings, it means they did not order coffee. Question: Can we conclude David asked for extra toppings? (No)
PBD_MP Conclusion of PBD: X MP: $(X \rightarrow Y) \wedge X \vdash Y$	Context: Jenny said the dog dug up the flower bed. Her brother said the dog did not dig up the flower bed. People usually tell the truth. Jenny is more trustworthy than her brother. If the dog dug up the flowers, it likely made a mess. Question: Can we conclude the dog made a mess? (Yes)
PBD_MT Conclusion of PBD: X MT: $(X \rightarrow Y) \wedge \neg Y \vdash \neg X$	Context: John said the shirt was blue. Mary said the shirt was not blue. Normally people are correct when they make assertions. John had a closer look at the shirt than Mary. If the shirt was purple, it could not be blue. Question: Can we conclude the shirt was purple? (No)
REI_MP Conclusion of REI: X MP: $(X \rightarrow Y) \wedge X \vdash Y$	Context: Ben, Mark, and Jacob took a history test. Students who study many hours usually pass history tests. Ben and Mark did not study many hours. If Jacob passed the history test, he must have paid attention in class. Question: Can we conclude Jacob paid attention in class? (Yes)
REI_MT Conclusion of REI: X MT: $(X \rightarrow Y) \wedge \neg Y \vdash \neg X$	Context: John, Peter and Kate are students in math class. Students in math class normally do homework. John and Peter did not do their math homework. If Kate missed class then she did not do her math homework. Question: Can we conclude Kate missed class? (No)
REII_MP Conclusion of REII: X MP: $(X \rightarrow Y) \wedge X \vdash Y$	Context: John bought a new phone. New phones usually come with a warranty. However, some new phones do not come with a warranty. If a phone has a warranty, then it has customer support. Question: Can we conclude John's new phone has customer support? (Yes)
REII_MT Conclusion of REII: X MT: $(X \rightarrow Y) \wedge \neg Y \vdash \neg X$	Context: Kate booked a room at hotel Y. Rooms at hotel Y are usually clean. There is at least one room at hotel Y that is not clean. If Kate's room has mold, then it is probably not clean. Question: Can we conclude Kate's room has mold? (No)

Table 4.10: Natural language examples of rule combinations of depth-2 for NM.

Rule	Examples
Rule: d3_1 Assumptions: 1: A and B are objects of type T and have property S. 2: Normally objects of type T with property S have property U. 3: if A has property U implies C has property D 4: if C has property D implies E has property F Question 1: Can we conclude if E does not have F then B has U? (YES) Question 2: Can we conclude if E does not have F then B does not have U? (NO)	Context: Smartphone A and Smartphone B both have GPS technology. Normally, smartphones with GPS technology also have internet connectivity. If smartphone A has internet connectivity, then Mike can access online maps. If Mike can access online maps, then Emily can get driving directions from Mike. Question 1: Can we conclude if Emily can not get driving directions from Mike, then smartphone B has internet connectivity? (Yes) Question 2: Can we conclude if Emily can not get driving directions from Mike, then smartphone B does not have internet connectivity? (No)
Rule: d3_2 Assumptions: 1: A and B are objects of type T and have property S. 2: Normally objects of type T with property S have property U. 3: if C has property G implies C has property D 4: if A has property U implies E has property F 5: either C has property G or E does not have property F or both Question 1: Can we conclude if C does not have D then B has U? (YES) Question 2: Can we conclude if C does not have D then B does not have U? (NO)	Context: Car A and car B are electric vehicles. Normally, electric vehicles (cars) have fast-charging capabilities. If car C is a hybrid, then car C has good fuel efficiency. If car A has a fast-charging capability, then it implies that the environment is very eco-friendly. Either car C is a hybrid or the environment is not very eco-friendly, or both. Question 1: Can we conclude if car C is not a good fuel efficient then Car B has a fast-charging capability? (Yes) Question 2: Can we conclude if car C is not a good fuel efficient then Car B does not have a fast-charging capability? (No)

Table 4.11: Natural language examples of rule combinations of depth-3 for NM.

Rule	Examples
Rule: d4_1 Assumptions: 1: A and B are objects of type T and have property S. 2: Normally objects of type T with property S have property U. 3: if C has property G implies C has property D 4: if E has property L implies E has property F 5: either C has property G or E has property F or both 6: if A has property U then E has property L Question 1: Can we conclude if C does not have D then B has U? (YES) Question 2: Can we conclude if C does not have D then B does not have U? (NO)	Context: Apple tree and Orange tree are fruit trees. Normally, fruit trees produce edible fruit. If Garden is regularly watered, then its plants are flourishing. If Orchard receives enough sunlight, then it yields high-quality fruit. Either Garden has regular watering or Orchard yields high-quality fruit or both. If the apple tree produces edible fruit, then Orchard receives enough sunlight. Question 1: Can we conclude if Garden does not have flourishing plants then the orange tree produces edible fruit? (Yes) Question 2: Can we conclude if Garden does not have flourishing plants then the orange tree does not produce edible fruit? (No)
Rule: d4_2 Assumptions: 1: A and B are objects of type T and have property S. 2: Normally objects of type T with property S have property U. 3: if C has property G implies C has property D 4: if E has property L implies E has property F 5: either C does not have property D or E does not have property F or both 6: if A has property U then E has property L Question 1: Can we conclude if C has property G then B has U? (YES) Question 2: Can we conclude if C does not have G then B does not have U? (NO)	Context: Assume A and B are plants of species T and they both produce flowers. Normally, flowering plants of species T also bear fruit. If an animal C is a bird, then it can fly. If an environment has a lot of sunlight, then it supports plant growth. Either the bird cannot fly or the environment does not support plant growth or both. If plant A bears fruit, then the environment has a lot of sunlight. Question 1: Can we conclude if the bird is capable of flying then plant B bears fruit? (Yes) Question 2: Can we conclude if the bird can fly then plant B does not bear fruit? (No)

Table 4.12: Natural language examples of rule combinations of depth-4 for NM.

Rule	Examples
Rule: d5_1 Assumptions: 1: A and B are objects of type T and have property S. 2: Normally objects of type T with property S have property U. 3: if C has property G implies C has property D 4: if E has property L implies E has property F 5: either C has property G or E has property F or both 6: if I has property H then E has property L 7: if A has property U then I has property H Question 1: Can we conclude if C does not have D then B has U? (YES) Question 2: Can we conclude if C does not have D then B does not have U? (NO)	Context: Rose and Lily are plants that flower. Normally, plants that flower also produce seeds. If a plant is a Cactus, and it has thorns, then it can survive in the desert. If a plant is an Orchid, and it has broad leaves, then it can grow in tropical areas. Either a Cactus has thorns, or an Orchid can grow in tropical areas, or both. If a Lotus has flowers, then an Orchid has broad leaves. If a Rose produces seeds, then a Lotus has flowers. Question 1: Can we conclude if a Cactus cannot survive in the desert then a Lily produces seeds? (YES) Question 2: Can we conclude if a Cactus cannot survive in the desert then a Lily does not produce seeds? (NO)

Table 4.13: Natural language examples of rule combinations of depth-5 for NM.

Prompt used to generate data instance for depth-2 using NM-BDR and PL-MP

Rule:

Assumptions:

- 1: A and B are objects of type T and have property P.
- 2: Normally objects of type T with property P have property Q.
- 3: A does not have property Q.
- 4: If B has property Q then it implies B has property C.

Question: Can we conclude B has property C?

Task 1:

Generate a short generic story that should only contain the natural language sentences for assumptions 1, 2, 3, and 4 using propositions to replace the labels A, B and so on. The story should not include labels like p or q and so on.

Task 2:

Generate the question by replacing them with the entities with respective propositions.

Prompt used to generate a data instance for depth-3 for NM

Rule: d3_1

Assumptions:

- 1: A and B are objects of type T and have property S.
- 2: Normally objects of type T with property S have property U.
- 3: if A has property U implies C has property D
- 4: if C has property D implies E has property F

Question 1:

Can we conclude if E does not have F then B has U? (YES)

Question 2:

Can we conclude if E does not have F then B does not have U? (NO)

Task 1: Generate a short context paragraph by replacing all the entity labels A, B, and so on in the above context with propositions and real entities. The generated context should have natural language sentences for all the sentences 1-4. It should not include label representations like A or B, and should not

mention the word "property".

Task 2: Generate questions 1 and 2 by replacing the respective labels from the generated context.

Example 1:

Assumptions:

Smartphone A and Smartphone B both have GPS technology.

Normally, smartphones with GPS technology also have internet connectivity.

If smartphone A has internet connectivity, then Mike can access online maps.

If Mike can access online maps, then Emily can get driving directions from Mike.

Question 1:

Can we conclude if Emily can not get driving directions from Mike, then smartphone B has internet connectivity?

Question 2:

Can we conclude if Emily can not get driving directions from Mike, then smartphone B does not have internet connectivity?

Prompt used to generate a data instance for depth-4 for NM

Rule: d4_1

Assumptions:

- 1: A and B are objects of type T and have property S.
- 2: Normally objects of type T with property S have property U.
- 3: if C has property G implies C has property D
- 4: if E has property L implies E has property F
- 5: either C has property G or E has property F or both
- 6: if A has property U then E has property L

Question 1:

Can we conclude if C does not have D then B has U? (YES)

Question 2:

Can we conclude if C does not have D then B does not have U? (NO)

Task 1: Generate a short context paragraph by replacing all the entity labels A, B, and so on in the above context with propositions and real entities. The generated context should have natural language sentences for all the sentences 1-4. It should not include label representations like A or B and should not mention the words "property".

Task 2: Generate questions 1 and 2 by replacing the respective labels from the generated context.

Example 1:**Assumptions:**

Apple tree and Orange tree are fruit trees.

Normally, fruit trees produce edible fruit.

If Garden is regularly watered, then its plants are flourishing.

If Orchard receives enough sunlight, then it yields high-quality fruit.

Either Garden has regular watering or Orchard yields high-quality fruit or both.

If the apple tree produces edible fruit, then Orchard receives enough sunlight.

Question 1: Can we conclude if Garden does not have flourishing plants then the orange tree produces edible fruit?

Question 2: Can we conclude if Garden does not have flourishing plants then the orange tree does not produce edible fruit?

Prompt used to generate a data instance for depth-5 for NM

Rule: d5_1

Assumptions:

- 1: A and B are objects of type T and have property S.
- 2: Normally objects of type T with property S have property U.
- 3: if C has property G implies C has property D
- 4: if E has property L implies E has property F
- 5: either C has property G or E has property F or both
- 6: if I has property H then E has property L
- 7: if A has property U then I has property H

Question 1:

Can we conclude if C does not have D then B has U? (YES)

Question 2:

Can we conclude if C does not have D then B does not have U? (NO)

Task 1: Generate a short context paragraph by replacing all the entity labels A, B,

and so on in the above context with propositions and real entities. The generated context

should have natural language sentences for all the sentences 1-4. It should not include label representations like A or B and should not mention the words "property".

Task 2: Generate questions 1 and 2 by replacing the respective labels from the generated context.

Example 1:**Assumptions:**

Rose and Lily are plants that flower.

Normally, plants that flower also produce seeds.

If a plant is a Cactus, and it has thorns, then it can survive in the desert.

If a plant is an Orchid, and it has broad leaves, then it can grow in tropical areas.

Either a Cactus has thorns, or an Orchid can grow in tropical areas, or both.

If a Lotus has flowers, then an Orchid has broad leaves.

If a Rose produces seeds, then a Lotus has flowers.

Question 1: Can we conclude if a Cactus cannot survive in the desert then a Lily produces seeds? (YES)

Question 2: Can we conclude if a Cactus cannot survive in the desert then a Lily does not produce seeds? (NO)

4.2.3 Qualitative Analysis

After data generation, we conducted a manual qualitative analysis, resulting in 1,552 high-quality samples for *Multi-LogicEval*.

Statistics *Multi-LogicEval* has 5 different logical reasoning depths. Table 4.14 shows the depth-wise statistics of samples present for each logic type after validation. After manual validation, from the generated data, we selected/updated high-quality 10 data instances for each inference rule in depth 1 and 15 or 20 data instances for each rule combination, which resulted in 410, 442, 310, 280, and 110 samples for depth-1, depth-2, depth-3, depth-4, and depth-5, respectively. For evaluation, of the total 1552 samples, 1126 samples have the answer *yes*, and the remaining 426 samples have the answer *no*.

Logic	Reasoning Depth					Total
	1	2	3	4	5	
PL	120	105	135	120	45	525
FOL	130	105	135	120	45	535
NM	160	232	40	40	20	492
Total	410	442	310	280	110	1552

Table 4.14: Statistics of *Multi-LogiEval*

Quality of Data Instances We examine each context for potential discrepancies throughout the data generation phase, ensuring they are logically correct and represent the intended logical relations. We also dedicated considerable effort to eliminating typos and validating the grammar. While validating, we encountered a few errors within the synthetically generated story-based context. We manually mitigate these errors to ensure integrity and utility.

Validation of Data Instances We involved four evaluators (who are also authors of this paper) for data validation. Each evaluator holds a graduate degree in computer science and has knowledge of logical reasoning. Each sample is evaluated by one evaluator to ensure its logical correctness. We categorized errors into three distinct groups. The categories of errors identified are (i) Incorrect Logical Premises (ILP) which indicates that premises generated by the model in the context are logically incorrect (i.e., did not align with the intended conclusion), (ii) Leaking Conclusion (LC) where the context inadvertently revealed the conclusion, bypassing the need for the logical deduction, and (iii) Repetition of Samples (RS) where identical or nearly identical contexts are present, reducing dataset diversity. We found $\sim 14.3\%$

(223 samples) of the total 1552 samples with ILP, $\sim 3.7\%$ (57 samples) with LC, and $\sim 3.7\%$ (57 samples) with RS. We mitigated all these errors manually from the generated data instances to provide a high-quality evaluation set. Furthermore, we also analyzed the number of samples we corrected for PL ($\sim 22\%$ - 115/525), FOL ($\sim 19\%$ - 102/535), and NM ($\sim 25.9\%$ - 127/492), highlighting the difficulty of generating instances for specific logics. Similarly, we also analyzed depth-wise instance correction where we corrected $\sim 17.8\%$ (73/410), $\sim 21\%$ (93/442), $\sim 23.5\%$ (73/310), $\sim 33.2\%$ (93/280), and $\sim 21\%$ (23/110) for the depth d_1 , d_2 , d_3 , d_4 , and d_5 , respectively, indicating the challenges of generating and validating multi-step reasoning context with increasing depth.

4.3 Results and Analysis

In this section, we present detailed information related to the experimental setup, our primary results, and a detailed analysis of the results.

4.3.1 Experimental Setup

Task Formulation We formulate a binary classification task using *Multi-LogiEval*. Let us consider a set of data instances $\mathcal{I}_{D,L}$ corresponding to depth D and logic type L . In this set, i^{th} instance is represented as $\mathcal{I}_{D,L}^i = \{(c_i, q_i)\}$ where c_i represents context and q_i represents question corresponding to i^{th} instance. Here, each context and question pair is created so that the conclusion provided in the question always entails context. However, you require different reasoning steps to conclude. We prompt the model to assign a label *Yes* if the conclusion logically entails the context; otherwise, *No*. To evaluate any LLMs, we provide $\langle p, c, q \rangle$ as input to predict a label *Yes* or *No* where p is a natural language prompt.

Experiments We evaluate a range of proprietary models (i.e., GPT-4, ChatGPT, and Gemini-Pro) and open-source models (i.e., Yi-34B-Chat, Orca-2-13B, and Mistral-7B-Instruct) on *Multi-LogiEval*. The evaluation is conducted on the versions of OpenAI and Google models released in April 2024. Each model is evaluated in a zero-shot-CoT setting. The prompt used for experiments is provided below. We evaluate LLMs in a zero-shot setting to show the logical reasoning ability of the model based on knowledge acquired during pre-training, since we can not expect in-context examples corresponding to different reasoning patterns and depths during inference. However, we also evaluate LLMs in a 3-shot setting.

Metrics Since the objective is to assess the model’s ability to arrive at the correct conclusion, we measure the accuracy associated with a *Yes* and *No* label. Apart from accuracy, we provide an in-depth analysis of reasoning chains in section 4.3.3 to gain insights into models’ performance. In addition, we would like to mention that the binary labels *Yes* and *No* indicate whether the conclusion presented in the question can be derived from the context. Hence, accuracy is an important evaluation metric, reflecting the model’s reasoning ability.

Prompt for Experiments

Given the context that contains rules of logical reasoning in natural language and a question, perform step-by-step reasoning to answer the question. Based on context and reasoning steps, answer the question ONLY in ‘yes’ or ‘no.’ Please use the format below:

Context: [text with logical rules]

Question: [question that is based on context]

Reasoning steps: [generate step-by-step reasoning]

Answer: Yes/No

4.3.2 Main Results

Objective Evaluation Table 4.15 illustrates the accuracy of reasoning at different depths for various LLMs, offering significant insights into their performance across distinct logic types and depths. From Table 4.15, experimental results reveal a consistent trend across PL and FOL, i.e., as the reasoning depth increases from 1 to 5, the models’ average performance drops. In particular, at depths 4 and 5, accuracy drops significantly for the majority of LLMs we evaluated. For instance, the accuracy of GPT-4, ChatGPT, and Gemini demonstrates a substantial drop from 89.17%, 91.67%, and 90% at d_1 to 66.67%, 44.44%, and 60.00% at d_5 for PL, respectively, indicating the challenge encountered even by larger-scale LLMs when handling longer chains of logical reasoning. In summary, for PL and FOL, LLMs perform well on d_1 compared to other depths. While they show competitive performance for d_2 and d_3 , there is a significant drop in performance for d_4 and d_5 in most cases. In contrast, moving on to NM, going from d_1 to d_5 , there is an increase in the performance of LLMs from an average of 35.63% to 55.83%.

Random Baseline We calculated a random baseline for each depth from MultiLogiEval as below:

$$\text{Acc}_{\text{random}} = p_{\text{yes}}^2 + p_{\text{no}}^2,$$

where p_{yes} and p_{no} represent the probabilities of predicting “yes” and “no,” respectively. The random baselines for depths d_1 , d_2 , d_3 , d_4 , and d_5 , with corresponding $\text{Acc}_{\text{random}}$ of 86.63%, 67.35%, 53.71%, 58.33%, and 83.33%, respectively. From Table 4.15, we can observe that these models perform lower in terms of average accuracy compared to the random baseline.

Models	Propositional					First-Order					Non-Monotonic				
	d_1	d_2	d_3	d_4	d_5	d_1	d_2	d_3	d_4	d_5	d_1	d_2	d_3	d_4	d_5
GPT-4	89.17	69.52	82.22	71.67	66.67	83.85	70.48	71.85	59.17	66.67	36.88	51.67	65.00	67.50	60.00
ChatGPT	91.67	56.19	63.70	62.50	44.44	97.69	59.05	57.78	50.83	37.78	33.75	41.11	50.00	62.50	60.00
Gemini	90.00	62.86	68.15	65.83	60.00	76.92	62.86	65.93	57.50	53.33	46.25	46.11	62.50	55.00	60.00
Yi-34B	85.00	65.71	58.52	46.67	26.67	90.00	55.24	57.94	48.33	13.33	37.50	41.11	55.00	62.50	65.00
Orca-13B	75.83	41.91	35.56	35.00	15.56	66.92	47.62	42.96	40.00	6.67	21.88	26.67	25.00	15.00	25.00
Mistral-7B	80.83	68.57	61.48	53.33	44.44	83.85	63.81	56.30	52.50	20.00	37.50	39.44	52.50	47.50	65.00
Avg	85.42	60.79	61.61	55.83	42.96	83.21	59.84	58.79	51.39	32.96	35.63	41.02	51.67	51.67	55.83

Table 4.15: Evaluation of LLMs in terms of accuracy on *Multi-LogiEval*.

Few-shot Results We evaluate models in a few-shot setting (specifically, 3-shot) on Multi-LogiEval, revealing a notable enhancement in performance, as depicted in Table 4.16. In the 3-shot evaluation results, we observe notable improvements in the performance of various LLMs. GPT-4 consistently exhibits high accuracy across all depths, particularly excelling in PL and FOL. Though showing significant enhancements compared to its zero-shot performance across all the models, they still underperform in NM, highlighting a persistent challenge in this area. Open-source models such as Yi-34B and Mistral-7B, while benefiting from the 3-shot setup, still display noticeable performance drops in higher depths. Comparing these findings to the zero-shot results from Table 4.15, we see a general trend of improved performance in the 3-shot setting, indicating the effectiveness of few-shot prompting. However, the observed performance drop from d_4 to d_5 in open-source models is comparable across both settings, suggesting that while few-shot examples enhance overall accuracy, they do not fully mitigate the inherent challenges these models face in higher depths. Moreover, the performance trends identified in the zero-shot evaluation, such as the consistent decrease in accuracy for larger open-source models and the superior performance of proprietary models such as GPT-4 and ChatGPT in PL and FOL, remain similar in the 3-shot setting.

Models	Propositional					First-Order					Non-Monotonic				
	d_1	d_2	d_3	d_4	d_5	d_1	d_2	d_3	d_4	d_5	d_1	d_2	d_3	d_4	d_5
GPT-4	90.00	85.71	84.44	79.17	73.33	97.78	84.76	73.33	68.33	73.33	54.38	56.11	75.00	90.00	75.00
ChatGPT	96.67	82.86	77.78	79.17	80.00	94.44	86.67	84.44	64.17	64.44	45.63	41.67	57.50	65.00	45.00
Gemini	92.22	73.33	81.48	88.33	77.78	90.00	83.81	81.48	76.67	57.78	59.38	42.78	75.00	62.50	75.00
Yi-34B	68.89	61.90	66.67	64.17	64.44	76.67	61.90	62.96	45.00	51.11	59.38	33.33	52.50	52.50	50.00
Orca-13B	85.56	80.00	72.59	75.83	68.89	91.11	73.33	63.70	55.00	42.22	56.88	46.67	60.00	50.00	50.00
Mistral-7B	80.00	64.76	71.11	73.33	66.67	93.33	71.43	62.96	62.50	42.22	37.50	36.11	45.00	57.50	70.00
Avg	84.22	75.05	74.52	74.33	70.67	90.67	75.62	69.48	59.00	54.66	50.75	42.78	60.83	62.92	60.83

Table 4.16: Few-shot Evaluation of LLMs in terms of accuracy on *Multi-LogiEval*.

Findings Table 4.15 reveal that open-source models experience a significant performance drop from d_4 to d_5 . Also, there is an increasing performance trend in NM. For PL and FOL, GPT-4, ChatGPT, and Gemini show improved performance from d_2 to d_3 , whereas the performance of open-source models consistently decreases. In addition, larger open-source models demonstrate decreasing performance. Furthermore, ChatGPT performs lower than GPT-4 and Gemini at d_5 in PL and FOL. Also, FOL performance is lower compared to PL at d_5 .

4.3.3 Analysis and Discussion

In this section, we manually analyze the generated reasoning chains ¹ by different LLMs and investigate the above-mentioned findings in detail.

Performance Improvement from d_2 to d_3 in PL and FOL for GPT-4, ChatGPT, and Gemini GPT-4, ChatGPT, and Gemini excel at d_3 for PL, with a performance decrease at d_4 and d_5 . This trend is also observed in FOL for the same models, except ChatGPT. Systematic analysis of all the reasoning chains with wrong predictions for PL and FOL shows these models reach incorrect conclusions

¹<https://github.com/Mihir3009/Multi-LogiEval>

often due to the wrong interpretation of evidence. In d_3 , increasing context length improves LLMs accuracy in information mapping, thus achieving peak performance (comparison with d_2 to d_5). At d_2 , around $\sim 27.4\%$ of reasoning chains with incorrect conclusions were due to the models’ failure to correctly map information, either from context to conclusion or the premise from one step to the next step. This number drops to $\sim 22\%$ at d_3 , and we observed that a larger context length at d_3 helps in reducing this problem. However, at d_4 and d_5 , the length of the reasoning chain increases further. Since longer reasoning steps are more prone to error propagation at later stages, causing the models to deviate further from the true conclusion, hence, lower performance at d_4 and d_5 .

Lower Performance of ChatGPT compared to GPT-4 and Gemini at Higher

Depths This pattern is particularly evident in FOL and PL at d_5 for ChatGPT compared to Gemini, and GPT-4. At d_5 , manual analysis shows that ChatGPT tends to generate longer reasoning chains compared to Gemini, and GPT-4 when answering question. For PL and FOL, the average reasoning chain length for ChatGPT at d_5 is 13.85, while for Gemini and GPT-4 at d_5 is 8.85 and 10.87, respectively. Longer reasoning chains do not necessarily correlate with better reasoning outcomes, highlighting the complexity of complex reasoning task. This suggests that optimizing reasoning chain length is crucial for improving model accuracy in complex scenarios.

Increasing Performance Trend in NM In our analysis of ChatGPT and the open-source model Yi-34B, we’ve observed consistent performance improvements with increasing depth in NM reasoning. This trend diverges from classical logic, PL and FOL. Specifically, at depths d_2 to d_5 , NM exhibits novel performance due to unique rule combinations in reasoning patterns. For instance, at d_2 , NM combines one PL rule

with one NM reasoning pattern, progressing to two PL rules with one NM pattern at d_3 , and so forth. The addition of NM reasoning patterns complements PL and FOL by providing supplementary evidence and improving contextual understanding. Notably, as depth increases, integrating basic classical rules with NM significantly enhances model accuracy, particularly evident at depths 4 and 5. This integration is pivotal for the notable performance gains observed in NM compared to classical logic at higher depths.

Larger Open-Source Models Show Decreased Performance Compared to

Smaller Models We examine Mistral-7B, Orca-13B, and Yi-34B, which differ significantly in parameter size. Mistral-7B, the smallest, performed best across various depths of classical logic, except at the simplest d_1 . As reasoning depth increased, Mistral-7B consistently outperformed Orca-13B and Yi-34B, with Yi-34B only marginally better (1.5%) at d_3 . For NM tasks, Mistral-7B and Yi-34B showed similar performance across all depths. At the most challenging depth (d_5) for both PL and FOL, Mistral-7B outperformed Orca-13B by achieving 3x performance despite Orca-13B’s larger size. We believe that this capability of Mistral is attributed to its architecture and training, as discussed in Jiang *et al.* [92]. In particular, the training of Mistral-7B focused on enhancing its reasoning capabilities.

Lower Average Performance in FOL than PL at d_1 to d_5 Upon observing the reasoning chains with wrong final predictions for the FOL and PL, we find that the generic rules in FOL contexts lead to deviations from the correct reasoning path. In some cases, it assigns predicates incorrectly to the FOL inference rule. This pattern is more prominent at d_5 , highlighting the large gap ($\sim 10\%$) in average performance between PL and FOL.

Lower Performance in d_1 of NM Reviewing the reasoning chains, we noted that models struggled to accurately map information. Interpreting various assumptions is crucial for effective reasoning at d_1 . However, we observed that models have difficulty concluding based solely on assumptions present in the context when explicit knowledge is absent.

Preliminary Discussion on Multi-variable FOL Since our work focuses on evaluating LLMs’ multi-step reasoning with simple FOL inference rules, we conducted only a preliminary study on their reasoning abilities for multi-variable FOL rules. This study reveals that creating natural language instances is challenging for this kind of setup. We evaluate the large-scale models GPT-4, ChatGPT, and Gemini. These models achieve an average accuracy of 80%, 84.3%, and 90%, respectively. This demonstrates that these LLMs can comprehend multi-variable FOL, but the rules currently involve only single-step reasoning. Our work also shows that these models perform well with single-step reasoning. Exploring multi-step reasoning with multi-variable FOL presents an interesting direction for future research.

Case Study on Evaluating *Multi-LogiEval* using Neural Symbolic Motivated by Olausson *et al.* [164], we evaluate GPT-4 using the neuro-symbolic approach where we utilized the Prover9². It first converts FOL statements into conjunctive normal form (CNF) and then performs resolution. Thus, we only evaluated data samples with FOL from *Multi-LogiEval*. For this study, we adapt the evaluation approach presented in Pan *et al.* [168] where GPT-4 is used to convert the context and question in a formal executable program and use Prover9 to solve it. We used an implementation with a similar GPT-4 version compatible with Pan *et al.* [168]. We use the following prompt to convert natural language to an executable logic program:

²<https://www.cs.unm.edu/~mccune/prover9/>

Prompt to convert natural language to an executable logic program

Given a problem description and a question. The task is to parse the problem and the question into first-order logic formulas. The grammar of the first-order logic formula is defined as follows :

1. logical conjunction: $expr1 \wedge expr2$
2. logical disjunction: $expr1 \vee expr2$
3. logical exclusive disjunction: $expr1 \oplus expr2$
4. logical negation: $\neg expr1$
5. $expr1$ implies $expr2$: $expr1 \rightarrow expr2$
6. $expr1$ if and only if $expr2$: $expr1 \leftrightarrow expr2$
7. logical universal quantification: $\forall x$
8. logical existential quantification: $\exists x$

Output format: $\langle \text{logic form} :: \text{description} \rangle$

We evaluate the performance in terms of the accuracy of selecting the correct answer. We also report the “Executable Rate”, which reflects the grammar correctness of the logical form, and the “Executable Accuracy” of the executable samples to measure the semantic correctness [168].

As the reasoning depth increases from d_1 to d_4 (Table 4.17), there is a general trend of decreasing overall accuracy and executable accuracy, indicating that higher reasoning depth poses more challenges for executing code correctly. Interestingly, at d_5 , both overall accuracy and executable accuracy show a significant improvement.

Human Evaluation

In this study, we performed a human evaluation on a selected subset of the *Multi-LogiEval*. Additionally, we explored potential strategies for enhancing LLMs’

Reasoning Depth	Overall Acc (%)	Executable Rate (%)	Executable Acc (%)
d_1	55.83	85.83	51.46
d_2	46.67	76.67	47.83
d_3	38.89	77.78	30.00
d_4	40.00	77.78	41.43
d_5	60.00	77.78	62.86

reasoning capabilities based on the findings of our cur-

Table 4.17: Performance Metrics by Reasoning Depths

rent analysis. We have conducted a human evaluation on a subset of Multi-LogiEval. Specifically, we selected 15 unique instances covering all 5 depths (5 instances for each logic type) from Multi-LogiEval. This selection resulted in a total of 75 <context, question> pairs. We hired three graduate student volunteers to provide the evaluations. Task instructions provided to all three annotators are similar to prompts provided to LLMs. Each instance pair is answered/annotated by three different annotators with 0.853 inter-annotator agreement (measured with raw/observed agreement). Here are the results for three logic types averaged across three annotators for each depth. The average accuracies are d_1 - 75.56, d_2 - 64.71, d_3 - 64.44, d_4 - 66.67, and d_5 - 64.44. From the results, we can observe that humans perform better at d_1 compared to higher depths. For higher depths, the human performance is low.

4.4 Conclusions and Future Works

In this work, we introduced *Multi-LogiEval*, a comprehensive multi-step logical reasoning benchmark consisting of three types of logic and over 60 combinations of inference rules. Our approach utilized a two-stage methodology to construct data instances for our benchmark, consisting of $\sim 1.6k$ data instances with 1 $\sim$ 5 reason-

ing depth. We evaluated a range of LLMs, including GPT-4, ChatGPT, Gemini, Yi, Orca, and Mistral on *Multi-LogiEval*. Experimental results revealed that these models struggle to perform logical reasoning, and their performance drops as the depth of logical reasoning increases (average accuracy of $\sim 68\%$ at d_1 to $\sim 43\%$ at d_5) for classical and non-classical logic. Furthermore, we systematically analyzed the reasoning chain generated by LLMs at various depths and presented interesting findings. We hope that *Multi-LogiEval* will facilitate future research in evaluating and enhancing the ability of existing and upcoming LLMs for multi-step logical reasoning.

Our results provide deeper insights into LLMs’ logical reasoning abilities by analyzing reasoning chains (§4.3.3) manually to some extent. Specifically, we analyzed where these models make mistakes and what their limitations are. We believe that such insights can help design better pre-training or alignment strategies to improve the reasoning abilities of LLMs. For instance, during pre-training, logical connectives can be treated differently at various stages of the transformer architecture. Additionally, in alignment techniques involving preference optimization, preference data can be created to prefer outputs with more logical correctness. The proposed approaches and findings in our paper can help in creating such datasets. Furthermore, exploring newer techniques such as DPO [188] and KTO [54] for their suitability in improving logical reasoning can also be an interesting future direction. Though *Multi-LogiEval* facilitates the evaluation of the multi-step logical reasoning ability of LLMs, the complexity of reasoning depth presented in *Multi-LogiEval* can be improved by adding reasoning depth beyond five steps. *Multi-LogiEval* can be further extended by incorporating other inference rules and logic types, for instance, the inference rules in first-order logic that capture n-ary relations between multiple variables. We also note that this research is limited to the English language and can be extended to multilingual scenarios for evaluating the logical reasoning ability of LLMs.

GRID PUZZLES: DIAGNOSING FAILURES IN LLM REASONING

5.1 Introduction

Recent advancements in LLMs such as GPT-4, Gemini, Claude-3 [5], Llama-2 [241], and Mistral [92] have achieved remarkable performance on a wide range of Natural Language Understanding (NLU) tasks previously thought to be exclusive to humans. Beyond NLU, exploring LLMs’ logical reasoning abilities [128, 210, 175, 176] on complex tasks such as puzzle-solving is under-explored. Past attempts have been made to evaluate models on logic-intensive grid-based puzzle-solving. However, they either do not focus on evaluating LLMs [156, 90] or do not evaluate LLMs independently, but rather use neuro-symbolic approaches [88] that use external specialized solvers on LLM outputs. Here, we aim to evaluate the puzzle-solving abilities of LLMs by themselves, without the use of any external logic solvers.

To understand the reasoning capabilities of LLMs, it is important to evaluate reasoning chains, rather than the final predicted answer. There have been works that evaluate reasoning chains using objective metrics such as ROSCOE [64], CTC [46], and BARTScore [302], however, they do not focus specifically on evaluating reasoning. Some prior works propose metrics for specific reasoning tasks, such as FOLIO [71] and ProntoQA [210]. However, these methods rely on reference-based evaluation, do not focus on puzzle-solving, and do not aim to identify fine-grained errors in reasoning chains. To address these limitations, we propose a reference-free manual and automated subjective evaluation of reasoning chains to understand various fine-grained errors in reasoning chains for grid-based puzzle-solving.

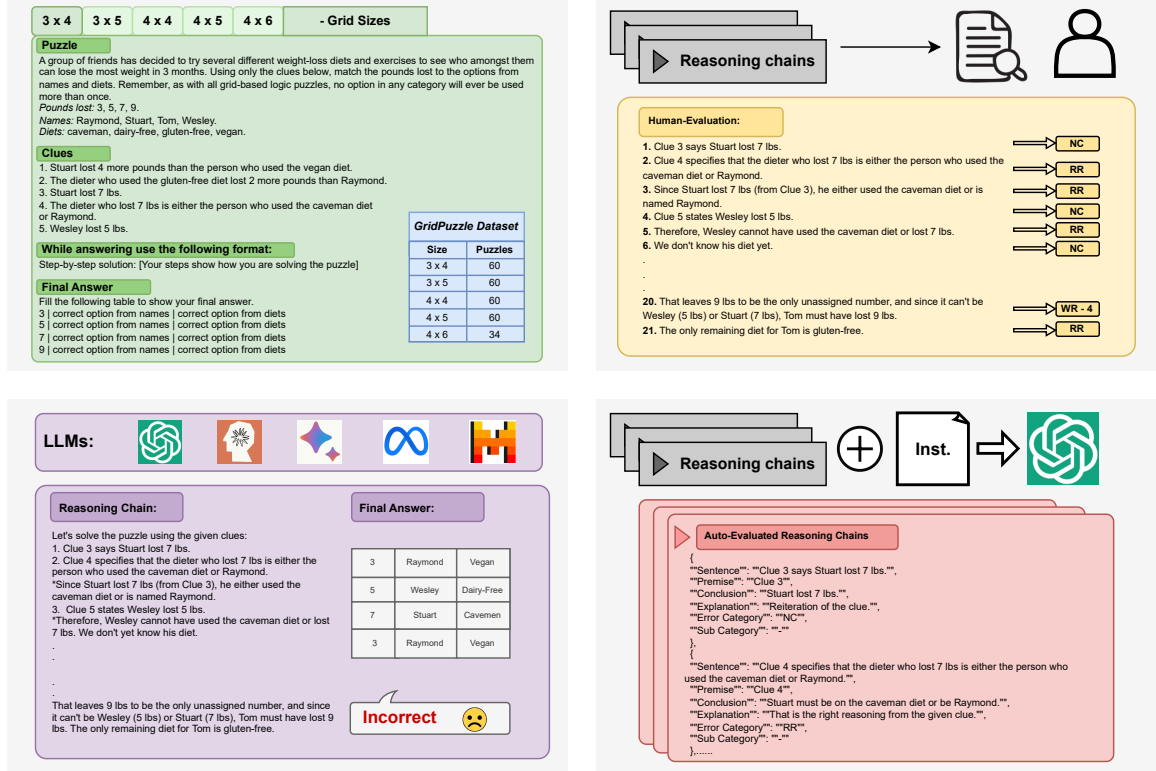


Figure 5.1: Schematic representation of proposed pipeline. Begins with the data collection of *GridPuzzle* dataset (top left) and evaluating various LLMs in zero-shot CoT setting (bottom left), then analyzing reasoning chains of LLMs manually to find various error types (top right) and automate this analysis process using LLM to check the correctness of reasoning chain by finding errors (bottom right).

Motivated by Mitra and Baral [156], we first develop *GridPuzzle* (Figure 5.1), a comprehensive evaluation dataset consisting of grid-based puzzles with grid-size of 3×4 , 3×5 , 4×4 , 4×5 , and 4×6 with three levels of difficulty (easy, medium, and hard). Then, we evaluate LLMs including GPT-4, Gemini-Pro, Claude-3, Llama-2, and Mistral on *GridPuzzle* in zero-shot-CoT setting (Figure 5.1). Experimental results show that LLMs do not fare well and achieve a maximum of 5.1% accuracy.

To investigate the reasoning chains, we manually analyze them (Figure 5.1) to find fine-grained errors (further details in §5.2.3). Based on this, we propose a new error taxonomy comprising five broad categories, and nine fine-grained sub-categories

(Table 5.1 and Table 5.2), providing deeper insights into the primary causes of the LLMs’ reasoning failures. However, scaling manual analysis to a larger set is time-consuming and laborious. Hence, we propose to leverage LLMs as auto-evaluators by creating prompts that utilize error taxonomy to automate the analysis of reasoning chains and help in identifying errors (Figure 5.1). While evaluating w.r.t. manual annotation, our auto-evaluator model achieves $\sim 86\%$ agreement, hence providing quality error categorization.

Evaluating reasoning chains based on error categorization reveals interesting findings such as LLMs show lower accuracy despite having more error-free reasoning steps, open-source models lack reasoning skills compared to closed-source models, and the most dominant error categories are wrong reasoning and elimination. Additionally, we employ existing prompting methods such as Plan-and-Solve and Self-discover, demonstrating that these methods do not improve performance on *GridPuzzle*. We believe that our findings will inspire future work in the automated in-depth evaluation of reasoning chains for reasoning tasks and enhance the reasoning abilities of models.

5.2 Evaluation of Reasoning Chains

5.2.1 *GridPuzzle*

To develop this dataset, we extract logic grid puzzles of various grid sizes from Puzzle Baron’s Logic Puzzles – <https://logic.puzzlebaron.com/>. Specifically, we compile logic grid puzzles of size 3×4 , 3×5 , 4×4 , 4×5 , and 4×6 . Each grid size has three levels of difficulty (easy, medium, and hard) except 4×6 . This particular grid size has only two difficulty levels (Easy and Medium). Statistics corresponding to each grid size are presented in Figure 5.1 (top left). The *GridPuzzle* dataset contains 274 puzzles of various grid sizes and complexity. A sample puzzle from the dataset

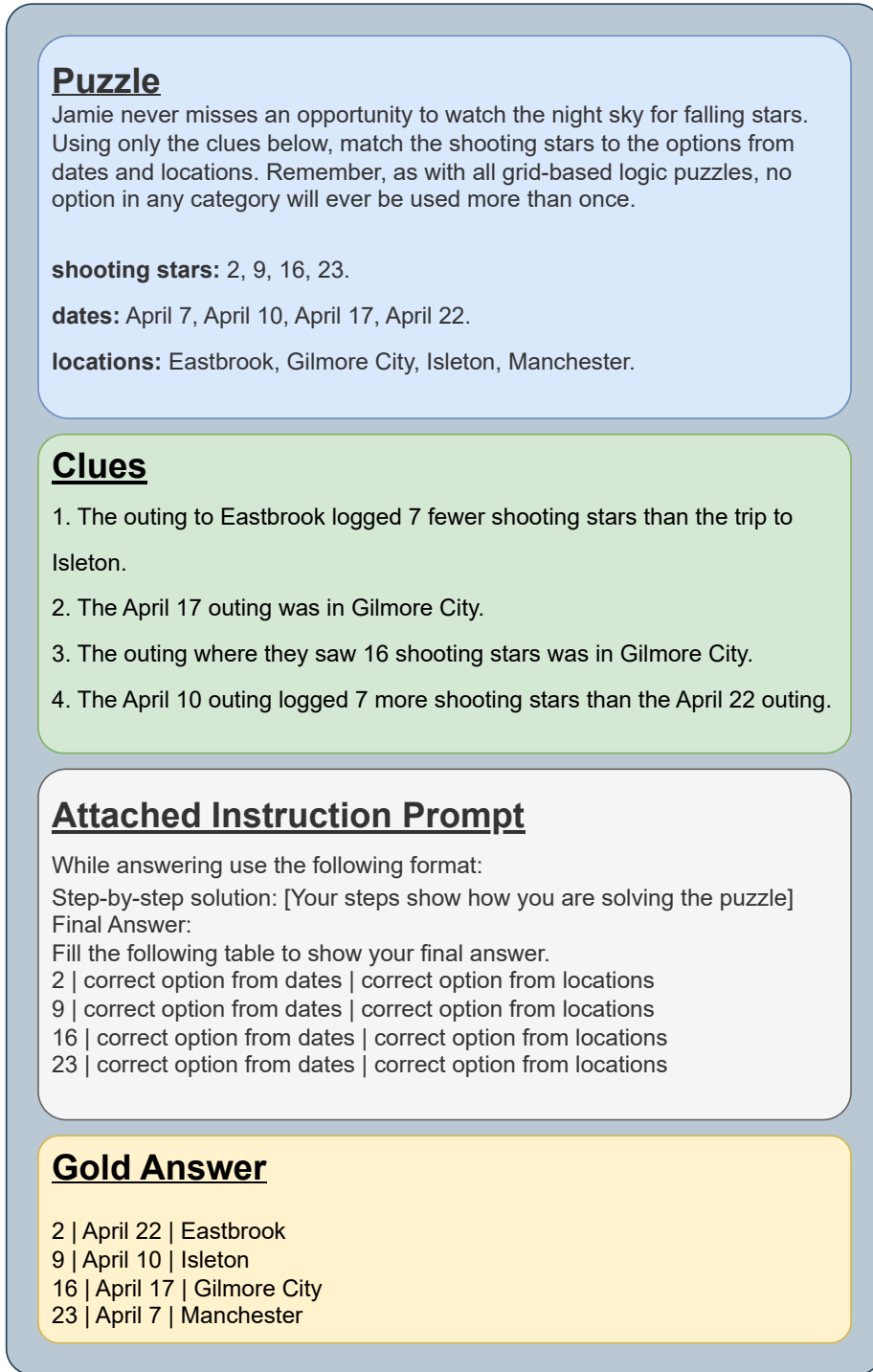


Figure 5.2: The prompt structure of a 4 x 4 grid size puzzle from *GridPuzzle*. Every Zero-shot-CoT prompt from the dataset consists of Puzzle, its corresponding Clues, the Instruction for solving the puzzle, along with the Gold solution of the Puzzle.

along with the Zero-shot-CoT prompt is described in Figure 5.2. All the puzzles in the dataset have a similar structure with varying numbers of clues.

5.2.2 *Manual Evaluation*

To explore where exactly these LLMs falter in performing reasoning, we conduct a detailed manual analysis of the reasoning chains generated by them while solving grid-based puzzles. To conduct the manual analysis of the reasoning chains, the annotators were provided the guidelines described in Figure 5.3. The same guideline was also used to create the system prompt for the GPT-4o Auto-evaluator. The annotation process was conducted by 5 annotators and the annotations were also cross-examined to resolve any discrepancies. Our manual analysis process consists of three steps. First, we begin by segmenting the reasoning chains into individual sentences, allowing us to categorize errors more precisely. Second, we identify the premise and conclusion for each sentence and determine their respective correctness. We refrain from subdividing sentences into multiple premises or conclusions to maintain simplicity for finding errors. At last, each sentence is categorized as either containing a single premise and conclusion or being a declarative statement without a conclusion. Then, we begin assessing potential issues or errors in the reasoning chains. Now, we follow an exhaustive approach to create fine-grained error categories. We begin with 30 reasoning chains (6 puzzles x 5 reasoning chains from LLMs) to manually identify potential errors. Next, we categorize these errors in a structured format. We then add another 30 reasoning chains to see if any new types of errors emerge. If new errors are identified, we refine our categories accordingly. This process is repeated until we evaluate a total of 150 reasoning chains and no new types of errors are found. Based on this method, we have carefully filtered and categorized several errors made by LLMs, presenting them as five broad categories and nine sub-categories.

12-step Annotation Guideline

1. Take your time to refer to the annotation example provided.
2. Make sure to break every step into individual sentences before doing the annotations and categorizations.
3. Analyze every sentence separated by a full stop and divide it into premise and conclusion (the conclusion is the derived information). If a sentence starts with words like "so" or "therefore", the premise will be the previous conclusion.
4. Take your time to refer to the error categories described in the Error Categories knowledge section below and Classify the derived information into specific error categories.
5. When classifying into the main Error Categories, the premise and conclusion correctness are determined independently from each other and it is only based on the answer given in the correct solution.
6. Compare the premise and conclusions with the correct solution first to determine if they are wrong or right.
7. Do not use the sub-category explanations to justify your main category (WW, WR, RR, RW, NC) classifications. Only after the error categories have been assigned you will look into the sub categories.
8. The premise is declared wrong only when it violates the data given in the correct solution. The conclusion is declared wrong only when it violates the data given in the correct solution.
9. If the category has a wrong premise, classify it further into a subcategory for the wrong premise.
10. If the category has a wrong conclusion, classify it further into a subcategory for the wrong conclusion.
11. Check your work and see if the chosen category is indeed the most appropriate category.
12. You are not required to solve the entire puzzle or correct any mistakes. You can take the help from the correct solution to conduct your analysis.

Figure 5.3: The 12-step guideline provided to the annotators to conduct manual evaluation of the reasoning chains.

5.2.3 Proposed Error Taxonomy

Based on manual analysis, Table 5.1 represents the broad categories and Table 5.2 shows sub-categories.

Broad Categories As shown in Table 5.1, we present five main categories: “WW” - *Wrong Premise Wrong Conclusion*, “WR” - *Wrong Premise Right Conclusion*, “RW” - *Right Premise Wrong Conclusion*, “RR” - *Right Premise Right Conclusion*, and “NC” - *No Conclusion*. These acronyms of broad categories are self-explanatory. For instance, the category “WW” comprises sentences where the sentence consists of a wrong premise as well as a wrong conclusion. Interestingly, we also find the “WR” category consists of instances where a wrong premise still leads to a correct conclusion. Additionally, sentences containing only information from clues or premises from previous steps fall under “NC”. The correctness of the premise and conclusions is determined by comparing them to the information provided in the puzzles and the gold solution only. We conduct further investigation as to why the premises and conclusions become incorrect.

Sub-categories: Wrong Premise As shown in Table 5.2, we identified the source of the premise to determine the origin of errors: (i) ‘*From Clues*’ – where the premise is directly borrowed from one of the clues without any further reasoning, and (ii) ‘*Derived*’ – where the premise is inferred from either the clues or the previous conclusions. From Table 5.2, there are six possible reasons associated with two different sources for the wrong premise. When the premise originates from the source (i), we find three types of errors: **Hallucination** – When some factual information from the clues is distorted or completely made up; **Incomplete information** – When the information is correctly borrowed from the clues but it is not sufficient to make a par-

Error Category	Description
WW	Wrong Premise and Wrong Conclusion
WR	Wrong Premise and Right Conclusion
RW	Right Premise and Wrong Conclusion
RR	Right Premise and Right Conclusion
NC	No Conclusion statement or no reasoning involved

Table 5.1: Proposed error taxonomy for broad categories based on manual analysis. If a statement starts with “so, therefore, hence, this means, this implies, etc.” and/or is not followed by any premise, consider the previous statement’s conclusion or the previous NC as the premise.

ticular conclusion; **Assumptions** – This is a special category where the premise is not derived but also not given exactly in the clues. It is often related to one of the clues and is of the form, “Let’s assume” or “Assuming that.” When source is *derived*, we find three different errors: **Error Propagation** – This occurs when a previously incorrect conclusion becomes the basis for a flawed premise, thereby extending the error from one conclusion to the next; **Incomplete information** – When the derived premise is not sufficient to make a particular conclusion; and lastly, **Wrong Assumption** – When the LLM reasoner clearly states that a premise was an assumption but it was incorrectly derived.

Sub-categories: Wrong Conclusion As shown in Table 5.2 (source), conclusions are always logically derived from a fixed set of premises. For having a wrong conclusion in any reasoning step, we find 3 errors responsible: **Error Propagation** – When a conclusion is wrong strictly due to some error in the preceding premise; **Wrong Elimination** – When the conclusion is wrong because the LLM reasoner failed to

Category	Source	Sub-Category	Description
Wrong Premise or No Conclusion	From the clues (Example: From clue 4,...)	(1) Hallucination	When information is completely out of context and not present in clues.
		(2) Incomplete Information	Lacks necessary information to make a particular conclusion.
		(3) Assumptions	Statements not derived from clues directly; might include assumed information relevant to the clue.
	Derived Conclusions using clues given in puzzle which was not inherently given in the clues.	(4) Error Propagation	Premise derived from a previous incorrect conclusion.
		(5) Incomplete Information	Lacks necessary information to make a particular conclusion.
		(6) Wrong Assumption	The derived assumption is incorrect.
Wrong Conclusion	Derived using the premise (which itself is either taken directly from the clues or derived)	(a) Wrong Reasoning	The reasoning is incorrect, regardless of the premise’s accuracy.
		(b) Error propagation	Conclusion is incorrect due to an erroneous premise.
		(c) Wrong Elimination	All premises are present, but not all conclusions are correctly derived.

Table 5.2: Proposed error taxonomy for sub-categories based on manual analysis. These sub-categories are defined for cases where either the conclusion or premise is incorrect (“RW” or “WR”) or both are incorrect (“WW”). For “WW”, the error sub-categories might appear in any combinations between (1-6) and (a-c) such as ‘1a’, ‘4b’, or ‘6c’.

eliminate all the unfit choices correctly. This case is specific to the grid-based puzzle task but is inherently an erroneous deduction on the LLM’s end; **Wrong reasoning**

- The remaining incorrect conclusions that did not fit in the above categories are classified under this label.

This detailed categorization not only shed light on the types of errors LLMs tend to make, but also guided the development of an auto-evaluator to expand the analysis of all reasoning chains generated from the *GridPuzzle* dataset.

5.2.4 Automated Evaluation

Manual analysis of reasoning chains provides a detailed categorization of errors; however, it is tedious and, therefore, challenging to scale for the entire dataset. However, analyzing the distribution of errors from our proposed taxonomy on the whole dataset is also crucial in understanding the shortcomings of LLMs’ reasoning ability.

Thus we develop an LLM-based auto-evaluator to automate the process of error evaluation. To this end, we prompt the GPT-4o model to identify and categorize errors in the given reasoning chain. Our prompt consists of a system instruction followed by a user prompt containing the reasoning chain to be evaluated along with the original puzzle and its gold solution. The structure of this system prompt is elaborated in Figure 5.4. The system prompt can be further dissected into 3 key components: the instructions, the knowledge, and an exemplar. The **instruction** contains all the rules that the GPT-4o needs to follow to conduct accurate evaluation and error categorization of the reasoning chains. It incorporates similar

sequential steps used during the manual evaluation of reasoning chains along with the required output format. The **knowledge** has a detailed description of our error

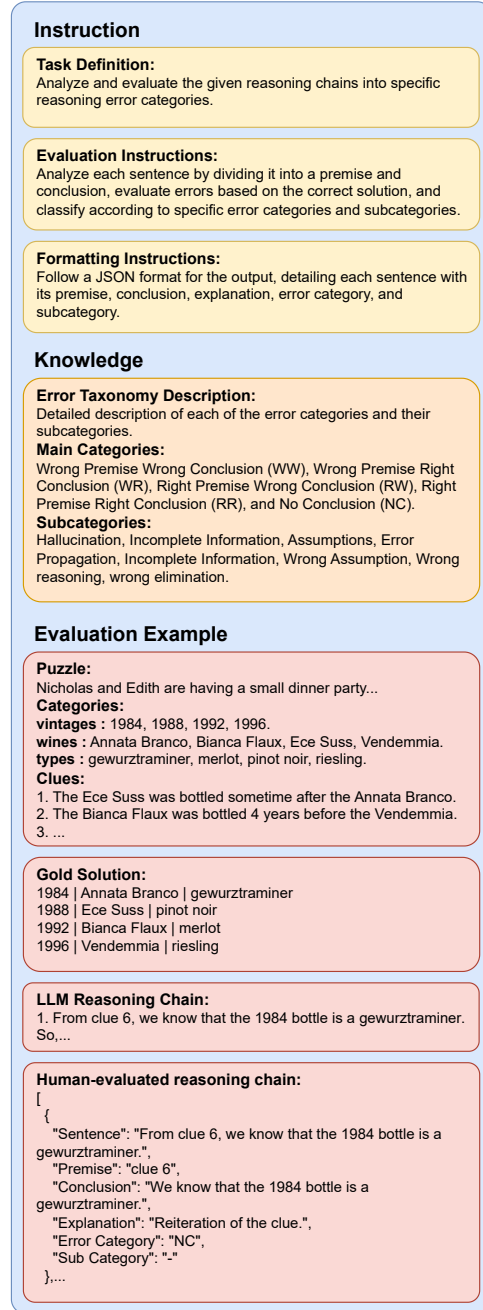


Figure 5.4: The prompt structure of auto-evaluator which is followed by a user prompt with the puzzle, its gold solution, and the reasoning chain to be evaluated.

Examples of reasoning chain evaluated by GPT-4o

Sentence: "Therefore, Zeno must be 69%, and UCLA must be 62%.",

Premise: "If Zeno were 55%, there would be no score 7% lower than 55% for UCLA.",

Conclusion: "Zeno must be 69%, and UCLA must be 62%",

Explanation: "The conclusion is incorrect as UCLA is already known to be 62% from clue 2.",

Error Category: "RW.",

Premise: "A",

Sentence: "Since the third performer used flashlights, it must be either Lora or Carmen.",

Premise: "The performer who used flashlights was either Lora or Carmen.",

Conclusion: "The third performer must be either Lora or Carmen.",

Explanation: "The conclusion is based on the incorrect premise that the third performer used flashlights.",

Error Category: "WW",

Sub Category "4B",

Table 5.3: Examples of reasoning chain evaluated by Auto-evaluator (GPT-4o).

taxonomy including the broad and sub-categories. We also provide a preference order for selecting categories along with the description to minimize any ambiguity in the evaluation process. Lastly, the **exemplar** consists of a puzzle, its correct solution, the original model-produced reasoning chain, and the manually evaluated reasoning chain with our error categories. We termed this LLM-based evaluator as “Auto-evaluator”. To expand the reasoning chain evaluation process we prompt the GPT-4o model with a detailed system prompt.

Using the Auto-evaluator, we evaluated a total of 1,370 reasoning chains generated

by five different LLMs for solving 274 puzzles. The application of our Auto-evaluator to this large dataset allowed us to analyze the distribution of error categories on a broader scale. To validate the accuracy of the evaluations performed by the Auto-evaluator, we randomly sampled 20 reasoning chains from the manually evaluated set. The authors then compared their error category assignments to those given by the Auto-evaluator. The agreement score for the total number of reasoning steps between the manual evaluation and the GPT-4o evaluation is $\sim 86\%$. Table 5.3 shows the example of reasoning steps evaluated by GPT-4o.

5.2.5 Example Annotated Reasoning Chains

In order to identify the error categories from the erroneous reasoning chains we conducted manual and auto-evaluation of the reasoning chains. The process of manual evaluation has been described in Figure 5.5 and the process of auto-evaluation using GPT-4o has been described in Figure 5.6.

5.3 Results and Analysis

5.3.1 Experiments

We evaluate a range of closed-source LLMs including GPT-4-Turbo, Claude-3-Opus, and Gemini-Pro, and open-source models Llama-2-13B, and Mistral-7B-Instruct on *GridPuzzle* in the Zero-shot-CoT setting [107]. We also conducted a scaling experiment on Llama-2-70B. Our *GridPuzzle* dataset consists of a set of instances denoted as $\mathcal{P} = \langle p_n^{i \times j}, a_n \rangle$, where $p_n^{i \times j}$ is n^{th} puzzle instance with grid size of $i \times j$ and a_n as a gold answer. We prompt each LLM to generate a reasoning chain before predicting answer $\hat{a}$. To evaluate each model in the Zero-shot-CoT setting, we provide $\langle I, p_n^{i \times j} \rangle$ as input to the model and predict an answer $\hat{a}$ where I is a

Prompt	Model Response
Puzzle: A group of friends has decided to try several different weight-loss diets and exercises to see who amongst them can lose the most weight in 3 months. Using only the clues below, match the pounds lost to the options from names and diets. Remember, as with all grid-based logic puzzles, no option in any category will ever be used more than once. pounds lost : 3, 5, 7, 9. names : Raymond, Stuart, Tom, Wesley. diets : caveman, dairy-free, gluten-free, vegan. Clues: 1. Stuart lost 4 more pounds than the person who used the vegan diet. 2. The dieter who used the gluten-free diet lost 2 more pounds than Raymond. 3. Stuart lost 7 lbs. 4. The dieter who lost 7 lbs is either the person who used the caveman diet or Raymond. 5. Wesley lost 5 lbs. While answering use the following format: Step-by-step solution: [Your steps show how you are solving the puzzle] Final Answer: Fill the following table to show your final answer. 3 correct option from names correct option from diets 5 correct option from names correct option from diets 7 correct option from names correct option from diets 9 correct option from names correct option from diets	Step-by-step solution: - From clue 3, we know Stuart lost 7 lbs. - From clue 1, since Stuart lost 7 lbs, the person on the vegan diet lost 3 lbs ($7 - 4 = 3$). - From clue 4, since Stuart lost 7 lbs, he is either on the caveman diet or he is Raymond. But we know Stuart's name, so he must be on the caveman diet. - From clue 2, the person on the gluten-free diet lost 2 more pounds than Raymond. Since we have the pounds lost as 3, 5, 7, and 9, and Stuart already lost 7 lbs, the only way for someone to lose 2 more pounds than another (excluding the 7 lbs already accounted for) is for the gluten-free dieter to have lost 9 lbs and Raymond to have lost 7 lbs. However, since Stuart lost 7 lbs and is not Raymond, this clue helps us deduce the sequence of weight loss but needs to be revisited for exact matching once we have more information. - From clue 5, Wesley lost 5 lbs. - Now, knowing Wesley lost 5 lbs, Stuart lost 7 lbs, and the vegan diet resulted in a 3 lbs loss, we can deduce that Tom, being the only person left, lost 9 lbs. - Since Tom lost 9 lbs and the gluten-free diet is associated with losing 2 more pounds than Raymond, and the only diet left for Wesley (who lost 5 lbs) is dairy-free, it fits that Tom is on the gluten-free diet. - This leaves the vegan diet for the person who lost 3 lbs, which hasn't been directly matched to a name yet. Since Stuart is on the caveman diet and lost 7 lbs, Wesley is left with dairy-free and lost 5 lbs, and Tom is on the gluten-free diet and lost 9 lbs, Raymond must be the one who lost 3 lbs and is on the vegan diet.
Gold Solution: 3 Raymond vegan 5 Wesley gluten-free 7 Stuart caveman 9 Tom dairy-free	Final Answer: 3 Raymond vegan 5 Wesley dairy-free 7 Stuart caveman 9 Tom gluten-free
Manually Evaluated Reasoning Steps	Error Categories
1. From clue 3, we know Stuart lost 7 lbs.	RR
2. From clue 1, since Stuart lost 7 lbs, the person on the vegan diet lost 3 lbs ($7 - 4 = 3$).	RR
From clue 4, since Stuart lost 7 lbs, he is either on the caveman diet or he is Raymond	RR
But we know Stuart's name, so he must be on the caveman diet.	RR
4. From clue 2, the person on the gluten-free diet lost 2 more pounds than Raymond.	RR
Since we have the pounds lost as 3, 5, 7, and 9, and Stuart already lost 7 lbs, the only way for someone to lose 2 more pounds than another (excluding the 7 lbs already accounted for) is for the gluten-free dieter to have lost 9 lbs and Raymond to have lost 7 lbs.	RW-C
However, since Stuart lost 7 lbs and is not Raymond, this clue helps us deduce the sequence of weight loss but needs to be revisited for exact matching once we have more information.	RR
5. From clue 5, Wesley lost 5 lbs.	RR
6. Now, knowing Wesley lost 5 lbs, Stuart lost 7 lbs, and the vegan diet resulted in a 3 lbs loss, we can deduce that Tom, being the only person left, lost 9 lbs.	WR-2
7. Since Tom lost 9 lbs and the gluten-free diet is associated with losing 2 more pounds than Raymond, and the only diet left for Wesley (who lost 5 lbs) is dairy-free, it fits that Tom is on the gluten-free diet.	RW-C
8. This leaves the vegan diet for the person who lost 3 lbs, which hasn't been directly matched to a name yet.	RR
Since Stuart is on the caveman diet and lost 7 lbs, Wesley is left with dairy-free and lost 5 lbs, and Tom is on the gluten-free diet and lost 9 lbs, Raymond must be the one who lost 3 lbs and is on the vegan diet.	WW-4B

Figure 5.5: The **top left** section of the figure consists of a 3x4 sample puzzle from the *GridPuzzle* dataset along with the Zero-shot-CoT prompt. Right below the prompt, we have the Gold solution for the corresponding puzzle. In the **top right** section of the figure, we have the Model-generated Reasoning chain to solve this puzzle along with the Final Answer. In this particular instance, the reasoning chain was generated by the GPT-4 model. In the **bottom half** of the figure, we have the manually evaluated reasoning chain, split into individual sentences. The corresponding error categories found in each step is given the **bottom right** column called ‘Error Categories.’

Prompt	Llama2-13B Response									
Puzzle: Using only the clues below, match the days to the options from guests and rooms. Remember, as with all grid-based logic puzzles, no option in any category will ever be used more than once. days : 2, 3, 4, 5. guests : Owens, Parker, Underwood, Zamora. rooms : 114, 124, 314, 320. Clues: - Underwood is in room 114. - The guest in room 320 is staying 1 day fewer than Underwood. - Owens is staying for 3 days. - Parker is staying 1 day longer than the client in room 314. While answering use the following format: Step-by-step solution: Your steps showing how you are solving the puzzle Final Answer: Fill the following table to show your final answer. <table><tr><td>2 correct option from guests correct option from rooms</td></tr><tr><td>3 correct option from guests correct option from rooms</td></tr><tr><td>4 correct option from guests correct option from rooms</td></tr><tr><td>5 correct option from guests correct option from rooms</td></tr></table>	2 correct option from guests correct option from rooms	3 correct option from guests correct option from rooms	4 correct option from guests correct option from rooms	5 correct option from guests correct option from rooms	Let's break down the clues and solve the puzzle step by step. Step 1: From clue 1, we know Underwood is in room 114. Since Underwood is staying for 2 days (as the days are 2, 3, 4, 5), we can conclude that Underwood is staying for 2 days. Step 2: From clue 2, we know the guest in room 320 is staying 1 day fewer than Underwood. Since Underwood is staying for 2 days, the guest in room 320 is staying for 1 day. This means the guest in room 320 is staying for day 2. Step 3: From clue 3, we know Owens is staying for 3 days. Since Owens is staying for 3 days, Owens must be staying for day 3. Step 4: From clue 4, we know Parker is staying 1 day longer than the client in room 314. Since the days are 2, 3, 4, 5, the client in room 314 must be staying for day 4. Parker is staying 1 day longer, so Parker is staying for day 5. Step 5: Now that we have the days for each guest, we can fill in the table: <table><tr><td>2 Zamora 320</td></tr><tr><td>3 Owens (not mentioned)</td></tr><tr><td>4 (not mentioned) 314</td></tr><tr><td>5 Parker (not mentioned)</td></tr></table>		2 Zamora 320	3 Owens (not mentioned)	4 (not mentioned) 314	5 Parker (not mentioned)
2 correct option from guests correct option from rooms										
3 correct option from guests correct option from rooms										
4 correct option from guests correct option from rooms										
5 correct option from guests correct option from rooms										
2 Zamora 320										
3 Owens (not mentioned)										
4 (not mentioned) 314										
5 Parker (not mentioned)										
Gold Solution: <table><tr><td>2 Zamora 124</td></tr><tr><td>3 Owens 314</td></tr><tr><td>4 Parker 320</td></tr><tr><td>5 Underwood 114</td></tr></table>	2 Zamora 124	3 Owens 314	4 Parker 320	5 Underwood 114	Final Answer: <table><tr><td>2 Zamora 320</td></tr><tr><td>3 Owens 124</td></tr><tr><td>4 Underwood 314</td></tr><tr><td>5 Parker 114</td></tr></table> Note: The rooms for days 3, 4, and 5 are not explicitly mentioned in the clues, but we can deduce them based on the information provided.		2 Zamora 320	3 Owens 124	4 Underwood 314	5 Parker 114
2 Zamora 124										
3 Owens 314										
4 Parker 320										
5 Underwood 114										
2 Zamora 320										
3 Owens 124										
4 Underwood 314										
5 Parker 114										
Auto-Evaluated Reasoning Steps		Error Categories	Sub Categories							
{ "Sentence": "From clue 1, we know Underwood is in room 114.", "Premise": "clue 1", "Conclusion": "we know Underwood is in room 114.", "Explanation": "Reiteration of the clue.", "Error Category": "NC", "Sub Category": "-" }		NC	-							
{ "Sentence": "Since Underwood is staying for 2 days (as the days are 2, 3, 4, 5), we can conclude that Underwood is staying for 2 days.", "Premise": "Underwood is in room 114.", "Conclusion": "Underwood is staying for 2 days.", "Explanation": "The conclusion is incorrect as it assumes Underwood is staying for 2 days without sufficient information.", "Error Category": "RW", "Sub Category": "a" }		RW	a							
{ "Sentence": "Parker is staying 1 day longer, so Parker is staying for day 5.", "Premise": "the client in room 314 must be staying for day 4.", "Conclusion": "Parker is staying for day 5.", "Explanation": "The conclusion is incorrect as it is based on the wrong premise that the client in room 314 is staying for day 4.", "Error Category": "WW", "Sub Category": "4b" }		WW	4b							

Figure 5.6: The **top left** section of the figure consists of a 3x4 sample puzzle from the *GridPuzzle* dataset along with the Zero-shot-CoT prompt. Right below the prompt, we have the Gold solution for the corresponding puzzle. In the **top right** section of the figure, we have the Model-generated Reasoning chain to solve this puzzle along with the Final Answer. In this instance, the reasoning chain was generated by the Llama2-13b model. In the **bottom half** of the figure, we have the GPT-4o Auto-Evaluated Reasoning chain. The auto-evaluation is done sentence-wise and the output is in a JSON-structured format consisting of 5 components: the *Sentence*, the *Premise*, the *Conclusion*, the *Error category* and the *Sub-category*. The corresponding error categories found in each sentence are given in the **bottom right** columns called ‘Error Categories’ and ‘Sub Categories.’

natural language instruction. The evaluation is conducted on the OpenAI, Google, and Anthropic model versions released in April 2024 with temperature setting 0 for deterministic predictions. NVIDIA A100 GPUs are used for conducting the inference of open-source models with a batch size of 4.

5.3.2 Metrics

We use accuracy to demonstrate the capability of LLMs in solving grid-based puzzles based on their ability to predict the final answer. To calculate this metric, we use the LLM-generated final answers and compare them with the available gold solution. The predicted answers and the gold solution are in the form of tables with the number of rows and columns equal to the grid size of the puzzle. We perform an Exact Match (EM) to compare the two tables and mark them as correct only when all the entries of the tables match. See the example of the final answer table in Figure 5.5 and Figure 5.6.

5.3.3 Objective Evaluation

To evaluate the performance of LLMs when solving grid-based puzzles, we assess the outputs of 5 LLMs using the accuracy. As shown in Figure 5.7, we found that all the models have low performance on the *GridPuzzle* dataset in terms of accuracy. The smaller open-source LLMs completely failed at the puzzle-solving task, with Llama-2 solving only one puzzle correctly. Close-source models with significantly larger parameter sizes also exhibited poor performance. GPT-4 had the highest accuracy at only 5.11% (14 puzzles out of 274). Despite the overall low performance of all LLMs, the closed-source models perform marginally better.

With the increase in the sizes of the grids, the complexity of the puzzles also rises, leading to a depreciating performance by the LLMs with larger grids. Overall

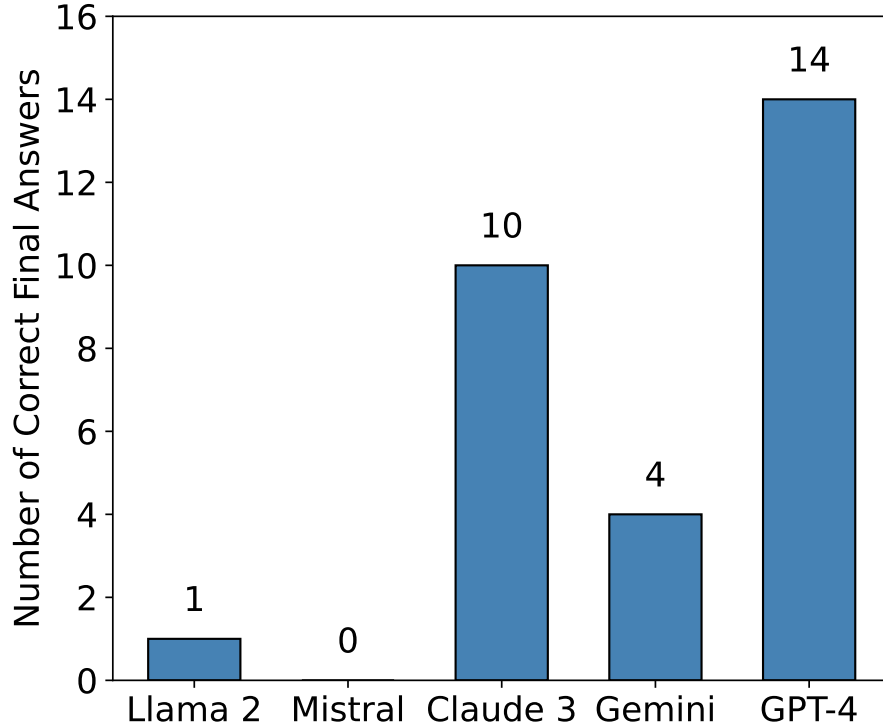


Figure 5.7: Performance of five different LLMs in terms of accuracy on the *Grid-Puzzle* dataset.

the performance of larger LLMs was much better than the small open-source models. It is interesting to note that the smaller LLMs have consistently low performance with the increase in the grid size of the puzzles but the larger LLMs have mixed performance.

5.3.4 Reasoning Chain Evaluation

We evaluated 1370 reasoning chains produced by 5 LLMs for the 274 puzzles in the *GridPuzzle* dataset using the GPT-4o Auto-evaluator.

The relative distribution of the broad error categories over the collective reasoning steps for each model is given in Figure 5.8. It is important to note that, despite using the same zero-shot-CoT setting, the GPT-4 and Llama-2 used significantly more reasoning steps ($> 5.5k$ steps) to solve the 274 puzzles compared to the other

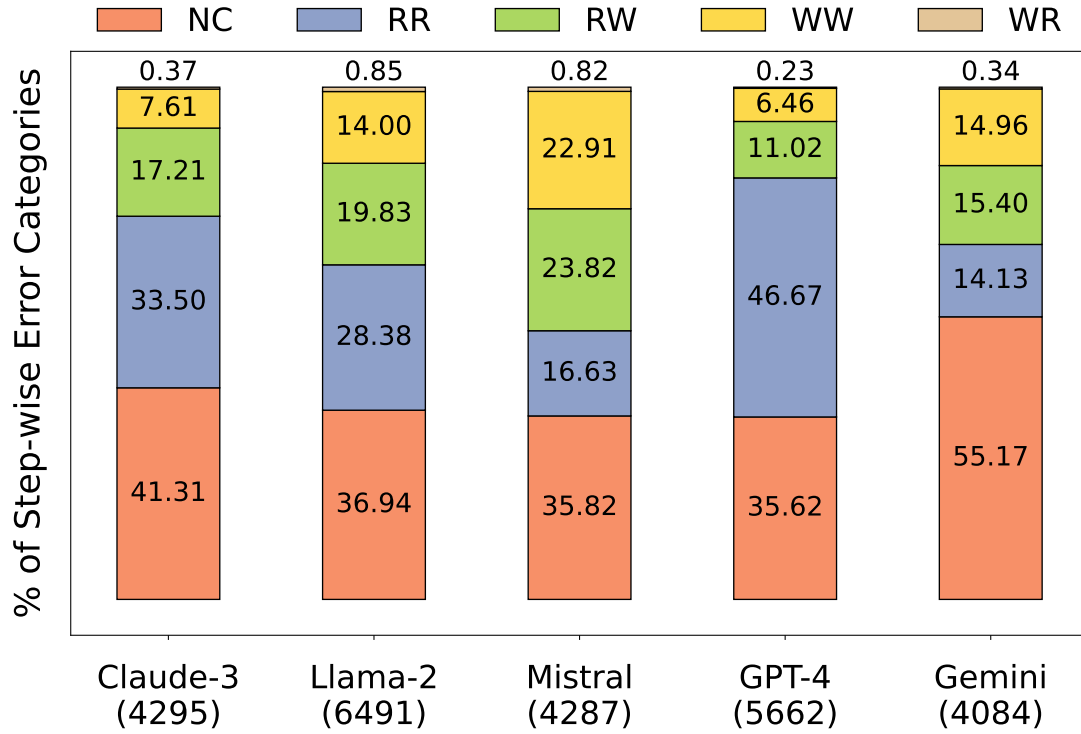


Figure 5.8: The percentage distribution of the broad error categories across the combined reasoning steps of all five LLMs. The total number of steps generated by each model is provided inside the round brackets below the model names.

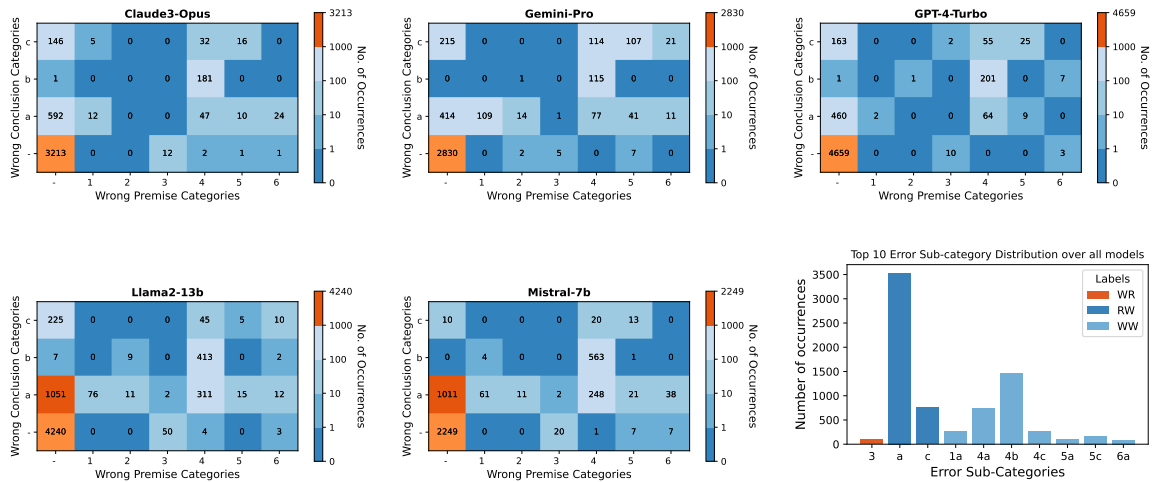


Figure 5.9: The first five sub-figures in the above section show the error Sub-category distribution over five LLMs. The last sub-figure denotes the top 10 error sub-category distribution across all model reasoning steps.

three models ($\sim 4k$ steps). The distribution of error sub-categories for each model is presented as heatmaps in the first five sub-figures in Figure 5.9. Here, we present several findings based on the evaluation of different error category distributions across *GridPuzzle*.

Majority of reasoning steps are error-free. Figure 5.8 shows that most reasoning steps for each model fall into the “NC” error category, indicating that many steps reiterate the facts or clues from the initial puzzle rather than focusing on reasoning. Over 55% of Gemini-Pro’s reasoning steps fall into this category, the most among all models, suggesting that Gemini spends the fewest steps on actual reasoning. The “RR” category comprises over 46% of GPT-4’s reasoning steps, highlighting its strong reasoning ability.

The accuracy is low despite the reasoning chains being mostly error-free. It has been observed that “RR” category reasoning steps mainly occur in the initial half of the chain. Conversely, errors in the “RW”, “WR”, and “WW” categories typically occur in the latter half, resulting in incorrect final answers and lower accuracy scores. Based on error taxonomy, “RW”, “WR”, and “WW” broad error categories have been further dissected into 6×3 error sub-categories, with their distribution across reasoning steps shown in Figure 5.9.

Dominant broad categories: RW and WW. The most common error sub-category across all heatmaps appears to be the “-”, the absence of errors. All the reasoning steps with “NC” and “RR” classifications fall in this category. To observe the actual overall trend across all 5 LLMs, the top 10 most common error sub-categories have been listed in the last sub-figure of Figure 5.9. The top categories ‘a’ and ‘c’ refer to the **Wrong Reasoning** and the **Wrong Elimination** sub-categories under

the “RW” category. These errors arise when the premise is correct but the LLMs fail to make accurate deductions from it. A number of the top 10 sub-error categories (‘1a’, ‘4a’, ‘4b’, ‘4c’, ‘5a’, ‘5c’, and ‘6a’) emerge from the “WW” category.

For the categories, ‘4a’, ‘4b’, and, ‘4c’ the errors in the premise are propagated from errors in previous reasoning steps showing how initially occurring errors in the chain can lead to more dependent errors. The ‘4b’ error category is the one where this behavior is maximized as here both the premise and conclusions were wrong because of previously propagated errors. The ‘5a’ and ‘5c’ errors occurred due to the incompleteness or lack of information in the premise and wrong reasoning or elimination in the corresponding conclusions. The ‘1a’ kind of error occurred when the premise consisted of hallucinated information. The only sub-category from the “WR” category making it in the top 10 is the ‘3’ category which is caused due to wrong assumptions in the premise. It can be noted here that the reasoning steps of the “WR” category do not deteriorate either of the evaluation metrics, as the conclusions ended up being correct, but rather indicate the inconsistency of the LLMs in reasoning over puzzle-solving.

Proprietary LLMs are way better at GridPuzzle than Open-Source LLMs.

From the results of objective and subjective metrics, it is evident that the open-source models have lower performance on the grid-puzzle-solving task than the proprietary models. The Llama-2 and Mistral models have the lowest accuracy values and their performance consistently degrades with the increase in the size and complexity of the grids. The Claude-3, Gemini, and GPT-4 models have higher values of accuracy. The disparity in the performance of both kinds of models can be attributed to the difference in their parameter sizes and the low instruction following capabilities of small open-source models.

Model Scaling Effect: Llama-70B We conducted a case study on the Llama models to analyze their performance on GridPuzzle with increasing model parameter size. We repeated the same experiment in the Zero-shot-CoT setting with the Llama-70B model. We found that the performance of the bigger model was marginally higher than the 13B model. The Accuracy went up from 1 correct final answer in the 13B model to 2 in the 70B model. However, despite the slight improvement, the Llama model’s performance was still inferior to GPT-4, Gemini, and Claude. We infer that even with the increasing model parameter size, the LLMs lack the intrinsic reasoning capabilities required to solve complex logic problems such as *GridPuzzle*.

5.3.5 Mitigation Strategies

We conducted a study on the 60, 3x4 puzzles present in *GridPuzzle* dataset to try and improve the reasoning abilities of LLMs when solving the grid-puzzle task. We used prompt-based methods, such as the Plan-and-Solve technique, which divides puzzle-solving into planning and solving steps. We also enhanced the solver with insights from our error taxonomy. The prompt structure for this technique is given in Figure 5.10.

Next, we devised our own strategy to improve LLM reasoning by using the top error categories from our findings and teaching the LLM to rectify those mistakes. This strategy termed as Feedback-learning makes use of a detailed system prompt that acts as a feedback-providing unit followed by a basic user prompt to solve the puzzle. The prompt structure for this strategy is shown in Figure 5.11.

We also implemented a code-based technique to solve GridPuzzle. We borrowed the PoT prompt from the original implementation to create a solver function to solve the puzzles. Next, we asked an LLM to implement this code and produce the Final Answer. Since the codes produced by the LLM may contain some errors we utilized

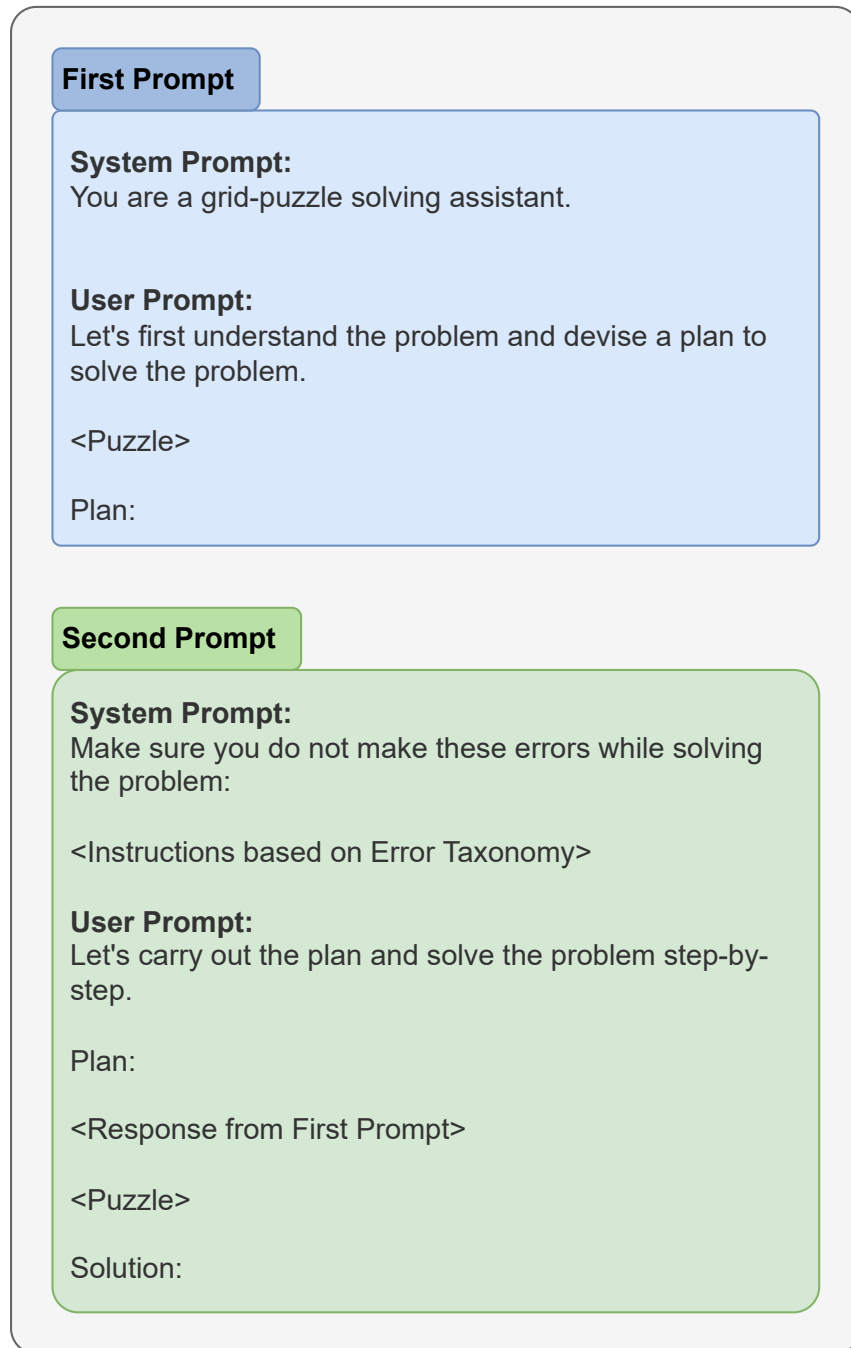


Figure 5.10: The prompt structure for the Plan-and-Solve strategy which is split into two prompts one for planning and the other for solving the puzzle.

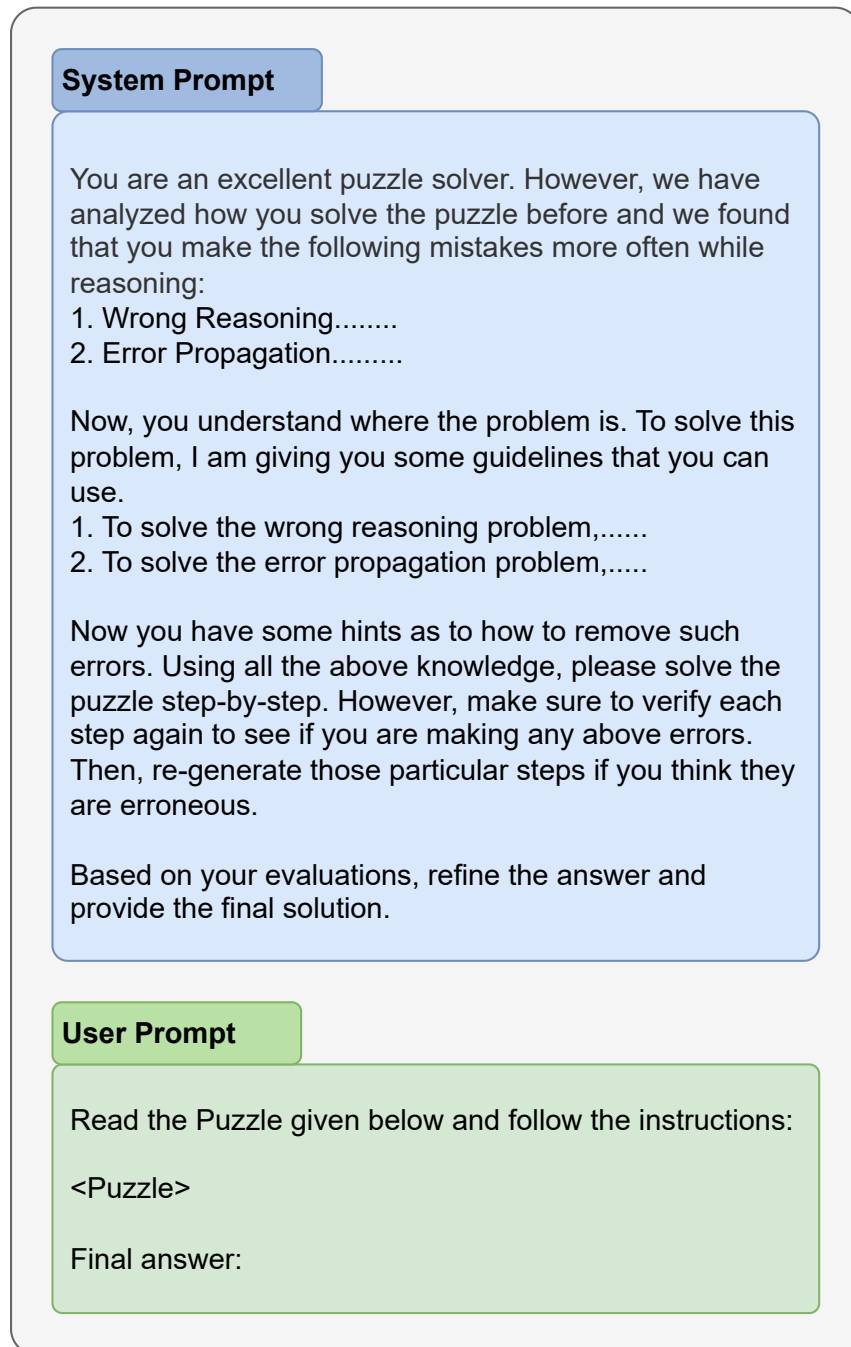


Figure 5.11: The prompt structure for the Feedback-learning strategy. The system prompts consist of instructions regarding the major errors as well as ways to rectify those errors.

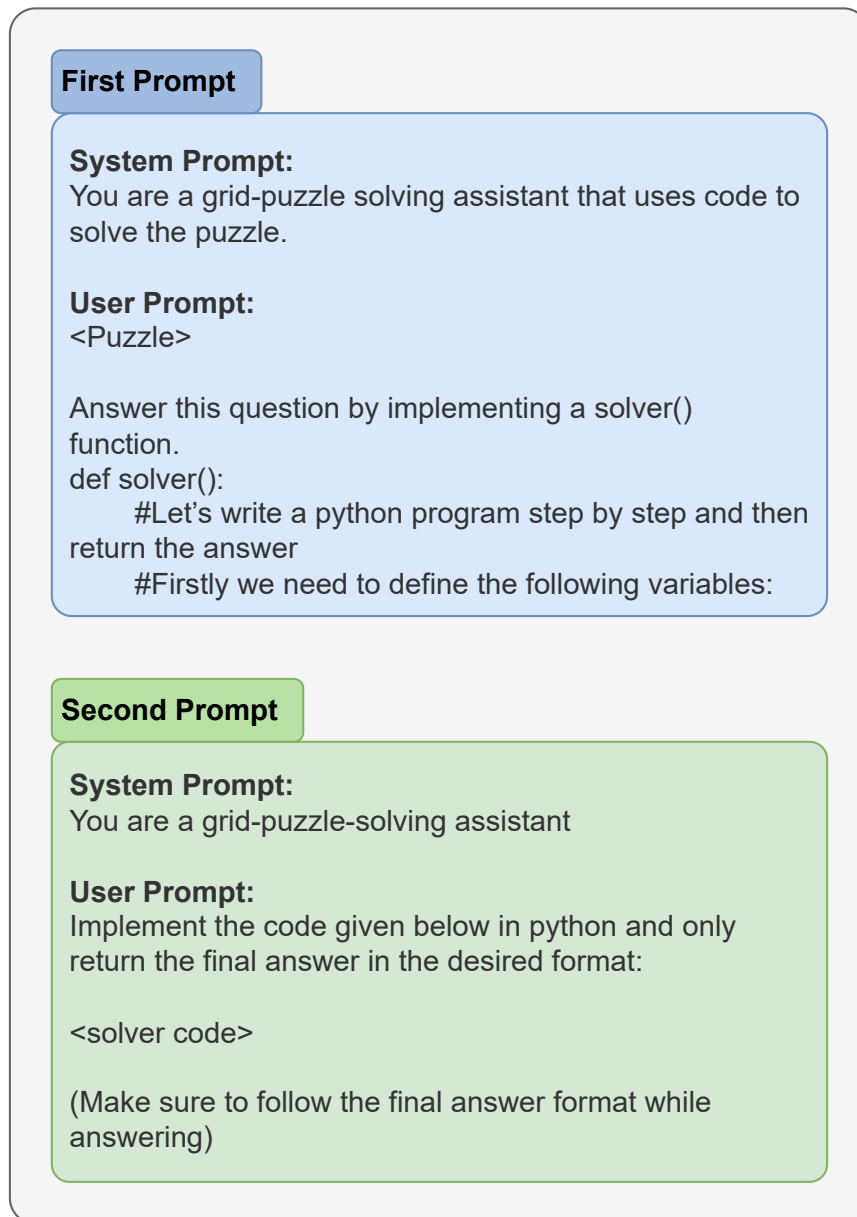


Figure 5.12: The prompt structure for the Program of Thought prompting technique. In the first part, we asked the LLM to generate a code to solve the given puzzle. In the second part, an LLM is prompted to implement the code produced in the first part to get the Final Answer table.

the LLM’s compiler to implement the code instead of a rigid Python compiler. The prompt structure is provided in figure 5.12. Next is the Self-correct strategy which merges Self-verify and Self-refine qualities to minimize LLM reasoning errors. It starts with solving the puzzle using a Zero-shot-CoT prompt, followed by prompting the LLM to verify and refine the solution. Finally, it integrates the model’s suggestions with insights from our error taxonomy to enhance the puzzle-solving response. The prompt structure for this strategy is shown in figure 5.13. Lastly, the Self-Discover strategy, depicted in figure 5.14, proved most effective in reducing LLM reasoning errors in puzzle-solving. This approach begins by having the model analyze the problem and potential errors, follows with a list of prescribed reasoning modules, prompts the LLM to select and apply the most suitable module, and concludes by using a structured prompt to solve the puzzle.

Popular reasoning error mitigation strategies do not improve LLMs on GridPuzzle.

We conduct a case study on a subset of *GridPuzzle*, focusing on a 3x4 grid size, utilizing commonly employed prompting techniques to enhance the rea-

soning capabilities of LLMs. In particular, we use five strategies: (1) Plan-and-Solve [260],

(2) Self-correct [308], (3) Self-discover [319], (4) Feedback-Learning, and (5) Program of Thought prompting [29]. We updated the prompts corresponding to these

Mitigation Strategy	Accuracy
Baseline	12
Plan-and-Solve	9
Self-correct	10
Self-discover	13
Feedback-Learning	10
Program-of-Thought	10

Table 5.4: The results for accuracy using GPT-4-Turbo, with and without mitigation strategies for the 60 samples of 3×4 grid-size.

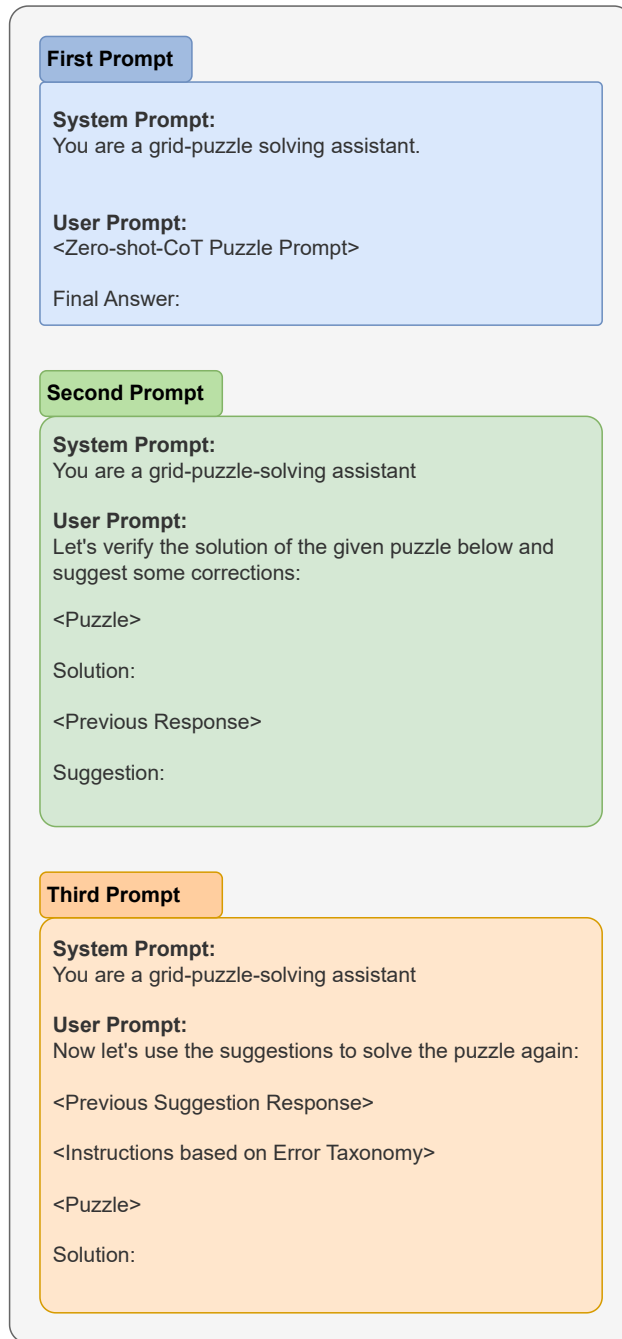


Figure 5.13: The prompt structure for the Self-Correct strategy is split into 3 parts. The first prompt solves the puzzle, the second prompt verifies the solution and gives suggestions to improve the solution, and the third prompt uses these suggestions along with error taxonomy-based instructions to refine the final solution.

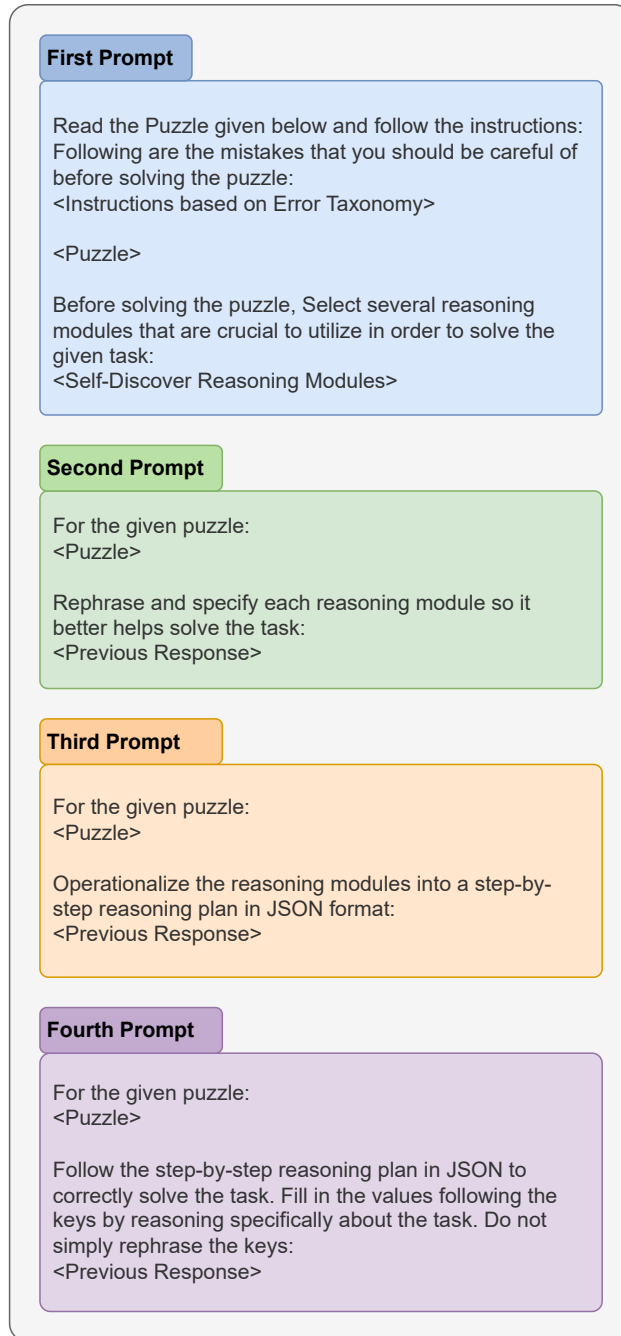


Figure 5.14: The prompt structure for the Self-Discover strategy. In the first part of this prompt the model is prompted to assess the problem and select the appropriate reasoning module to solve it. Then the module is modified to give a structured plan to solve the puzzle. In the second part, the model uses this structured plan along with instructions from our error taxonomy to solve the puzzle.

techniques to include some of our major findings from the reasoning chain evaluations and error categorization analysis as precautionary instructions.

The first strategy Plan-and-Solve prompts the model to first generate a plan to solve the given problem and then follow those steps. The second strategy is inspired by the Self-correct method which uses a combination of self-verification and self-refine to improve reasoning. Next, we used the Self-discover technique which is a 2-step structured reasoning process. We created our prompting technique called “Feedback-Learning” by providing specific feedback system instructions to the LLM based on our error taxonomy. Lastly, we also implemented a code-style prompting technique that implements a code to solve the puzzle but does not give a reasoning chain. The results of these strategies are in Table 5.4. It is evident from the results that prompting-based strategies are not sufficient to significantly improve the LLM reasoning on the grid-puzzle-solving task. Compared to the rest of the strategies, Self-Discover marginally improves the performance. These results indicate the need to develop techniques beyond prompting by having deeper insights from LLMs’ reasoning chains.

5.4 Conclusion

In this work, we evaluated the logical reasoning abilities of LLMs through the lens of a grid-based puzzle-solving task. We introduced *GridPuzzle*, an evaluation dataset of 274 puzzles with various grid sizes. From a manual evaluation of reasoning chains generated by five different LLMs on *GridPuzzle*, we developed a fine-grained error taxonomy with five broad categories and nine sub-categories. We then created an Auto-evaluator to automate the identification of error categories, providing broader insights into error distributions across the dataset. Additionally, we proposed *PuzzleEval*, a reference-free metric to objectively evaluate the correctness of reasoning chains for grid-based puzzles. Our analysis of error distributions in *GridPuzzle* re-

vealed several interesting findings and insights into the logical reasoning abilities of different LLMs. We further evaluated existing reasoning-specific prompting methods, such as self-discover and self-correct, finding that they do not improve results on *GridPuzzle*. We believe our work offers a challenging dataset, highlights where these LLMs make mistakes, and provides insights to develop better logical reasoning systems for complex tasks such as grid puzzle-solving.

While *GridPuzzle* facilitates the evaluation of LLMs’ logical reasoning abilities, the complexity of the puzzles can be enhanced by incorporating further complex grid sizes beyond 4x6. Additionally, this study can be extended to different types of puzzles, such as Sudoku, Game of 24, and commonsense puzzles. Though our study provides fine-grained error categories, it can be further refined by mapping to formal logic to identify more detailed and atomic errors, offering a deeper understanding of LLMs’ reasoning failures. Although we propose an effective automatic method for error identification to reduce manual analysis, exploring other automated methods using smaller-scale supervised learning could be a promising future research direction. We also note that this research is currently limited to the English language and can be extended to multilingual scenarios to evaluate the logical reasoning abilities of LLMs.

The dataset, *GridPuzzle*, used for this study is based on 274 puzzles from the open-source platform. No personal information from data creators has been collected during the creation of the dataset. The data collection process strictly adheres to the terms of use and privacy policies of the platform.

PLANGEN: A MULTI-AGENT FRAMEWORK FOR COMPLEX REASONING TASKS

6.1 Introduction

In many real-world tasks, we often encounter the word “plan for” or “plan to”. For example, prompting large language models (LLMs) with *“Let us make a plan to ...”* yields completions such as “travel the world”, “schedule a meeting”, or “organize a visit”. To show open-ended, “everyday” plans from LLMs, we show below two prompt templates and their ChatGPT-4o (checkpoint: May 2025) completions.

Example Prompts for “Let us make a plan”

Prompt: “Let us make a plan to ...”

- Let us make a plan to travel the world on a budget.
- Let us make a plan to surprise our parents with a visit.
- Let us make a plan to start our own small business.
- Let us make a plan to eat healthier without giving up flavor.
- Let us make a plan to finish that project we started months ago.
- Let us make a plan to learn a new language together.
- Let us make a plan to volunteer in our local community.
- Let us make a plan to save enough money for a down payment.
- Let us make a plan to reconnect with old friends.
- Let us make a plan to spend more quality time offline.

Prompt: “Let us make a plan for . . .”

- Let us make a plan for our next big adventure.
- Let us make a plan for retirement that actually excites us.
- Let us make a plan for the weekend so it doesn’t slip away.
- Let us make a plan for dealing with unexpected emergencies.
- Let us make a plan for hosting the perfect dinner party.
- Let us make a plan for our child’s education journey.
- Let us make a plan for how to reach our fitness goals.
- Let us make a plan for moving into our dream home.
- Let us make a plan for celebrating our anniversary in style.

The tasks related to such prompts are referred to as “natural planning” which is different from the traditional classical AI planning, where given an initial state, goal, descriptions of executability and effect of actions, one has to come up with a sequence of executable actions that achieves the goal. Classical AI planning, which in early settings had inputs in specific formats (i.e., not in natural language), has high computational complexity [21]. Recent works [98, 251] have shown that LLMs are not good at such planning, and natural planning with LLMs offers a promising direction, aligning better with real-world tasks [73, 309, 269, 95]. Thus, our focus is only on natural planning [314] and its effect on downstream reasoning tasks where the formulation does not necessarily match with a classical planning setting.

In recent years, LLM agents have shown impressive abilities to solve complex reasoning problems [290, 282, 259]. Orthogonal to this exploration, scaling a search space during inference-time (i.e., test-time scaling) [222, 274] has gained popularity in tackling difficult problems such as mathematical reasoning [304] and code generation [257]. Despite the success of these frameworks, we hypothesize that they often struggle

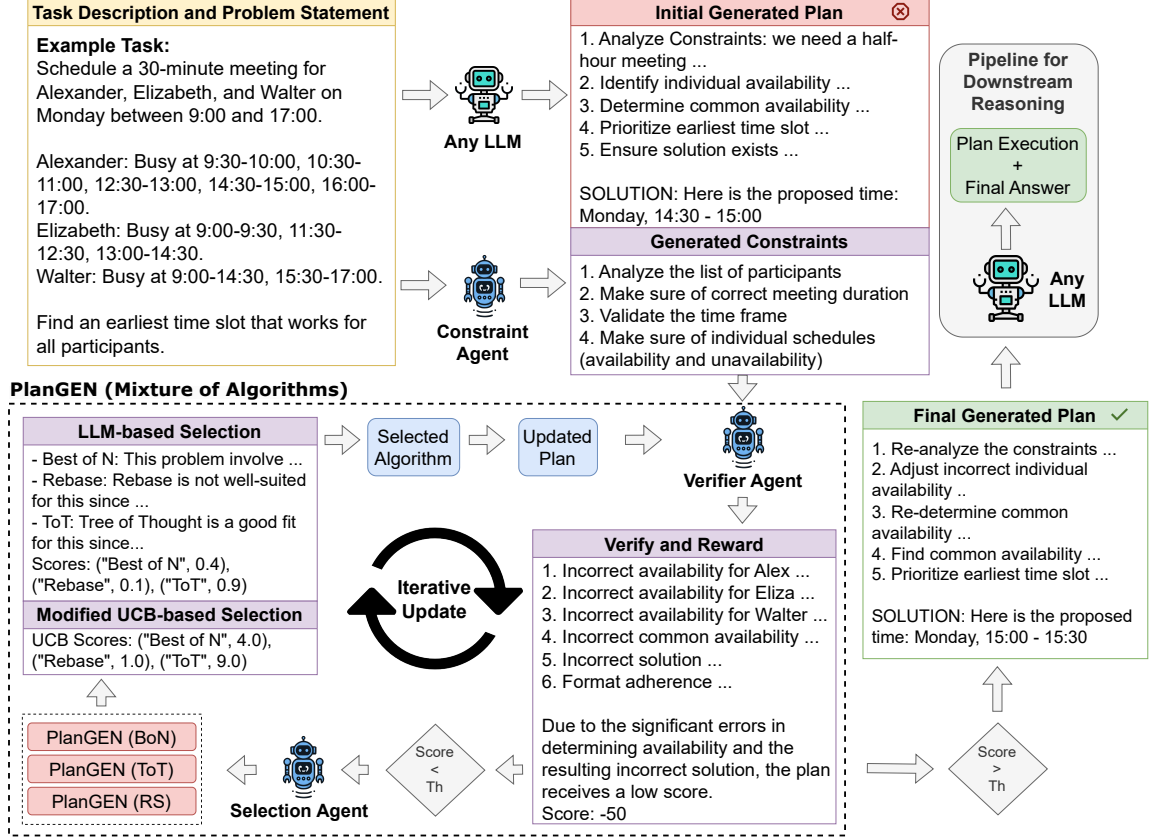


Figure 6.1: Schematic representation of PlanGEN (Mixture of Algorithms). An initial plan and constraints guide iterative plan refinement. The verification agent provides reward scores for plan quality, and the selection agent chooses inference algorithms until the highest-reward plan is found and used for downstream reasoning (if needed). UCB: Upper Confidence Bound, BoN: Best of $\mathcal{N}$, ToT: Tree-of-Thought, RS: REBASE.

with complex planning and reasoning due to the lack of a better verification module and a failure to account for instance-level complexity across single tasks. Furthermore, although some initial explorations exist [16, 116], the effectiveness of these frameworks for natural planning is under-explored. Motivated by these, we proposed PlanGEN, a model-agnostic, easily scalable, multi-agent framework for effective natural plan generation.

PlanGEN consists of three specialized agents: *constraint agent*, *verification agent*,

and *selection agent*. The constraint agent extracts instance-specific constraints (e.g., budget, concepts, rules, etc.); the verification agent evaluates plan quality and provides a reward score considering the constraints; and the selection agent dynamically chooses the best inference algorithm using an improved Upper Confidence Bound (UCB) formula [70] for instances of different complexity. We explore popular and widely used three inference algorithms within **PlanGEN**: Best of $\mathcal{N}$ [18], Tree-of-Thought (ToT) [289], and REward-BAlanced SEarch (REBASE) [280]. We combine our agents with these algorithms, yielding four frameworks: (1) **PlanGEN** (Best of $\mathcal{N}$), (2) **PlanGEN** (ToT), (3) **PlanGEN** (REBASE), and (4) **PlanGEN** (Mixture of Algorithms). In **PlanGEN**, “Multi-Agent” signifies using the constraint and verification agents for the first three approaches, and all three agents for the “Mixture of Algorithms” (Figure 6.1). Figure 6.1 shows an example from NATURAL PLAN (Calendar scheduling), and §6.2 provides more examples.

We perform all experiments using Gemini-1.5-Pro [236] as the underlying model in **PlanGEN**. We further present a case study on Gemini-2.5-Pro, Gemini-2.0-Flash, and GPT-4o [87] to show the model-agnostic nature. We evaluate natural planning ability on NATURAL PLAN [314], scientific/mathematical reasoning on GPQA [198] and OlympiadBench [75], and financial reasoning on DocFinQA [197]. Performance is compared against Zero-shot Chain-of-Thought (CoT) and a vanilla multi-agent baseline. We achieve state-of-the-art results on NATURAL PLAN ($\sim 8\%\uparrow$ average across all categories), OlympiadBench (text-only) ($\sim 5\%\uparrow$ on MATH, $\sim 4\%\uparrow$ on PHYSICS), and DocFinQA ($\sim 7\%\uparrow$). On GPQA, we outperform Gemini-1.5-Pro ($\sim 13\%\uparrow$), GPT-4o ($\sim 12\%\uparrow$), and Claude-3.5-Opus ($\sim 9\%\uparrow$), while achieving competitive performance compared to the vanilla multi-agent baseline ($\sim 1\%\uparrow$). Further analysis reveals that the simplest method (i.e., **PlanGEN** (Best of $\mathcal{N}$)) achieves the best performance on NATURAL PLAN (Figure 6.6). **PlanGEN** (Mixture of Algorithms) achieves the best

performance for complex problems (Figure 6.7), including GPQA, and Olympiad-Bench(MATH). Our case study on QwQ-32B [238] demonstrates PlanGEN’s compatibility with recent open-source reasoning models. Additionally, our comparison with Buffer-of-Thought (BOT) [287] shows that PlanGEN achieves superior performance compared to reasoning-based prompting. We further conduct a thorough analysis of the results, which reveals several important findings. In summary, our contributions are: (1) PlanGEN, a novel, model-agnostic, and scalable multi-agent framework for enhancing LLM natural planning; (2) SOTA results on several complex planning and reasoning benchmarks; and (3) a novel approach to constraint-based verification and instance-level complexity-based inference algorithm selection.

6.2 PlanGEN

This section provides detailed discussion on each proposed agents as well as proposed frameworks using those agents and three popular inference-time algorithms, i.e., Best of $\mathcal{N}$, ToT, and REBASE.

6.2.1 Proposed LLM Agents

PlanGEN comprises three specialized LLM agents: a *constraint agent*, a *verification agent*, and a *selection agent*. Each agent utilizes an off-the-shelf LLM (e.g., Gemini, GPT) which is equipped with task-specific prompts for efficient performance.

6.2.1.1 Constraint Agent

We define “constraints” as the instance-specific criteria necessary for verifying solutions to planning problems. For instance, in the calendar scheduling from NATURAL PLAN, relevant constraints include ‘individual schedules’, ‘availabilities’, and ‘preferences’. In a scientific reasoning problems from GPQA, constraints might be the

‘concepts used’, ‘calculation correctness’, and ‘formula selection’. We argue that careful extraction of these constraints is critical for successful verification. The constraint agent serves as a preprocessing component in the framework, designed to extract instance-specific constraints from the problem description. By analyzing the input problem, this agent identifies maximum possible constraints that are required for verifying generated plans to improve the overall relevance and quality of the planning process. The prompt used by the constraint agent enables it to systematically identify constraints by asking the underlying LLM to focus on specific aspects of the problem. This ensures that no critical information is overlooked and that the resulting constraints are comprehensive. While constraint agent is effective in capturing the majority of constraints, it does not offer a formal guarantee of exhaustiveness in constraint extraction.

Prompt for Constraint Agent To enable systematic extraction of constraints, we design a task-specific prompt for the constraint agent provided below. Based on the prompts, we define the types of constraints used in the NATURAL PLAN benchmark for different planning tasks: calendar scheduling, meeting planning, and trip planning. For DocFinQA, we provide a set of formatting instructions to ensure structured constraint generation. For GPQA and OlympiadBench, the constraint extraction follows the general prompt outlined here.

You are an expert in understanding an input problem and generating set of constraints. Analyze the input problem and extract all relevant instance-specific constraints and contextual details necessary for accurate and feasible planning.

(Optional) These constraints may include:

<You may provide any specific type of constraints>

<You may provide any formatting instruction>

Input Problem: <problem statement>

Examples for Constraint Agent To illustrate the output of our constraint agent, Table 6.1, Table 6.2, Table 6.3, and Table 6.4 present representative examples of generated constraints. These tables highlight the diverse constraints generated for problem instances of different tasks.

6.2.1.2 Verification Agent

The verification agent plays a critical role in the framework by assessing the quality of generated plans based on constraints. This agent ensures that plans are aligned with task objectives, adhere to constraints, and progress logically toward a correct and complete solution. This agent has two key components: (i) feedback generation, and (ii) numerical reward score generation based on feedback.

Feedback Generation While verifying each generated plan against different constraints, the verification agent generates detailed natural language reasoning regarding plan quality. We consider this explanation as “feedback”, offering interpretability and actionable next step towards improvement.

Numerical Reward Generation Motivated by Zhang *et al.* [304], we instruct the agent to evaluate the plan against various constraints and assign a reward score on a scale of -100 to 100 . The scoring mechanism is designed to enforce strict quality

Benchmark: Calendar Scheduling

You are an expert at scheduling meetings. You are given a few constraints on the existing schedule of each participant, the meeting duration, and possibly some preferences on the meeting time. Note there exists a solution that works with existing schedule of every participant.

TASK: You need to schedule a meeting for Michelle, Steven and Jerry for one hour between the work hours of 9:00 to 17:00 on Monday.

Here are the existing schedules for everyone during the day:

- Michelle has meetings on Monday during 11:00 to 12:00; - Steven has blocked their calendar on Monday during 9:00 to 9:30, 11:30 to 12:00, 13:30 to 14:00, 15:30 to 16:00; - Jerry has blocked their calendar on Monday during 9:00 to 9:30, 10:00 to 11:00, 11:30 to 12:30, 13:00 to 14:30, 15:30 to 16:00, 16:30 to 17:00;

Find a time that works for everyone's schedule and constraints.

Instance-Specific Constraints and Details:

Participants: Michelle, Steven, Jerry - The meeting must include all three.

Meeting Duration: 1 hour - The meeting requires a contiguous 60-minute slot.

Work Hours: 9:00 to 17:00 (Monday) - The meeting must start and end within these working hours. This acts as a global constraint.

Individual Schedules (Existing Meetings/Blocked Times):

- Michelle: Busy from 11:00 to 12:00 on Monday. - Steven: Busy from 9:00 to 9:30, 11:30 to 12:00, 13:30 to 14:00, and 15:30 to 16:00 on Monday. - Jerry: Busy from 9:00 to 9:30, 10:00 to 11:00, 11:30 to 12:30, 13:00 to 14:30, and 15:30 to 17:00 on Monday (note this constraint ends at the end of the workday).

While the problem states "Note there exists a solution," this isn't a constraint in itself; it's a guarantee that a feasible solution exists given the constraints. The actual constraints are the schedules, meeting duration, and working hours.

Table 6.1: Example of constraint generation for the calendar scheduling benchmark from NATURAL PLAN.

Benchmark: Meeting Planning

You are visiting San Francisco for the day and want to meet as many friends as possible. Solve the problem by considering various different schedules and picking the best one to optimize your goals.

Travel distances (in minutes):

Bayview to Golden Gate Park: 22.

Golden Gate Park to Bayview: 23.

CONSTRAINTS: You arrive at Bayview at 9:00AM. Barbara will be at Golden Gate Park from 8:00AM to 11:30AM. You'd like to meet Barbara for a minimum of 90 minutes.

Your response should start with 'SOLUTION:'

Instance-Specific Constraints and Details:

Travel Times: - Bayview to Golden Gate Park: 22 minutes. - Golden Gate Park to Bayview: 23 minutes (Note: This information is provided but not strictly necessary for this specific problem as the return trip isn't relevant to meeting Barbara).

Locations: - Starting Location: Bayview - Meeting Location: Golden Gate Park

Scheduled Meeting Times/Participant Availability: - Barbara is available at Golden Gate Park from 8:00 AM to 11:30 AM.

Arrival Time: - You arrive at Bayview at 9:00 AM.

Minimum Meeting Duration: - The meeting with Barbara must be at least 90 minutes long.

Sequence Requirements: - Travel (Bayview to Golden Gate Park) → Wait (if necessary, due to arriving before Barbara's availability ends) → Meeting (with Barbara)

Objective: Maximize the number of friends met (in this case, it's simplified to just meeting Barbara, so the objective is to determine IF a meeting is possible given the constraints).

Table 6.2: Example of constraint generation for the meeting planning benchmark from NATURAL PLAN.

Question: Two quantum states with energies E_1 and E_2 have a lifetime of 10^{-9} sec and 10^{-8} sec, respectively. We want to clearly distinguish these two energy levels. Which one of the following options could be their energy difference so that they can be clearly resolved?

- (A) 10^{-11} eV
 - (B) 10^{-8} eV
 - (C) 10^{-9} eV
 - (D) 10^{-4} eV
-

Constraints for Verifying the Energy Difference Solution for Resolvable Quantum States

This problem revolves around the energy-time uncertainty principle, which dictates the resolvability of energy levels given their lifetimes. To verify a proposed energy difference ($\Delta E = |E_2 - E_1|$) that allows clear resolution, we need constraints encompassing the following:

1. Uncertainty Principle Adherence:

Primary Constraint: The product of the uncertainty in energy (ΔE) and the uncertainty in time (Δt) must satisfy the energy-time uncertainty relation: $\Delta E \Delta t \geq \frac{\hbar}{2}$, where $\hbar$ is the reduced Planck constant.

Specific Constraint: Since we have two states, we must consider the *combined* uncertainty in time. We can approximate this as the average of the individual lifetimes: $\Delta t \approx \frac{\tau_1 + \tau_2}{2}$, where τ_1 and τ_2 are the lifetimes of the two states.

Operational Constraint: For "clear resolution," we typically require ΔE to be significantly greater than the uncertainty dictated by the uncertainty principle. A common rule of thumb is $\Delta E \geq \frac{\hbar}{\Delta t}$. A stricter constraint might use a larger factor (e.g., $\Delta E \geq \frac{2\hbar}{\Delta t}$) for even clearer resolution.

2. Units Consistency:

Unit Constraint: All calculations must use consistent units. Ensure that energy (ΔE), time (Δt), and $\hbar$ are expressed in compatible units (e.g., Joules and seconds, or eV and seconds). Clearly state the chosen unit system.

3. Numerical Accuracy:

Precision Constraint: Calculations should be performed with sufficient numerical precision to avoid rounding errors. Define the acceptable level of precision.

Constant Accuracy: Use a precise value for the reduced Planck constant ($\hbar$).

4. Solution Justification:

Explanation Constraint: The solution must not only provide a numerical value for ΔE but also clearly explain why this value satisfies the constraints and allows for clear resolution. This explanation should reference the uncertainty principle and the chosen resolution criterion.

5. Alternative Solutions Consideration (Optional):

Exploration Constraint: Explore whether multiple values of ΔE could satisfy the constraints. If so, discuss the trade-offs between different solutions.

Example Implementation of Constraints:

Let's assume we're working in SI units (Joules and seconds).

1. Uncertainty Principle: $\Delta E \geq \frac{\hbar}{[(10^{-9} \text{ s} + 10^{-8} \text{ s})/2]}$, where $\hbar \approx 1.054 \times 10^{-34}$ J s.
 2. Units: All calculations are in Joules and seconds.
 3. Numerical Accuracy: Calculations will be performed to at least 4 significant figures.
 4. Justification: The solution will explain how the calculated ΔE is significantly larger than the uncertainty, ensuring clear resolution.
-

Table 6.3: Examples of constraints generated by the constraint agent given reasoning question. The example illustrates the constraint generation for solving physics question from GPQA

Question: For a positive integer a , define a sequence of integers $x_1, x_2, \dots$ by letting $x_1 = a$ and $x_{n+1} = 2x_n + 1$ for $n \geq 1$. Let $y_n = 2^{x_n} - 1$. Determine the largest possible k such that, for some positive integer a , the numbers $y_1, \dots, y_k$ are all prime.

Constraints for Verifying the Solution Plan for Prime Sequence Problem

This problem requires a solution plan that likely involves a combination of mathematical reasoning and computational verification. Here's a breakdown of constraints to verify and evaluate such a plan:

I. Mathematical Reasoning Constraints:

1. **Correct Recurrence Relation:** The plan must correctly interpret and utilize the given recurrence relations for x_n and y_n . Verify that the plan doesn't misinterpret or modify these definitions.
2. **Mersenne Primes Understanding:** The plan should demonstrate an understanding that y_n are Mersenne numbers (numbers of the form $2^p - 1$). It should acknowledge that for y_n to be prime, x_n must itself be prime (though this is a necessary but not sufficient condition).
3. **Logical Flow and Justification:** The plan's reasoning should be clear, logical, and well-justified. Each step should follow from the previous ones, with clear explanations for any claims or deductions made. Avoid leaps of logic or unsubstantiated assertions.
4. **Handling of Base Case ($k = 1$):** The plan should explicitly address the trivial case where $k = 1$. Since $y_1 = 2^a - 1$, any prime value of 'a' will result in y_1 being a potential Mersenne prime. This confirms that $k \geq 1$.
5. **Strategy for Finding Maximum k :** The plan must outline a clear strategy for determining the *largest* possible k . This likely involves exploring increasing values of k and checking the primality of the resulting y_n sequence. The plan should explain how it avoids infinite search or unnecessary computations.
6. **Optimality Argument:** The plan should include an argument (or at least a strategy for developing an argument) for why the found value of k is indeed the *largest* possible. This might involve demonstrating that for $k + 1$, no suitable 'a' exists.

II. Computational Verification Constraints:

1. **Primality Testing Algorithm:** If the plan involves computational primality testing, it should specify the algorithm used (e.g., probabilistic tests like Miller-Rabin or deterministic tests like AKS). The plan should justify the choice of algorithm based on the size of the numbers being tested and the desired level of certainty.
2. **Correct Implementation:** If code is involved, it must be correct and free of errors. The plan should include testing procedures to ensure the code accurately implements the chosen primality test and the recurrence relations.
3. **Computational Resources:** The plan should consider the computational resources required for the primality testing. For large values of x_n , testing the primality of y_n can be computationally expensive. The plan should address how it manages these computational challenges.
4. **Range of 'a' considered:** The plan should specify the range of values for 'a' that will be explored. It should justify this range and explain why exploring values outside this range is unnecessary.
5. **Termination Condition:** The plan must have a clear termination condition for its computational search. It should explain how it determines when to stop searching for larger values of 'k'.

III. Presentation and Clarity Constraints:

1. **Clear and Concise Explanation:** The plan should be presented in a clear, concise, and easy-to-understand manner. Avoid unnecessary jargon or overly complex language.
2. **Well-Organized Structure:** The plan should be well-organized, with a logical flow of ideas. Use headings, subheadings, and bullet points to improve readability.
3. **Reproducibility:** The plan should be sufficiently detailed to allow another person to reproduce the results. All assumptions, parameters, and procedures should be clearly stated.

Table 6.4: Examples of constraints generated by the constraint agent given mathematical reasoning problem. The example illustrates the constraint generation for solving MATH question from OlympiadBench

standards, with a threshold (e.g., a score of 95 or higher) indicating a verified, high-quality plan.

Prompts for Verification Agent The prompt for the verification agent is designed to be task-agnostic, meaning it can be applied across different problem domains without modification. By enforcing strict evaluation criteria, this agent enhances the reliability of PlanGEN, making it robust for various planning and reasoning tasks. In this prompt, list of constraints are generated using constraint agent. Notably, the list of constraints used in the verification prompt is dynamically generated by the constraint agent. This ensures that the verification process is based on instance-specific constraints rather than relying on predefined, static rules.

Provide a reward score between -100 and 100 for the quality of the provided plan steps, using strict evaluation standards. Ensure the reward reflects how effectively the plan contributes to progressing toward the correct solution.

Problem Statement:

{problem}

Plan:

{plan}

Consider the following constraints while evaluating:

- [Constraint 1]
- [Constraint 2]
- [Constraint 3]

Provide feedback in the following format:

[Step-by-step reasoning for the reward score]

Score: [Strictly provide an integer reward score between -100 and 100]

Example for Verification Agent To illustrate the output of our verification agent, Table 6.5 presents representative examples of the verification process for NATURAL PLAN (calendar scheduling). This table highlights how the verification agent verifies the generated plan using constraints.

6.2.1.3 Selection Agent

The selection agent dynamically determines the most suitable inference algorithm for solving a given problem instance based on its complexity. It leverages a combination of historical performance, diversity, and recovery scores, and guidance from a LLM to adaptively select the best algorithm (among three) for the given instance. To create the selection agent, we utilize a modified UCB policy. The policy combines multiple factors, including normalized rewards, exploration bonuses, diversity adjustments, and recovery scores. Additionally, the agent incorporates LLM-guided priors, which provide algorithm suitability scores based on the problem statement, task requirements, and previous plan (if available). These priors enable the agent to align its selections with the input instance complexity and corresponding constraints, improving the relevance of the chosen algorithm.

$$\begin{aligned} \text{UCB}(a) = & \frac{R(a)}{N(a)} + \sqrt{\frac{2 \log(T+1)}{N(a)}} \\ & + \lambda_{\text{prior}} \cdot \text{Prior}(a) \\ & + \frac{\alpha_{\text{diversity}}}{N(a) + 1} \\ & + \alpha_{\text{recovery}} \cdot S_{\text{recovery}}(a) \end{aligned}$$

Modified UCB Policy equation combines several terms to balance exploitation and exploration when selecting the best algorithm for a given task instance. To design

Problem Statement: You are an expert at scheduling meetings. You are given a few constraints on the existing schedule of each participant, the meeting duration, and possibly some preferences on the meeting time. Note there exists a solution that works with existing schedule of every participant. TASK: You need to schedule a meeting for Michelle, Steven and Jerry for one hour between the work hours of 9:00 to 17:00 on Monday. Here are the existing schedules for everyone during the day: - Michelle has meetings on Monday during 11:00 to 12:00; - Steven has blocked their calendar on Monday during 9:00 to 9:30, 11:30 to 12:00, 13:30 to 14:00, 15:30 to 16:00; -Jerry has blocked their calendar on Monday during 9:00 to 9:30, 10:00 to 11:00, 11:30 to 12:30, 13:00 to 14:30, 15:30 to 16:00, 16:30 to 17:00; Find a time that works for everyone's schedule and constraints.
Instance-Specific Constraints and Details: Participants: Michelle, Steven, Jerry - The meeting must include all three. Meeting Duration: 1 hour - The meeting requires a contiguous 60-minute slot. Work Hours: 9:00 to 17:00 (Monday) - The meeting must start and end within these working hours. This acts as a global constraint. Individual Schedules (Existing Meetings/Blocked Times): - Michelle: Busy from 11:00 to 12:00 on Monday. - Steven: Busy from 9:00 to 9:30, 11:30 to 12:00, 13:30 to 14:00, and 15:30 to 16:00 on Monday. - Jerry: Busy from 9:00 to 9:30, 10:00 to 11:00, 11:30 to 12:30, 13:00 to 14:30, and 15:30 to 17:00 on Monday (note this constraint ends at the end of the workday). While the problem states "Note there exists a solution," this isn't a constraint in itself; it's a guarantee that a feasible solution exists given the constraints. The actual constraints are the schedules, meeting duration, and working hours.
Generated Plan: - Analyze Constraints: - Participants: Michelle, Steven, Jerry - Duration: 1 hour - Work hours: 9:00 - 17:00 - Day: Monday - Identify Individual Availability: - Michelle: 9:00-11:00, 12:00-17:00 - Steven: 9:30-11:30, 12:00-13:30, 14:00-15:30, 16:00-17:00 - Jerry: 9:30-10:00, 11:00-11:30, 12:30-13:00, 14:30-15:30, 16:00-16:30 - Determine Common Availability: Looking for 1-hour slots: - Comparing Michelle and Steven: 9:30-11:00, 12:00-13:30, 14:00-15:30, 16:00-17:00 - Comparing the above with Jerry: 14:30-15:30 - Prioritize Earliest Time Slot: The earliest 1-hour slot is 14:30-15:30 - Ensure Solution Exists: The slot 14:30-15:30 works for all three. - Strict Format Requirement: SOLUTION: Here is the proposed time: Monday, 14:30 - 15:30
Verification: The plan demonstrates significant improvement compared to the initial plan. It correctly identifies individual availabilities and successfully finds the two viable one-hour slots: 9:30-10:30 and 14:30-15:30. It avoids exploring irrelevant shorter slots. The plan is well-structured and clearly explains each step. It's on the verge of providing the correct solution. However, it has not yet explicitly stated the final answer (the prioritized earliest time slot) which is a key part of the task. Therefore, while much improved, it doesn't quite merit a top score due to this omission. Score: 95

Table 6.5: Examples of verification generated by the verification agent given a generated plan and constraints. This example illustrates the verification process for calendar scheduling from NATURAL PLAN

our selection agent, we conducted an ablation study evaluating modifications to the UCB formula, shown in Figure 6.2. Initially, we replaced the selection agent with a simple sequential strategy, termed “Multi-Agent (Sequential)”, where algorithms execute in sequence, and the verification agent selects the highest-scoring plan. Next, we implemented a UCB selection agent, but excluded the ‘diversity bonus’ and ‘recovery term’ introduced in our proposed formulation in the main paper, denoted as “Multi-Agent (UCB w/o div. and rec.)”. Finally, we implemented the complete selection agent incorporating our proposed UCB, labeled “Multi-Agent (UCB)”. As shown in Figure 6.2, the inclusion of the diversity bonus and recovery terms in the UCB formula (“Multi-Agent (UCB)”) resulted in $\sim 3.5\%$ performance gain compared to the UCB variant without these terms, further enhancing overall results. Note that the LLM-guided priors are still part of Multi-Agent (UCB w/o div. and rec.) and Multi-Agent (UCB).

All terms in the equation are calculated across one evaluation run. Here, the cost of calculation is negligible since it only utilizes reward values from previous runs, but only one LLM call is required to get a score for $\text{Prior}(a)$. The first term, $\frac{R(a)}{N(a)}$, represents the average reward for algorithm a , where $R(a)$ is the total reward accumulated by the algorithm, and $N(a)$ is the number of times the algorithm has been selected. This term ensures that algorithms with higher historical performance are prioritized. The second term, $\sqrt{\frac{2\log(T+1)}{N(a)}}$, serves as the exploration component, encouraging the selection of algorithms with fewer trials, denoted as T . This term ensures that under-explored options are adequately evaluated. Furthermore, $\lambda_{\text{prior}} \cdot \text{Prior}(a)$, which leverages LLM-guided priors to align algorithm selection with the instance-specific complexity. Here, λ_{prior} is a dynamically decaying weight defined as $\frac{\lambda_{\text{prior}}}{1+T}$, where T represents the total number of trials. This decay gradually shifts the focus from initial priors to historical performance as trials progress. The diversity

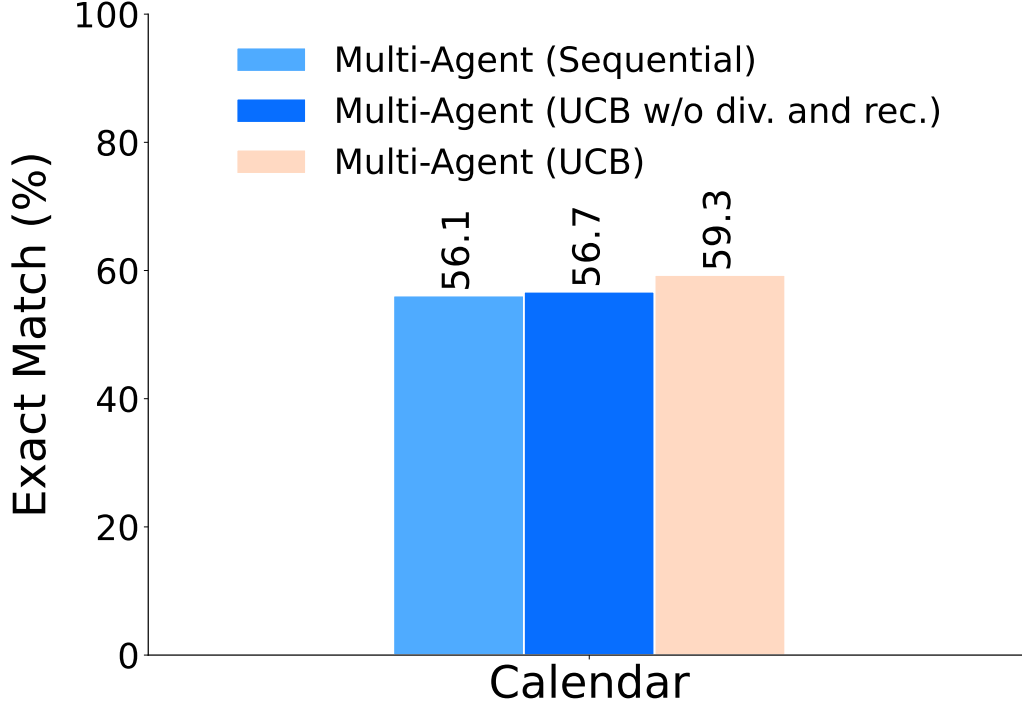


Figure 6.2: Ablation Study of UCB Modifications on Selection Agent and its impact on PlanGEN(Mixture of Algorithms). div.: diversity bonus, rec: recovery term.

bonus, $\frac{\alpha_{\text{diversity}}}{N(a)+1}$, penalizes overused algorithms, ensuring balanced exploration across all options. Finally, the recovery term, $\alpha_{\text{recovery}} \cdot S_{\text{recovery}}(a)$, rewards algorithms that recover effectively from failures, with $S_{\text{recovery}}(a)$ representing the recovery score for algorithm a .

Selection Process This process begins by initializing algorithm-specific variables, such as accumulated rewards, selection counts, and failure counts. The algorithm (Algorithm 1) presented is a modified UCB selection strategy that incorporates additional factors for exploration, diversity, and recovery. It initializes each algorithm with basic statistics like reward ($R(a)$), count of trials ($C(a)$), and recovery score ($Rec(a)$). The algorithm computes a normalized reward $\bar{R}_{\text{norm}}(a)$ for each option, balancing the reward with exploration ($E(a)$), which encourages trying less-used algorithms. A diversity bonus $D(a)$ penalizes overused algorithms, while a recovery

Algorithm 1 Selection using Modified UCB with LLM-Guided Priors

```
1: Initialize:  $R(a) \leftarrow 0$ ,  $C(a) \leftarrow 1$ ,  $Rec(a) \leftarrow 0$ ,  $F(a) \leftarrow 0$ ,  $D(a) \leftarrow 1$ ,  $T \leftarrow 0$ 
2: Set  $\lambda_{\text{prior}}$ ,  $\alpha_{\text{diversity}}$ ,  $\alpha_{\text{recovery}}$ 
3: Load LLM-guided priors
4: procedure SELECTALGORITHM( $args$ )
5:   Compute prior decay:  $\lambda_{\text{prior}} \leftarrow \frac{\lambda_{\text{prior}}}{1+T}$  ▷ Reduces as trials increase
6:   Set max exploration term  $M \leftarrow 5$ 
7:   Obtain LLM prior scores:  $LLM\_prior \leftarrow \text{LLM\_Guided\_Prior}(args)$ 
8:   Compute max reward:  $R_{\text{max}} \leftarrow \max(R(a))$  (set to 1 if all rewards are 0)
9:   for each algorithm  $a$  do
10:     Compute normalized reward:

$$\bar{R}_{\text{norm}}(a) \leftarrow \frac{R(a)}{C(a)R_{\text{max}}}$$

▷ Scales rewards between 0 and 1 for comparability
11:     Compute exploration term:

$$E(a) \leftarrow \min \left( \sqrt{\frac{2 \log(T+1)}{C(a)}}, M \right)$$

▷ Encourages trying less-used algorithms, capped at  $M$ 
12:     Compute diversity bonus:

$$D(a) \leftarrow \frac{\alpha_{\text{diversity}}}{C(a) + 1}$$

▷ Penalizes frequently used algorithms to encourage variety
13:     Compute recovery bonus:

$$RecB(a) \leftarrow \alpha_{\text{recovery}} \cdot Rec(a)$$

▷ Rewards algorithms that perform well after failures
14:     Compute final UCB score:

$$UCB(a) \leftarrow \bar{R}_{\text{norm}}(a) + E(a) + \lambda_{\text{prior}} LLM\_prior(a) + D(a) + RecB(a)$$

▷ Balances exploitation, exploration, diversity, and recovery
15:   end for
16:   Select best algorithm:

$$a^* \leftarrow \arg \max_a UCB(a)$$

17:   return  $(a^*, UCB(a^*))$ 
18: end procedure
```

bonus $RecB(a)$ rewards algorithms that perform well after prior failures. LLM-guided priors (LLM_prior) are used to influence the selection process based on prior knowledge. The final selection is made by maximizing the UCB score, which combines these factors to balance exploitation and exploration. The agent then incorporates LLM-guided priors to generate algorithm suitability scores based on the problem statement and any provided feedback. These priors are derived from a LLM, and serve as initial estimates to adjust the UCB [70] values.

Prompts for Selection Agent The prompt for the selection agent is task-agnostic, allowing it to be applied across various domains without modification. It processes feedback from the verification agent and contextual information from the problem statement to assign suitability scores to different inference-time algorithms.

Analyze the following planning problem and explain your reasoning for assigning priority scores to the algorithms based on their suitability. Scores should be between 0 and 1, where 1 represents the most suitable algorithm for the given problem.

Problem Statement: <problem statement>

Requirements: <feedback>

Context: <context if context else ‘None provided’>

Start by providing a brief reasoning for each algorithm’s suitability based on problem complexity. Then, **ONLY output your response strictly as a list** with the exact format below:

Reasoning:

- **Best of N:** [Explain why this algorithm is or isn’t suitable]
- **Rebase:** [Explain why this algorithm is or isn’t suitable]

- **ToT:** [Explain why this algorithm is or isn't suitable]

Scores:

```
[("Best of N", float),
 ("Rebase", float),
 ("ToT", float)]
```

6.2.2 Proposed Frameworks

Within PlanGEN, we propose four different frameworks: (1) PlanGEN (Best of $\mathcal{N}$) (Figure 6.3), (2) PlanGEN (ToT) (Figure 6.4), and (3) PlanGEN (REBASE) (Figure 6.5), and (4) PlanGEN (Mixture of Algorithms) (Figure 6.1). Here, we present a detailed description of each framework.

6.2.2.1 PlanGEN (Best of $\mathcal{N}$)

Motivated by Brown *et al.* [18], we adapted the Best of $\mathcal{N}$ algorithm and modified it using our constraint and verification agents as illustrated in Figure 6.3. The framework generates $\mathcal{N}$ candidate plans (Plan 1, Plan 2, ..., Plan n), and each plan is assessed by a verification agent based on a set of constraints. Then, a corresponding reward (Reward 1, Reward 2, ..., Reward n) gets assigned by the verification agent. Finally, the plan with the highest reward from generated candidates is selected, resulting in a plan that aligns with the problem constraints.

6.2.2.2 PlanGEN (ToT)

ToT algorithm has been studied in detail for solving many complex problems [289]. As shown in Figure 6.4, we modify the ToT algorithm with our constraint and verification agents. The method begins by initializing a root node that represents the problem and

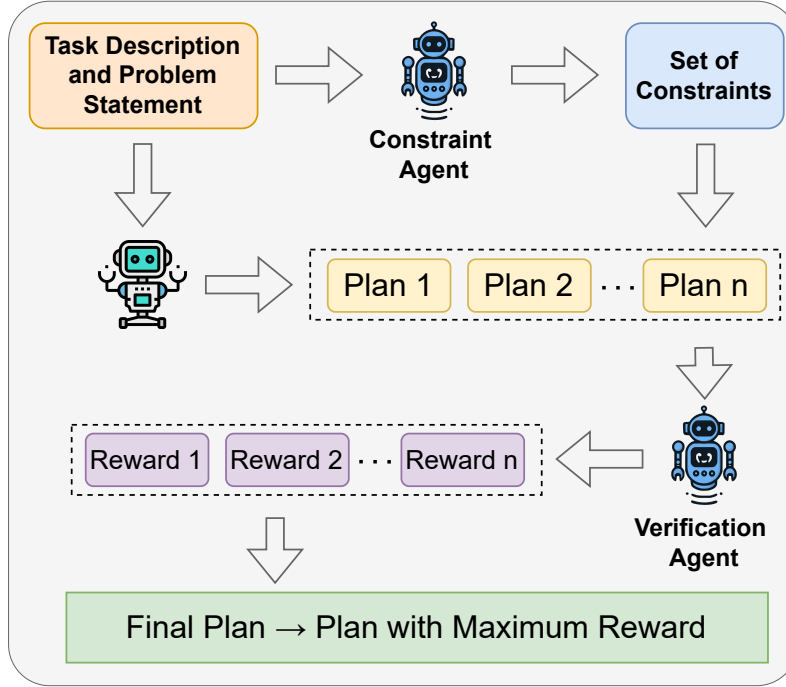


Figure 6.3: Schematic representation of PlanGEN (BoN).

generating multiple potential next steps, creating a tree-like structure. The generated steps are verified using a verification agent that assigns reward scores based on a set of constraints. The iterative process involves evaluating all possible steps at a given depth, selecting the most promising path based on reward scores, and expanding it further by generating new steps. This process continues until a valid solution is identified or a pre-defined limit on iterations is reached.

6.2.2.3 PlanGEN (REBASE)

The REBASE tree search method inherits the exploitation and pruning properties of tree search and is well-studied for mathematical reasoning [280]. As shown in Figure 6.5, the framework incorporates a dynamic selection and expansion strategy to iteratively refine solutions. At each depth of the tree, candidate nodes are ranked based on their assigned reward scores (obtained using a verification agent), ensuring

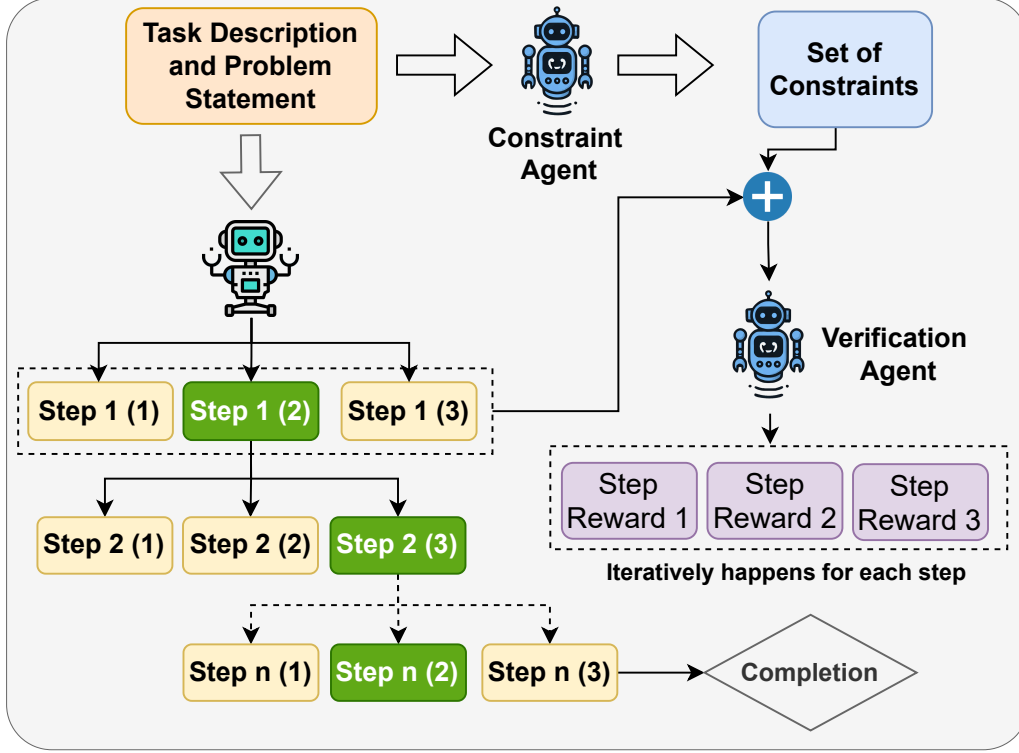


Figure 6.4: Schematic representation of PlanGEN (ToT). Highest-reward steps are highlighted in green.

that the most promising candidates are explored first. Even steps with lower rewards are considered, but with a reduced number of children, meaning that their exploration depth is limited. This hierarchical pruning helps maintain efficiency, thereby reducing unnecessary exploration of weaker nodes. This process continues until either a valid, complete solution is found or a predefined depth or width limit is reached. Also, there is a completion check similar to ToT, which identifies nodes that represent complete solutions, enabling REBASE to terminate early once a satisfactory outcome is identified.

Prompts used for ToT and REBASE PlanGEN (ToT) and PlanGEN (REBASE) employ three prompt types: (1) step prompt, (2) step reward prompt, and (3) completion prompt. Step prompt guides the model to generate subsequent steps based

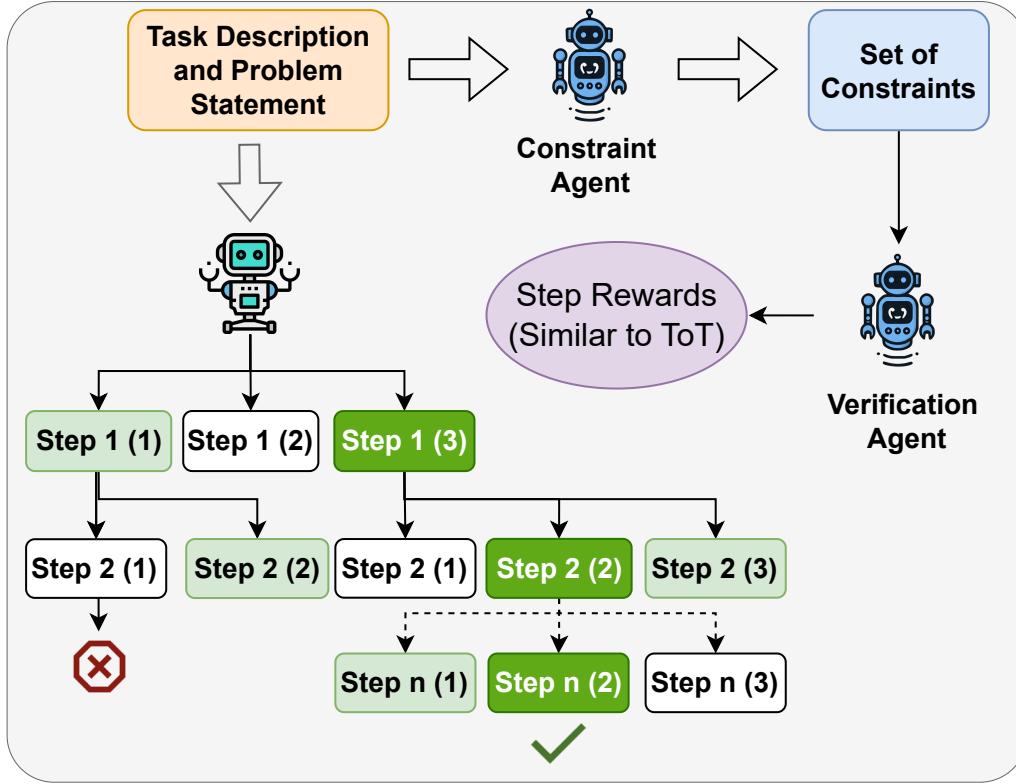


Figure 6.5: Schematic representation of PlanGEN (REBASE). Green shading indicates step reward (darker = higher). Darker steps are prioritized for exploration.

on the problem statement and previously generated steps. Step reward prompt evaluate each intermediate step against the problem statement and constraints, similar to the prompts used by a verification agent. Completion prompt check for a complete solution after each step. If a solution is found, exploration terminates; otherwise, the process continues until a solution is reached.

Step Prompt

You are an expert assistant for generating a step-by-step plan to solve a given question using specified tools. Given the problem and any intermediate steps, output only the next step in the plan. Ensure that the next action helps in moving toward the correct plan to solve the given question. Do not provide the full plan. Keep responses concise, focusing solely on the immediate next step that

is most effective in progressing toward the correct plan.

<problem>

{Add a problem statement here}

</problem>

<intermediate_step>

{Append previously generated steps}

</intermediate_step>

Completion Prompt

You are an assistant tasked with verifying if the final, complete plan to solve the given question has been achieved within the intermediate steps. Output only ‘1’ if the intermediate steps contain the full solution needed to solve the question. If the full plan has not yet been reached, output only ‘0’. Provide no additional commentary—return exclusively ‘1’ or ‘0’.

<problem>

{Add a problem statement here}

</problem>

<intermediate_step>

{Append previously generated steps}

</intermediate_step>

6.2.2.4 PlanGEN (Mixture of Algorithms)

The Mixture of Algorithms framework (Figure 6.1) introduces a selection agent (§6.2.1.3) which dynamically selects the best possible inference-time algorithms proposed in the above sections based on instance-level complexity. The framework operates in a modular and iterative manner, ensuring adaptability in addressing planning and reasoning problems of different complexities effectively.

Orchestration The process begins with generating an initial plan using LLM based on the task description and problem statement. Along with this, the constraint agent (§6.2.1.1) is employed to generate an instance-specific set of constraints. Based on the constraints, the verification agent (§6.2.1.2) evaluates the quality of the initial plan and provides a reward score (indicated as ‘Score’ in Figure 6.1). If the initial plan meets the required threshold (denoted T_h), it is acceptable as the “Final Plan”. Otherwise, the iterative refinement process begins.

Iterative Refinement The refinement loop is driven by a suite of inference algorithms as shown in Figure 6.1. During this iterative refinement, the selection agent (§6.2.1.3) determines the most suitable algorithm based on the instance-specific complexity and historical UCB values. The selected algorithm produces an updated plan, which is then re-evaluated by the verification agent. To ensure continual improvement, the framework incorporates feedback generated by a verification agent that provides guidance, and this feedback loop enables the system to refine the plan incrementally.

6.3 Experiments and Results

In this section, we provide details about datasets, experimental setups, and discussion around the main results.

6.3.1 Experimental Setup

Datasets To demonstrate improvement in natural planning abilities, we utilize the NATURAL PLAN [314]. After improving the planning, we show that this significantly enhances the reasoning capabilities of LLMs on two benchmarks: GPQA [198] and OlympiadBench (text-only) [75]. Additionally, we show that PlanGEN improves performance on a domain-specific dataset, DocFinQA [197]. For evaluation, we utilize evaluation sets of all four benchmarks. For NATURAL PLAN, we employed the provided evaluation sets, consisting of 1000 instances each for Calendar Scheduling and Meeting Planning, and 1600 instances for Trip Planning. The GPQA evaluation was conducted using the Diamond set, which comprises 198 highly challenging instances. From OlympiadBench, we selected the text-only problems, excluding those requiring a theorem prover, resulting in 674 instances for the MATH category and 236 for the PHY category. We also used 922 instances from the DocFinQA evaluation set.

Models Our primary evaluations use Gemini-1.5-Pro for all the experiments. We also present a case study with Gemini-2.0-Flash, Gemini-2.5-Pro, GPT-4o, and QwQ-32B (recent reasoning model) to showcase the model-agnostic nature and generalizability of PlanGEN. For all models, we utilize a checkpoint from January 2025.

Metrics We use task-specific metrics for all evaluations. Specifically, we use Exact Match (EM) for NATURAL PLAN similar to Zheng *et al.* [314], micro-average accuracy for OlympiadBench similar to He *et al.* [75], and accuracy for GPQA and DocFinQA (along with F1-Score for DocFinQA).

Baselines and Our Frameworks We develop two baselines: (i) Zero-shot CoT [108] and (ii) a Vanilla Multi-Agent Baseline. In the Zero-shot CoT, we provide an

input prompt to the model, which generates outputs in the form of `<CoT reasoning, Answer>`. For the “Multi-Agent Baseline”, the same model is called iteratively across multiple iterations. The system repeatedly refines its outputs through feedback loops, where the feedback is generated based on a self-reflective prompt designed to improve reasoning. We evaluate all proposed frameworks (§6.2.2) on all benchmarks. For reasoning tasks, we use a two-stage approach: (1) generating plan using `PlanGEN`, and (2) executing the plan to produce the final answer (Figure 6.1).

Hyper-parameters for Experiments To ensure deterministic behavior, we set the temperature of all models to 0 for each agent. For the inference-time algorithms, we used the following settings: `PlanGEN` (Best of $\mathcal{N}$) with five samples at a temperature of 0.7; Tree of Thoughts (ToT) with three children per root node, generated at a temperature of 0.7; and REBASE, initialized with width 10 at temperature of 0.7, decremented by 1 after each call to expand. For our experiments, we set $\lambda_{\text{prior}} = 10$, $\alpha_{\text{diversity}} = 1$, $\alpha_{\text{recovery}} = 1$, and capped the exploration term at $M = 5$ (retaining the standard 2 under the square root). All these hyperparameters are selected based on an empirical study.

Feedback prompt for Multi-Agent Baseline In the multi-agent baseline, we employ a feedback prompt to iteratively generate improved and refined outputs. The prompt is provided below:

Feedback Prompt

Analyze the following planning problem and explain your reasoning for assigning priority scores. You are an intelligent assistant capable of self-reflection and refinement. I will provide you with your last response, and your task is to improve it, if necessary.

Here is your previous response:

{previous_response}

Analyze and refine your response step-by-step:

1. Reflect on your reasoning process. Where might it be unclear or incorrect?

Improve it.

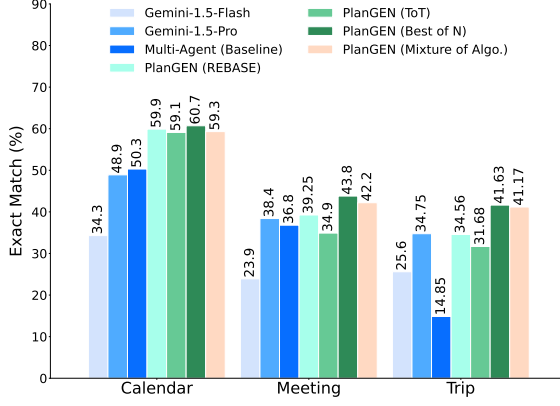
2. Revise the explanation to address any identified issues and make it more logical and comprehensive.

3. Ensure the final answer is correct, supported by clear reasoning.

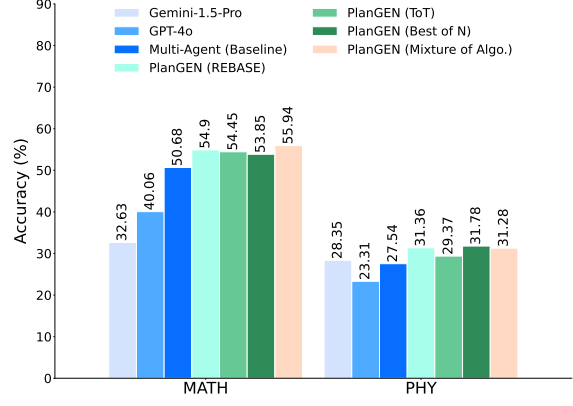
6.3.2 Main Results

Figure 6.6 compares the performance of PlanGEN frameworks across various baselines (varies across benchmarks - some single-agent baselines for GPQA are obtained from <https://klu.ai/glossary/gpqa-eval>), showing that multi-agent frameworks are consistently outperforming the baselines.

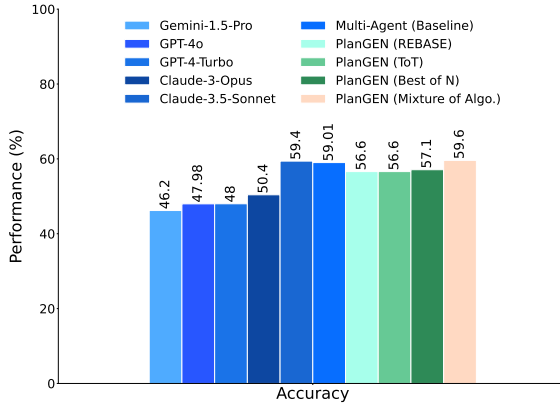
Performance on NATURAL PLAN From Figure 6.3.2, PlanGEN (Best of $\mathcal{N}$) achieves the highest EM scores across all tasks: 60.70 (Calendar), 43.80 (Meeting), and 41.63 (Trip). In calendar scheduling, all four frameworks surpass the strongest baseline (Multi-Agent) by $\sim 10\%$. For meeting and trip planning, all except ToT outperform the best baseline (Gemini-1.5-Pro) by $\sim 6\%$ and $\sim 7\%$, respectively. PlanGEN (Mixture of Algo.) achieves the second-highest performance in meeting and trip planning while remains competitive in calendar scheduling. These results demonstrate the effectiveness of our frameworks in handling diverse natural language planning tasks and establishing SOTA for all three categories of NATURAL PLAN.



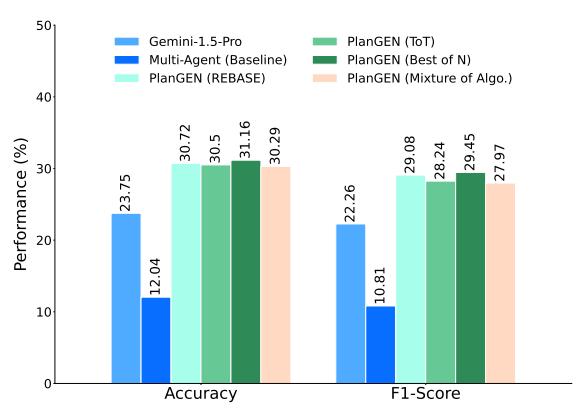
(a) NATURAL PLAN



(b) OlympiadBench



(c) GPQA



(d) DocFinQA

Figure 6.6: Performance comparison of the proposed multi-agent frameworks against baselines across four benchmarks. All experiments are conducted using Gemini-1.5-Pro. Algo: Algorithms, PHY: Physics.

Performance on OlympiadBench From Figure 6.3.2, PlanGEN (Mixture of Algo.) achieves the highest accuracy in the MATH (55.94%), outperforming the Multi-Agent Baseline (50.68%) by $\sim 5\%$. The superior performance of the PlanGEN in MATH highlights its effectiveness in complex mathematical reasoning, setting a SOTA for the MATH. In the PHY, all PlanGEN frameworks surpass Gemini-1.5-Flash (strongest baseline), with PlanGEN (Best of $\mathcal{N}$) achieving the highest accuracy (31.78%), setting a SOTA for the PHY.

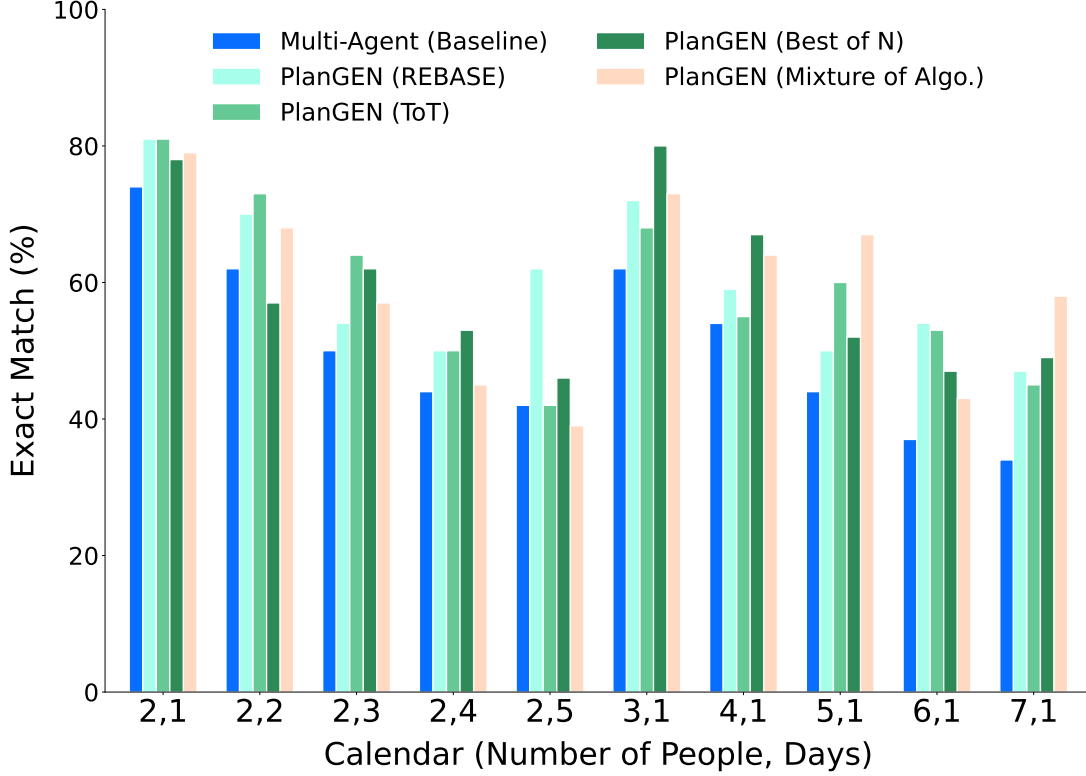


Figure 6.7: Performance comparison of inference-time algorithms across different complexity levels for calendar scheduling from NATURAL PLAN.

Performance on GPQA From Figure 6.3.2, the PlanGEN (Mixture of Algo.) achieves the highest accuracy (59.6%). The individual inference-time algorithms achieve a lower performance, indicating the usefulness of selection. All proposed frameworks outperform Gemini-1.5-Pro (46.2%), GPT models ($\sim 48\%$), and Claude-3-Opus (50.4%) by a large margin. While Claude-3.5-Sonnet, and Multi-Agent Baseline perform competitively ($\sim 59\%$) compared to PlanGEN (Mixture of Algo.).

Performance on DocFinQA From Figure 6.3.2, our frameworks significantly improve performance on DocFinQA, with PlanGEN (Best of $\mathcal{N}$) achieving the highest accuracy (31.16%) and F1-Score (29.45%), setting SOTA for the task. All our frameworks outperform the Gemini-1.5-Pro (strong baseline) by $\sim 7\%$. These results highlight the effectiveness of PlanGEN in performing financial reasoning.

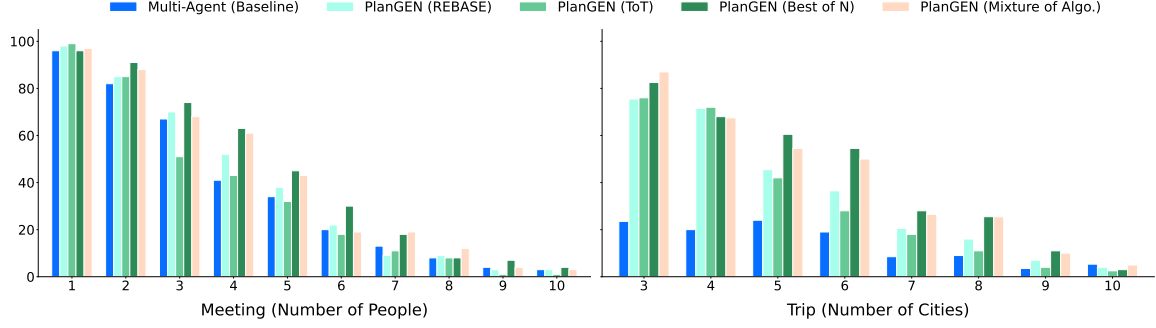


Figure 6.8: Performance comparison of inference-time algorithms across different complexity levels for meeting and trip planning from NATURAL PLAN

Performance of PlanGEN w.r.t. different complexity As shown in Figure 6.7, we conduct a case study on calendar scheduling tasks from NATURAL PLAN to analyze the impact of varying complexity levels on the performance of different frameworks. For the calendar scheduling, we observe that PlanGEN (ToT) performs best for simple problems, while PlanGEN (Best of $\mathcal{N}$) is more effective for intermediate problems. As complexity increases, a PlanGEN (Mixture of Algo.) proves to be the most effective approach. We further conduct a similar analysis for meeting and trip planning from NATURAL PLAN.

From Figure 6.8, in the meeting planning, PlanGEN (Best of $\mathcal{N}$) excels in both simple and intermediate problems, whereas a PlanGEN (Mixture of Algo.) performs better for complex problems. The trip planning presents a different trend, where PlanGEN (Best of $\mathcal{N}$) and a PlanGEN (Mixture of Algo.) consistently outperform other approaches across all complexity levels. Nonetheless, in very complex problems for meeting and trip planning, all algorithms exhibit poor performance.

Main Findings Compared to single-agent systems, PlanGEN consistently outperforms in generating better planning trajectories (Figure 6.6). Furthermore, Multi-Agent (Baseline) is not always the strongest benchmark, as self-correction can in-

introduce challenges, as shown in Huang *et al.* [83]. Thus, different agents within the system require distinct handling strategies similar to our **PlanGEN**. Additionally, even in **PlanGEN** frameworks, relying on a single inference-time algorithm proves insufficient for more complex problems (Figure 6.7). A **PlanGEN** (Mixture of Algo.) offers substantial advantages for solving complex reasoning problems, highlighting the importance of algorithm selection based on instance-specific complexity (Figure 6.1).

Discussion on LLM calls *vs.* Performance (%) Figure 6.9 shows the relationship between the number of LLM calls and task performance across baselines (single-agent and multi-agent) and proposed frameworks, using OlympiadBench (MATH category). The single-agent system, zero-shot CoT, requires only one LLM call. The multi-agent baseline requires the same number of calls as **PlanGEN** (Best of $\mathcal{N}$), but our framework outperforms the multi-agent baseline. For **PlanGEN** (ToT) and **PlanGEN** (REBASE), we focus on LLM calls during the tree expansion phase. **PlanGEN** (ToT) involves dynamic exploration, where each explored path requires three LLM calls: step generation, reward evaluation, and completion verification. The total cost is the per-path cost multiplied by the number of paths explored, constrained by either the number of steps generated for each problem or a predefined iteration budget (i.e., 20). For **PlanGEN** (REBASE), the number of LLM calls depends on the search width (i.e., 10). Each solution path expansion involves three calls: step generation, quality evaluation, and completion verification, thus giving a maximum of 30 LLM calls for a single problem. For **PlanGEN** (Mixture of Algo.), we estimate the average LLM calls by summing the estimated calls for each selected algorithm per problem, then dividing by the total number of problems. As shown in Figure 6.9, the single-agent system exhibits the lowest performance despite requiring just one LLM call. Multi-agent approaches show improved performance, with **PlanGEN** (ToT) and **PlanGEN**

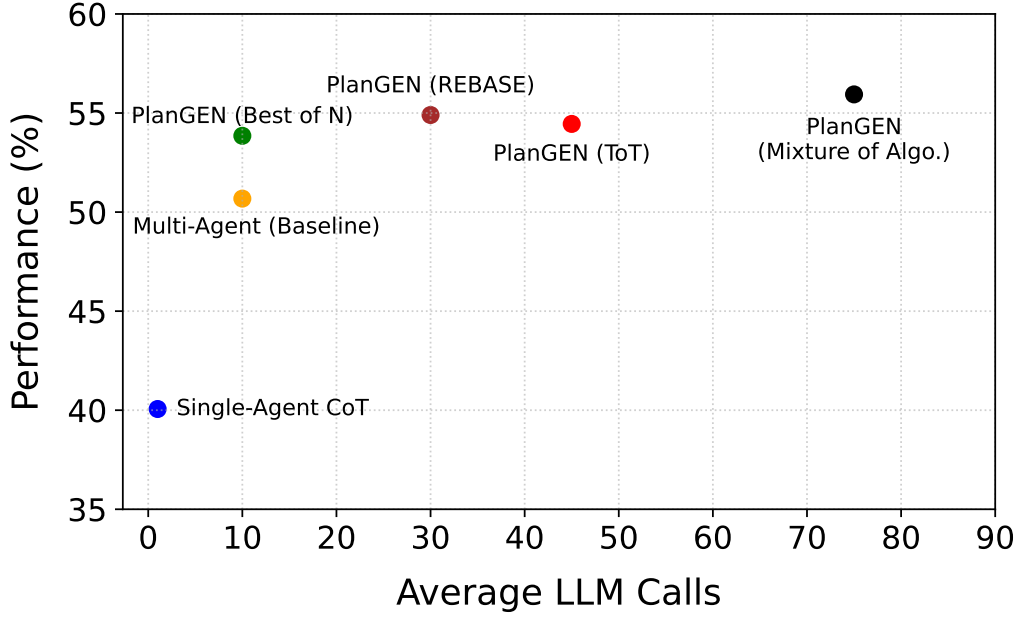


Figure 6.9: Comparison of baselines and our frameworks, showing the trade-off between LLM calls and performance (%) for OlympiadBench (MATH).

(REBASE) balancing LLM call efficiency and accuracy. The **PlanGEN** (Mixture of Algo.) method achieves the highest performance, suggesting that combining diverse planning strategies enhances efficiency.

Figure 6.10 shows the relationship between the time for running per instance and task performance across baselines (single-agent and multi-agent) and proposed frameworks, using OlympiadBench (MATH category). Since the algorithms used with the **PlanGEN** framework are dynamic and abstract from transient issues like the existing quota limit and retrying, we calculate the average seconds per instance based on the latency of a single call (denoted as `latency_of_single_call`). For example, for a method that supports running LLM calls in parallel, the total time for 10 calls would be approximately $1 \times \text{latency_of_single_call}$. In contrast, for methods that process calls sequentially, the total time would be closer to $10 \times \text{latency_of_single_call}$. This shows a similar trend to Figure 6.9.

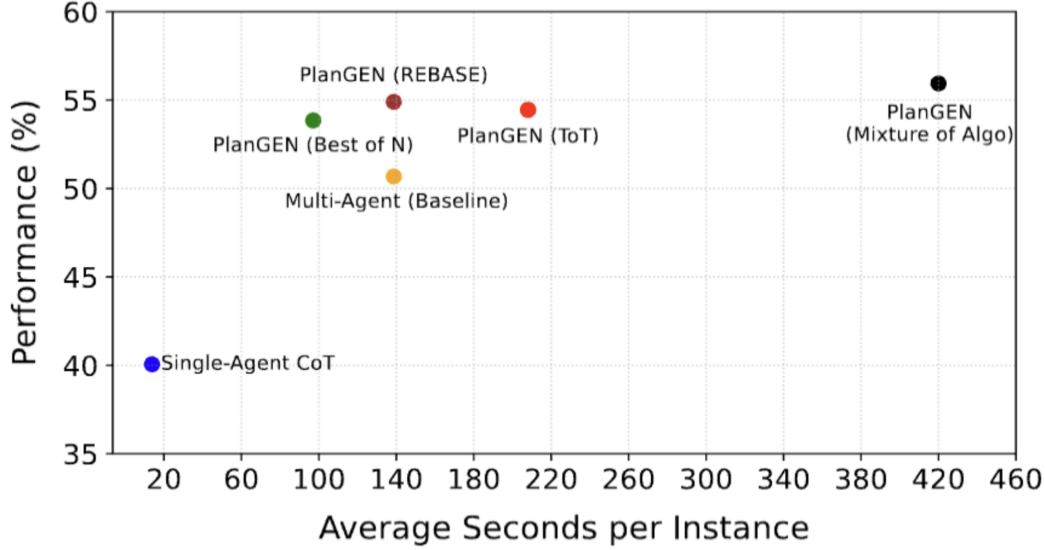


Figure 6.10: Comparison of baselines and our frameworks, showing the trade-off between time (seconds) and performance (%) for OlympiadBench (MATH).

6.4 Analysis and Discussion

Importance of Verification Agent Figure 6.11 demonstrates the verification agent’s crucial role in PlanGEN by showing a strong correlation between assigned reward values and prediction correctness (1 for correct, 0 for incorrect). The plotted points represent the average correctness rate for data buckets of varying reward values, each bucket containing hundreds of samples. A logistic regression model trained on DocFinQA and GPQA data (~ 1100 total samples) reveals a sigmoidal trend: higher rewards correlate with increased success probability, highlighting the agent’s effectiveness. This reinforces the importance of constraint-guided verification for improving inference-time algorithms.

As shown in Figure 6.12, the kernel density estimation (KDE) plot visualizes the distribution of reward values assigned to two distinct outcomes: “Success” (green) and “Failure” (red). The plot reveals a clear separation between the reward distributions, with “Success” outcomes strongly associated with high reward values (around 80-100) and “Failure” outcomes primarily associated with low reward values (around

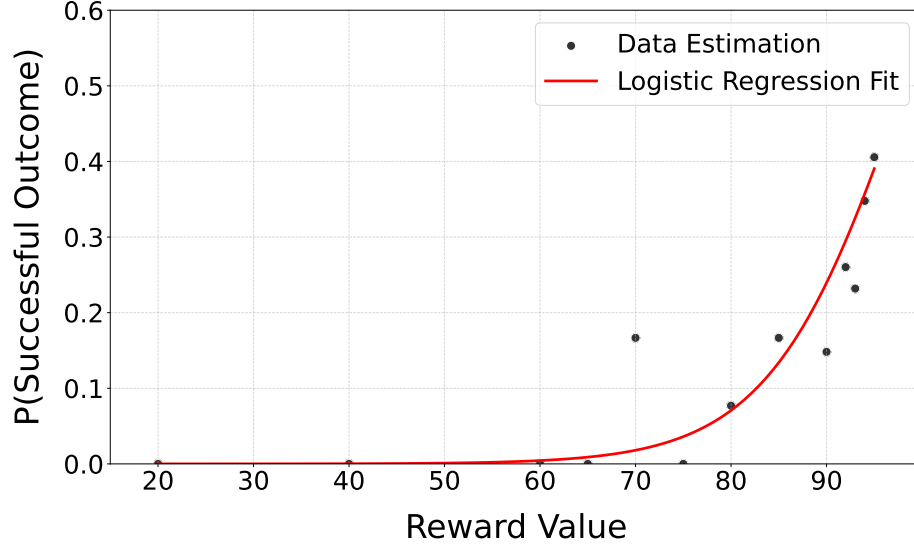


Figure 6.11: Logistic regression plot showing verification agent’s positive performance impact. $P(\text{Successful Outcome})$ = probability of prediction being correct.

20-40). The sharply peaked green curve suggests consistent and high rewards for successful outcomes, while the broader red curve reflects more variability in rewards assigned to failures. However, a small bump in the red curve at high reward values (around 80-90) suggests a few instances where failures received unexpectedly high rewards, warranting further investigation. This observation is further supported by a statistically significant difference between the reward distributions, a Mann-Whitney U test ($U = 116128.0$, $p < 0.0001$). The low p-value ($3.42\text{e-}09$) provides evidence that the difference in reward distributions is statistically significant.

Importance of Selection Agent Figure 6.13 illustrates the importance of the selection agent by comparing the performance on the NATURAL PLAN. Here, Multi-Agent (Ver.) includes only the verification agent, while Multi-Agent (Ver. + Selection) further includes a selection agent. The results highlight the progressive impact of these components. For example, in calendar scheduling, Multi-Agent (Ver.) improves performance to 56.1 EM compared to Multi-Agent (Baseline). However, Multi-Agent

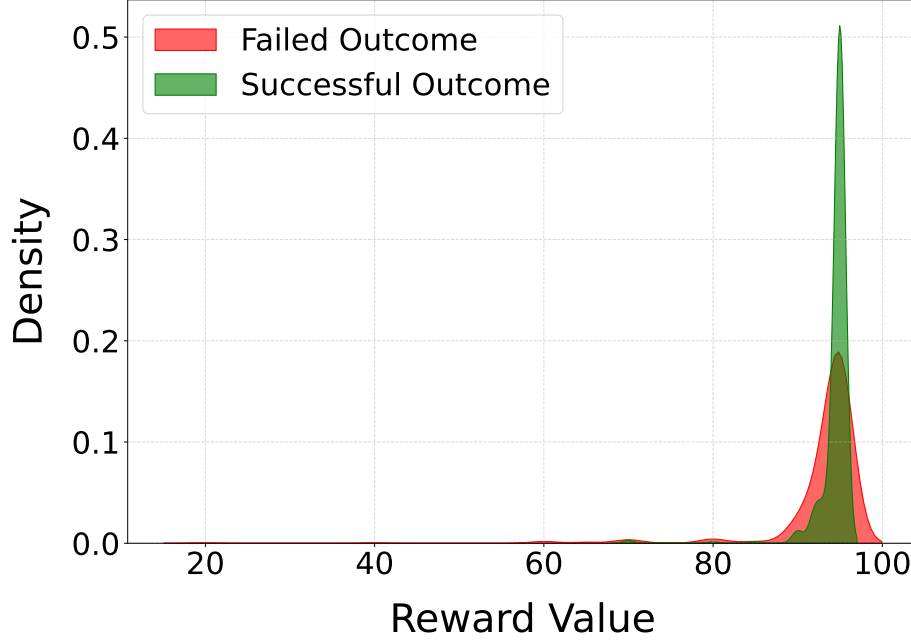


Figure 6.12: KDE plot illustrating the relationship between reward value and outcome (success/failure).

(Ver. + Selection) achieves 59.3 EM, demonstrating the additional benefit of algorithm selection. A similar trend is observed in trip planning, where Multi-Agent (Ver. + Selection) outperforms Multi-Agent (Ver.) (41.17 EM vs. 35.44 EM) and the Multi-Agent (Baseline). For meeting planning, Multi-Agent (Ver.) achieves 43.1 EM compared to 36.8 EM of Multi-Agent (Baseline), whereas Multi-Agent (Ver. + Selection) achieves competitive performance. Together, verification and selection agents drive significant improvements over single-agent and multi-agent baselines.

Model-Agnostic Nature The results from Table 6.6 demonstrate the model-agnostic nature of PlanGEN frameworks. While the primary experiments were conducted using Gemini-1.5-Pro, the framework’s effectiveness holds across different underlying models, such as Gemini-2.5-Pro, Gemini-2.0-Flash, and GPT-4o. For instance, in the NATURAL PLAN (calendar scheduling), the PlanGEN (Best of $\mathcal{N}$) framework achieves a significant improvement, reaching 68.90 EM, outperforming

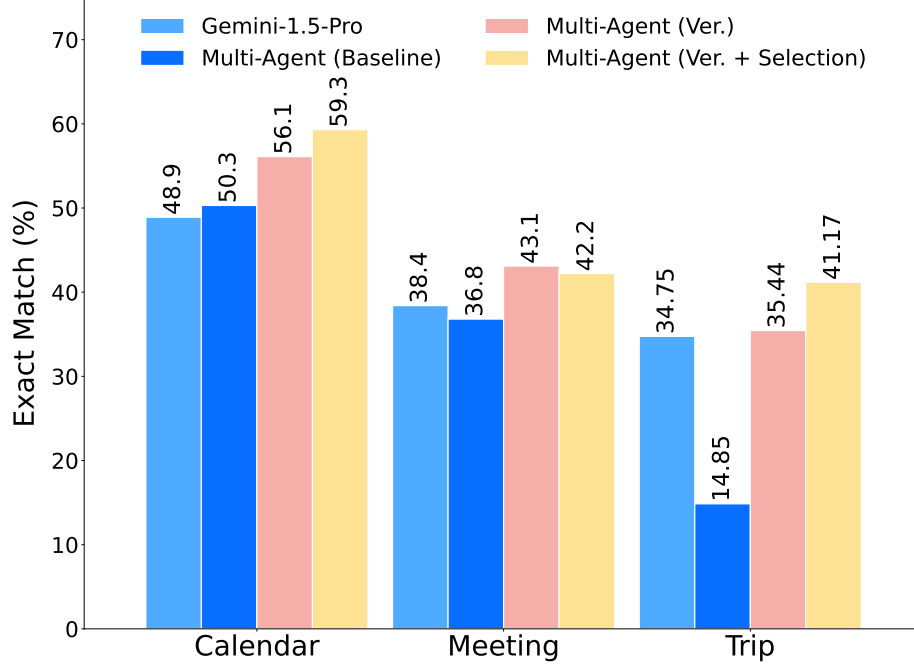


Figure 6.13: Case study on NATURAL PLAN, showing the impact of selection agent. Ver.: Verification

Gemini-2.0-Flash (61.10 EM). Similarly, in OlympiadBench, the **PlanGEN** (Mixture of Algo.) achieves the highest scores in MATH (64.10) and PHY (37.29), surpassing Gemini-2.0-Flash (52.13 MATH, 27.54 PHY). Note that the Mixture of Algo. outperforms the other three frameworks, showing the effectiveness of the selection agent. On GPQA, Mixture of Algo. (49.40), **PlanGEN** (REBASE) (64.14), and **PlanGEN** (Best of $\mathcal{N}$) (77.27) outperform GPT-4o (47.98), Gemini-2.0-Flash (60.10), and Gemini-2.5-Pro (53.03), respectively. These results highlight that regardless of the underlying model, our frameworks consistently enhance performance by leveraging multi-agent collaboration.

Different hyper-parameters of inference-time algorithms vs. their performance We conduct a case study on OlympiadBench, where we analyze the impact of varying hyperparameters on the performance of different inference-time algorithms. The results (Table 6.7) indicate that while increasing the number of samples (Best of

Methods	GPQA	NATURAL PLAN (Calendar)	OlympiadBench MATH PHY	Methods	GPQA	NATURAL PLAN (Calendar)	OlympiadBench MATH PHY	Methods	GPQA	Methods	GPQA
Gemini-1.5-Pro	46.20	48.90	32.63 28.35	Gemini-2.0-Flash	60.10	61.10	51.13 27.54	GPT-4o	47.98	Gemini-2.5-Pro	53.03
PlanGEN (BoN) Gemini-1.5-Pro	56.60	60.70	53.85 31.78	PlanGEN (BoN) Gemini-2.0-Flash	56.83	68.90	59.90 35.60	PlanGEN (BoN) GPT-4o	40.40	PlanGEN (BoN) Gemini-2.5	77.27
PlanGEN (ToT) Gemini-1.5-Pro	56.60	59.10	54.45 29.37	PlanGEN (ToT) Gemini-2.0-Flash	59.18	62.30	60.30 35.70	PlanGEN (ToT) GPT-4o	46.70	PlanGEN (ToT) Gemini-2.5-Pro	75.25
PlanGEN (REBASE) Gemini-1.5-Pro	57.10	59.90	54.90 31.36	PlanGEN (REBASE) Gemini-2.0-Flash	64.14	61.50	60.98 36.02	PlanGEN (REBASE) GPT-4o	41.40	PlanGEN (REBASE) Gemini-2.5-Pro	71.72
PlanGEN (MoA) Gemini-1.5-Pro	59.60	59.30	55.94 31.28	PlanGEN (MoA) Gemini-2.0-Flash	63.64	66.55	64.10 37.29	PlanGEN (MoA) GPT-4o	49.40	PlanGEN (MoA) Gemini-2.5-Pro	68.19

Table 6.6: Performance comparison for model-agnostic nature of PlanGEN. We utilize Gemini-1.5-Pro, Gemini-2.0-Flash, GPT-4o, and Gemini-2.5-Pro as baseline and underlying models in PlanGEN frameworks. Comparing methods that use the same base and underlying model for a fair assessment. MoA: Mixture of Algorithms.

$\mathcal{N}$), steps (ToT), or refinements (REBASE) lead to marginal improvements, the overall differences remain relatively small. Given this, we opted for lower hyperparameter values across all inference-time algorithms to balance efficiency and performance.

Frequency of inference-time algorithm selection across datasets For the PlanGEN (Mixture of Algo.) method, we analyze how frequently each inference-time algorithm (Best of $\mathcal{N}$, ToT, and REBASE) is selected across different datasets. The results (shown in Table 6.8) show that PlanGEN (ToT) is the most frequently chosen algorithm in NATURAL PLAN, OlympiadBench, and GPQA, indicating its effectiveness in these domains. In contrast, for DocFinQA, PlanGEN (Best of $\mathcal{N}$) is the dominant choice, suggesting that its strategy aligns better with financial reasoning tasks. PlanGEN (REBASE) is selected the least across all datasets, implying that its refinements are less favored by the selection mechanism. These findings highlight the dataset-dependent nature of inference-time algorithm effectiveness and the adaptability of the mixture approach in dynamically choosing the most suitable method.

PlanGEN with Recent Reasoning Models We conducted a case study using the latest QwQ-32B model [238] on GPQA. In a zero-shot setting, it achieves 65.15%

Methods	OlympiadBench	
	MATH	PHY
PlanGEN (Best of $\mathcal{N}$) (5)	53.26	32.63
PlanGEN (Best of $\mathcal{N}$) (10)	54.90	31.36
PlanGEN (Best of $\mathcal{N}$) (20)	53.22	29.38
PlanGEN (ToT) (3)	52.97	31.36
PlanGEN (ToT) (5)	55.20	32.05
PlanGEN (ToT) (10)	55.79	32.52
PlanGEN (REBASE) (10)	54.45	31.78
PlanGEN (REBASE) (20)	54.45	29.37
PlanGEN (REBASE) (30)	55.04	30.28

Table 6.7: Performance impact of hyper-parameters on inference-time algorithms in OlympiadBench

Frameworks	NATURAL PLAN	OlympiadBench	GPQA	DocFinQA
PlanGEN (BoN)	19.55%	7.09%	8.56%	81.03%
PlanGEN (ToT)	68.85%	90.09%	85.59%	12.5%
PlanGEN (REBASE)	11.6%	2.82%	5.86%	6.47%

Table 6.8: Algorithm Selection Frequency by Dataset

accuracy. When plugged into our PlanGEN frameworks, its performance further improves—reaching 68.01% under PlanGEN (Best of $\mathcal{N}$), 70.00% under PlanGEN (ToT), and 60.71% under PlanGEN (REBASE)—demonstrating that PlanGEN consistently boosts even state-of-the-art reasoning models. Hence, we can say that PlanGEN is model-agnostic (Table 6.6) and compatible with recent reasoning models.

PlanGEN vs. BOT We incorporate Buffer-of-Thoughts (BOT) [287]—a recent reasoning method—into our evaluation. We implement BOT using Gemini-1.5-Pro (to

Dataset	Baseline	Best Framework
GPQA	58.08%	59.60%
OlympiadBench(PHY)	25.85%	31.78%

Table 6.9: Comparison between a constraints-based CoT baseline and best-performing PlanGEN framework.

match our PlanGEN setup), and we measured accuracy across four benchmarks. BOT scores 55.50% on GPQA, 52.96% on MATH, and 27.11% on PHY. As shown in Figure 6.6, all of our PlanGEN variants outperform BOT by $\sim 4\%$ on GPQA and OlympiadBench. These results show that PlanGEN remains superior to state-of-the-art single-agent reasoning methods.

Constraints-based CoT *vs.* PlanGEN This baseline first prompts to generate constraints, then verifies them, and finally revises the final solution, *without* employing independent agents for these stages, similar to PlanGEN. We evaluated this baseline on two datasets, GPQA and OlympiadBench (PHY). For fair comparison we used the same model as in our main results (Gemini-1.5-Pro; Figure 6.6). As shown in Table 6.9, our full framework, which assigns dedicated agents for constraint extraction, verification, and strategy selection, consistently outperforms baseline on both benchmarks. These results indicate that explicit modularization of planning steps is more effective than a monolithic iterative revision approach. For creating baseline, prompts for each stage are used as shown in §6.2.

Performance of PlanGEN on Blocksworld (Plan Generation) We evaluated our approach on the *Blocksworld* domain (versions v1 and v3) for the *Plan Generation* category from Valmeekam *et al.* [249]. In PlanBench, the Plan Generation task requires generating a valid sequence of grounded actions that transforms

Method	Blocksworld (v1)	Blocksworld (v3)
Gemini-2.5-Flash	88.40%	92.93%
PlanGEN (BoN)	85.40%	95.96%
PlanGEN (REBASE)	95.54%	98.98%

Table 6.10: Accuracy on blocksworld v1 and v3 across different methods for plan generation task.

a given initial state into a specified goal state while satisfying all domain constraints. For experiments, we utilize default prompts from Valmeekam *et al.* [249]. The results, summarized in Table 6.10, compare a strong LLM baseline (Gemini-2.5-Flash) with two variants of our framework: PlanGEN(Best of $\mathcal{N}$) and PlanGEN(REBASE). As shown in Table 6.10, PlanGEN(REBASE)¹ achieves the highest accuracy across both Blocksworld versions, outperforming Gemini-2.5-Flash by about 7.1% points on v1 and 6.0% points on v3. Even the simpler PlanGEN(Best of $\mathcal{N}$) variant surpasses Gemini-2.5-Flash on the more challenging v3, highlighting the value of diverse plan exploration. These results demonstrate that PlanGEN’s structured, multi-agent approach—combining diverse plan sampling with iterative verification and constraint enforcement—substantially mitigates the weaknesses of standard LLMs identified by PlanBench and yields robust improvements.

Examples of Generated Plans To demonstrate the plan generation process, Table 6.11, Table 6.12, Table 6.13, Table 6.14, and Table 6.15 present example plans for NATURAL PLAN, GPQA, DocFinQA, and OlympiadBench. Generated using PlanGEN (Best of $\mathcal{N}$), these tables highlight the varied nature of plans produced across different task types. For GPQA, DocFinQA, and OlympiadBench (i.e., downstream

¹Results on REBASE are calculated for only 315 samples for v1 due to resource constraints, whereas we utilize all 500 samples for v1 and 100 samples for v3.

reasoning tasks), the examples additionally illustrate how these generated plans are executed to derive the final answer.

6.5 Conclusions

In this work, we proposed **PlanGEN**, an easily scalable multi-agent approach incorporating three key components: constraint, verification, and selection agents. We leveraged these agents to improve the verification process of existing inference algorithms and proposed three frameworks: Multi-Agent Best of $\mathcal{N}$, ToT, and REBASE. Further, we introduced a Mixture of Algorithms, an iterative framework that integrates the selection agent (Figure 5.1) to dynamically choose the best algorithm. We evaluated our frameworks on NATURAL PLAN, OlympiadBench, GPQA, and DocFinQA. Experimental results demonstrate that **PlanGEN** outperforms strong baselines, achieving SOTA results across datasets. Furthermore, our findings suggest that the proposed frameworks are scalable and generalizable to different LLMs, improving their natural language planning ability.

Despite the strong performance of our frameworks, an area of improvement is the reliance on predefined heuristics for selecting inference-time algorithms, which may not always generalize optimally across all tasks and domains. Additionally, while our frameworks demonstrate strong performance, their computational overhead could be further optimized for efficiency in real-world applications. We believe that our frameworks can be useful in further boosting the planning and reasoning capabilities of existing models such as o1 and Gemini-thinking. In addition, the use of RL techniques to dynamically adapt agent strategies based on task complexity could be an interesting area to explore. Moreover, broadening the scope to multi-modal reasoning would significantly expand the applicability of our approach, and exploring the use of generated planning trajectories for model training offers valuable direction.

Category: Calendar Scheduling

You are an expert at scheduling meetings. You are given a few constraints on the existing schedule of each participant, the meeting duration, and possibly some preferences on the meeting time. Note there exists a solution that works with existing schedule of every participant.

TASK: You need to schedule a meeting for Michelle, Steven and Jerry for one hour between the work hours of 9:00 to 17:00 on Monday.

Here are the existing schedules for everyone during the day:

- Michelle has meetings on Monday during 11:00 to 12:00; - Steven has blocked their calendar on Monday during 9:00 to 9:30, 11:30 to 12:00, 13:30 to 14:00, 15:30 to 16:00; - Jerry has blocked their calendar on Monday during 9:00 to 9:30, 10:00 to 11:00, 11:30 to 12:30, 13:00 to 14:30, 15:30 to 16:00, 16:30 to 17:00;

Find a time that works for everyone's schedule and constraints.

1. Analyze Constraints:

- Participants: Michelle, Steven, Jerry - Duration: 1 hour - Work hours: 9:00 - 17:00 - Day: Monday

2. Identify Individual Availability:

- Michelle: 9:00-11:00, 12:00-17:00 - Steven: 9:30-11:30, 12:00-13:30, 14:00-15:30, 16:00-17:00 - Jerry: 9:30-10:00, 11:00-11:30, 12:30-13:00, 14:30-15:30, 16:00-16:30

3. Determine Common Availability: Looking for 1-hour slots:

- Comparing Michelle and Steven: 9:30-11:00, 12:00-13:30, 14:00-15:30, 16:00-17:00 - Comparing the above with Jerry: 14:30-15:30

4. Prioritize Earliest Time Slot: The earliest 1-hour slot is 14:30-15:30

5. Ensure Solution Exists: The slot 14:30-15:30 works for all three.

6. Strict Format Requirement:

SOLUTION: Here is the proposed time: Monday, 14:30 - 15:30

Table 6.11: Example of generated plan for the calendar scheduling category from NATURAL PLAN.

Category: Meeting Planning

You are visiting San Francisco for the day and want to meet as many friends as possible. Solve the problem by considering various different schedules and picking the best one to optimize your goals.

Travel distances (in minutes):

Bayview to Golden Gate Park: 22.

Golden Gate Park to Bayview: 23.

CONSTRAINTS: You arrive at Bayview at 9:00AM. Barbara will be at Golden Gate Park from 8:00AM to 11:30AM. You'd like to meet Barbara for a minimum of 90 minutes.

You leave Bayview at 9:00 AM and travel 22 minutes to Golden Gate Park, arriving at 9:22 AM.

You meet Barbara from 9:22 AM – 10:52 AM, fulfilling the 90-minute requirement.

Table 6.12: Example of generated plan for the meeting planning category from NATURAL PLAN.

Question: Two quantum states with energies E_1 and E_2 have a lifetime of 10^{-9} sec and 10^{-8} sec, respectively. We want to clearly distinguish these two energy levels. Which one of the following options could be their energy difference so that they can be clearly resolved?

Plan to solve the problem:

1. Identify the relevant physics principle: This problem involves the energy-time uncertainty principle, which states that the product of the uncertainty in energy (ΔE) and the uncertainty in time (Δt) is greater than or equal to $\hbar/2$ ($\hbar/2$). The uncertainty in time is related to the lifetime of the states. To clearly distinguish the energy levels, the energy difference between them must be greater than the uncertainty in their energies.
 2. Relate lifetimes to time uncertainty: The lifetimes given (10^{-9} s and 10^{-8} s) represent the uncertainties in the time for each state (Δt_1 and Δt_2). We need to consider which lifetime is more restrictive for resolving the energy levels. Since we want to *clearly* distinguish them, we should use the *smaller* lifetime as the relevant Δt . This is because the shorter lifetime leads to a larger energy uncertainty, making the levels harder to resolve.
 3. Calculate the minimum energy difference: Use the energy-time uncertainty principle ($\Delta E * \Delta t \geq \hbar/2$) with the smaller lifetime to calculate the minimum resolvable energy difference (ΔE). We can approximate this by setting $\Delta E * \Delta t \approx \hbar/2$. We'll need the value of $\hbar$ (Planck's constant divided by 2π) in eV·s.
 4. Compare the calculated ΔE with the choices: Compare the calculated minimum resolvable energy difference (ΔE) with the given options (A) through (D). The correct answer will be the option that is *greater* than the calculated ΔE , ensuring that the energy difference is large enough to resolve the two levels.
 5. Select the correct answer: Based on the comparison in step 4, choose the option that represents an energy difference greater than the minimum resolvable energy difference.
-

Plan Execution:

1. Identify the relevant physics principle: The energy-time uncertainty principle ($\Delta E * \Delta t \geq \hbar/2$) governs this situation.
2. Relate lifetimes to time uncertainty: The shorter lifetime, 10^{-9} s, is the relevant Δt because it leads to a larger energy uncertainty, making resolution more difficult. So, $\Delta t = 10^{-9}$ s.
3. Calculate the minimum energy difference:
We have $\Delta E * \Delta t = \hbar/2$. We need $\hbar$ in eV·s. $\hbar = 6.582 \times 10^{-16}$ eV·s.
 $\Delta E = (\hbar/2)/\Delta t$
 $\Delta E = (6.582 \times 10^{-16} \text{ eV·s}/2)/(10^{-9} \text{ s})$
 $\Delta E = 3.291 \times 10^{-7} \text{ eV}$
This is approximately 3×10^{-7} eV.
4. Compare the calculated ΔE with the choices: We need an energy difference greater than 3×10^{-7} eV to clearly resolve the levels.
(A) 10^{-11} eV - Too small
(B) 10^{-8} eV - Too small
(C) 10^{-9} eV - Too small
(D) 10^{-4} eV - Large enough
5. Select the correct answer: Option (D) is the only one significantly larger than the calculated minimum energy difference.

Answer: D

Table 6.13: Examples of generated plans by PlanGEN (Best of $\mathcal{N}$) given physics problem. The example illustrates the plan generation and its execution for a physics problem from GPQA.

<Entire Document>

Question: What is the percentage change in cash flow hedges in 2011 compare to the 2010?

Plan to solve the problem:

1. Locate the information on cash flow hedges: Search the document for "cash flow hedges" or similar terms related to derivative instruments used for hedging cash flows. Focus on sections discussing derivative instruments, hedging activities, and related notional amounts.
 2. Identify the notional amounts for 2011 and 2010: Once the relevant section is found (Note 2i), extract the notional amounts of the cash flow hedges specifically for fiscal year 2011 and fiscal year 2010.
 3. Calculate the percentage change: Use the formula for percentage change: $[(\text{Notional Amount 2011} - \text{Notional Amount 2010}) / \text{Notional Amount 2010}] * 100$. This will provide the percentage change in cash flow hedges from 2010 to 2011.
-

Plan Execution:

1. Locate the information on cash flow hedges: The information on cash flow hedges was located in a section discussing derivative instruments and hedging activities. Specifically, it was found within a sentence detailing the notional amounts of various hedging instruments.

Code (if needed):

```
“python
# No code needed for this step.
“
```

Result: Information on cash flow hedges located successfully.

2. Identify the notional amounts for 2011 and 2010: The notional amounts for cash flow hedges were identified as \$153.7 million for 2011 and \$139.9 million for 2010.

Code (if needed):

```
“python
# No code needed for this step.
“
```

Result: 2011 notional amount: \$153.7 million; 2010 notional amount: \$139.9 million.

3. Calculate the percentage change:

Code (if needed):

```
“python
notional_2011 = 153.7
notional_2010 = 139.9
percentage_change = ((notional_2011 - notional_2010) / notional_2010) * 100
rounded_percentage_change = round(percentage_change, 1)
print(f"rounded_percentage_change")
“
```

Answer: 9.9%

Table 6.14: Examples of generated plans by PlanGEN (Best of $\mathcal{N}$) given financial reasoning problem. The example illustrates the plan generation and its execution for a problem from DocFinQA.

Question: For a positive integer a , define a sequence of integers $x_1, x_2, \dots$ by letting $x_1 = a$ and $x_{n+1} = 2x_n + 1$ for $n \geq 1$. Let $y_n = 2^{x_n} - 1$. Determine the largest possible k such that, for some positive integer a , the numbers $y_1, \dots, y_k$ are all prime.

Plan for Solving the IMO Problem:

1. Problem Understanding: The problem defines a recursive sequence ' x_n ' based on an initial value ' a ', and a related sequence ' y_n ' based on ' x_n '. The goal is to find the largest ' k ' such that ' y_1 ', ' y_2 ', ..., ' y_k ' can all be prime numbers for some choice of ' a '.

2. Key Information and Variables:

' a ': A positive integer, the starting value for the sequence ' x_n '. This is the variable we need to choose optimally.

' x_n ': Defined recursively as ' $x_1 = a$ ' and ' $x_{n+1} = 2x_n + 1$ '.

' y_n ': Defined as ' $y_n = 2^{x_n} - 1$ '. This is the sequence we need to check for primality.

' k ': The length of the subsequence of ' y_n ' consisting only of primes. We want to maximize this.

3. Problem Type and Method: This is a number theory problem involving sequences, primality, and recursion. The method will involve a combination of:

Analyzing the structure of ' x_n ' and ' y_n ': Finding a closed-form expression for ' x_n ' in terms of ' a ' and ' n ' will be crucial. This will help us understand the structure of ' y_n '.

Primality testing/properties: We'll need to leverage known properties of prime numbers, particularly those related to Mersenne primes (primes of the form $2^p - 1$, where p is prime).

Case analysis/exploration: We might need to explore small values of ' a ' and ' k ' to identify patterns and formulate conjectures.

Proof by contradiction: To establish the largest possible ' k ', we may need to assume a larger value is possible and derive a contradiction.

4. Solution Process:

Step 1: Derive a closed-form expression for ' x_n '. The recursive definition suggests a linear recurrence relation. We can solve it to express ' x_n ' directly in terms of ' a ' and ' n '.

Step 2: Substitute ' x_n ' into the definition of ' y_n '. This gives us a formula for ' y_n ' in terms of ' a ' and ' n '.

Step 3: Analyze the primality conditions for ' y_n '. Given that ' y_n ' has the form $2^m - 1$, we know that for ' y_n ' to be prime, ' m ' (which is ' x_n ') must itself be prime. This suggests focusing on values of ' a ' that lead to prime values of ' x_n '.

Step 4: Explore small values of ' a ' and ' k '. This helps to identify patterns and limitations on the possible values of ' k '.

Step 5: Investigate divisibility properties. Consider the factors of ' y_n ' and how they might relate to previous terms in the sequence. Look for patterns in the factors that might prevent multiple consecutive ' y_n ' from being prime.

Step 6: Formulate a conjecture for the maximum ' k '. Based on the observations from the previous steps, propose a value for the maximum ' k '.

Step 7: Prove the conjecture. Use a proof by contradiction. Assume a larger ' k ' is possible and demonstrate that this leads to a contradiction based on the divisibility properties of ' y_n '. This will establish the proposed value of ' k ' as the maximum.

Plan Execution:

1. Closed-form for x_n : The recurrence relation $x_{n+1} = 2x_n + 1$ with $x_1 = a$ can be solved as follows:

$$x_2 = 2a + 1$$

$$x_3 = 2(2a + 1) + 1 = 4a + 3 = 2^2a + 2^2 - 1$$

$$x_4 = 2(4a + 3) + 1 = 8a + 7 = 2^3a + 2^3 - 1$$

By induction, we can show that $x_n = 2^{n-1}a + 2^{n-1} - 1 = 2^{n-1}(a + 1) - 1$.

2. y_n in terms of a and n : Substituting the expression for x_n into $y_n = 2^{x_n} - 1$, we get $y_n = 2^{2^{n-1}(a+1)-1} - 1$.

3. Primality of y_n : For y_n to be prime, x_n must be prime. Let $x_n = p_n$, where p_n is a prime number.

4. Exploring small values:

If $a = 1$, $x_1 = 1$, $y_1 = 2^1 - 1 = 1$, which is not prime.

If $a = 2$, $x_1 = 2$, $y_1 = 2^2 - 1 = 3$, which is prime.

$x_2 = 2(2) + 1 = 5$, $y_2 = 2^5 - 1 = 31$, which is prime.

$x_3 = 2(5) + 1 = 11$, $y_3 = 2^{11} - 1 = 2047 = 23 \cdot 89$, which is not prime. So, for $a = 2$, $k = 2$.

5. Divisibility:

If $x_1 = a$ is prime, say $a = p$, then $x_2 = 2p + 1$. If x_2 is also prime, then y_1 and y_2 are potentially Mersenne primes. However, $x_3 = 2(2p + 1) + 1 = 4p + 3$. If $p = 2$, $x_3 = 11$, and y_3 is not prime. If $p = 3$, $x_2 = 7$, $x_3 = 15$, which is not prime. If $p = 5$, $x_2 = 11$, $x_3 = 23$, $x_4 = 47$, $x_5 = 95$, which is not prime.

If $k = 3$, we need $x_1 = a$, $x_2 = 2a + 1$, and $x_3 = 4a + 3$ to all be prime. If $a = 2$, $x_1 = 2$, $x_2 = 5$, $x_3 = 11$. But $y_3 = 2^{11} - 1 = 2047 = 23 \cdot 89$, which is not prime.

6. Conjecture: The largest possible k is 2.

7. Proof: We have shown that for $a = 2$, y_1 and y_2 are prime. We need to show that y_3 cannot be prime. $x_3 = 4a + 3$. If $a \equiv 2 \pmod{3}$, then $4a + 3 \equiv 4(2) + 3 \equiv 11 \equiv 2 \pmod{3}$. If $a \equiv 1 \pmod{3}$, then $4a + 3 \equiv 4(1) + 3 \equiv 7 \equiv 1 \pmod{3}$. If $a \equiv 0 \pmod{3}$, then $4a + 3 \equiv 3 \equiv 0 \pmod{3}$. So, if $a > 3$ and $a \equiv 0 \pmod{3}$, then x_3 is divisible by 3 and greater than 3, so it is not prime. If $a = 3$, $x_3 = 15$, not prime. If $a = 2$, $x_3 = 11$, and $y_3 = 2047 = 23 \cdot 89$.

So the final answer is $\boxed{2}$.

Table 6.15: Examples of generated plans by PlanGEN (Best of $\mathcal{N}$) given maths problem. The example illustrates the plan generation and its execution for a problem from OlympiadBench (MATH).

PLAN-TUNING: POST-TRAINING FOR IMPROVING REASONING CAPABILITIES OF LLMs

7.1 Introduction

Natural planning aligns more with real-world tasks such as trip or meeting planning [314]. Decomposing complex problems into simpler subtasks is a key to human-like natural planning [95, 171]. For instance, when prompted with “*How to plan a trip to ...*”, recent LLMs naturally generate subtasks such as selecting a mode of transport, estimating a budget, and determining the trip duration. These prompts exemplify everyday reasoning tasks that require multi-step planning and decision-making, making them ideal for evaluating LLMs’ ability to perform structured, goal-oriented reasoning. From organizing a trip or scheduling a meeting to planning a birthday party or solving a math problem, each scenario involves decomposing a high-level objective into coherent subtasks. The corresponding responses reflect how a well-structured plan can guide users through complex decisions, highlighting both procedural thinking and domain knowledge. Such prompts can be used as qualitative case studies to assess models’ competence in natural planning, temporal sequencing, and commonsense reasoning.

How to plan a trip to Japan

1. **Decide on Travel Dates** – Choose the season (e.g., cherry blossom in spring or autumn foliage) and trip duration.
2. **Set a Budget** – Estimate total costs including flights, transport, lodging, food, and attractions.

3. **Check Entry Requirements** – Look into visa policies, passport validity, and any travel restrictions.
4. **Choose Destinations** – Select cities and regions (e.g., Tokyo, Kyoto, Osaka, Nara, Hokkaido, Okinawa) based on your interests.
5. **Book Flights and Accommodation** – Use platforms like Google Flights, Booking.com, or Airbnb; book early in peak seasons.
6. **Plan Transportation Within Japan** – Consider a Japan Rail Pass and IC cards (Suica, Pasmo) for local travel.
7. **Create an Itinerary** – Map out a day-by-day plan including travel time, attractions, and rest periods.
8. **Arrange Travel Essentials** – Pocket Wi-Fi/SIM card, power adapter (Type A/B), Japanese yen (cash is still widely used).
9. **Learn Local Etiquette** – Study basic customs (e.g., removing shoes indoors, quiet on trains, no tipping).
10. **Pack Appropriately** – Bring weather-appropriate clothing, walking shoes, and a small daypack for city exploration.

How to schedule a meeting with your PhD colleague

1. **Define the Purpose** – Clarify the topic (e.g., paper discussion, experiment planning).
2. **Propose a Timeframe** – Suggest a few potential date/time slots that work for you.
3. **Use Scheduling Tools** – Use Doodle, Google Calendar, or Outlook to find mutual availability.
4. **Check Their Availability** – Send a quick message (email/Slack/WhatsApp) asking

for their availability.

5. **Suggest a Duration** – Be clear about how long you expect the meeting to take (30 mins, 1 hour, etc.).
6. **Decide the Format** – In-person or virtual (Zoom, Teams, Google Meet).
7. **Send Calendar Invite** – Include topic, agenda (if needed), and location or meeting link.
8. **Follow Up Politely** – If they don't respond in a day or two, send a short, polite reminder.
9. **Prepare Materials** – If needed, send any relevant documents or reading material beforehand.
10. **Confirm on the Day** – Send a quick message confirming the meeting a few hours before, especially for virtual calls.

How to organize a party for the birthday

1. **Set a Budget** – Decide how much you want to spend overall.
2. **Choose a Date & Time** – Pick a convenient day for the birthday person and guests.
3. **Pick a Venue** – At home, a restaurant, park, or rented space—based on budget and guest size.
4. **Decide a Theme (optional)** – Themes like retro, beach, movie night, or favorite colors add fun.
5. **Make a Guest List** – Know how many people you're inviting to plan food and space.
6. **Send Invitations** – Use digital invites (WhatsApp, email, Evite) or printed cards.
7. **Plan Food & Drinks** – Homemade, catered, or ordered—match it to the guest

profile and theme.

8. **Arrange Decorations & Cake** – Buy or DIY decorations and order a cake in advance.
9. **Plan Activities or Entertainment** – Music, games, speeches, or photo booth to keep guests engaged.
10. **Prepare a Backup Plan** – Especially for outdoor events, have a Plan B in case of weather or other issues.

How to solve a given mathematical reasoning problem

1. **Understand the Problem** – Read carefully and identify what is being asked.
2. **Identify Key Information** – Extract numbers, relationships, and conditions from the problem.
3. **Break Down the Problem** – Divide the complex problem into smaller, manageable parts.
4. **Visualize or Represent** – Use diagrams, equations, or tables if helpful.
5. **Recall Relevant Concepts** – Think of formulas, theorems, or reasoning strategies that apply.
6. **Formulate a Plan** – Decide on the method (e.g., algebra, logic, geometry) to approach the problem.
7. **Execute Step-by-Step** – Carry out your plan methodically, solving subparts if needed.
8. **Check Units and Logic** – Make sure the answer is in the correct form and logically consistent.

9. **Verify Your Answer** – Plug it back into the original problem to confirm it satisfies all conditions.
10. **Reflect on the Method** – Consider if there’s a more efficient or alternative solution strategy.

Recent large-scale LLMs such as o4 [316], Gemini-2.0-Pro [235], and Deepseek-v3 [126] have demonstrated this ability, and it significantly boosts their performance on complex reasoning tasks [198, 76]. Figure 7.1 illustrates a synthetic planning trajectory distilled from Gemini-2.0-Pro: a complex problem is first decomposed into intermediate subgoals (e.g.,

“identify relevant quantities,” “formulate equations,”

“solve subexpressions”), which guide the model through a structured solution path.

However, these capabilities have largely been explored at inference time and on large-scale proprietary models [256, 171]; smaller open-source LLMs still struggle to leverage the decomposition step from natural planning effectively, limiting their performance on complex tasks. Thus, we address the research question: “Can we

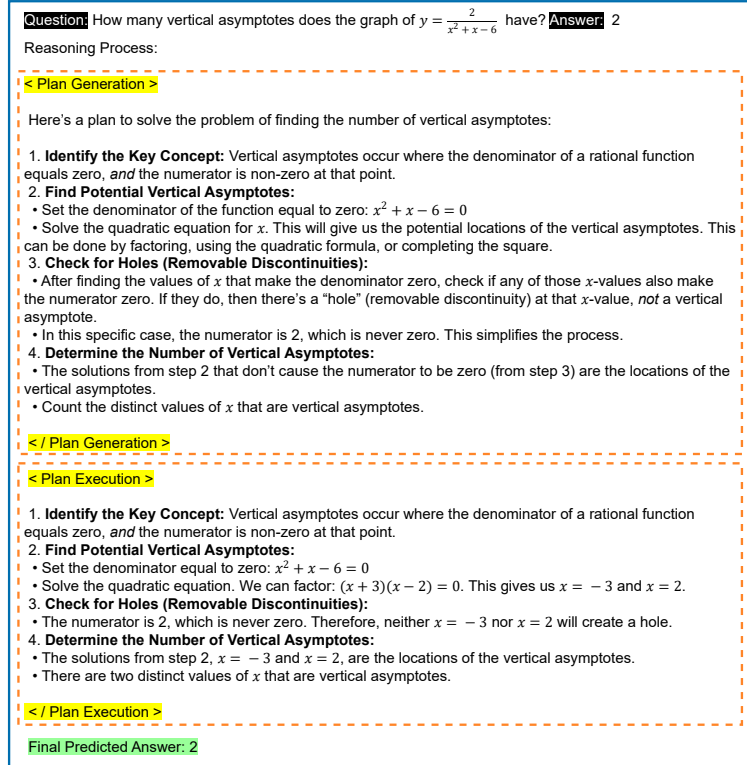


Figure 7.1: An example planning trajectory for a problem adapted from the MATH [78].

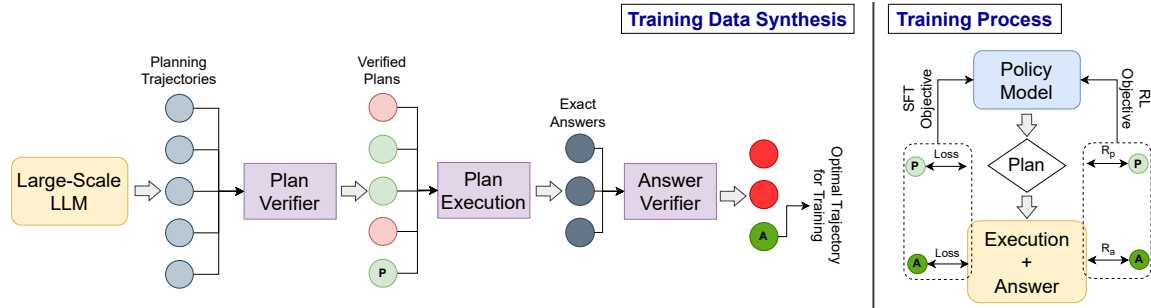


Figure 7.2: Left: Large-scale LLM generates multiple planning candidates (green = high-quality, red = low-quality). A *Plan Verifier* scores plans, and an *Answer Verifier* confirms the final answer; only trajectories passing both become the training corpus. **Right:** These trajectories train the policy model via SFT and RL objectives. P and A denote gold synthetic planning and answer trajectories, while R_p and R_a are their respective rewards.

improve the performance of smaller LLMs on complex tasks by incorporating such capabilities through post-training, rather than relying solely on inference-time prompts?” To this end, we propose PLAN-TUNING, a post-training method that uses synthetic planning trajectories—sequences of natural decomposition steps—to teach a model how to plan as part of its parameterized knowledge. PLAN-TUNING incorporates planning trajectories in supervised and Reinforcement Learning (RL) settings, with customized objectives (loss) and reward functions to improve planning capabilities. Our plan-tuned smaller LLMs show improved reasoning skills, focused on mathematical reasoning.

As illustrated in Figure 7.2, for distilling high-quality planning trajectories from large-scale LLM, we leverage the Best-of- $\mathcal{N}$ approach from the recent PlanGEN framework proposed by Parmar *et al.* [171]. For the MATH and GSM8k [40] training sets, we generate five candidate plans per problem. Each plan is then evaluated by the verification agent from PlanGEN, and only those exceeding a predefined quality threshold are retained. Because gold final answers are available for training data, we next execute every retained plan and verify that it produces the correct final answer. At last,

we include only those trajectories in PLAN-TUNING that both pass the agent-based scoring threshold and yield the correct solution; all others are discarded.

In PLAN-TUNING, we explore two post-training paradigms, supervised fine-tuning (SFT) and RL to incorporate planning in smaller LLMs. In SFT, we examine (1) an end-to-end setting where the model learns to map problem statements directly to a step-by-step plan and final answer, and (2) a two-stage pipeline that first generates only the plan and then executes it to derive the solution. In RL, we introduce Group Relative Policy Optimization (GRPO) [220], in which we augment the RL objective with planning-specific rewards to incorporate high-quality plan generation. We evaluate PLAN-TUNING against two strong baselines: an SFT model trained using reasoning chains and answers, and a vanilla GRPO model that optimizes preferences without any planning-based reward.

We evaluate four mathematical reasoning benchmarks—two in-domain (GSM8k and MATH) for both training and evaluation, and two out-of-domain (OlympiadBench [76] and AIME [228]) to assess generalization. For SFT, we use Gemma-3-12B-it [237] and Qwen3-8B [286], and for RL, we use Gemma-3-1B-it, and Qwen3-4B. Across both in-domain tasks, plan-tuned models consistently outperform these baselines, yielding $\sim 7\% \uparrow$ on GSM8k and $\sim 20\% \uparrow$ on MATH. They also demonstrate stronger reasoning-based generalization, achieving $\sim 10\% \uparrow$ and $\sim 12\% \uparrow$ on OlympiadBench and AIME, respectively. Moreover, our detailed analysis results in several interesting findings, such as mixing GSM8k and MATH during plan-tuning actually degrades performance, and plan-tuned models substantially mitigate the token-repetition issues. In summary, our work proposes PLAN-TUNING, a method to distill planning trajectories from large-scale LLMs and fine-tune smaller LLMs to incorporate this ability to solve complex problems. We believe that PLAN-TUNING can be an effective method for improving complex reasoning in smaller LLMs.

7.2 Proposed Method

In this section, we present task formulation, background, and detailed discussion on our proposed post-training methods.

7.2.1 Task Formulation

Defining Task Decomposition as Part of Natural Planning In this work, we formalize natural planning—distinct from classical AI planning (which is not the focus of this work)—as the process of decomposing an input problem x into an ordered sequence of intermediate subgoals that guide its solution. Formally, we introduce a latent state space $\mathcal{S}$ of partial reasoning states, with the initial state $s_0 = x$. A planning trajectory $\tau = (\tau_1, \dots, \tau_K)$ is then defined as a sequence of operators $\tau_k : \mathcal{S} \rightarrow \mathcal{S}$ such that

$$s_k = \tau_k(s_{k-1}) \quad \text{for } k = 1, \dots, K, \quad (7.1)$$

The final state s_K encodes sufficient information to extract the correct answer y . We model the planner as a conditional distribution as below:

$$\pi_\theta(\tau \mid x) = \prod_{k=1}^K \pi_\theta(\tau_k \mid s_{k-1}), \quad (7.2)$$

This equation indicates that the policy model π_θ assigns high probability to trajectories that (i) follow high-quality, human-like decomposition patterns and (ii) lead to a correct final solution. For the scope of this work, we use smaller LLMs as policy models for generating planning trajectories.

Problem Statement We frame mathematical reasoning with planning as learning a conditional model that, given a problem statement $x \in \mathcal{X}$, jointly generates a

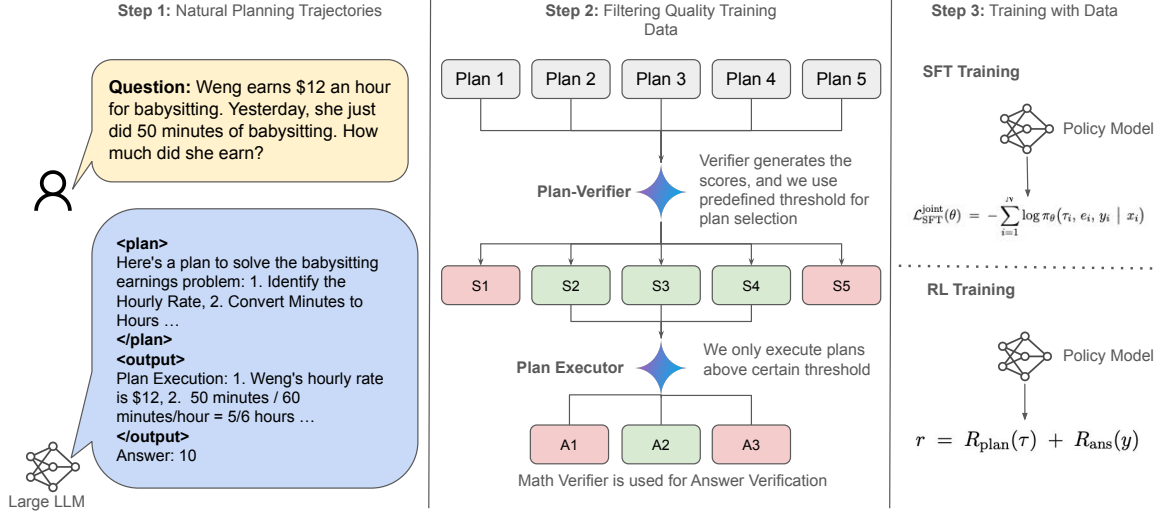


Figure 7.3: Overview of the PLAN-TUNING pipeline. First, a large LLM generates multiple candidate natural-language planning trajectories for each problem. Next, a Plan Verifier scores and filters these trajectories, and a Math Verifier executes and validates only those above a quality threshold. Finally, the curated plan-answer pairs are used to train the target model via both SFT and RL (GRPO) objectives.

planning trajectory $\tau = (\tau_1, \dots, \tau_K) \in \mathcal{T}$ and a final answer $y \in \mathcal{Y}$, as shown in the below equation.

$$\pi_{\theta}(\tau, y \mid x) = \pi_{\theta}(y \mid \tau, x) \pi_{\theta}(\tau \mid x) \quad (7.3)$$

The above equation is the policy of our plan-tuned LLM, parameterized by θ . We collect a training corpus $\mathcal{D} = \{(x_i, \tau_i, y_i)\}_{i=1}^N$ of synthetic, high-quality trajectories paired with gold answers. For this purpose, we use large-scale LLMs such as Gemini-2.0-Flash along with a filtering module to synthesize high-quality data. Our learning process thus train model to first decompose problem into solvable sub-problems through natural planning, and leverage two different objectives: supervised fine-tuning, which aligns $\pi_{\theta}(\tau, y \mid x)$ with the observed (τ_i, y_i) pairs, and reinforcement learning, which maximizes expected rewards to encourage better generation, execution and correct answers.

7.2.2 Data Generation

Figure 7.1 and Figure 7.3 show illustrative examples of planning trajectories and their execution to get the final answer for the given problem¹. Here, we provide a detailed discussion about the distillation of these high-quality planning trajectories using Gemini-2.0-Flash. For this purpose, we utilize the training sets of GSM-8k and MATH.

Data Synthesis Let us denote the training set as $\{x_i\}_{i=1}^N$. Motivated by Parmar *et al.* [171], for each problem x_i , we employ a method similar to the PlanGEN (Best of $\mathcal{N}$) (with $\mathcal{N} = 5$ and temperature to 0.7 for the underlying LLM $\mathcal{M}$ for diversity) to synthesize five distinct natural-language planning trajectories:

$$\{\tau_i^{(1)}, \dots, \tau_i^{(5)}\} \sim \mathcal{M}(\cdot \mid x_i). \quad (7.4)$$

Each trajectory $\tau_i^{(n)}$ is passed through a constraint-based verification agent—adapted from Parmar *et al.* [171]—to compute a plan quality score.

$$s_i^{(n)} = R_{\text{ver}}(\tau_i^{(n)}), \quad (7.5)$$

This score assesses coherence, logical soundness, and alignment with human-like decomposition patterns. Simultaneously, we execute each $\tau_i^{(n)}$ by feeding it into our execution module (another underlying LLM) to obtain an answer $y_i^{(n)}$. This yields a candidate set as below, capturing both plan quality and solution correctness.

$$\left\{ x_i, \tau_i^{(n)}, y_i^{(n)}, s_i^{(n)} \right\}_{\substack{i=1, \dots, N, \\ n=1, \dots, 5}}$$

¹More examples are provided at <https://anonymous.4open.science/r/plan-tuning-6113>

The whole process of doing this data synthesis is presented in Figure 7.2, and a detailed example is presented in Figure 7.3.

Data Synthesis Prompts These prompts form a structured framework for evaluating and improving mathematical reasoning in LLMs. The *Plan Generation Prompt* encourages models to decompose complex math problems into step-by-step solution strategies, fostering procedural thinking. The *Constraints Generation Prompt* identifies key logical and mathematical conditions that must be satisfied by any valid solution plan, serving as a verification checklist. Finally, the *Plan Verification Prompt* introduces a rigorous reward-based scoring scheme, allowing evaluators to assign interpretable, constraint-aware scores to the quality of generated plans.

Plan Generation Prompt

Analyze the given maths question, and create a plan to solve it:

<question>

{question}

</question>

Feel free to break down the problem in whatever way you think is most effective. Consider key concepts, formulas, relevant facts, or any logical approach that would help solve this. Your task is to only provide a plan and not solve it during this process.

Constraints Generation Prompt

You are an expert in identifying explicit and implicit constraints for verifying plans generated to solve complex maths problems. Your job is to generate those constraints for the following question, which can be helpful in verifying and evaluating the given plan.

<question>

{question}
</question>

Make sure to identify all constraints in the question. Please output the constraints as a list. DO NOT include any other text in your response.

Plan Verification Prompt

Provide a reward score between -100 and 100 for the plan quality, using very strict standards. Do not give a full score above 95. Make sure the reward score is an integer.

Input:

{input}

Generated plan to evaluate:

{response}

Consider the constraints below while evaluating:

{verification_prompt}

Make sure to check all the constraints before giving the reward score.

Please provide a reward in the format below:

- Steps: [step-by-step reasoning for the reward score]
- Score: [Strictly provide the reward score as an integer between -100 and 100]

Training Data Quality To ensure high-quality supervision, we apply a two-stage filtering process. First, we retain only trajectories whose verification score exceeds a threshold $\alpha = 80$:

$$s_i^{(n)} \geq 80, \quad (7.6)$$

This threshold we decided based on manual analysis presented in Parmar *et al.* [171] that plans above this score have a high likelihood of yielding correct solutions. Now, for the training set, we have the gold final answer available. Hence, we validate each selected trajectory by checking execution correctness, i.e., $y_i^{(n)} = y_i^*$, where y_i^* is the gold answer. Only those $(x_i, \tau_i^{(n)}, y_i^{(n)})$ triples satisfying both criteria are included in the final training corpus; all remaining candidates are discarded. This selection yields a dataset of `<problem, plan, plan execution, final answer>` that balances plan quality with solution accuracy, creating high-quality training data.

7.2.3 PLAN-TUNING

We utilize two post-training methods: (i) supervised fine-tuning (SFT), and (ii) reinforcement learning via Gradient-based Reward Policy Optimization (GRPO)—each aimed to incorporate planning abilities in underlying LLMs.

7.2.3.1 SFT Training

In SFT, we compare two approaches: (1) joint plan and answer Generation teaches models to produce complete solutions (plans, step-by-step execution, and final answers) from problems, while (2) plan-only generation focuses exclusively on creating high-quality plans. These methods use different loss functions to optimize model parameters, with the joint approach minimizing negative log-likelihood across all solution components and the plan-only approach focusing only on plan quality.

Method 1 The model learns to map each problem x_i to the concatenated sequence (τ_i, e_i, y_i) , where τ_i is the plan, e_i is the step-by-step execution of that plan, and y_i is

the final answer. We minimize the negative log-likelihood using below:

$$\mathcal{L}_{\text{SFT}}^{\text{joint}}(\theta) = - \sum_{i=1}^N \log \pi_{\theta}(\tau_i, e_i, y_i \mid x_i) \quad (7.7)$$

Method 2 The model focuses exclusively on generating high-quality plans τ_i . The objective we use to optimize is given below:

$$\mathcal{L}_{\text{SFT}}^{\text{plan}}(\theta) = - \sum_{i=1}^N \log \pi_{\theta}(\tau_i \mid x_i) \quad (7.8)$$

Once we have generated a high-quality plan τ_i , we use the same off-the-shelf base LLM to execute that plan and produce the final answer. No additional training is performed on this execution model. In the rest of the paper, we refer to method 1 as $\mathcal{M}_1$, and method 2 as $\mathcal{M}_2$.

7.2.3.2 GRPO Training

In this approach, we apply GRPO, a policy-gradient algorithm that directly maximizes sequence-level returns. Let q denote a problem statement and $o = (o_1, \dots, o_{|o|})$ the model’s generated output/rollouts (the plan, its execution, and the final answer). We define a combined reward for each sampled trajectory as below:

$$r(.) = R_{\text{plan}}(\tau) + R_{\text{ans}}(y), \quad (7.9)$$

where $R_{\text{plan}}(\tau)$ is the plan-quality score produced by our similarity function (discussed later in the section), and $R_{\text{ans}}(y)$ is a binary correctness indicator (2 if y matches the gold answer, 0 otherwise).

Background GRPO, a PPO [216] variant, estimates the advantage by aggregating reward scores of a group of n sampled responses to a given query q . Formally, let π_{θ}

and $\pi_{\theta_{old}}$ be the current and old policy models respectively. Let q and o_i be the query and i^{th} response sampled from the dataset and the old policy, respectively. Let $r(\cdot)$ be the reward function, which measures the correctness of a given response. Then, the GRPO objective is defined as follows:

$$\begin{aligned} \mathcal{J}_{GRPO}(\theta) = \mathbb{E} \bigg[& q \sim \mathcal{D}, \{o_i\}_{i=1}^n \sim \pi_{\theta_{old}}(O \mid q) \bigg] \\ & \left\{ \frac{1}{n} \sum_{i=1}^n \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left[\frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | q, o_{i,<t})} \hat{A}_{i,t}, \right. \right. \\ & \left. \left. \text{clip} \left(\frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] \right. \\ & \left. - \beta D_{KL}[\pi_{\theta} \parallel \pi_{\text{ref}}] \right\} \end{aligned}$$

Here, the advantage is calculated as the normalized reward, i.e., $\hat{A}_{i,t} = \tilde{r}(o_i) = \frac{r(o_i) - \text{mean}(r)}{\text{std}(r)}$. This $\hat{A}_{i,t}$ centers and scales each trajectory's return r across the sampled batch. By weighting each token's log-probability gradient by $\hat{A}$, GRPO amplifies updates for outputs yielding above-average combined reward and suppresses those with below-average reward.

Our Modification (Plan GRPO) In the above formulation, our focus is to modify $r(\cdot)$ function, and incorporate planning-based reward. So, the final reward function consists: (1) plan-quality reward $R_{\text{plan}}(\tau)$ ensures that generated trajectories closely match high-quality synthetic plans distilled from large-scale LLMs, and (2) answer correctness reward $R_{\text{ans}}(y)$ guarantees that following these plans leads to right solutions. Thus, the final reward is presented in Equation 7.9. By normalizing and combining these signals, GRPO fine-tunes the policy model π_{θ} to generate both good plans, accurate execution, and a correct final answer.

Details on Planning Reward $R_{\text{plan}}(\tau)$ is computed by measuring how closely a model-generated plan τ matches its reference τ^* using our Gemini-based similarity scorer. Here, reference τ^* is the plan synthesized from the above section. It is distilled from large-scale LLMs (Gemini-2.0 in our case). We first extract the plan segments. We parse each model completion to pull out only the `<plan>` portion. Then, we prompt Gemini for similarity. We construct a natural-language prompt comparing the generated plan to the gold plan and send it to Gemini-2.0-Flash, asking it to rate plan similarity on a 0–1 scale. At last, we parse the numeric score. We apply a flexible regular expression to the returned text to extract the score.

$$R_{\text{plan}}(\tau_i) = \text{Score}_{\text{Gemini}}(\tau_i, \tau_i^*) \in [0, 1].$$

These per-example rewards are then fed into our GRPO objective $\mathcal{J}_{\text{GRPO}}$, so that higher-quality plans receive proportionally larger policy-gradient updates. The prompt for calculating $R_{\text{plan}}(\tau)$ is presented below. The box below defines the exact evaluation prompt we use to score planning quality in GRPO. It asks the model to compare a generated plan against a gold plan, provide a brief similarity analysis, and emit a single scalar score on a 0–1 scale. This score then serves directly as the planning reward during RL fine-tuning.

Prompt for Planning Reward

You are an expert evaluator of problem-solving plans. Compare the following two plans and rate their similarity on a scale from 0 to 1.

0.0 = Completely different plans with no shared approach or reasoning steps. 0.25 = Minimal similarity with some overlapping concepts but fundamentally different approaches. 0.5 = Moderate similarity with shared key ideas but significant differences in execution or reasoning. 0.75 = High similarity with mostly aligned reasoning and steps, with minor differences. 1.0 = Nearly identical plans that follow the same approach and reasoning

steps.

Generated Plan:

{generated plan}

Gold Plan:

{gold plan}

First, provide a brief analysis of the similarity, then output only a single float number between 0 and 1, representing the similarity score.

Please **STRICTLY** use the format below:

Analysis: [brief analysis]

Score: [float number between 0 and 1]

(Note: this score will be used as the planning reward in GRPO.)

7.3 Results and Analysis

In this section, we provide details about datasets, experiments, a discussion on main results, and a thorough analysis of results.

7.3.1 Experimental Setup

Datasets We evaluate our PLAN-TUNING on four mathematical reasoning benchmarks. As shown in Table 7.1, GSM-8k [40] and MATH [78] provide in-domain datasets for both training and evaluation. After filtering out lower-quality examples, the GSM-8k training set contains 6,586 problems (from an original 7,500), with 1,319 held out for evaluation. The MATH training set comprises 10,000 problems (from 12,000), with 500 held-out for evaluation. To evaluate out-of-domain generalization, we use the text-only version of OlympiadBench (MATH) (674 problems) [76] and

AIME (933 problems), for evaluation purposes only.

Models For data synthesis, we use **Gemini-2.0-Flash** (Checkpoint: April 2025). We fine-tune four pretrained HuggingFace checkpoints: **Gemma-3-12B-It** and **Qwen3-8B** for PLAN-TUNING: SFT; and **Gemma-3-1B-It** and **Qwen3-4B** for PLAN-TUNING: GRPO.

Baselines Our two baselines for these training paradigms: (i) an SFT model that learns to output conventional chain-

of-thought reasoning and final answers, and (ii) a “vanilla” GRPO model optimized only on answer correctness $R_{\text{ans}}(y)$ without any planning-specific rewards $R_{\text{plan}}(\tau)$.

Proposed Experiments For PLAN-TUNING: SFT, we experiment with both methods: (i) $\mathcal{M}_1$, a joint plan-and-answer formulation, where the model maps each input x to the tuple $\langle \text{Plan}, \text{Execution}, \text{Answer} \rangle$, and (ii) $\mathcal{M}_2$, a plan-only variant in which the model simply generates $\langle \text{Plan} \rangle$. Both SFT variants use a batch size of 8, an adaptive learning rate of 5×10^{-6} , a single training epoch, and a cosine learning-rate scheduler. Second, we apply GRPO Training: a vanilla GRPO baseline that optimizes only the answer correctness reward $R_{\text{ans}}(y)$, and a planning-specific GRPO that additionally incorporates the Gemini-based planning reward $R_{\text{plan}}(\tau)$ into the objective $R_{\text{plan}} + R_{\text{ans}}$. For GRPO, we use a batch size of 32, the same learning rate and scheduler as SFT, one epoch, 4 rollouts per policy update, and a KL-coefficient

Dataset	Train	Eval
GSM-8k	6,586	1,319
MATH	10,000	500
OlympiadBench	–	674
AIME	–	933

Table 7.1: Statistics of the datasets used in our experiments. Training set sizes are shown for in-domain benchmarks, and evaluation set sizes for both in-domain and out-of-domain benchmarks.

Models	Methods	Datasets Evaluated using $\mathcal{M}(\text{GSM8k})$			Datasets evaluated using $\mathcal{M}(\text{MATH})$		
		GSM8k	OlympiadBench (MATH)	AIME 2024	MATH	OlympiadBench (MATH)	AIME 2024
Gemma-3	Baseline SFT	81.43	31.45	28.62	65.40	22.11	14.04
	Vanilla GRPO	19.94	02.52	00.00			
	$\mathcal{M}_1$	86.20	32.20	32.27	74.20	27.00	20.04
	$\mathcal{M}_2$	86.35	34.27	33.98	83.80	31.75	29.37
	Plan GRPO	28.35	04.30	03.00			
Qwen-3	Baseline SFT	80.74	28.78	29.05	53.20	12.02	05.57
	Vanilla GRPO	84.27	24.63	17.68			
	$\mathcal{M}_1$	87.11	30.27	32.05	73.80	23.44	19.08
	$\mathcal{M}_2$	85.67	30.12	31.94	79.40	31.01	31.51
	Plan GRPO	86.57	25.07	15.22			

Table 7.2: This table reports the accuracy (%) of the base LLM and its two plan-tuned variants (custom SFT and GRPO) on four mathematical reasoning benchmarks. Columns 1–2 show in-domain performance on GSM8K and MATH. Columns 3–4 present out-of-domain generalization on OlympiadBench and AIME 2024. These results demonstrate that leveraging synthetic planning trajectories via both SFT and RL objectives improves reasoning accuracy in smaller LLMs. $\mathcal{M}$ (GSM8k): Model trained using GSM8k dataset, $\mathcal{M}$ (MATH): Similar for MATH.

of 0.04. Due to resource and time constraints, our GRPO experiments are limited to the GSM8k dataset.

Metrics We report dataset-specific accuracy on each benchmark to assess in-domain performance and out-of-domain generalization. In particular, we use micro-average accuracy for OlympiadBench similar to He *et al.* [76], and Exact Match (EM) for all other datasets.

7.3.2 Main Results

Baseline Performance on In-Domain Tasks From Table 7.2, the off-the-shelf SFT model trained on GSM8K achieves 81.43% accuracy when evaluated on GSM8K

and 65.4% on the MATH benchmark for the Gemma-3. In comparison, the Qwen-3 SFT baseline shows 80.74% on GSM8K and only 53.2% on MATH. This is a $\sim 15\%$ drop between the two in-domain datasets, highlighting that, without explicit natural planning, smaller LLMs struggle to generalize from more complex and constrained math word problems where the diverse, multi-step reasoning is required.

Supervised Fine-Tuning Variants Introducing supervised planning trajectories yields consistent gains across both model families. For Gemma-3, the joint plan-and-answer SFT ($\mathcal{M}_1$) improves GSM8K accuracy to 86.2% (+4.8%) and MATH to 74.2% (+8.8%), while the plan-only SFT ($\mathcal{M}_2$) further boosts these to 86.35% on GSM8K and an 83.8% on MATH. Qwen-3 exhibits similar trends: joint SFT improves up to 87.11% on GSM8K (+6.4%) and 73.8% on MATH (+20.6%), whereas plan-only fine-tuning yields 85.67% and 79.4%, respectively. In particular, large gains on MATH suggest that guiding the model to focus purely on plan generation better incorporates the structured decomposition strategies needed for complex, multi-step reasoning tasks.

Out-of-Domain Generalization When evaluated on Olympiad-level math benchmarks, the importance of PLAN-TUNING is even more prominent. Gemma-3 baseline SFT achieves only 31.45% on OlympiadBench and 14.04% on AIME. Joint SFT (Method 1) improves it to 32.2% and 20.04%; plan-only SFT (Method 2) improves them further to 34.27% and 29.37%. Qwen-3 follows the same trend: baseline SFT is 28.78%/5.57% (OlympiadBench/AIME), joint SFT 30.27%/19.08%, and plan-only 30.12%/31.51%. These gains, often improving twice or more AIME performance, demonstrate that high-quality planning exemplars for training are especially critical for tackling novel, complex Olympiad-level math problems.

Improvements with Plan-GRPO From Table 7.2, we present results for vanilla-GRPO and Plan-GRPO where the model is trained using the GSM8k dataset. From the results, we can observe that, for Gemma-3-1B-It, plan-GRPO improves performance on GSM8k to 28.35% compared to vanilla-GRPO (19.94%). A similar trend can be observed for the Qwen3-4B model in terms of GSM8k. Now, on out-of-domain datasets, for Gemma-3, plan-tuned models are achieving better generalization. However, for Qwen-3 models, we are seeing a performance drop for AIME. Training with GRPO is highly dependent on reward functions. Also, the lower performance on OlympiadBench and AIME is subject to the smaller sizes of LLMs, 1B and 4B, used for GRPO training.

7.3.3 Analysis

Synthesis and Distribution-shift Considerations Together, these results indicate that embedding explicit natural planning into smaller LLMs—first via supervised trajectories and then through an RL-based policy refinement—yields substantial improvements in both in-domain accuracy and out-of-domain robustness. The significant improvements on AIME highlight how enforcing intermediate correctness reduces arithmetic drift, while consistent gains across two model families indicate the generality of our PLAN-TUNING post-training approach in bridging distributional gaps between training and evaluation domains.

Longer Reasoning Chains on OlympiadBench The reason behind the higher performance of PLAN-TUNED models is that SFT provides clear templates for breaking down multi-step proofs, giving the model a reliable blueprint for structured decomposition. Plan-GRPO builds on this by rewarding diverse, high-quality plans, encouraging the model to flexibly combine reasoning fragments when it encounters

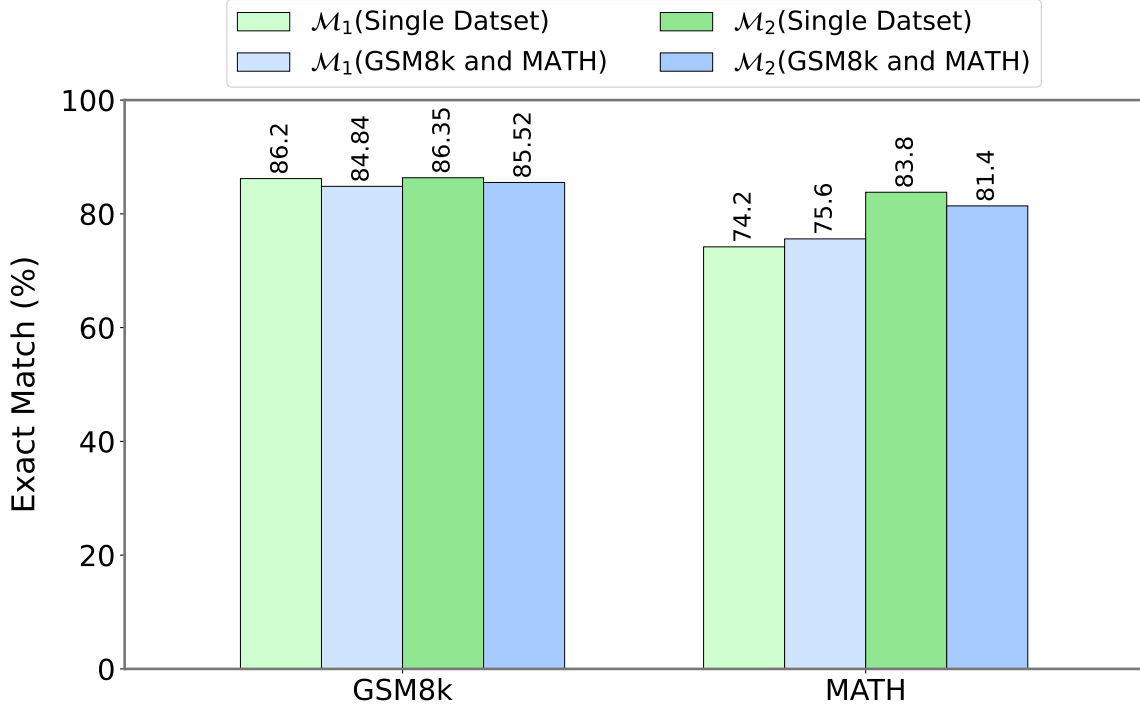


Figure 7.4: Comparison of plan-tuning accuracy on GSM8K and MATH when trained on each dataset individually *vs.* their combined corpus. $\mathcal{M}_i$ (Single Dataset) indicates that the respective method is trained on a given dataset (green), while other indicates that the respective method is trained on both datasets (blue).

novel or unexpected subgoals.

Variance in Plan Quality For short, arithmetic-focused GSM8K tasks, both methods quickly converge to high accuracy using straightforward reasoning chains. On more complex olympiad-level tasks—like geometry or combinatorics subcases—the RL objective in Plan-GRPO improves the impact of diverse, intermediate-valid plans, enabling the discovery of novel solution paths.

Mixing GSM8k and MATH during PLAN-TUNING Figure 7.4 compares PLAN-TUNING on GSM8K and MATH when training on each dataset separately *vs.* training on their combination. In both the joint plan-and-answer ($\mathcal{M}_1$) and plan-only ($\mathcal{M}_2$))

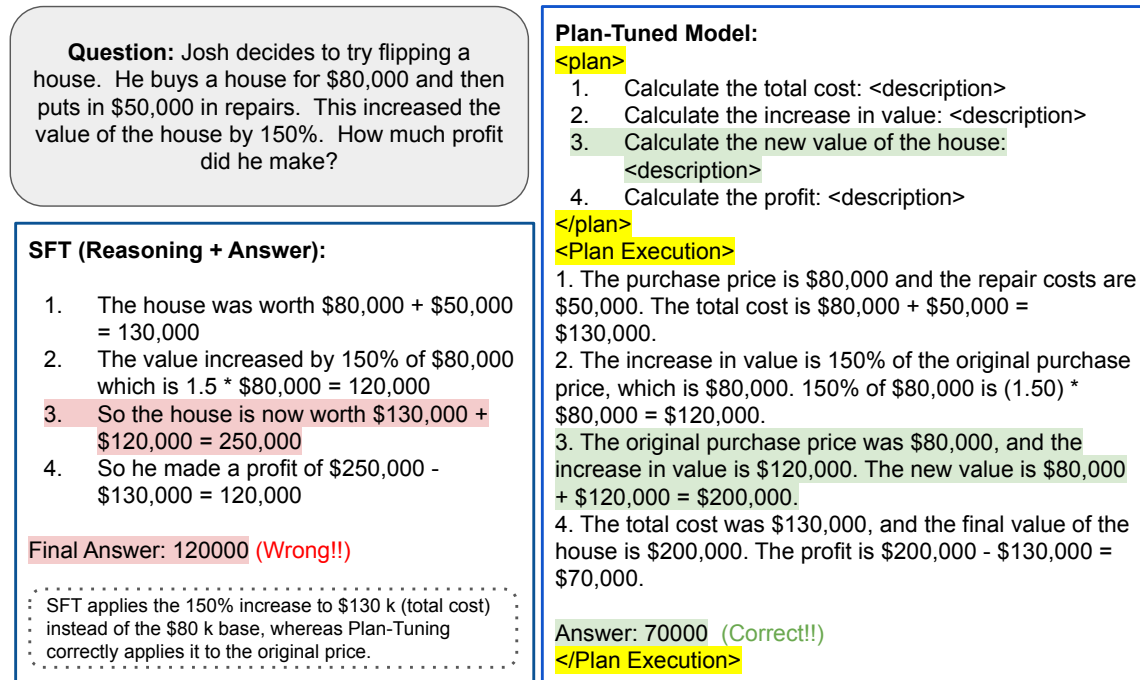


Figure 7.5: Qualitative Example: Comparison of SFT vs. PLAN-TUNING on a sample problem from GSM8k.

variants, tuning on a single dataset yields the best accuracy on its own benchmark, 86.2% on GSM8K and 83.8% on MATH—whereas mixing the two corpora in a sample batch drops GSM8K performance by $\sim 2\%$ and fails to improve MATH beyond its single-dataset result. This consistent degradation indicates that PLAN-TUNING relies on dataset-specific patterns of problem structure and reasoning style; when these patterns become heterogeneous, the model struggles to internalize a coherent planning policy, so domain-focused tuning can be more effective.

Qualitative Analysis In this example, we demonstrate how planning trajectories steer the model toward a structured solution. The vanilla SFT model misapplies the 150% increase to the combined cost, yielding an incorrect profit of \$120,000. In contrast, the plan-tuned model explicitly decomposes the task into four subtasks—total

cost, value increase, new house value, and profit—and arrives at the correct answer of \$70,000. The SFT pipeline collapses subtasks and propagates an early error, whereas our PLAN-TUNING framework enforces step-by-step reasoning aligned with human planning. This case highlights the importance of decomposing complex problems into clear intermediate goals to improve both accuracy and interoperability.

7.4 Conclusions

We introduce PLAN-TUNING, a novel post-training method that incorporates synthetic natural planning trajectories into smaller LLMs’ parameters, rather than relying solely on inference-time prompts. We develop two complementary post-training strategies—SFT to imitate high-quality plan decompositions and GRPO to reinforce plan quality alongside answer correctness—thereby teaching models both how to plan and how to execute. Across two in-domain datasets (GSM8K, MATH), PLAN-TUNED models achieve an average $\sim 7\%$ accuracy boost over strong baselines; on out-of-domain benchmarks (OlympiadBench, AIME), they significantly improved the performance. Through detailed analyses, we show (i) that a good plan relies on dataset-specific consistency—mixing heterogeneous sources degrades performance, and (ii) that plan-tuning substantially mitigates the token-repetition issue.

Our approach relies on a large base LLM to generate and verify planning trajectories; errors or biases in that model can propagate into training data and limit downstream gains. Generating, filtering, and executing multiple candidate plans per example incurs computational cost and implementation complexity, which may hinder large-scale or real-time applications. All experiments focus on mathematical reasoning; it remains to be validated whether PLAN-TUNING generalizes to other problem types (e.g., commonsense, code synthesis) without substantial adaptation. Key thresholds (e.g., plan-quality cutoff, reward weighting) require manual tuning.

DON'T BLAME ANNOTATORS

8.1 Introduction

Benchmarks have been proven pivotal for driving progress in Natural Language Understanding (NLU) in recent years [201, 7, 268]. Nowadays, NLU benchmarks are mostly created through crowdsourcing, where crowdworkers write examples following annotation instructions crafted by dataset creators [22, 313, 227]. The instructions typically include a short description of the task, along with several examples [44, 317, 207].

Despite the vast success of this method, past studies have shown that data collected through crowdsourcing often exhibit various biases that lead to overestimation of model performance [219, 68, 182, 245, 113, 150, 149, 80]. Such biases are often attributed to annotator-related biases, such as writing style and background knowledge [68, 60] (see more discussion on related work in §2.4).

In this work, we propose that biases in crowdsourced NLU benchmarks often originate at an early stage in the data collection process of designing the annotation task. In particular, we hypothesize that task instructions provided by dataset creators, which serve as the guiding principles for annotators to complete the task, often influence crowdworkers to follow specific patterns, which are then propagated to the dataset and subsequently over-represented in the collected data. For instance, $\sim 36\%$ of the instruction examples for the QUOREF dataset [44] start with “*What is the name*”, and this same pattern can be observed in $\sim 59\%$ of the collected instances.

To test our hypothesis, we conduct a broad study of this form of bias, termed *in-*

instruction bias, in 14 recent NLU benchmarks. We find that instruction bias is evident in most of these datasets, showing that $\sim 73\%$ of instruction examples on average share a few clear patterns. Moreover, we find that these patterns are propagated by annotators to the collected data, covering $\sim 61\%$ of the instances on average. This suggests that instruction examples play a critical role in the data collection process and the resulting example distribution.

It is difficult to represent a task with a few examples, and bias in instruction examples makes it even more difficult since a task and its associated reasoning have a larger scope than instruction patterns. For example co-reference resolution, temporal commonsense reasoning, and numerical reasoning are much broader tasks than the prevalent patterns in QUOREF (“*what is the name...*”), MC-TACO (“*how long...*”) and DROP (“*how many field goals...*”) datasets.

We investigate the effect of instruction bias on model performance, showing that performance is overestimated by instruction bias and that models often fail to generalize beyond instruction patterns. Moreover, we observe that a higher frequency of instruction patterns in the training set often increases the model performance gap on pattern and non-pattern examples and that large models are generally less sensitive to instruction bias.

In conclusion, our work shows that instruction bias widely exists in NLU benchmarks, often leading to an overestimation of model performance. Based on our study, we derive concrete recommendations for monitoring and alleviating this bias in future data collection efforts. From a broader perspective, our findings also have implications on the recent learning-by-instructions paradigm [51, 153], where crowdsourcing instructions are used in model training.

8.2 Instruction Bias in NLU Benchmarks

Instructions are the primary resource for educating crowdworkers on how to perform their task [161]. Bias in the instructions, dubbed *instruction bias*, could lead crowdworkers to propagate specific patterns to the collected data.

Here, we study instruction bias in NLU benchmarks¹, focusing on two research questions: (a) Do crowdsourcing instructions exhibit patterns that annotators can pick up on? and (b) Are such patterns propagated by crowdworkers to the collected data? In our study, we use the instructions of 14 recent NLU benchmarks:² (1) CLARIQ [3], (2) COSMOSQA [85], (3) DROP [49], (4) DUORC [206], and (5) HOTPOTQA [288] (6) HYBRIDQA [30], (7) MC-TACO [317], (8) MULTIRC [104], (9) PIQA [15], (10) QASC [106], (11) QUOREF [44], (12) ROPES [123], (13) SCIQA [273], (14) WINOGRANDE [207]. These benchmarks were created through different crowdsourcing protocols to evaluate diverse tasks [153]. Table 8.1 describes the statistics of the train and evaluation sets of datasets used in our experiments. Here, we can observe that each selected dataset differs in terms of the number of training samples, % of instruction patterns, and tasks.

Table 8.2 provides the number of examples present in crowdsourcing instructions of each dataset. From Table 8.2, we can observe that our analysis involves a wide range of different tasks. Also, we believe that the lower number of examples in crowdsourcing instructions might be limiting the imagination of annotators while creating samples, resulting in instruction bias.

¹All benchmarks are in English.

²The instructions were obtained from Mishra *et al.* [153], who have collected those from the dataset authors.

Dataset	Train			Test		
	$\mathcal{S}_{\text{train}}$	$\mathcal{S}_{\text{train}}^p$	$\mathcal{S}_{\text{train}}^{-p}$	$\mathcal{S}_{\text{test}}$	$\mathcal{S}_{\text{test}}^p$	$\mathcal{S}_{\text{test}}^{-p}$
CLARIQ	8566	7286 85.1%	1280 14.9%	4499	4006 89%	493 11%
DROP	77409	48422 62.5%	28987 37.5%	9536	5960 62.5%	3576 37.3%
MULTIRC	5131	1972 38.4%	3159 61.6%	953	395 41.5%	558 58.6%
PIQA	17171	7508 43.7%	9663 56.3%	3268	1401 42.9%	1867 57.1%
QUOREF	19399	11052 57%	8347 43%	2418	1451 60%	967 40%
ROPES	1412	1046 74.1%	366 25.9%	203	42 20.7%	161 79.3%
SCIQA	11679	9765 83.61%	1914 16.4%	1000	845 84.5%	155 15.5%
Total	140767	87051 61.8%	53716 38.2%	21877	14100 64.5%	7777 35.6%

Table 8.1: Statistics of number of train and test examples with and without instruction patterns. $\mathcal{S}_{\text{train}}$: set of examples in train set, $\mathcal{S}_{\text{train}}^p$: set of examples in train set with instruction pattern, $\mathcal{S}_{\text{train}}^{-p}$: set of examples in train set without instruction pattern, $\mathcal{S}_{\text{test}}$: set of examples in test set, $\mathcal{S}_{\text{test}}^p$: set of examples in test set with instruction pattern, $\mathcal{S}_{\text{test}}^{-p}$: set of examples in test set without instruction pattern.

8.2.1 Patterns in Crowdsourcing Instructions

Our goal is to quantify biases in instruction examples that propagate to collected data instances. In this study, we focus on an intuitive form of bias of recurring word patterns, which crowdworkers can easily pick up on. To find such patterns, we manually analyze the instruction examples of each dataset to find a *dominant pattern*, using the following procedure: (a) identifying repeating patterns of $n \geq 2$ words, (b) merging patterns that are semantically similar or have a substantial word overlap, and (c) selecting the most frequent pattern as the dominant pattern.

Pattern Extraction Method Here, we describe an example to show how we extract the dominant pattern from the crowdsourcing instructions and subsequently identify the same pattern in the dataset. We try to find recurring word patterns such

Dataset	Task	# of Examples
CLARIQ	Clarification QA	18
COSMOSQA	Commonsense Reasoning	8
DROP	Numerical Reasoning	10
DUORC	Paraphrased RC	10
HOTPOTQA	Multi-hop QA	8
HYBRIDQA	QA	17
MC-TACO	Event Duration	3
	Event Ordering	2
	Frequency	2
	Stationary	2
	Absolute Point	2
MULTIRC	Complex QA	7
PIQA	Physical Interaction QA	6
QASC	Complex QA	7
QUOREF	Coreference QA	11
ROPES	RC	14
SCIQA	Science-based QA	6
WINOGRANDE	Commonsense Reasoning	19
Average		8.4

Table 8.2: Tasks of each dataset and number of examples in crowdsourcing instruction of each dataset. RC: Reading Comprehension, QA: Question Answering.

as “Are you...”, “how many points...”, “Was... still...”, “since... the...”. For example, MC-TACO (event duration) has 3 examples in crowdsourcing instructions: (1) how long did Jack play basketball?, (2) how long did he do his homework?, and (3) how long did it take for him to get the Visa? In step (a), we analyze examples manually

and find *dominant pattern*. Here, we can see that all examples contain tri-gram pattern, i.e., “*how long did*”. In step (b), we try to generate more possible patterns that are semantically similar to the *dominant pattern* or have a significant word overlap. Here, “*how long did*” can be “*how long was*”, “*how long does*”, etc. (i.e, **How long AUX**). In step (c), we look for all these possible patterns in datasets using simple word-matching techniques.

Table 8.3 shows the dominant pattern in the instruction examples of each dataset. On average, 72.7% of the instruction examples used to create a dataset exhibit the same dominant pattern, and for 10 out of 14 datasets, the dominant pattern covers more than half of the instruction examples. This suggests that crowdsourcing instructions demonstrate a small set of repeating “shallow” patterns. Moreover, the short length of the patterns (2-4 words) and the typical low number of instruction examples (Table 8.2) make the patterns easily visible to crowdworkers, who can end up following them.

Notably, our results are an underestimation of the actual instruction bias, since (a) we only consider the dominant pattern for each dataset (b) our manual analysis over instruction examples has a preference for short patterns (c) we do not consider paraphrased patterns (beyond the shallow paraphrases which are visible in annotation instructions), and (d) datasets may include implicit patterns (e.g. writing style and biases from the annotator’s background knowledge) that also contribute to instruction bias. Accounting for such patterns is expected to increase the bias percentage in Tab. 8.3 further.

8.2.2 Instruction Bias Propagation to Datasets

We now turn to investigate whether patterns in instruction examples are further propagated by crowdworkers to the collected data. We analyze the train and test

Dataset	Pattern	% Ins.	% $\mathcal{S}_{\text{train}}$	% $\mathcal{S}_{\text{test}}$
CLARIQ	[Are Would Do] you	72.2	85.1	89
COSMOSQA	What AUX	87.5	45.1	38.4
DROP	How many [field goals years yards points touchdowns]	70	62.5	62.5
DUORC	[How old How What Who] AUX	70	85.1	84
HOTPOTQA	[In Of From _] [Which What] AUX	87.5	53.8	54.2
HYBRIDQA	Which AUX	29.4	25.7	15.1
MC-TACO	How long AUX	100	-	87.6
	What AUX	100	-	90.1
	How often AUX	100	-	85.3
	AUX... [still always by the time]	100	-	67.3
	When did / What time	100	-	83.4
MULTIRC	What AUX	14.3	38.4	41.5
PIQA	How [do can]	66.7	43.7	42.9
QASC	What AUX	57.1	49.3	47
QUOREF	What AUX the [_ full real first last] name	36.4	57	60
ROPES	Which AUX	42.9	74.1	20.7
SCIQA	What AUX	100	83.6	84.5
WINO-GRANDE	[because so while since but] ... the	73.7	63.4	63.1
Average		72.7	59	62

Table 8.3: Portion of patterns in instruction examples (Ins.) and in the corresponding train ($\mathcal{S}_{\text{train}}$) and test ($\mathcal{S}_{\text{test}}$) sets of NLU datasets. $\text{AUX} \in \{\text{am, is, are, was, were, has, have, had, do, does, did, will, would, can, could, may, might, shall, should, must}\}$, and $_$ is an empty string. MC-TACO has 5 different data subsets corresponding to different types of temporal reasoning (see Tab. 8.2), hence, the sum of percentages for this dataset exceeds 100%.

sets of each benchmark³ to find the same patterns, using simple string matching. To account for syntactic modifications in identified patterns based on some examples from dataset, we also consider synonym words where appropriate and match the paraphrased version of each pattern.

Table 8.3 shows the results. Across all datasets, instruction patterns are ubiquitous in the collected data, occurring in 60.5% of the instances on average, with similar presence in training (59%) and test (62%) examples. While the dominant pattern’s frequency in the data is typically not higher than in the instructions, for CLARIQ, DUORC, MULTIRC, QUOREF and ROPES, the pattern frequency was amplified by the crowdworkers. Interestingly, these datasets used a relatively large number of instruction examples (Table 8.2), suggesting that more examples do not necessarily alleviate the propagation of instruction bias. Example data instances with instruction patterns are provided in Table 8.4.

A natural question that arises is whether patterns in collected data reflect the true task distribution rather than a bias in the instructions. We argue that this is highly unlikely. First, while the space of possible patterns for a NLU task is arguably large, the dataset patterns are imbalanced proportionately to the patterns in the instructions for collecting it. For example, questions for assessing temporal commonsense reasoning could have various forms, such as “*What is the duration of...*”, “*For how much time...*”, and “*How long...*”. However, MC-TACO (event duration) is heavily dominated (87.6%) by questions with the pattern “*how long*”, which appears in 100% of the instruction examples (Table 8.3). In addition, datasets of similar tasks have different dominant patterns, while each dataset’s dominant pattern correlates with the pattern in the corresponding instructions; the pattern ‘What is’ appears in 48% of the questions in QASC and in 57% of the instruction examples, but it is

³If no explicit test set exists, we use the validation set.

Dataset	Pattern	Examples
CLARIQ	[Are Would Do] you	Are you looking for a specific web site?
		What kind of train are you looking for?
		Do you want to watch news videos or read the news?
		Would you like the location of the ritz carlton lake las vegas?
COSMOSQA	What AUX	What may happen after the young man makes his call?
		What might happen if you have him for the whole day?
		What's a possible reason the writer doesn't look disabled on the outside?
DROP	How many [field goals years yards points touchdowns]	How many touchdowns did Jones have?
		How many field goals did Kris Brown kick
		How many yards was the longest touchdown of the game?
		After Akers 32-yard field goal, how many points behind was Washington?
HOTPOTQA	[in of from _]	Which franchise was founded in 1978, Chuck E. Cheese's or Jet's Pizza?
	[Which What]	Busan, in the area surrounding the mountain of Geumjeongsan, is the second most populated city in which country?
	AUX	What is the name of the third album from singer Selena Quintanilla-Pérez?
MC-TACO	How long AUX	How long was his mother ill?
	What AUX	What did the government decide after the 9/11 attack?
	How often AUX	How often would one family be able to do something like this?
	AUX...	Will electronic espionage always be happening in the U.S.?
	[still always]	Is she still gone?
	When did / What time	What time did the planes crash into the World Trade Center? When did Durer die?
MULTIRC	What AUX	What was Poe's first published work?
		What is the full name of the person described?
		What kind of career does Christie Brinkley have?
PIQA	How [do can]	How do I make orange icing if I have store-bought white frosting?
		How can I make popsicles for dogs?
		Are you nervous about giving a speech or doing something? How can you calm yourself?
QUOREF	What AUX the [full real first last] name	What is the first name of the person who purchases a revolver?
		What is the full name of the person who is calmly asked to leave?
		What was the name of the house where Appleton Water Tower was built?
		What is the last name of the person who convinces the girls to help him look for the treasure?
ROPES	Which AUX	Which area would be less likely to experience a drought and have better chance at a new growth?
		Which hair spray brand should Greg buy to be environmentally friendly?
		Which markalong was produced asexually?
SciQA	What AUX	What are by far the most common type of invertebrate?
		What do waves deposit to form sandbars and barrier islands?
		What is the term for the total kinetic energy of moving particles of matter?
WINO-GRANDE	[because so while since but] ... the	The dog didn't like its collar but was okay with its leash because the _ was loose on it.
		Hunter took Benjamin's clothes to the laundromat, since _ had the day off that day.
		James sang his song at the top of his voice so as to be heard over the noise but the _ is louder.

Table 8.4: Examples of data instances from original dataset that contain instruction patterns. $AUX \in \{\text{am, is, are, was, were, has, have, had, do, does, did, will, would, can, could, may, might, shall, should, must}\}$. $_$: <blank>.

entirely different from the dominant pattern of HOTPOTQA, which is another QA dataset for multi-hop reasoning.

We further validate the propagation of bias in instruction examples by comparing the pattern distributions of collected instances when the instructions include and do not include examples. We conduct this experiment for MC-TACO and QUOREF and find that, without any examples provided, the dominant pattern is substantially less frequent, showing that instruction bias is propagated during data collection. Propagation of instruction bias to the test set raises concerns regarding its reliability for evaluation, which we address next.

To study the effect of bias in instruction examples on collected data, we asked NLP graduate students to write five questions for each of (1) temporal reasoning (event duration) and (2) coreference resolution, based on the crowdsourcing instructions of MC-TACO and QUOREF, respectively. For each task, we conduct two surveys, where the instructions include and do not include any examples.

We collected responses from 10 participants. The dominant patterns of MC-TACO ('how long') and QUOREF ('What is the name') only contribute to 38% and 8% of our collected data where examples are not given, in contrast to 68% ($\uparrow 79\%$) and 32% ($\uparrow 300\%$) in collected data where examples are given. This indicates that crowdsourcing examples bias crowdworkers to follow certain patterns, whereas showing no examples increases the creativity of crowdworkers.

In addition, our collected responses where examples are not given contain 10 and 9 unique patterns for MC-TACO (event duration) and QUOREF respectively, in contrast to only 4 and 5 unique patterns in collected data where examples are given. Our finding shows that there is substantial linguistic diversity associated with the NLP tasks, unlike the patterns covered in instruction examples that get propagated to corresponding datasets.

The task instructions and collected annotations are available at <https://github.com/Mihir3009/instruction-bias/blob/main/SURVEY.md>.

8.3 Effect on Model Learning

Let $\mathcal{S}_{\text{train}}$ ($\mathcal{S}_{\text{test}}$) be the set of training (test) examples, and denote by $\mathcal{S}_{\text{train}}^p$ ($\mathcal{S}_{\text{test}}^p$) and $\mathcal{S}_{\text{train}}^{-p}$ ($\mathcal{S}_{\text{test}}^{-p}$) its disjoint subsets of examples with and without instruction patterns, respectively. We conduct two experiments where we fine-tune models on (a) $\mathcal{S}_{\text{train}}^p$ and (b) $\mathcal{S}_{\text{train}}^p \cup \mathcal{S}_{\text{train}}^{-p}$, and evaluate them on $\mathcal{S}_{\text{test}}^{-p}$ and $\mathcal{S}_{\text{test}}^p$. This is to assess to what extent models generalize from instruction patterns to the downstream task (a), and to compare model performance on instances with and without instruction patterns (b).

8.3.1 Experimental Setting

Datasets Since model training is computationally expensive, we select a subset of seven datasets from those analyzed in §8.2: (1) CLARIQ, (2) DROP, (3) MULTIRC, (4) PIQA, (5) QUOREF, (6) ROPES, and (7) SCIQA. These datasets cover a variety of tasks, different types and levels of instruction bias (Table 8.3), and are different in size (Table 8.1).

Models For all datasets except DROP, we evaluate T5-base and T5-large [189], and BART-base and BART-large [118]. For DROP, we use Numnet+ [194], a RoBERTa model [132] with specialized output heads for numerical reasoning. Numnet+ has 355M parameters, which is closer to T5-base (220M) than to T5-large (770M) in size. All experiments were performed on GTX-1080 and V100 GPUs.

Evaluation We evaluate model performance using the standard F_1 evaluation score, and report the average score over three random seeds.

	Base			Large		
	$\mathcal{S}_{\text{test}}^p$		$\mathcal{S}_{\text{test}}^{-p}$	$\mathcal{S}_{\text{test}}^p$		$\mathcal{S}_{\text{test}}^{-p}$
CLARIQ	30.5	27.4	10.2% ↓	30.3	26.3	13.2% ↓
DROP	75.7	31.8	58% ↓			
MULTIRC	40.5	33.7	16.8% ↓	42	37.4	11% ↓
PIQA	20.7	15	27.5% ↓	21.8	15.3	29.8% ↓
QUOREF	85.9	66	23.2% ↓	92.1	81.1	11.9% ↓
ROPES	57	42	26.3% ↓	57.8	58.2	0.7% ↑
SCIQA	80.7	79.7	1.2% ↓	82.8	81.9	1.1% ↓
Average	55.9	42.2	24.5% ↓	54.5	50	8.3% ↓

Table 8.5: Performance of T5 models on $\mathcal{S}_{\text{test}}^p$ vs. $\mathcal{S}_{\text{test}}^{-p}$ of models trained on data instances containing instruction patterns ($\mathcal{S}_{\text{train}}^p$).

8.3.2 Results

We observe similar results for T5 and BART, and thus, present only the results for T5 in detail and present quick discussion on results for BART.

Models often fail to generalize beyond instruction patterns. Table 8.5 shows the performance on $\mathcal{S}_{\text{test}}^p$ and $\mathcal{S}_{\text{test}}^{-p}$ when training only on examples with instruction patterns. Across all experiments, there are large performance gaps, reaching to 58% in DROP and $> 10\%$ in both base and large models for CLARIQ, MULTIRC, PIQA, and QUOREF. This indicates that models trained only on examples with instruction patterns fail to generalize to other task examples, and stresses that instruction bias should be monitored and avoided during data collection. Notably, the gap is lower for large models than for base ones, showing that large models are less sensitive to instruction bias. This might be attributed to their larger capacity to capture knowledge and skills during pre-training.

	Base			Large		
	$\mathcal{S}_{\text{test}}^p$	$\mathcal{S}_{\text{test}}^{-p}$		$\mathcal{S}_{\text{test}}^p$	$\mathcal{S}_{\text{test}}^{-p}$	
CLARIQ	30.5	26.9	11.8% ↓	30.7	26.9	12.4% ↓
DROP	76	78.9	3.8% ↑			
MULTIRC	40.5	39	3.7% ↓	42.4	44.5	5% ↑
PIQA	20.7	19.8	4.4% ↓	21.9	20.6	5.9% ↓
QUOREF	86.7	73.1	15.7% ↓	92.1	81.1	11.9% ↓
ROPES	59.9	48.7	18.7% ↓	59	64.2	8.8% ↑
SCIQA	80.6	80.2	0.5% ↓	82.8	82.9	0.1% ↑
Average	56.4	52.4	7.1% ↓	54.8	53.4	2.6% ↓

Table 8.6: Performance of T5 models on $\mathcal{S}_{\text{test}}^p$ vs. $\mathcal{S}_{\text{test}}^{-p}$ of models trained on $\mathcal{S}_{\text{train}}$.

Model performance is overestimated by instruction bias. We compare the performance on $\mathcal{S}_{\text{test}}^p$ and $\mathcal{S}_{\text{test}}^{-p}$ of models trained on the full training set (Table 8.6). The average performance across all datasets is higher on examples that exhibit instruction patterns by $\sim 7\%$ and $\sim 3\%$ for the base and large models, respectively. Specifically, base models perform worse on $\mathcal{S}_{\text{test}}^{-p}$ than on $\mathcal{S}_{\text{test}}^p$ for all datasets except DROP, in some cases by a dramatic gap of $> 15\%$ (e.g. 18.7% in ROPES and 15.7% in QUOREF). In contrast, results for the large models vary across datasets, while the performance gap is generally smaller in magnitude. This shows that model performance is often overestimated by instructions bias, and reiterates that large models are generally less sensitive to instruction patterns.

Results on BART Table 8.7 and Table 8.8 show the performance of BART on $\mathcal{S}_{\text{test}}^p$ and $\mathcal{S}_{\text{test}}^{-p}$ when training only on examples with instruction patterns and the full training set, respectively. From Table 8.7, there are large performance gaps reaching 45.9% in MULTIRC and $> 20\%$ in both base and large models for QUOREF,

	Base			Large		
	$\mathcal{S}_{\text{test}}^p$		$\mathcal{S}_{\text{test}}^{-p}$	$\mathcal{S}_{\text{test}}^p$		$\mathcal{S}_{\text{test}}^{-p}$
CLARIQ	30	26.2	12.7% ↓	29.2	25.6	12.3% ↓
MULTIRC	27.9	15.1	45.9% ↓	31.1	21	32.5% ↓
PIQA	21.9	15.3	30.1% ↓	23	16.1	30% ↓
QUOREF	78.8	45.8	41.9% ↓	87	58.6	32.6% ↓
ROPES	43.9	33.1	24.6% ↓	47.2	39.1	17.2% ↓
SCIQA	76.8	66.1	13.9% ↓	77.5	69.6	10.2% ↓
Average	46.6	33.6	27.9% ↓	49.2	38.3	22.2% ↓

Table 8.7: Performance of BART models on $\mathcal{S}_{\text{test}}^p$ vs. $\mathcal{S}_{\text{test}}^{-p}$ of models trained on data instances containing instruction patterns ($\mathcal{S}_{\text{train}}^p$).

	Base			Large		
	$\mathcal{S}_{\text{test}}^p$		$\mathcal{S}_{\text{test}}^{-p}$	$\mathcal{S}_{\text{test}}^p$		$\mathcal{S}_{\text{test}}^{-p}$
CLARIQ	29.8	26	12.8% ↓	29.4	26.3	10.5% ↓
MULTIRC	28.5	29.8	4.6% ↑	41.5	33.9	18.3% ↓
PIQA	22.3	20.5	8.1% ↓	23.1	21.6	6.5% ↓
QUOREF	80.5	61.2	24% ↓	87.8	73.4	16.4% ↓
ROPES	44	44.1	0.2% ↑	47.9	46.5	2.9% ↓
SCIQA	76.5	70.6	7.7% ↓	52.2	50.8	2.7% ↓
Average	46.9	42	10.5% ↓	47	42.1	10.4% ↓

Table 8.8: Performance of BART on $\mathcal{S}_{\text{test}}^p$ vs. $\mathcal{S}_{\text{test}}^{-p}$ of models trained on $\mathcal{S}_{\text{train}}$.

and PIQA. Overall, the average performance across all datasets is 27.9% and 22.2% higher on $\mathcal{S}_{\text{test}}^p$ for the base and large models, respectively. This indicates that both base and large models often fail to generalize beyond instruction patterns.

From Table 8.8, the average performance across all datasets is higher on examples that exhibit instruction patterns by $\sim 10.5\%$ for both base and large models. Thus, we can conclude that the model’s performance is overestimated by instruction bias.

8.4 Conclusions and Discussion

We identify a prominent source of bias in crowdsourced NLU datasets, called *instruction bias*, which originates in annotation instructions written by dataset creators. We study this bias in 14 NLU benchmarks, showing that instruction examples used to create NLU benchmarks often exhibit clear patterns that are propagated by annotators to the collected data. In addition, we investigate the effect of instruction bias on model performance, showing that instruction patterns can lead to overestimated performance as well as limit the ability of models to generalize to other task examples.

Based on our findings, we derive three recommendations for future crowdsourced NLU benchmarks: (1) Crowdsourcing instructions should be diverse; this could be achieved, for example, by having a large number of instructive examples, rephrasing examples using neural models, or periodically sampling examples from a *diverse* set of previously collected examples. The latter could be done, for example, by maintaining a pool of diverse examples during the collection process and then presenting every annotator with a different random sample from this growing pool. (2) Word patterns in collected instances should be analyzed during data collection, as well as possible correspondence to instruction examples. Such analysis will help researchers monitor the collection process and the quality of the resulting data. (3) Correlation between model performance and input patterns should be checked during evaluation.

This work covers 14 NLU datasets, for which annotation instructions are publicly available. However, most of these datasets are QA datasets. Our analysis can be extended to other NLU task categories, such as Natural Language Inference (NLI) and Relation Extraction (RE). Our study reveals a concrete bias that skews the collected data distribution toward specific patterns. While the effect of instruction examples

on collected data is prominent, it is hard to quantify how different the distribution of crowdsourced examples is from the natural distribution of the task. Concretely, to conduct a study that compares the distributions of crowdsourced versus natural complex reasoning questions, datasets of complex natural questions are needed. However, to the best of our knowledge, as of today, no such datasets exist.

In our analysis, we focused on shallow patterns based on word matching, however, it is known that there are other types of biases that are implicit in the text. Exploring these kinds of biases can be an interesting future direction. In addition, our analysis of model performance is based on splitting dataset instances based on the dominant pattern. However, it might be possible that there are more patterns, and the non-pattern subset might include other less frequent patterns. Hence, exploring the effect of different less frequent patterns on model learning can be a future work. Last, our work studied the effect of instruction bias on widely used generative models (i.e., T5 and BART); it would be valuable to investigate whether our findings hold in encoder-only models, such as BERT [47] and RoBERTa [132].

TARGEN: TARGETED DATA GENERATION

9.1 Introduction

Large Language Models (LLMs) like ChatGPT, Llama [240, 242], and Mistral [92] have showcased impressive results across a plethora of tasks [165, 20]. As LLM capabilities advance, the tools to test the extent of these capabilities become insufficient [130, 77, 250, 26]. This is particularly true for domain-specific datasets, as the creation of expertly curated evaluation benchmarks is time and labor-intensive [38, 230, 267]. Several synthetic dataset creation methods, such as Self-Instruct [266], AttrPrompt [301], and ZeroGen [291] have been proposed primarily for text classification tasks. These approaches employ in-context learning to generate synthetic data points that resemble their prompt exemplars, thereby inherently constraining their ability to produce diverse examples. Additionally, these approaches are often uniquely tailored to certain downstream tasks or datasets, reducing their adaptability. Consequently, they may be unsuited to synthesizing domain-specific tasks/benchmarks evaluating novel tasks where there is a lack of sufficient data samples to use as prompt exemplars.

To mitigate these issues, we introduce TarGEN, a multi-step prompting strategy (Figure 9.1). This approach consists of four steps. First, we initialize a set of contexts to inject semantic diversity, followed by the generation of task-specific elements we call “instance seeds”- elements that form the unique basis of each instance. These seeds can be sentences, passages, or more atomic elements, but are not input exemplars. Then, for each “instance seed”, we formulate a label constraint that uses these seeds to generate a data instance attributable to the constrained label. Finally, we leverage

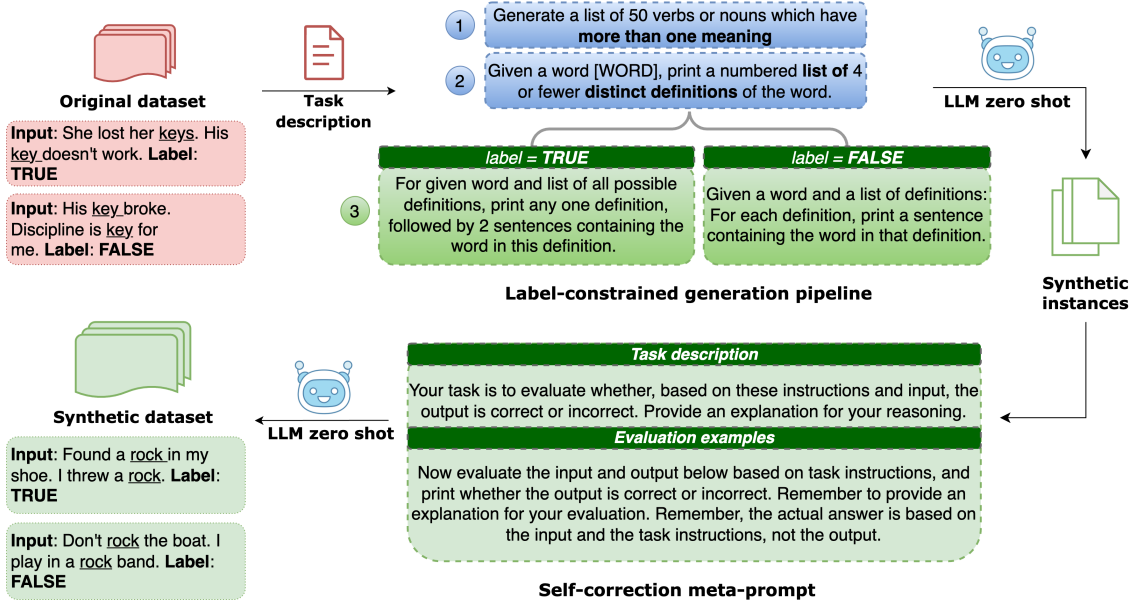


Figure 9.1: An overview of using TarGEN to generate instances for the WiC task. We first create a set of prompts (boxes 1, and 2 in the Figure) to generate instance seeds, or linguistic components unique to each task instance. Next, we create label-specific prompts (box 3) that generate instances based on instance seeds and the relationship implied by the label for this task. We use zero-shot LLM inference to generate an initial set of synthetic instances. The instances are then passed to our *self-correction* module, consisting of a single meta-prompt which allows us to re-label mislabeled data instances, helping us reduce noise. Hence, based on the task description, we obtain high-quality synthetic instances to evaluate a task.

our evaluator model for *self-correction* over the generated instances, and re-label them wherever necessary (§9.2.4). This method offers the benefit of not necessitating pre-existing task instances as starting points for generation. Consequently, it is suitable for domain-specific tasks and the generation of novel benchmarks, relying only on the provided task descriptions. The self-correction module integrated into the pipeline contributes to noise reduction and enhances the overall quality of generated content. Using TarGEN, we can produce high-quality, diverse datasets with precise labels.

To evaluate, we create a synthetic variant of the SuperGLUE [254] dataset using ChatGPT. We train several models from different families (encoder only, encoder-decoder, and decoder only) on the synthetically generated train set and the original

SuperGLUE train set, and evaluate these models on the original test set. We find that models trained on the synthetic train set perform at par as opposed to models trained on the original train set. Instruction tuning results in a 3.42% increase for Flan T5 models and a 3.24% improvement for Pythia GPT models (See detailed results in §9.3.1). We also conduct a comparative study between TarGEN and Self-Instruct [266] by pre-finetuning Llama 2 (7B) [242] on both datasets and evaluating them on the OpenLLM benchmark [38, 303, 78, 124]. Our findings indicate that Llama 2 pre-finetuned on our dataset outperforms Llama 2 trained using Self-Instruct by 2.62% points (49.20% vs 46.58% respectively).

We perform a thorough analysis of the datasets, which reveals the robustness of our synthetically generated SuperGLUE dataset in terms of dataset difficulty, diversity, and bias. It exhibits equivalent or higher dataset difficulty, as indicated by lower $\mathcal{V}$ -usable information [53], showcasing the adequate complexity of our datasets. Furthermore, our datasets have comparable lexical diversity [301] and consistently display lower cosine similarity between intra-dataset text pairs, highlighting the datasets’ rich and distinct content. Additionally, we measure the datasets’ lexical diversity in terms of vocabulary count. Our results indicate that the datasets generated using TarGEN consistently exhibit higher lexical diversity than their original counterparts. In terms of bias, our datasets align closely with the original datasets, demonstrating a balanced representation of various named entities such as Geopolitical Entities (GPE), Person Entities, and Nationalities or Religious or Political groups (NORP). A detailed analysis is present in §9.4.

9.2 TarGEN

In this section, we formulate the problem statement and explain the data generation pipeline. Also, we describe the datasets we intend to recreate from scratch.

9.2.1 Problem Formulation

Given a dataset for a language task t , its data points can be expressed as (d, l) such that there exists a function $f : \mathcal{D} \rightarrow \mathcal{L}$

$$f_t(d) = l, \forall d \in \mathcal{D}, l \in \mathcal{L} \quad (9.1)$$

where f_t is a mathematical representation of the task t , $d \in \mathcal{D}$ is the instance input, and l is the instance label from $\mathcal{L}$, the label space for the given task. We formalize dataset generation as a sequence of label-constrained text generation problems, where the generation of an instance is constrained by its label value. This allows us to control label distribution in our synthetic dataset and craft high-quality instances by clearly defining the relationships between instances and labels. This circumvents the need for any seed instances from the original dataset; i.e., for any given task, a dataset can be generated from scratch by formulating its generation function from the task description. The task-specific label-constrained dataset generation approach is given below:

$$\bigcup_{l \in \mathcal{L}} \bigcup_{n=1}^{N_l} (G_{l,t,n}(l, i_n), L = l) \quad (9.2)$$

where N_l is the number of samples for the label l , t is the task, and $G_{l,t}$ is an inverse function such that $f_t(G_{l,t}(l)) = l$, and i_n is the n^{th} instance seed. Formulating these task- and label-specific prompting strategies forms the crux of our simplified data synthesis pipeline. While these individual prompts are task-specific, the nature of these prompts and the sequence they occur follow a framework engineered to create diversity and improve coverage. The stages of this framework are as follows:

Step 1 We generate a set $\mathcal{C}$ of “contexts”, or “settings” that provide unique semantic scope, such as “geopolitical news”, “book review”, or “movie script”. These provide

contexts within which a model can simulate a naturally occurring excerpt, to maintain semantic diversity.

Step 2 We generate a set of passages, sentences, or task-specific elements, which we call “instance seeds”. These seeds form the basis for each instance of the task.

Step 3 We use the instance seeds as inputs to the generation prompt. This prompt is a descriptive formulation of the inverse generation function $G_{l,t}$, allowing us to generate data instances for the task.

Step 4 We pass all generated data instances through a single-step *self-correction* process, where we use a task-specific prompt to reinforce the task instructions and identify and correct any mislabeled instances. This helps reduce noise and improve overall dataset quality.

9.2.2 Task-specific prompting strategies

We study the pattern of each task and create multi-step prompting strategies tailored to each task. We generate a common set of contexts (Step 1) for almost all tasks.

Dataset Statistics We chose the following tasks from the SuperGLUE benchmark: 1. CommitmentBank (CB). 2. Choice of Plausible Alternatives (COPA). 3. Recognizing Textual Entailment (RTE). 4. Word-in-Context (WiC). 5. Winograd Schema Challenge (WSC). 6. BoolQ. 7. Reading Comprehension with Commonsense Reasoning (ReCoRD). and 8. Winogender Diagnostics (AX-g). Given the challenges posed by each dataset, we employ TarGEN to create synthetic instances that can be used to evaluate language model performance. Table 9.1 showcases the instances

Datasets	Task Type	Instances Split
Axg	NLI	Not Ent: 146 Ent: 138
Boolq	Bin. Class.	True: 2535 False: 1764
CB	NLI	Cont:119 Ent:115 Neut:16
Copa	Bin. Class.	Choice 1: 195 Choice 2: 107
Record	MCQ	1778 MCQs
RTE	NLI	Not Ent: 1241 Ent: 1249
Wic	Bin. Class.	True: 2433 False: 2410
Wsc	Bin. Class.	True: 259 False: 285

Table 9.1: Statistics of the SuperGLUE dataset. The following abbreviations are used Bin. Class: Binary Classification, Ent: Entailment, Cont: Contradiction, NLI: Natural Language inference

split used in the original and synthetic datasets. Since label generation is controlled, synthetic SuperGLUE was created to match the exact number of original instances, while maintaining a balanced label distribution¹. The exact data synthesis pipeline and prompts used for each dataset can be found in §9.2.4.

CB [45] We generate sample pairs that share the relationship between label $l \in \{\textit{entailment}, \textit{neutral}, \textit{contradiction}\}$, and based on a given context $c \in \mathcal{C}$.

COPA [200] We generate instances for CAUSE and EFFECT relations. In this case, Step 3 involves generating a premise and 2 hypotheses for a given context. For each relationship r , we generate (1) sentence pairs (P, C) such that the premise and hypothesis share the relationship specified, and (2) an alternate hypothesis C_{alt}

¹Due to ChatGPT budget constraint, ReCoRD dataset was truncated to have 1778 instances, BoolQ dataset had 4299 instances, and MultiRC dataset was skipped.

which explicitly does not share the specified relationship with the premise P . Thus $(P, C) \in r, (P, C_{alt}) \notin r$. The label space for this task is defined as $\mathcal{L} = \{C1, C2\}$. To ensure an even label split, we alternatively select attribute instances of C as $C1$ and C_{alt} as $C2$ and vice versa.

RTE A collection of textual entailment challenges. For this task, step 2 consists of generating a set of premises $\mathcal{P}$ as instance seeds. For each $p \in \mathcal{P}$, we then generate hypotheses that are either logically sound ($l = \textit{entailment}$) or logically unsound, i.e. do not follow the premise ($l = \textit{notentailment}$).

WiC [180] We curate a list of homonyms ($\mathcal{S}$) along with all possible definitions ($\mathcal{M}_s \forall s \in \mathcal{S}$). These words act as instance seeds. For each $m \in \mathcal{M}_s$, given the label *True*, we generate a pair of sentences ($d1, d2$) containing the word s , such that the definition of s in $d1$ and $d2$ is m . For the label *False*, we randomly choose $m1, m2 \in \mathcal{M}_s, m1 \neq m2$ and generate $d1, d2$ such that the definitions of s in $d1$ and $d2$ are $m1$ and $m2$ respectively, making them distinct in word sense.

WSC [117] For each context, we generate pairs of distinct noun phrases (N_1, N_2) with identical plurality. This pronoun and noun phrases act as instance seeds. For each pair ($N1, N2$), we then generate texts s containing $N1$ and $N2$ with all pronouns labeled with coreferent noun phrases. We randomly select an ambiguously coreferent pronoun P from the text. Based on this pronoun and the label constraint, we affix a noun phrase to the input instance.

BoolQ [36] For this task, we generate passages with multiple entities and inter-entity relations, which act as instance seeds in Step 2. In Step 3, we generate a query, based on the passage $p \in \mathcal{P}$ and the label constraint $l \in \{Yes, No\}$. In the case of

$l = \text{Yes}$, the query is information that can be inferred from the passage. For $l = \text{No}$, we generate a query that contradicts the passage.

ReCoRD [307] This dataset evaluates understanding implied entailment relationships. In Step 2, we generate a set of passages $\mathcal{A}$ to act as instance seeds. Next, for each article $a \in \mathcal{A}$, we generate a complex, context-relevant sentence, and subsequently obscure a single entity reference.

AX-g [203] For this task, we generate 10 subject pairs that are from the same domain space in Step 2. For each subject pair, we then generate an independent clause containing these subjects. These independent clauses are then used to generate dependent clauses coreferent with each subject, to act as instance seeds. In step 3, we use these subject-specific dependent clauses and the subject pairs to generate gender-agnostic hypotheses based on label constraints.

9.2.3 Self-correction

Despite learning capabilities, LLMs demonstrate inconsistent reasoning [295]. We remedy them by implementing *self-correction*, an evaluation strategy that corrects inconsistent labels in the data synthesis process. We leverage an LLM as an evaluator model (ChatGPT in this case) to verify the alignment between the generated instances and their labels, as well as the alignment between these instances and the task description. *Self-correction* consists of a single meta-prompt that is common to all tasks. The task instructions and task-specific validation examples are used to augment the meta-prompt and tailor it to each generated dataset. This meta-prompt and the task instructions are given below. Based on the provided input, the meta-prompt helps evaluate the correctness of its attributed label. If this label is deemed

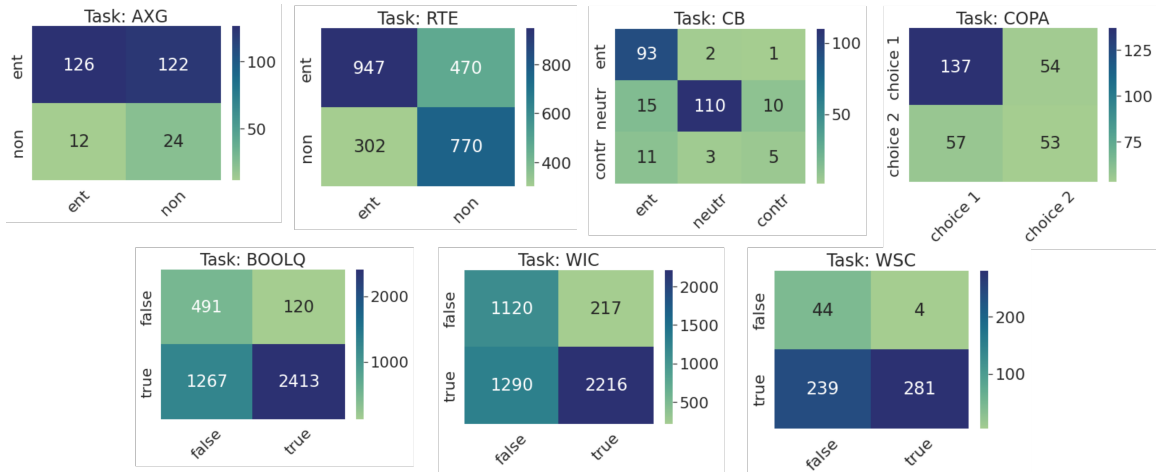


Figure 9.2: Matrices showing the effect of the *self-correction* step across various datasets of SuperGLUE. The row values show the number of labels that were originally assigned to that label (ent, non: entailment, non-entailment; neutr, contr: neutral, contradiction). The number in a cell (i, j) reflects the number of labels originally assigned to label i which were re-labeled to label j after self-correction. While the majority of the instances had their labels reaffirmed by self-correction, a significant number of instances were re-labeled as a result of this step.

not correct, the evaluator model generates the correct label based on the instructions and instance input. Figure 9.2 shows that a significant number of instances require relabeling, especially for high complexity tasks such as AX-g and WSC.

Evaluation Instructions

These are the *task instructions* you are given to accomplish a task:

insert task instructions here.

Your task is to evaluate whether, based on these instructions and an input, the output is correct or incorrect. Also provide an explanation for your reasoning.

insert task-specific self-correction samples here.

Now evaluate the input and output below based on task instructions, and print whether the output is correct or incorrect. Remember to provide an explanation for your evalua-

tion. Remember, the actual answer is based on the input and the task instructions, not the output.

Input:

Output:

Evaluation:

Task-specific instructions can be found in § 9.2.5. Task-specific self-correction examples follow the following schema:

For correct outputs

Actual result: *correct output*

Output: *correct output*

Based on this input and the given task instructions, the output is **CORRECT**.

Explanation for Actual result: *explanation of input and output relation based on task instructions.*

Actual result: *correct output*

Output: *correct output*

Actual result matches the output, so the output is **CORRECT**.

For incorrect outputs

Actual result: *correct output*

Output: *incorrect output*

Based on this input and the given task instructions, the output is **INCORRECT**.

Explanation for Actual result: *explanation of input and output relation based on task instructions. Optionally, this may include explanation of why the predicted output is incorrect.*

Actual result: *correct output*

Output: *incorrect output*

Actual result does not match the output, so the output is **INCORRECT**.

9.2.4 Task-specific Instance Generation Pipelines

COPA

Step 1: Generate a list of domains or settings in which events can take place.

Step 2: Generate N sentences describing events that could take place in the domain $[DOMAIN]$.

Step 3:

$query = CAUSE$. Add “What was the *CAUSE* of this?” to the premise during post-processing.

For the given sentence, generate 2 hypotheses (Hypothesis 1, Hypothesis 2), such that Hypothesis 1 is a probable cause of the sentence. Hypothesis 2 is very unlikely to be the cause of the sentence.

Example:

Sentence: I cast a long shadow.

Hypothesis 1: The sun was low in the sky.

Hypothesis 2: The grass was tall.

Explanation: Hypothesis 1, the low position of the sun is more likely to cause a long shadow. The height of the grass has nothing to do with the long shadow, and thus is unlikely to be a cause.

Sentence: $[SENTENCE]$

$query = RESULT$. Add “What was the *RESULT* of this?” to the premise during post-processing.

For the given sentence, generate 2 hypotheses (Hypothesis 1, Hypothesis 2), such that Hypothesis 1 is a probable result of the sentence.

Hypothesis 2 is very unlikely to be the result of the sentence.

Example:

Sentence: I fell down the stairs.

Hypothesis 1: I injured myself.

Hypothesis 2: My mother bought a new car.

Explanation: Hypothesis 1, the injury is more likely to be a result of the fall. The buying of a car is not implied by the sentence which talks about falling down stairs — hence it is less likely to be the result of the sentence.

Sentence: [SENTENCE]

Mathematical formulation of Step 3:

$$\begin{aligned} G_{l,t,r}(l, r = \text{result}) &= (P, C1, C2) \\ &: \begin{cases} P \Rightarrow C1, P \not\Rightarrow C2 & l = C1 \\ P \Rightarrow C2, P \not\Rightarrow C1 & l = C2 \end{cases} \\ G_{l,t,r}(l, r = \text{cause}) &= (P, C1, C2) \\ &: \begin{cases} C1 \Rightarrow P, C2 \not\Rightarrow P & l = C1 \\ C2 \Rightarrow P, C1 \not\Rightarrow P & l = C2 \end{cases} \end{aligned}$$

The relationship $r \in \{\text{cause}, \text{result}\}$ is affixed to the premise P as a query. The generated triplet $(P, C1, C2)$ and the constrained label $l \in \{C1, C2\}$ that answers the query form an instance of this dataset.

Self-correction

Instructions:

You are given a premise and 2 possible hypotheses (Choice 1 and Choice 2) as input.
Select the hypothesis which is more likely to have a causal link to the sentence.

If the premise asks for CAUSE: If the premise is more likely to be the result of Choice 1, output Choice 1. Otherwise, output Choice 2.

If the premise asks for RESULT: If Choice 1 is more likely to be the result of the premise, output Choice 1. Otherwise, output Choice 2.

CommitmentBank

Step 1: *Generate a list of domains or settings in which events can take place.*

Step 2: *N/A*

Step 3: *label = entailment*

For the given domain [DOMAIN], generate [N] pairs of sentences (Sentence 1, Sentence 2) such that Sentence 2 logically follows, or is implied by, Sentence 1.

Example:

Sentence 1: The singer was very nervous.

Sentence 2: The singer saw critics in the front row.

Now generate N such sentence pairs.

label = neutral

For the given domain [DOMAIN], generate [N] pairs of sentences (Sentence 1, Sentence 2) such that Sentence 2 has no relation with Sentence 1, and cannot be derived from it.

Example:

Sentence 1: I made coffee.

Sentence 2: My assignment is due tomorrow.

Now generate N such sentence pairs.

label = contradiction

For the given domain $[DOMAIN]$, generate $[N]$ pairs of sentences (Sentence 1, Sentence 2) such that Sentence 2 is explicitly contradicted by Sentence 1.

Example:

Sentence 1: The musician was not on time for the gig.

Sentence 2: The promoter praised the musician for being on time.

Now generate N such sentence pairs.

Mathematical formulation of Step 3:

$$G_{l,t}(l) = (P, H): \begin{cases} P \Rightarrow H & l = \text{entailment} \\ P \not\Rightarrow H & l = \text{neutral} \\ P \Rightarrow \neg H & l = \text{contradiction} \end{cases}$$

Each generated pair (P, H) of premise and hypothesis, along with its constrained label $l \in \{\text{entailment, neutral, contradiction}\}$, constitutes a synthetic instance for this dataset.

Self-correction

Instructions:

The input contains a premise and a hypothesis. Assume the premise is always true.

1. If the hypothesis logically follows the premise and the hypothesis can be derived from the information in the premise, output “entailment”.
2. If the hypothesis directly contradicts information in the premise, output “contradiction”.
3. If the hypothesis is unrelated to the premise, or cannot be sufficiently proven from the information in the premise, output “neutral”.

MultiRC

Step 1: List of categories: News*; Wikipedia*; History and anthropology*; Society, law, and justice*; Elementary school science textbooks*; 9/11 reports*; Fiction - literature or movie plots*; art; computer science; biological processes; physics; chemistry; linguistics; psychiatry and psychology; supernatural phenomena (* - present in original dataset)

Step 2: Generate 50 unique topics or titles in the category [CATEGORY]. Generate 7 short paragraphs on the following topic: [TOPIC]

Step 3: Given a paragraph, frame a question that requires information from multiple sentences of the paragraph to be answered correctly. Then, generate a set of options. The correct answer may be a combination of one, some, or all options. Also include options that do not answer the above question. Finally, output the combination of options that form the correct answer.

Paragraph: [PARAGRAPH]

Mathematical formulation of Step 3: Not applicable in this case

Self-correction

Instructions:

You are given a passage followed by a question and a list of options. Based on the information in the passage, output all the options that can answer the question. The output must not include options that do not answer the question. The output must not lack any options that answer the question.

RTE

Step 1: Generate a list of topics or domains to talk about.

Step 2: For the given domain [DOMAIN], generate N complex sentences containing information relevant to this domain.

Step 3:

label = entailment

Given a sentence as premise, add a logically sound hypothesis concerning the information in the premise.

Premise: A place of sorrow after Pope John Paul II died became a place of celebration as Roman Catholic faithful gathered in downtown Chicago to mark the installation of new Pope Benedict XVI.

Hypothesis: Benedict XVI is the new Catholic Pope.

Premise: [SENTENCE]

label = non-entailment

Given a sentence as premise, add a logically unsound hypothesis concerning the information in the premise.

Premise: A place of sorrow after Pope John Paul II died became a place of celebration as Roman Catholic faithful gathered in downtown Chicago to mark the installation of new Pope Benedict XVI.

Hypothesis: Benedict XVI died recently.

Premise: [SENTENCE]

Mathematical formulation of Step 3:

$$G_{l,t,p}(l,p) = (P = p, H): \begin{cases} P \Rightarrow H & l = \text{True} \\ P \not\Rightarrow H & l = \text{False} \end{cases}$$

The generated premise and hypothesis pairs (P, H) , as well as their associated label $l \in \{\text{True}, \text{False}\}$, form the binary labeled instances for this task.

Self-correction

Instructions:

You are given a premise and a hypothesis. If the hypothesis logically follows from the premise, output entailment. If the hypothesis cannot be logically derived from the information in the premise, output not entailment.

WSC

Step 1: *Generate a list of domains or settings in which events can take place and where people can interact. Examples: “A restaurant”, “a museum”, “a lively rock concert”, “An opera house”*

Step 2: *Identify a pair of subjects (Subject 1, Subject 2) in the context of [DOMAIN]. For example, in the context of “a classroom”, a pair of subjects could be (Subject1 : teacher, Subject 2 : student), or (Subject1: students, Subject2: rivals). For the given domain, generate [10] such subject pairs.*

For 5 pairs, both subjects must be singular.

For 5 pairs, both subjects must be plural.

Both subjects must be humans or groups of humans.

Output all 10 pairs in a numbered list.

Step 3: *For a given subject pair (Subject 1, Subject 2), generate 2 passages, S1 and S2. Each passage must be 2 sentences or fewer. You are also given the gender of both subjects.*

For every pronoun in the sentences, identify which subject is being referred to.

Example:

Input:

Subject 1: Teacher, Subject 2: Student

Pronouns: He/him

Output:

S1: The teacher was disappointed in the student because [he=teacher] had

high hopes for [him=student].

S2: The teacher and the student are not on good terms. [He=student] is very rebellious, and does not show up to classes.

Explanation: In S1: It is clear that the student has disappointed the teacher, who had high hopes for the student. Hence, “he had high hopes” — “he” refers to the teacher, “for him” — “him” refers to the student. In S2: The student is rebellious and does not show up to classes — hence the “he” in “he is very rebellious” refers to the student.

Input:

Subject 1: [SUBJECT 1], Subject 2: [SUBJECT 2]

Pronouns: (randomly chosen)

Output:

Mathematical formulation of Step 3:

$$G_l = \begin{cases} (s, N1, P): N1 \leftrightarrow P & l = \text{True} \\ (s, N2, P): N2 \leftrightarrow P & l = \text{True} \\ (s, N1, P): N2 \leftrightarrow P & l = \text{False} \\ (s, N2, P): N2 \leftrightarrow P & l = \text{False} \end{cases}$$

where s contains $N1$, $N2$, and P . We denote coreference with the $\leftrightarrow$ operator.

Self-correction

Instructions:

The given input talks about 2 noun phrases (Subject 1 and Subject 2). You are given Subject 1 and a Pronoun that occurs in the input.

Rules:

1. If the pronoun refers to the noun phrase Subject 1, the task output is *TRUE*.
2. If the pronoun refers to the noun phrase Subject 2, the task output is *FALSE*.

BoolQ

Step 1: Generate a list of domains to write an article in.

Step 2: Generate 50 unique topics or titles in the category [DOMAIN]. Generate 7 short paragraphs on the following topic: [TOPIC]

Step 3: label = *TRUE*

You are given a passage. Generate a boolean query. The answer to this query, based on the passage, must be *YES*.

Example:

Passage: The Millennium Falcon, a legendary starship piloted by Han Solo and Chewbacca, has become an iconic symbol of rebellion and hope in the struggle against the oppressive Galactic Empire. May the Force be with you, as the epic adventures of Luke Skywalker, Princess Leia, and Darth Vader remind us that even in the darkest times, there is always a glimmer of light and a chance for redemption in the Star Wars universe.

Query: Does Han Solo work with Chewbacca?

Answer: YES, Han Solo and Chewbacca pilot the Falcon together.

Similarly, generate a query for the following passage.

Passage: [PASSAGE]

Query:

label = *FALSE*

You are given a passage. Generate a boolean query. The answer to this query, based on the passage, must be *NO*.

Example:

Passage: The Millennium Falcon, a legendary starship piloted by Han Solo and Chewbacca, has become an iconic symbol of rebellion and hope in the struggle against the oppressive Galactic Empire. May the Force be with you, as the epic adventures of Luke Skywalker, Princess Leia, and Darth Vader remind us that even in the darkest times, there is always a glimmer of light and a chance for redemption in the Star Wars universe.

Query: Does Han Solo work alone?

Answer: NO, Han Solo works with Chewbacca.

Similarly, generate a query for the following passage.

Passage: [PASSAGE]

Query:

Mathematical formulation of Step 3:

$$G_{l,t,p}(l,p) = (P = p, Q): \begin{cases} P \Rightarrow Q & l = \text{Yes} \\ P \Rightarrow \neg Q & l = \text{No} \end{cases}$$

Self-correction

Instructions:

You are given a passage followed by a question. The questions ask for confirmation of a fact that may or may not be present in the passage. If the passage explicitly confirms the fact being asked in the question, output TRUE as the answer to the question. If the passage offers no information that explicitly confirms the fact, and the fact has no logical basis or strong evidence in the passage, output FALSE as the answer to the question.

Step 1: Generate a list of domains to write an article in.

Step 2: Generate 50 unique topics or titles in the category [DOMAIN]. Generate 7 short paragraphs on the following topic: [TOPIC]

Step 3: Example:

As a spring breeze wafted into his trench French commander Georges Lamour saw something surreal drift his way - a yellow-green cloud. 'All my trenches are choked,' he cried into the field telephone to headquarters. 'I am falling myself!' Chlorine gas — carried by favourable winds over Flanders Fields from German positions — had been used for the first time. It was April 22, 1915. Scroll down for video Chlorine gas — carried by favourable winds over Flanders Fields from German positions — sowed terror and agony for the first time on April 22, 1915. Above, German Red Cross workers carry bottles of water to help revive troops. German forces launched first attack using gas on April 22, 1915. 150,000 tons of gas were used by German and Allied forces in WW1.

Query: Had they been able to peer a bit further across no-man's land they would have seen how [X] troops had dug in, under cover of night, more than 5,000 gas cylinders with tubes pointing their way.

Answer: German

Explanation: The query fits the context of WW1 and talks about both entities involved in the war. The paragraph mentions German forces using gas. German is the entity replaced by [X] in the query.

Instruction: Generate a complex sentence that fits the context of the given paragraph.

The generated query must be a statement about the events in the paragraph. Put [X] in place of any one entity mention. The query must not contain any events mentioned in the paragraph. The answer must contain the entity mention that can replace [X].

Query:

Mathematical formulation of Step 3:

$$G_{a,t}(a) = (A = a, s, e) : s, e \in a, l = e$$

where a is the article, and s, e are the paraphrased sentence (with one entity e obscured) as well as the entity e obscured acting as the label.

Self-correction

Instructions:

You are given a passage, followed by a query. The query contains $[X]$ in place of any one entity mentioned. Output the entity that could logically replace $[X]$ in the query.

AXg

Step 1: *Generate a list of domains or settings in which events can take place and where people can interact. Examples: “A restaurant”, “a museum”, “a lively rock concert”, “An opera house”*

Step 2: Identify a pair of subjects (Subject 1, Subject 2) in the context of [DOMAIN]. For example, in the context of “a classroom”, a pair of subjects could be (Subject1: teacher, Subject 2: student), or (Subject1: students, Subject2: rivals). For the given domain, generate [10] such subject pairs.

For a given pair of subjects, generate 4 sentences with the following specifications:

Initial clause: A clause containing Subject 1 and Subject 2

Sentence 1: Initial clause with a dependent clause containing a gendered pronoun. Dependent clause should refer to Subject 1, not Subject 2.

Sentence 2: Completely identical to sentence 1 but with a different gendered pronoun

Sentence 3: Initial clause with a dependent clause containing a gendered pronoun. Dependent clause should refer to Subject 2, not Subject 1.

Sentence 4: Identical to sentence 3 but with a different gendered pronoun

Ensure the dependent clauses in Sentence 1 and Sentence 2 refer to Subject 1 and Subject 2 respectively. Sentences 1 and 2 must be identical in terms of dependent, with nothing but the pronoun changed. Similarly, Sentences 3 and 4 must be identical, with only the pronoun changed.

Sentences 1 to 4 must start with the same clause.

Only one gendered pronoun must be present in the dependent clauses.

All sentences should make logical sense.

Now generate 4 sentences according to these specifications for the following subjects.

Subject 1: [SUBJECT 1]

Subject 2: [SUBJECT 2]

Step 3:

label = entailment Given a sentence, (*independent clause + dependent clause*), and the subject being referred to in its dependent clause: generate a sentence containing the subject, which logically follows the sentence. This generated sentence should have no gendered pronouns.

label = non-entailment Given a sentence, (*independent clause + dependent clause*), and the subject being referred to in its dependent clause: generate a sentence containing the subject, which does not logically follow the sentence. This generated sentence should have no gendered pronouns.

Mathematical formulation of Step 3:

$$G_{l,t}(s, i, d) = (i, d, h)$$

$$\Rightarrow \begin{cases} (s, d) \Rightarrow h & l = \text{entailment} \\ (s, d) \not\Rightarrow h & l = \text{not entailment} \end{cases}$$

where i and d refer to the independent and dependent clauses respectively, s is the subject that the dependent clause is coreferent with, and h is the generated gender-agnostic hypothesis based on the subject s and instance label $l \in \{\text{entailment}, \text{not entailment}\}$.

Self-correction

Instructions:

You are given a premise and a hypothesis. If the hypothesis logically follows from the premise, output entailment. If the hypothesis cannot be logically derived from the information in the premise, output not entailment.

WiC

Step 1: N/A

Step 2: *Generate a list of 50 verbs or nouns which have more than one meaning. Given a word [WORD], print a numbered list of 4 or fewer distinct definitions of the word.*

Example:

Word: shoot

Definitions:

- 1. to fire a bullet*
- 2. click a picture*
- 3. record on video*
- 4. a movie set.*

Word: [WORD]

Definitions:

Step 3: *label = TRUE*

For given word and list of all possible definitions, print any one definition, followed by 2 sentences containing the word in this definition.

Example:

Word: key

Definitions:

1. a piece of shaped metal used to open or close a lock
2. a button or lever on a keyboard or musical instrument
3. a crucial or central element
4. to provide something with a key or identifying code

Chosen definition:

1. a piece of shaped metal used to open or close a lock

Sentences:

1. I lost my key yesterday
2. He shouldn't steal people's keys.

Word: [WORD]

Definitions: [DEFINITIONS]

Chosen definition:

label = FALSE

Given a word and a list of definitions: For each definition, print a sentence containing the word in that definition.

Example:

Word: key

Definitions:

1. a piece of shaped metal used to open or close a lock
2. a button or lever on a keyboard or musical instrument
3. a crucial or central element
4. to provide something with a key or identifying code

Sentences:

1. I lost my key yesterday
2. This key on the piano is out of tune.
3. The key to victory is planning ahead.
4. I don't know what to key in to gain access.

Explanation:

1. I lost my [key] yesterday - here [key] means 1. a piece of shaped metal used to open or close a lock
2. This [key] on the piano is out of tune. - here [key] means 2. a button or lever on a keyboard or musical instrument
3. The [key] to victory is planning ahead. - here [key] means 3. a crucial or central element
4. I don't know what to [key] in to gain access. - here [key] means 4. to provide something with a key or identifying code

Word: [WORD]

Definitions: [DEFINITIONS]

Sentences:

Mathematical formulation of Step 3:

$$G_{l,t}(l) = (d_1, d_2)$$

$$\Rightarrow \begin{cases} f(s, d_1) = f(s, d_2) & l = \text{True} \\ f(s, d_1) \neq f(s, d_2) & l = \text{False} \end{cases}$$

where $s \in \mathcal{S}$ and $f(s, d_1) = m \in \mathcal{M}_s$ is the word sense of s in the context of d_1 .

Self-correction

Instructions:

You are given a word (Keyword) and 2 sentences, both containing the Keyword.

- 1. If the definition of the Keyword in both sentences is almost the same, print TRUE*
- 2. If the Keyword means something different in sentence 1 than in sentence 2, print FALSE.*

9.2.5 Prompts for Instruction Tuning

CommitmentBank Instructions

The input contains a premise and a hypothesis.

1. If the hypothesis logically follows the premise and the hypothesis can be derived from the information in the premise, Output “entailment”
2. If the hypothesis directly contradicts information in the premise, Output “contradiction”
3. If the hypothesis is unrelated to the premise, or cannot be sufficiently proven from the information in the premise, Output “neutral”

Example:

Premise: “There is no food in the larder, we never went shopping! We could starve!”, she cried.

Hypothesis: She did not go shopping for food.

Output: entailment

Explanation: The speaker is complaining about the lack of food due to not going shopping. The hypothesis is strongly supported by the information in the premise. Hence, the output is entailment, i.e. the premise entails the hypothesis.

Premise: The diplomatic summit was a failure. The hostility between the representatives led to a breakdown in the talks. The world watched as they traded jibes and ended the meeting on bad terms.

Hypothesis: A diplomatic solution was reached through negotiation.

Output: contradiction

Explanation: The premise relates the failure of diplomatic talks, and implies lack of negotiation, contradicting the hypothesis. Hence, the output is contradiction.

Premise: "I hope the cat is well," I said. I was lying, because I wished the cat to be dead.

Hypothesis: The cat is dead.

Output: neutral

Explanation: The premise mentions the speaker wishes the cat was dead, but says they hope it is alive. From the premise, no conclusion can be drawn about the actual state of the cat. Since the premise cannot provide evidence to confirm or deny the hypothesis, the output is neutral.

Premise: [PREMISE]

Hypothesis:

COPA Instructions

You are given a premise and 2 possible hypotheses (Choice 1 and Choice 2) as input. Select the hypothesis which is more likely to have a causal link to the sentence.

If the premise asks for CAUSE:

If the premise is more likely to be the result of Choice 1, output Choice 1. Otherwise, output Choice 2.

If the premise asks for RESULT:

If Choice 1 is more likely to be the result of the premise, output Choice 1. Otherwise, output Choice 2.

Example:

Premise: My body cast a shadow over the grass. What was the CAUSE of this?

Choice 1: The sun was rising.

Choice 2: The grass was cut.

Output: Choice 1

Explanation: The premise asks for CAUSE. Out of the 2 choices, choice 1 describing the position of the sun is more likely to cause a shadow. The grass being cut has no relation.

Example:

Premise: The elderly woman suffered a stroke. What happened as a RESULT?

Choice 1: The woman's daughter came over to clean her house.

Choice 2: The woman's daughter moved in to take care of her.

Output: Choice 2

Explanation: The premise asks for RESULT. Choice 2 is more likely to logically follow the premise of the sick elderly woman. There is nothing to suggest cleaning the house.

Premise: [PREMISE]

Choice 1: [CHOICE 1]

Choice 2: [CHOICE 2]

MultiRC Instructions

You are given a passage followed by a question and a list of options. Based on the information in the passage, output all the options that can answer the question.

Example:

Passage:

Bioluminescence is a captivating natural phenomenon that illuminates the depths of our oceans and various terrestrial environments. It is the mesmerizing ability of living organisms to produce light through chemical reactions within their bodies. This enchanting light emission occurs primarily in marine creatures such as glowing plankton, jellyfish, and deep-sea creatures, turning the oceanic world into a dazzling light show. The mesmerizing glow serves a range of purposes, from attracting prey and mates to warding off predators. Bioluminescence not only adds a magical touch to the hidden realms of the Earth but also remains an essential area of scientific research, offering insights into evolutionary adaptations and potential biomedical applications. Exploring the mysteries of bioluminescence continues to unveil the secrets of these glowing organisms and further ignites our curiosity about the wonders of the natural world.

Question: Which organisms demonstrate bioluminescence?

Options:

- A) Glowing plankton
- B) Deep-sea creatures
- C) Terrestrial environments
- D) Jellyfish
- E) Land-based organisms

Output:

- A) Glowing plankton
- B) Deep-sea creatures
- D) Jellyfish

Passage: [PARAGRAPH]

Question: [QUERY]

Options: [LIST OF OPTIONS]

Output:

RTE Instructions

You are given a premise and a hypothesis. If the hypothesis logically follows from the premise, output entailment. If the hypothesis cannot be logically derived from the information in the premise, output not entailment.

Premise: My car ran out of diesel and I had to walk 6 miles to my house.

Hypothesis: My car needs diesel to run.

Output: entailment

Premise: The diplomacy talks mark a gradual lessening of tensions between the UK and Argentina.

Hypothesis: Argentina and UK are heading towards a war.

Output: not entailment

Premise: [PREMISE]

Hypothesis: [HYPOTHESIS]

Output:

WiC and WSC Instructions

You are given a word (Keyword) and 2 sentences, both containing the Keyword.

1. If the definition of the Keyword in both sentences is almost the same, print TRUE.
2. If the Keyword means something different in sentence 1 than in sentence 2, print FALSE.

Example:

Word: shoot

Sentence 1: he shot the wedding with a handheld camera

Sentence 2: he shot me with a gun

Output: FALSE

Example:

Word: shoot

Sentence 1: the shoot was suspended due to the actor's absence

Sentence 2: the director wrapped the shoot up by evening.

Output: TRUE

Word: [WORD]

Sentence 1: [SENTENCE 1]

Sentence 2: [SENTENCE 2]

Output:

The given input talks about 2 noun phrases (Subject 1 and Subject 2). You are given Subject 1 and a Pronoun that occurs in the input.

Rules:

1. If the pronoun refers to the noun phrase Subject 1, output LABEL: TRUE.
2. If the pronoun refers to the noun phrase Subject 2, output LABEL: FALSE.

Explain your reasoning.

Example:

Input: The city councilmen refused the demonstrators a permit because they feared violence.

Subject 1: The city councilmen.

Pronoun: They.

Output: TRUE

Explanation:

Noun phrases: city councilmen (Subject 1); demonstrators (Subject 2).

The pronoun “they” occurs in the phrase “they feared violence.” Out of the two subjects, demonstrators are more likely to commit violence, and city councilmen are more likely to fear the violence. This phrase must be talking of the city councilmen (Subject 1). This follows Rule 1.

Example:

Input: The scientist studied the lion because he was paid to do so.

Subject 1: lion.

Pronoun: he

Output: FALSE

Explanation:

Noun phrases: lion (Subject 1); scientist (Subject 2).

The pronoun “he” occurs in the phrase “he was paid to do so.” This is unlikely to refer to the lion, as the scientist (Subject 2) is more likely to be paid to study something. Thus, the pronoun must refer to Subject 2.

Input: [INPUT]

Output:

BoolQ Instructions

You are given a passage followed by a question. The questions ask for confirmation of a fact that may or not be present in the passage. If the passage explicitly confirms the fact being asked in the question, output TRUE as the answer to the question. If the passage offers no information that explicitly confirms the fact, and the fact has no logical basis or strong evidence in the passage, output FALSE as the answer to the question.

Passage:

In J. R. R. Tolkien's Middle-earth, the Half-elven (Sindarin singular Peredhel, plural Peredhil, Quenya singular Perelda) are the children of the union of Elves and Men. Of these, the most significant were the products of couplings between the Eldar (the Elves who followed the Call to Valinor) and the Edain (the Men of the Three Houses of early Men who allied themselves with the Eldar in their war against Morgoth).

Question:

can elves and humans mate lord of the rings

Answer: TRUE

Explanation: The fact needing confirmation is whether elves and humans can mate, in the context of Lord of the Rings, by J R R Tolkien. The passage clearly mentions that in his universe, children of elves and humans or men exist, thus proving that elves and humans can mate in this universe. Hence, the answer to the question is TRUE.

Passage:

Over the years, the Movie Maker has undergone various updates that added improvements to the software. It even integrates with other tools to provide a more advanced level of editing. However, it is still a basic video editing app. If you are looking to create a more professional-looking movie, apps such as Adobe Premiere Pro are your best option.

Question:

Is the movie maker app used by the film industry

Answer: FALSE

Explanation: The question asks to confirm the fact whether the movie maker app is used by the film industry. The passage clearly states that movie maker is a basic video editing app. Furthermore, the passage mentions Adobe Premiere Pro specifically as a professional tool. As any industry uses professional tools, the passage clearly contradicts the fact presented in the question. Hence, the answer is FALSE.

Passage: [PARAGRAPH]

Question: [QUERY]

Answer:

ReCoRD Instructions

You are given a passage, followed by a query. The query contains [X] in place of any one entity mentioned. Output the entity that could logically replace [X] in the query.

Example:

Passage: Georges Lamour saw something drift his way - a yellow-green cloud. 'All my trenches are choked,' he cried. Chlorine gas was carried by winds over Flanders Fields from German positions. Chlorine gas sowed terror and agony for the first time on April 22, 1915. Red Cross workers carried bottles of water. German forces launched their first attack using gas on April 22, 1915. 150,000 tons of gas were used by German and Allied forces in WW1.

Query: Had they been able to peer a bit further, they would have seen how [X] troops had dug in, under cover of night, more than 5,000 gas cylinders with tubes pointing their way.

Output:

[X]: German

Passage:

Query: [QUERY]

Output:

[X]:

AXg Instructions

You are given a premise and a hypothesis. If the hypothesis logically follows from the premise, output entailment. If the hypothesis cannot be logically derived from the information in the premise, output not entailment.

Example:

Input:

Premise: The paralegal forgot to file the client's paperwork, so he was fired.

Hypothesis: The client was fired.

Output: not entailment

Explanation: The paralegal made a mistake here, and clients are not likely to be fired.

Hence, the hypothesis does not make logical sense. Hence, output "non-entailment".

Example:

Input:

Premise: The athlete stole the cheerleader's uniform so she could wear it to a party.

Hypothesis: The athlete was going to a party.

Output: entailment

Explanation: Despite gender stereotypes, "she" here refers to the athlete, not the cheer-

leader. According to the premise, the athlete stole the uniform so that the athlete could wear it to a party, not the cheerleader, who does not have the outfit any longer. The hypothesis that the athlete was strongly evidenced by the premise. Hence, output “entailment”.

Input:

Premise: [PREMISE]

Hypothesis: [HYPOTHESIS]

Output:

9.3 Experiments and Results

We train five models in the single-task learning (STL) setting for original and synthetic datasets separately. We evaluate them on the original test set to measure performance. The above experiments are repeated in instruction tuning settings as well. We also perform a multi-task learning (MTL) experiment where a T5-3B is finetuned on all original and synthetic datasets separately in a multi-task fashion [154]. The results in §9.3.1 are the average of five runs. We use 6xNVIDIA Tesla P40 GPUs. Batch Size: 16 for STL and 1 MTL setting. Gradient Accumulation Steps: 1, Learning rate: 5e-5, Number of Epochs: 5 for STL and 1 MTL setting.

Models The following models are used: RoBERTa Large (354M) [132], Pythia GPT (410M) [14], Cerebras GPT (590M) [48], Flan T5 Large (780 M) [35], T5 Large (780M) [190] and T5-3B in MTL setting.

Metric Following the SuperGLUE benchmark, we use accuracy for all dataset tasks.

9.3.1 Results

Table 9.2 showcases the results of all the models trained on both individually on original and the synthetic dataset. In the traditional finetuning setting, the best synthetic datasets’ scores are nearly the same or slightly higher compared to the original datasets. However, there is a considerable jump of $\sim 3\%$ in performance when instruction tuning [154, 271, 212] is used. Table 9.3 gives the average model-wise results for the same. Table 9.4 denotes the results when the datasets are trained in a multitask fashion on the T5-3B model. Since we use a larger model, the results are significantly better than single-task learning results. Flan T5 gets the highest jump from 81.12 in the original dataset to 84.54 in the synthetic, with instructions which could be attributed to the degree of instruction tuning Flan has gone through. RoBERTa has the lowest jumps between scores, with just 79.98 in original to 82.08 in synthetic with instructions.

9.4 Analysis

In this section, we present a comprehensive analysis of the synthetic data generated, examining both quantitative and qualitative aspects.

Lexical Dataset Diversity We analyze the dataset diversity, along the lines presented in [301]. We initiate our exploration with a straightforward vocabulary-based examination to assess lexical diversity, as summarized in Table 9.5. Notably, our TarGEN synthetic data exhibits an average lexical diversity that is 25% higher across various dataset tasks.

Semantic Dataset Diversity To analyze the semantic dataset diversity, we examine the cosine similarity distribution of SentenceBERT embeddings of within-dataset

	Asg				Boolq			
Models	Og	Syn	Og-I	Syn-I	Og	Syn	Og-I	Syn-I
Cerebras	73.23	76.00	72.80	77.63	84.12	86.11	84.77	88.61
Pythia	74.90	78.43	74.57	79.80	83.29	84.02	82.61	85.05
T5	77.12	79.14	77.65	79.84	84.43	85.64	84.16	86.87
Flan	78.03	80.23	76.20	81.70	84.43	86.01	85.08	88.21
RoBERTa	77.10	78.17	76.35	82.14	84.43	84.56	84.07	87.56
	Copa				RTE			
Cerebras	80.56	81.98	80.01	82.11	81.12	83.25	81.02	85.42
Pythia	79.98	80.74	80.54	83.27	82.98	83.65	82.85	87.87
T5	82.12	83.34	82.32	85.91	86.18	86.10	86.79	88.92
Flan	81.78	82.41	82.36	86.24	88.91	90.19	89.86	89.22
RoBERTa	82.12	83.98	80.99	83.16	88.20	89.01	88.32	88.84
	CB				Record			
Cerebras	88.21	89.15	88.60	88.93	67.88	68.12	68.42	71.45
Pythia	91.93	89.63	89.74	93.03	68.83	69.19	68.61	73.36
T5	90.32	92.43	89.84	92.05	71.13	72.23	70.44	72.07
Flan	89.02	93.21	88.06	94.32	70.21	70.11	71.58	76.83
RoBERTa	87.86	90.12	92.36	90.48	69.88	70.21	70.29	70.02
	Wic				Wsc			
Cerebras	66.78	68.72	66.70	70.22	82.12	85.28	86.31	90.66
Pythia	68.33	71.77	70.53	70.84	86.71	87.32	86.44	88.09
T5	68.01	71.13	68.35	69.41	85.56	85.95	85.91	88.43
Flan	70.12	72.45	69.29	71.43	86.23	88.13	88.08	88.35
RoBERTa	69.90	70.06	70.16	71.23	87.21	85.65	82.37	83.21

Table 9.2: Performance of various models over SuperGLUE tasks. For each dataset, we compare the performance of these models (Cerebras, Pythia, T5, FLAN, RoBERTa) when trained on 4 distinct variants: Og (original train data), Syn (synthetic train data), Og-I (inst. tuning original train data), Syn-I (inst. tuning synthetic train data). For each task, we denote the highest performing model trained on original train data in green, and the highest performing model trained on synthetic train data in blue. All results are presented in %.

Model	Og	Syn	Og-I	Syn-I
Cerebras	78.64	79.83	78.80	81.88
Pythia	79.48	80.59	79.49	82.66
T5	80.72	82.00	80.83	82.94
Flan	81.12	82.84	81.49	84.54
RoBERTa	79.98	81.47	80.08	82.08

Table 9.3: Avg. performance across tasks, models, and data variants. Og, Syn, and I represent Original, Synthetic, and Instruction tuning, respectively.

Dataset	Boolq	Wic	CB	Axg	Record	RTE	Wsc	Copa
Original	89.02	68.42	84.54	41.66	68.52	87.72	72.11	94.65
Synthetic	90.04	74.78	93.22	49.13	76.59	92.41	73.07	95.20

Table 9.4: Results using a T5-3B model trained in a multi-task fashion. Original: Using a combined set of original datasets to train the model. Synthetic: Using synthetic versions to train the model. All numbers are in %. We measure performance in accuracy, except in the case of ReCoRD, where we use the Rouge-L score. Higher score is highlighted. We find that model trained on synthetic data is significantly better than the one trained on original data.

sample pairs as presented in Figure 9.3. Across most SuperGLUE tasks, the TarGEN datasets consistently display lower cosine similarity than the original dataset, indicating reduced semantic similarity of within-dataset samples and, consequently, higher semantic diversity. This observation underscores our approach’s intrinsic capability to generate diverse samples. Moreover, our findings of higher cosine similarity of the original datasets align with those of [172], where the authors highlight the propensity for crowdsourced datasets to exhibit high bias and low diversity. This phenomenon arises as crowdsourced workers often adhere to patterns provided by dataset creators. The dataset generated using our approach can generate samples with more diversity.

Dataset	Boolq	Wic	CB	Axg	Record	RTE	Wsc	Copa
Original	251.8k	65.8k	6.4k	3.2k	160.3k	79.1k	8.8k	5.1k
Synthetic	190.6k	76.4k	16.6k	4.4k	236.7k	49.4k	8.9k	5.2k

Table 9.5: Lexical diversity of the dataset. Figures in red correspond to where the diversity of the synthetic dataset is limited. Blue corresponds to where the synthetic dataset has more diversity than the original.

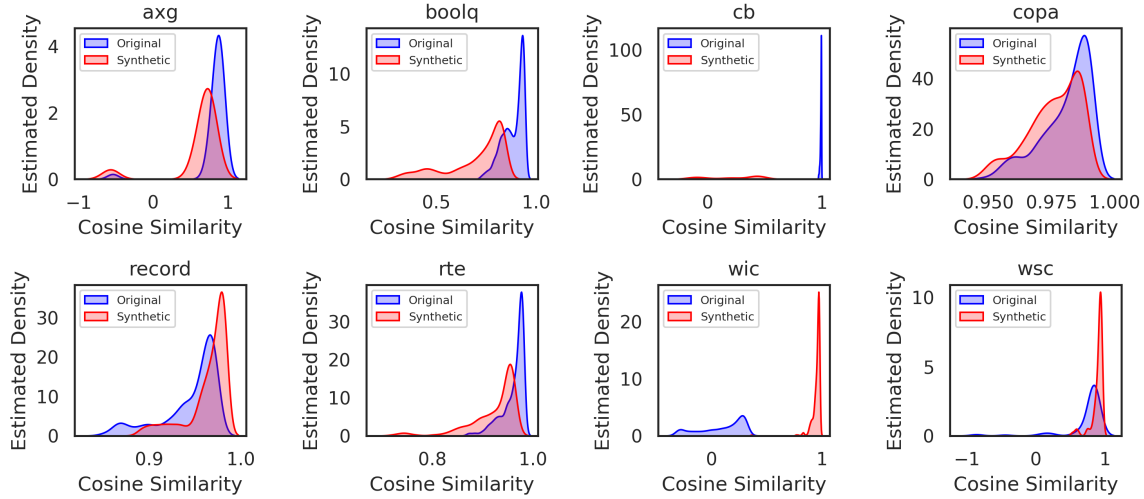


Figure 9.3: Comparison of semantic diversity across datasets among the original and the synthetically generated dataset. It can be seen that the original datasets’ cosine similarity is higher for most tasks as compared to the synthetic datasets, which have a consistently lower cosine similarity, indicating higher semantic diversity.

Dataset Difficulty To estimate the dataset difficulty, we use $\mathcal{V}$ -usable information [53], which estimates the information an input X holds for predicting the target Y across a family of models $\mathcal{V}$. Lower $\mathcal{V}$ -usable information indicates higher dataset difficulty for $\mathcal{V}$. Figure 9.4 offers a comparative view of dataset difficulty between the original and synthetically generated datasets by TarGEN. Notably, the synthetic datasets exhibit a diverse range of samples with varying pointwise $\mathcal{V}$ -usable information, showcasing their diversity in terms of difficulty. Furthermore, the absence of mislabeled samples, indicated by positive $\mathcal{V}$ -usable information in the synthetically

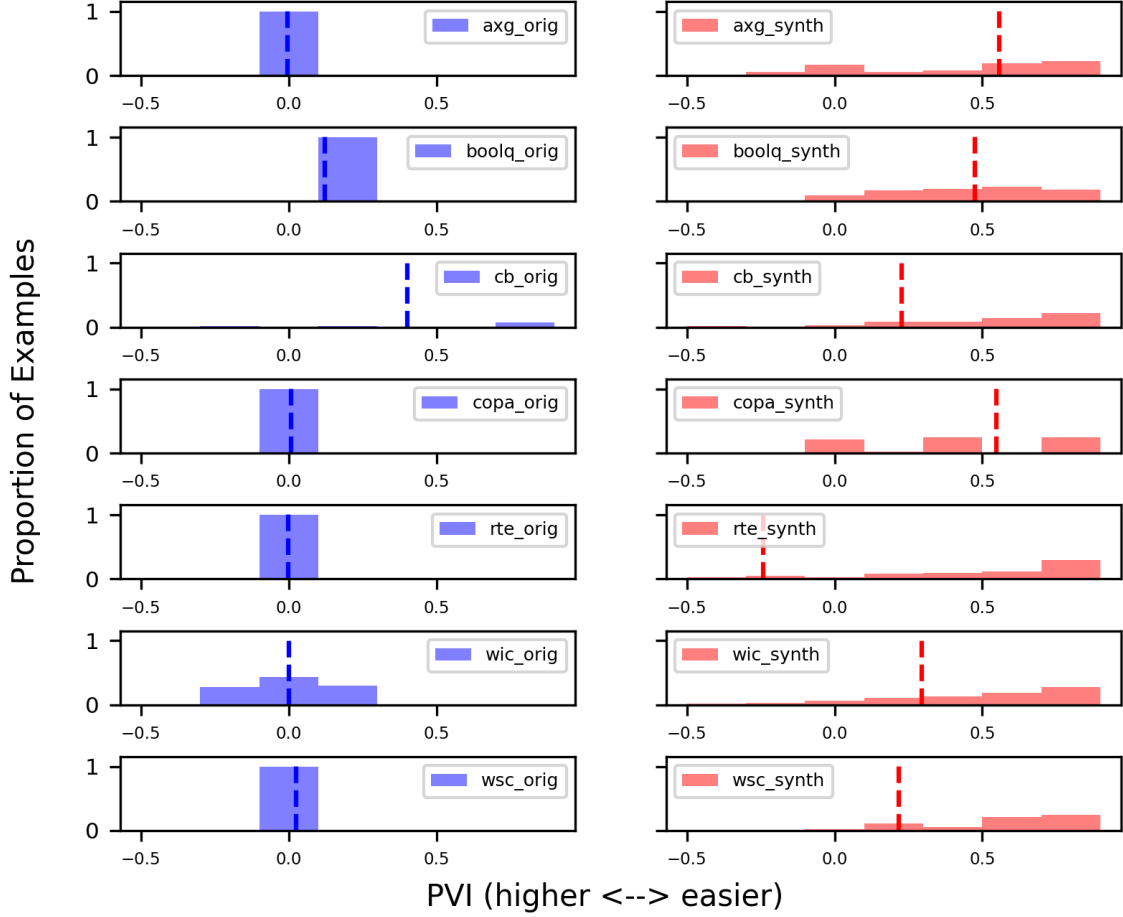


Figure 9.4: Comparison of PVI ($\mathcal{V}$ -usable information) across datasets for the original dataset and the synthetically generated dataset. Synthetic data seems to have better quality as Original datasets PVI is concentrated around -0.1 to 0.1 whereas the synthetic data generated has a diverse mix of difficulty level among the samples.

generated datasets, underscores the effectiveness of *self-correction* prompts.

Dataset Bias To estimate the dataset bias, we visualize the distribution of tokens related to named entities, belonging to Geopolitical Entities (GPE), Products, and Nationalities or Religious or Political groups (NORP). The distribution of input tokens for the original and synthetically generated BoolQ dataset is presented in Figure 9.5. Our results reveal that the distribution of GPE, Product, and NORP entities in

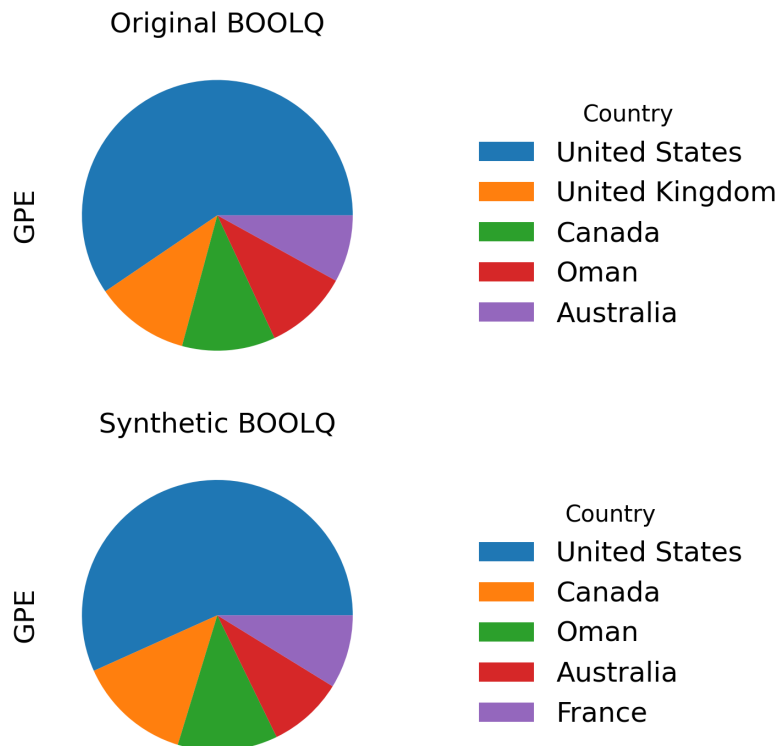


Figure 9.5: Comparison of dataset bias for the BoolQ dataset and the synthetically generated BoolQ dataset. This represents the distribution for the GPE (Geo Political Entity) named entity tag. The distribution for other entities can be found in the appendix section.

the TarGEN dataset closely resembles that of the original dataset². The further plots for dataset bias for other datasets are also presented here.

Comparison with Self-Instruct To compare the synthetic SuperGLUE dataset with other synthetic instruction following benchmarks, we choose Self-Instruct [266], a popular synthetic dataset generation framework. We choose Llama 2 (7B) and pre-finetune using synthetic SuperGLUE. We refer to this tuned model as L2SSG. We refer to the Llama 2 model trained on the Self-Instruct dataset as L2SI. Both models are evaluated on the OpenLLM benchmark. Table 9.6 showcases the results

²To ensure a broad-scale examination across all samples, we utilized the Spacy library and its *en_core_web_sm* model to extract named-entity tags.

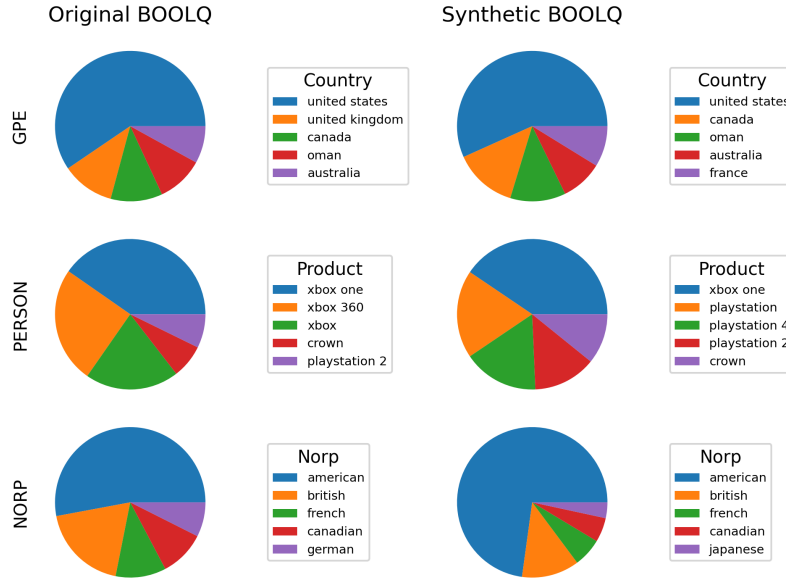


Figure 9.6: Comparison of dataset bias for the BoolQ dataset and the synthetically generated BoolQ dataset.

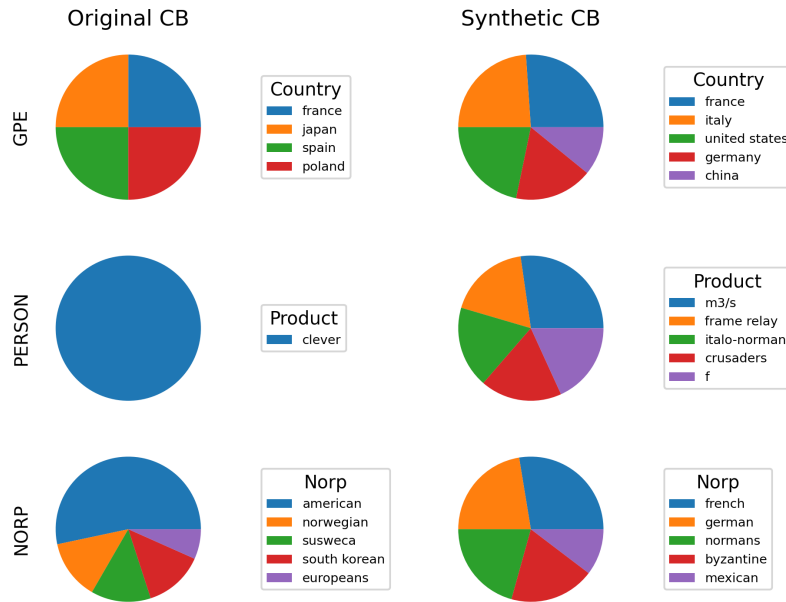


Figure 9.7: Comparison of dataset bias for the CB dataset and the synthetically generated CB dataset.

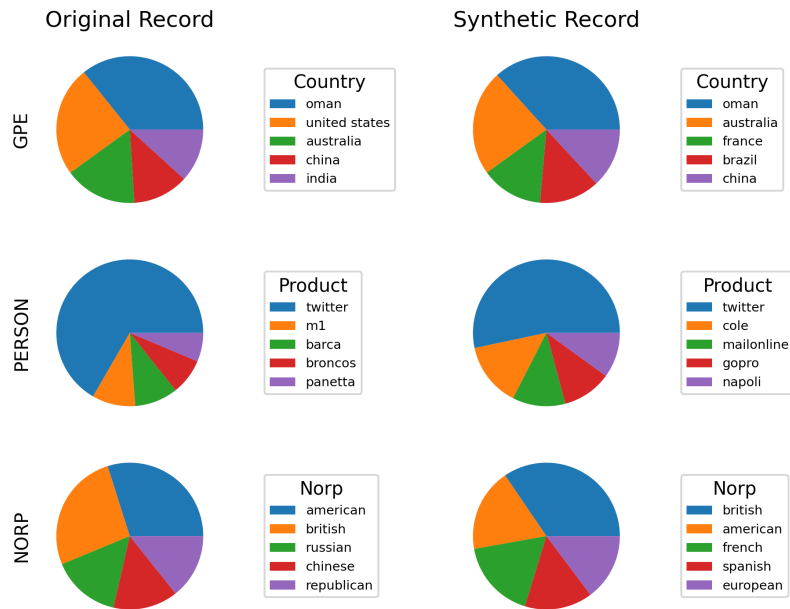


Figure 9.8: Comparison of dataset bias for the Record dataset and the synthetically generated Record dataset.

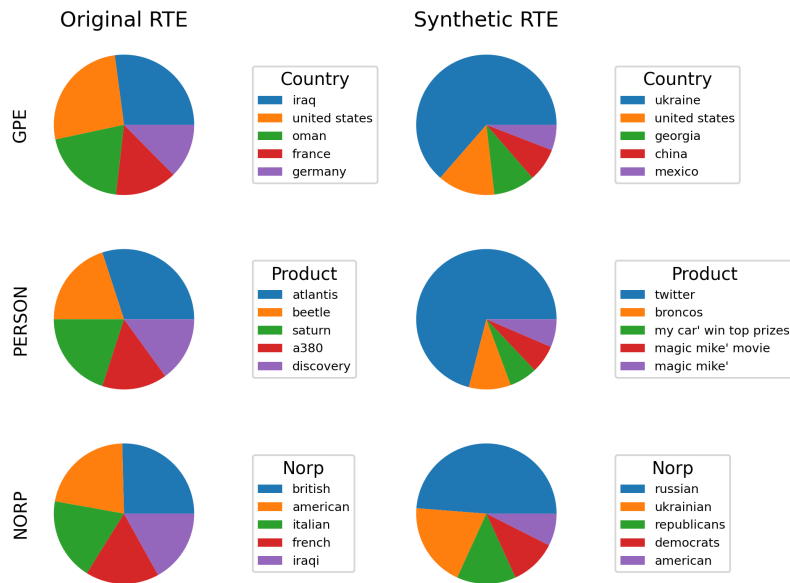


Figure 9.9: Comparison of dataset bias for the RTE dataset and the synthetically generated RTE dataset.

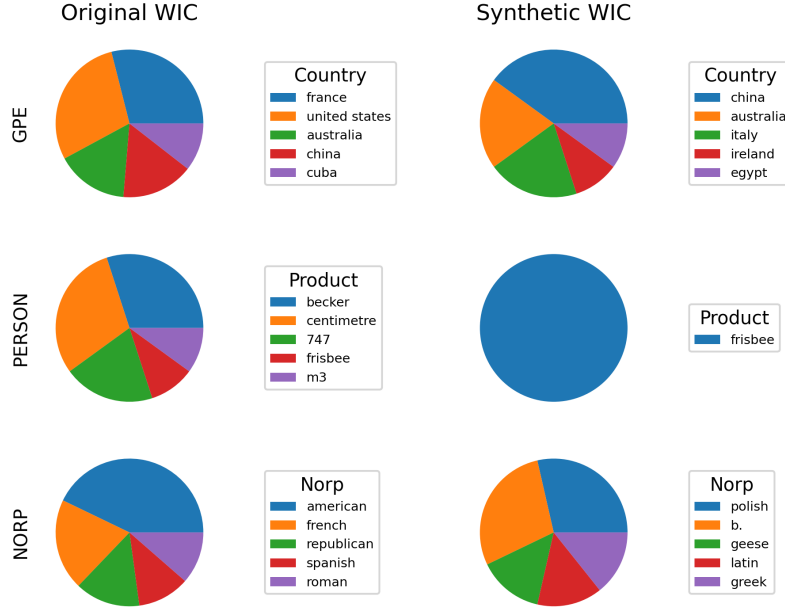


Figure 9.10: Comparison of dataset bias for the WIC dataset and the synthetically generated WIC dataset.

for the benchmark. L2SSG scores 2.62% higher than L2SI, underscoring the quality of targeted dataset generation by the TarGEN framework over Self-Instruct.

9.5 Conclusion

In this work, we introduced TarGEN, a multi-step prompting strategy for generating high-quality and diverse synthetic datasets utilizing LLMs without any human supervision. We described a step-by-step methodology for TarGEN to synthesize a dataset from instructions without any task exemplars. To evaluate our proposed framework, we emulated eight tasks from the SuperGLUE benchmark and compared them with the original SuperGLUE by training different families of models. Experimental results reveal that models fine-tuned on our synthetic SuperGLUE outperform models fine-tuned on the original SuperGLUE. A comprehensive analysis of the synthetic benchmark with respect to the original benchmark resulted in several interesting findings, such as the fact that data instances in our synthesized benchmark

Dataset	L2SSG	L2SI
ARC	54.65	52.30
HellaSwag	79.25	77.09
MMLU	45.63	41.60
TruthFulQA	42.34	37.58
Winogrande	71.37	69.46
GSM8K	1.98	1.44
Average	49.20	46.58

Table 9.6: Comparison of Synthetic SuperGLUE and Self-Instruct Dataset. Llama 2 (7B) is pre-finetuned on both datasets individually and evaluated on the OpenLLM leaderboard.

are more difficult and diverse compared to the original benchmark, and also exhibit similar dataset bias.

Further comparison with Self-Instruct and AttrPrompt revealed that synthetic SuperGLUE served as better pre-finetuning corpora when evaluated on the OpenLLM benchmark, resulting in an impressive performance using (inc: T5-3B) Llama 2. Though TarGEN facilitates high-quality data generation, we believe that it is important to assess our proposed frameworks within a multi-lingual context and also on additional benchmarks, including BigBench, LILA, and HELM. Furthermore, TarGEN currently relies on the ChatGPT model for synthesizing the benchmark, but our plans involve exploring the impact of other LLMs such as GPT-4, Llama 2, and Falcon when employed with TarGEN. We believe that TarGEN can serve as a valuable tool for enhancing the quality of data generation, thus reducing human effort.

IN-BOXBART: INSTRUCTION-TUNING FOR BIOMEDICAL TASKS

10.1 Introduction

For long, task-specific models have played a central role in achieving state-of-the-art performance in both general and biomedical NLP [255, 9]. During 2017-2019, the pre-train and fine-tune paradigm [129] became the prevalent approach in NLP. Due to the success of Language Models (LMs) in the biomedical domain, such as BioBERT [115], ClinicalXLNET [84], and others [4, 109, 179], this paradigm is widely used for creating many task-specific models [255, 9]. However, task-specific models have limitations to real-world applications because this approach is computationally expensive (i.e., requires large computational resources) and time-consuming [225, 218]. Hence, there is a need for generalization where a single model can perform various tasks, leading to a computationally efficient approach. Past attempts have been made in general-domain NLP to achieve generalization across tasks such as MQAN [143], UNICORN [134], and UnifiedQA [105]. However, approaches to achieve generalization across various biomedical NLP tasks have not been systematically studied. Hence, this chapter studies the multi-tasking approach that can generalize over different biomedical NLP tasks. Figure 10.1 shows the overview of our proposed multi-tasking approach for various biomedical NLP tasks.

Recently, prompt-based models have been widely used because of their ability to achieve generalization instead of task-specific models [129]. Mishra *et al.* [153], Wei *et al.* [271] and Sanh *et al.* [209] show the effectiveness of instructional prompts in generalizing on seen as well as unseen general-domain NLP tasks. In this chapter, we

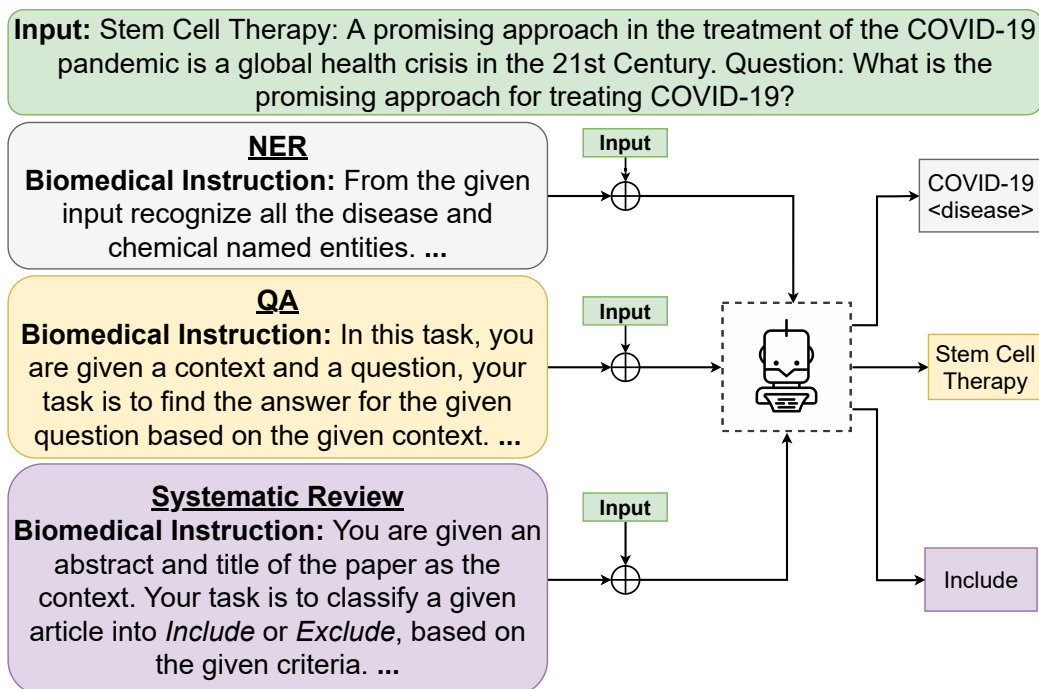


Figure 10.1: Schematic representation of multi-tasking in the biomedical domain using instructional prompts. In this approach, a model is allowed to utilize tasks to get familiar with instructions and use them to map a given input to its corresponding output.

adapt this instructional prompt-based approach for the first time to achieve generalization across various biomedical NLP tasks. To this extent, this chapter introduces a collection of 32 instruction tasks for **Biomedical NLP** across (**X**) various categories (BoX) and proposes a unified model that can generalize over 32 different biomedical NLP tasks. The proposed unified model (i.e., In-BoXBART) is trained on the instruction-based meta-dataset (i.e., BoX) and evaluated on each task individually from the BoX.

To evaluate the proposed approach, we compare our model (i.e., In-BoXBART) with two baselines: (1) single-task models (i.e., models trained on one task and evaluated on the same task), and (2) a multi-task model (i.e., a single model trained on a combination of all tasks) without instructions. Experimental results show that In-BoXBART outperforms the single-task baseline by $\sim 3\%$, and the multi-task baseline

by $\sim 18\%$. We also analyze a few-shot learning scenario using In-BoXBART since obtaining annotated data in the biomedical domain is costly and time-consuming [139]. In the few-shot setting (i.e., 32 instances per task), In-BoXBART outperforms the single-task baseline by 23.33%. This indicates that Multi-Task Learning (MTL) and instruction-tuning have an advantage in low-resource settings. Although the performance of the In-BoxBART is promising, our analysis reveals that there is still room for improvement on some tasks, implying the scope for future research direction. Concisely, our contributions can be summarized in three folds:

1. This chapter introduces the first benchmark meta-dataset in the biomedical domain, i.e., BoX: a collection of 32 instruction tasks for Biomedical NLP across (X) various categories. Each task is processed in a unified format and equipped with instructions that can be used to train sequence-to-sequence models.
2. Using this meta-dataset, we propose an instruction-tuned Bidirectional and Auto-Regressive Transformer (BART) model, termed as In-BoXBART. The comparison of In-BoxBART and two baselines shows that In-BoxBART outperforms the single-task baseline by $\sim 3\%$ and the multi-task (without instruction) baseline by $\sim 18\%$.
3. In the few-shot setting, we show that In-BoxBART significantly outperforms the single-task baseline by $\sim 23\%$. This indicates the potential application of instruction-tuning in the biomedical domain, where annotated data is difficult to obtain.

10.2 BoX

We use 29 existing, widely adopted biomedical NLP datasets collected from various challenges, platforms, and organizations to create BoX. We define the BoX as a benchmark dataset for biomedical MTL across 9 different categories. In the BoX, we reframed all the datasets as text generation tasks (see examples in Table 10.2) and created 32 instruction tasks. BoX consists of high-quality human-authored Biomed-

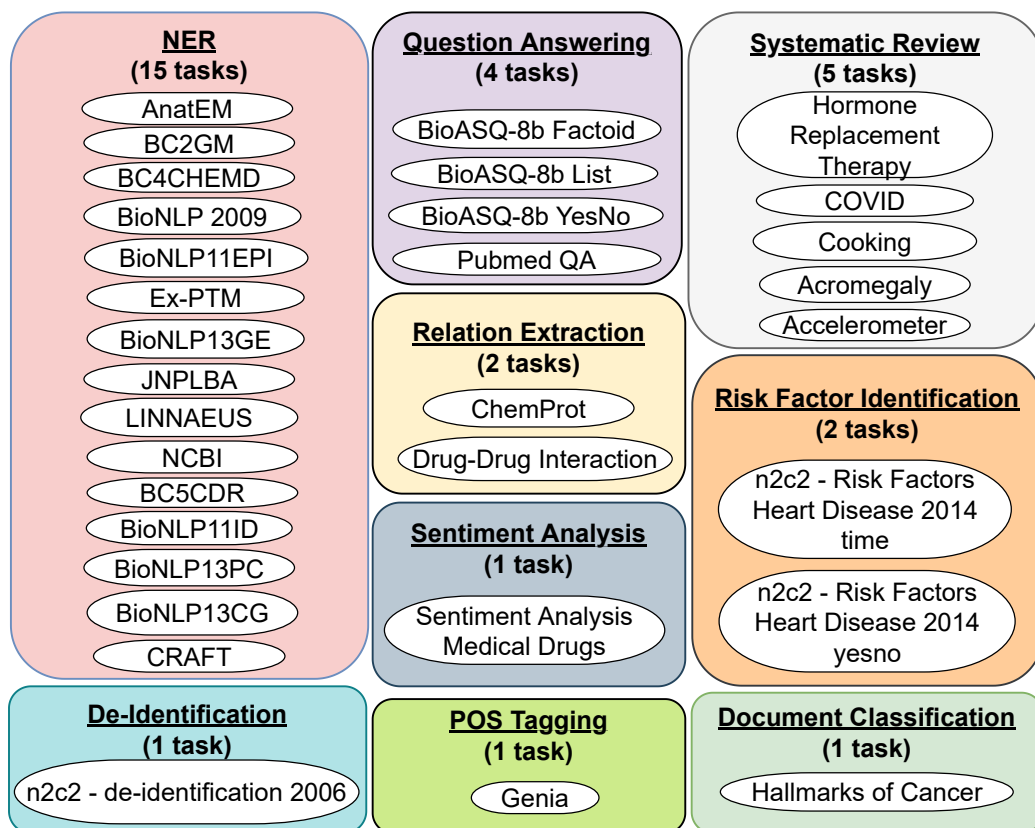


Figure 10.2: Schematic representation of 9 categories of tasks: each block represents one category with various tasks equipped with instruction.

ical Instructions (BIs) for all 32 tasks. Figure 10.2 shows the 9 different categories and corresponding generated tasks. Each category is defined as a colored box, and each box contains instruction tasks re-purposed from original datasets.

10.2.1 Tasks

Table 10.1 provides all the statistics of training, validation, and inference data used for experiments. All the number of instances provided in Table 10.1 are calculated after discarding the instances with more than 1024 token length as described in the §10.4.1. We have divided the dataset into standard 70/10/20 splits for train/validation/test if there is no separate validation and testing set provided in the dataset. Each category and corresponding tasks from the BoX are defined as below:

Category	Tasks	# of Instances		
		Train	Dev	Test
NER	AnatEM	3507	1121	2303
	BC2GM	6427	1291	2570
	BC4CHEMD	14466	14568	12397
	BC5CDR	4940	4940	5158
	BioNLP11EPI	3796	1242	2836
	BioNLP11ID	2466	780	1869
	BioNLP13CG	4591	1489	2759
	BioNLP13GE	1503	1663	1937
	BioNLP13PC	2945	1070	1997
	BioNLP09	4710	1013	1699
	CRAFT	12839	4423	8882
	Ex-PTM	855	278	1160
	JNLPBA	15124	1533	3152
	NCBI	2922	488	538
	linnaeus	1484	524	993
DI	DI 2006	106	22	27
POS	Genia	16323	2174	2035
QA	BioASQ8b (factoid)	695	16	115
	BioASQ8b (list)	373	8	45
	BioASQ8b (yesno)	543	16	115
	PubMedQA	4167	500	473
RE	ChemProt	3350	2415	2660
	DDI	20009	2780	2660
SA	Medical Drugs	2860	526	804
SR	Accelerometer	499	58	142
	Acromegaly	663	80	192
	COVID	2385	300	675
	Cooking	735	84	205
	HRT	1479	171	410
DC	HoC	3119	445	890
RFI	RFHD 2014 (yesno)	834	360	451
	RFHD 2014 (time-riskfactor)	152	177	69
Total	-	140795	46554	64561

Table 10.1: Statistics of training (i.e., Train), validation (i.e., Dev) and evaluation (i.e., Test) data for all instruction tasks from the BoX. RFHD: Risk Factor for Heart Disease.

Named Entity Recognition (NER) NER has been considered a necessary first step in processing literature for biomedical text mining, where the model helps in identifying named entities such as protein, gene, chemical, disease, and treatment. We use fifteen publicly available biomedical NER datasets [41] to create instruction tasks.

De-Identification (DI) In this task, the model takes medical discharge records of a patient as input and identifies Private Health Information (PHI) such as organizations, persons, locations, and dates. We use n2c2 2006 de-identification challenge dataset [248] to perform this task.

Part-Of-Speech (POS) Tagging The goal of this task is to identify various POS tags from the biomedical text. We use the GENIA corpus [234] built from MEDLINE abstracts for the POS tagging task.

Question-Answering (QA) QA models receive a question and a corresponding context as input and output the relevant answer from the given context. To execute this task, we used the BioASQ-8b dataset [162] for different question types, i.e., yes/no, factoid, and list-type questions. We created three different tasks from this dataset. Also, we use the PubMedQA dataset [96] for this task.

Relation Extraction (RE) We used two datasets for this task: (1) CHEMPROT corpus from biocreative VI precision medicine track [89], and (2) Drug-Drug Interaction (DDI) corpus from SemEval 2013 DDI Extraction challenge [79]. Here, we only consider binary RE tasks without any label describing the type of the relation.

Systematic Review (SR) The relentless growth in clinical research and published articles has created a need for automation to expedite the process of SRs and to enable

Living Systematic Reviews (LSRs). A crucial step in both SRs and LSRs is the title and abstract-based screening of the articles. A new dataset was developed from six SRs in the clinical domain by Mayo Clinic physicians. In this study, we used data from the following five SRs that were conducted using the traditional (manual) process and published in relevant venues: (1) Hormone Replacement Therapy (HRT), (2) Cooking, (3) Accelerometer, (4) Acromegaly, and (5) COVID from Parmar [169]. The initial bibliographic search was designed and conducted by an experienced librarian with guidance from the principal investigators for the respective studies. The search was conducted in different bibliographic databases like PubMed, PubMed Central (PMC), Embase, EBM Reviews, and Ovid MEDLINE(R). Each article in the bibliographic search results was categorized by two physicians with domain expertise as “Include” or “Exclude”, by reading the title and abstract of the article. When there was a disagreement between two annotators, a positive class (i.e., “Include”) was preferred.

Sentiment Analysis (SA) Analyzing the sentiment of people towards medical drugs is an essential task in the biomedical domain. To that effect, we use a medical drug sentiment analysis dataset¹ to identify one of three sentiments: (1) positive, (2) negative, and (3) neutral.

Document Classification (DC) We have used the Hallmarks of Cancer (HoC) dataset [8] for this task.

Risk Factor Identification (RFI) The goal of this task is to identify risk factors for Coronary Artery Disease (CAD) in diabetic patients over time. For this, we used n2c2 2014 shared task track 2 dataset [110] with two different purposes: (1) identify if the risk factor is present in the medical discharge summary, and (2) the time of risk

¹<https://www.kaggle.com/arbazkhan971/analyticvidhyadatasetsentiment>

factor present in the discharge records.

10.2.2 Biomedical Instructions

Motivated by Mishra *et al.* [153], we have used a similar approach to create Biomedical Instructions (BIs). BI consists of natural language instructions that describe a task and contain instances of that task. Here, we introduce a unified schema to present BI and describe how we can construct BI for each task given in the BoX. Figure 10.3 illustrates the schematic representation of the schema, and Figure 10.4 shows an example of BI that describes a “Named Entity Recognition (NER)” task, accompanied with a few positive examples.

10.2.2.1 Unified Schema

All BIs are mapped to the unified schema. As shown in Figure 10.3, the unified schema consists of a definition, prompt, and positive examples. This schema helps in organizing each BI. Each of the elements of the schema is explained below:

Definition contains the core explanation about the task and detailed instruction to the model on what needs to be done in the given task.

Prompt is the short explanation of the task that needs to be done.

Examples contain the input/output pairs of the task instance along with the explanation of how the output is generated. Generally, we provide 2-3 examples for each task.

Instances contain the input/output pairs of training samples from the task datasets.

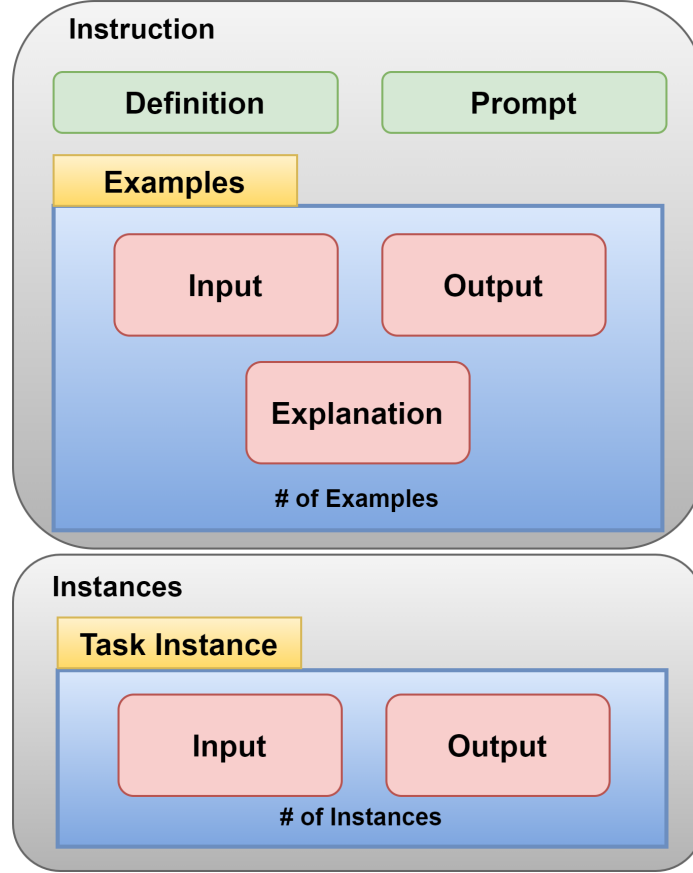


Figure 10.3: Unified schema used to create a Biomedical Instruction (BI).

10.2.2.2 Construction of BI

We have created a BI for each dataset given in the BoX. To create BI, we manually fill in the fields of the unified instruction schema (Figure 10.3). For each dataset, the BI is created by one author and verified by other authors.

Quality of BIs In the instruction verification process, we edit BIs if needed in terms of grammar, typos, ambiguity, etc., to improve the quality. According to [11], concise instructions are more beneficial compared to repetition, hence, we also redact repetition from BIs. As a result, our BIs consist of high-quality, short, and meaningful task definitions and prompts.

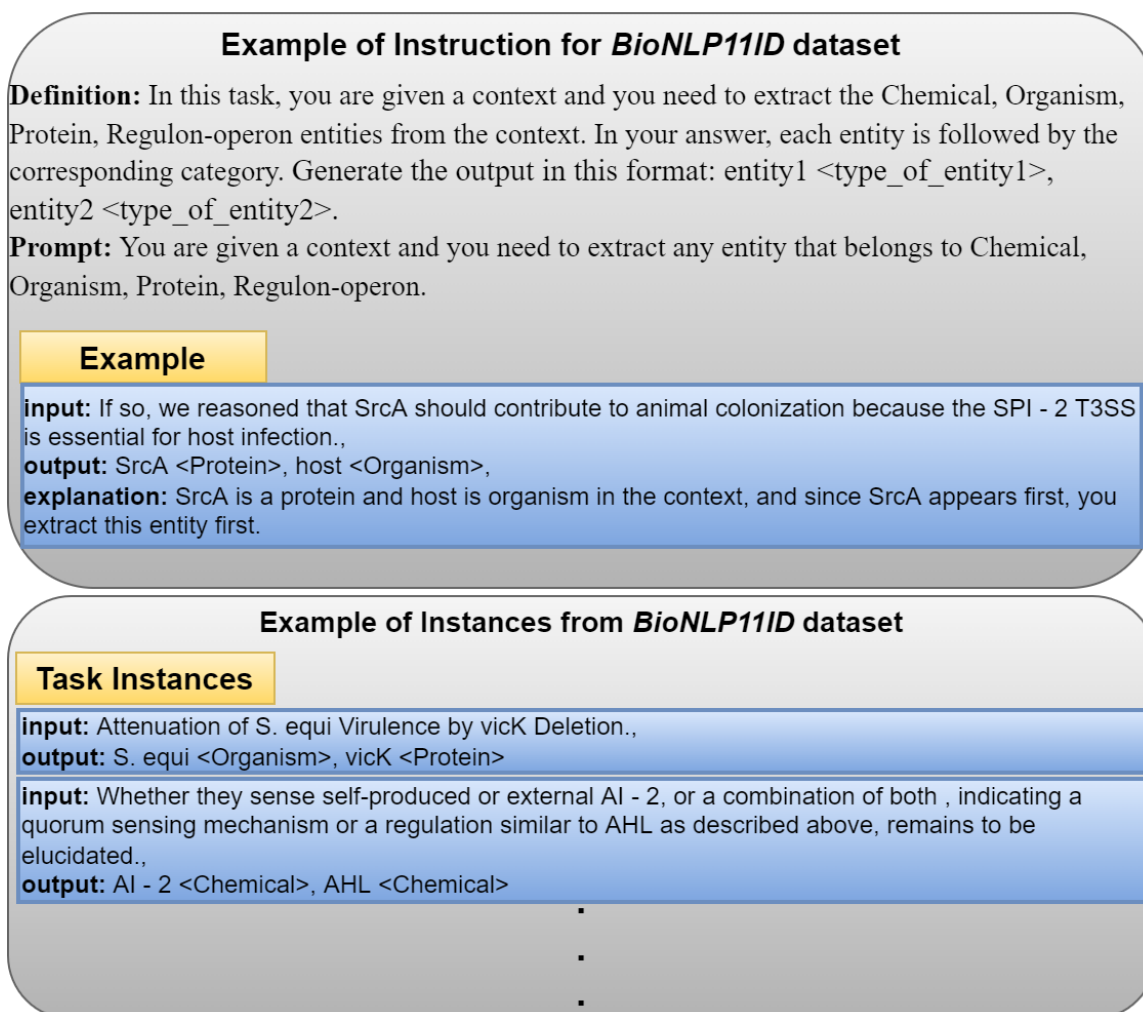


Figure 10.4: Example of Biomedical Instruction (BI) and task instances from BioNLP11ID (NER) dataset.

Positive examples and its explanation For each dataset, we have provided 2-3 positive examples and corresponding explanations to give an idea of how to perform the given task. As we know, the selection of examples has an impact on model performance [135]. To that extent, we have been careful in selecting examples for text generation and classification tasks. For text generation, we have provided 2-3 examples with a detailed explanation about how the output is generated. For text classification tasks, we have included examples corresponding to each class with an

explanation of why the particular class is assigned to a given input instance. All positive examples are drawn from training instances and have been removed from training in order to avoid repetition. All the explanations of examples pass through the verification process to maintain high quality.

Collection of input/output instances Since each biomedical NLP dataset included in the BoX has its own annotated input/output instances, we converted them into text-to-text format [134]. To build all the models (baselines, proposed model, and few-shot learning), we adapt the unified text-to-text format for all the tasks of BoX. Table 10.2 shows an example of input and output from each category. Moreover, we have also re-purposed some biomedical datasets to create more than one task as described in §10.2.1.

10.3 Problem Setup and Models

10.3.1 Problem setup

Let us assume we have an input/output instances pair (X_t, Y_t) for a given task t . Along with that, each task is described in terms of its instruction BI_t .

Single-task models Traditional supervised models learn a mapping function (f_M) between input (x) and output (y), where $(x, y) \in (X_t^{\text{train}}, Y_t^{\text{train}})$ and are evaluated on the same task $(X_t^{\text{test}}, Y_t^{\text{test}})$. We refer to this setup as single-task learning.

Multi-task models In this setup, we combined the training data and the corresponding biomedical instruction of all tasks together. The goal of multi-task learning models is to learn mapping function (f_M) between input (x), output (y) and biomedical instruction BI_t , i.e., $f_M(BI_t, x) = y$, where $(x, y) \in (X_t, Y_t)$. This model is

Category	Task	Input	Output
NER	BC5CDR	Such interactions may result in serious cardiovascular complications even after cessation of an infusion of ritodrine.	cardiovascular complications <Disease>, ritodrine <Chemical>
de-identification	DI2006	757085252 HLGMC 1228824 18705/6o5b 3/25/1993 12:00:00 AM CONGESTIVE HEART FAILURE . Unsigned DIS Report Status : Unsigned ADMISSION DATE : 3/25/93 DISCHARGE DATE : 4/4/93 PRINCIPAL DIAGNOSIS : congestive heart failure . ASSOCIATED DIAGNOSIS : aortic stenosis ; coronary artery disease , status post multi vessel coronary artery bypass graft surgery , ... , M.D. TR : go / bmot DD : 4/4/93 TD : 04/06/93 CC : [report_end]	3/25 <DATE>, 18705/6o5b <ID>, 757085252 <ID>, go / bmot <DOCTOR>, 4/4 <DATE>
POS-Tagging	Genia	Binding sites were mapped for each factor.	Binding <VBG> sites <NNS> were <VBD> mapped <VBN> for <IN> each <DT> factor <NN>.
QA	BioASQsb (factoid)	Context: Hyperosmia is suspected in pregnancy; however, no empirical study using validated measures of olfactory function has clearly confirmed the anecdotal reports of this phenomenon. subjective hyperosmia is associated with primarily negative odor-related experiences. Hyperosmia is increased olfactory acuity \n Question: What is hyperosmia	Hyperosmia is increased olfactory acuity.
RE	Drug-Drug Interaction	Context: Antacids may interfere with the absorption of LEVSIN. Drug_1: Antacids Drug_2: LEVSIN	true
Sentiment Analysis	Medical Drugs	Why don't more folk opt for Cladribine? \n Drug: cladribine \n Option1: Neutral Option2: Positive Option3: Negative	Positive
Systematic Review	Acromegaly	No greater incidence or worsening of cardiac valve regurgitation with somatostatin analog treatment of acromegaly CONTEXT: Excess GH and IGF-I in acromegaly are associated with reduced life expectancy due to cardiovascular complications. Option_1: Include, Option_2: Exclude.	Include
Document Classification	Hallmarks of Cancer (HoC)	Studies of cell-cycle progression showed that the anti-proliferative effect of Fan was associated with an increase in the G1/S phase of PC3 cells.	Evading growth suppressors, Sustaining proliferative signaling
Risk Factor Identification	n2c2 - Risk Factors Heart Disease 2014 (yesno)	Context: Record date: 2157-08-27 History of Present Illness ID:Admitted from cardiac cath lab. HPI:Mr. Doty is a 80 y.o. male with h/o HTN, DM, PVD, elevated cholesterol who presents with 6 month h/o chest and upper extremity discomfort on exertion along with SOB. He has limited his activities to prevent symptoms. ... \n Risk Factor: Diabetes	Yes

Table 10.2: Examples of one instruction tasks converted into text-to-text format for each category

evaluated on task-specific instances $(x, y) \in (X_t^{\text{test}}, Y_t^{\text{test}})$ In contrast to single-task models, a single model is used here to solve various tasks, hence, achieving generalization. We refer to this setup as MTL.

10.3.2 Models

We propose an instruction-based model to achieve multi-tasking and compare it with two baselines: (1) single-task models, and (2) multi-task models without

instructions. We have fine-tuned the BART (base) model [119] to build baselines as well as the proposed model.

10.3.2.1 Baselines

Single-Task models As formulated in the single-task problem setup, we have trained the BART model on each task from the BoX and evaluated it on the same task.

Multi-task without instruction To build this baseline, we have combined the training data of each task from the BoX together without appending BIs and trained a single model on the combined data. We refer to this model as Vanilla-BoXBART. This model is evaluated on each task of the BoX.

10.3.2.2 Proposed Model

As formulated in the multi-task problem setup, we have combined training data and the corresponding BI of each task. To combine instruction with input instances, we map a BI and an input (x) into the textual format and obtain $enc(BI_t, x)$. After that, the BART model is used to predict an output (y) using a mapping function $f_M : enc(BI_t, x) \rightarrow y$. To perform encoding, a standard NLP paradigm of mapping is used, i.e., mapping an input to text. Here, we map each element of BI (i.e., definition and positive examples as shown in the schema) to a textual format and append it before the input instances. After appending the BI of each task to the instances, we combined all the training data of each task. Now, we fine-tuned the BART model with this combined instruction meta-dataset. We refer to this instruction-tuned model as In-BoXBART.

10.4 Experiments and Analysis

10.4.1 Experimental Setup

We have used the BART (base) model to build all baselines and the proposed model. All the experiments are performed using *Quadro RTX 8000* GPU. All models are trained for 3 epochs. In particular, we have used *huggingface implementation* [277] of the BART and its pre-defined functions for the training and evaluation with default parameters.

Instance Selection As we know, BART (base) can accept the input of a maximum 1024 token length. Since there are a few instances in some datasets that exceed this limit (after including instructions), we have discarded those instances while creating instruction tasks. We have also removed the same instances while training two baselines to do a fair comparison. We have discarded long samples (>1024 token length) from validation and testing data as well.

Example Selection As discussed in Lu *et al.* [135], the selection and order of the examples included in instructions matter mainly for classification tasks and affect the performance of the model. We empirically conclude that the proposed model benefits from ignoring examples from biomedical instructions for classification tasks during training and evaluation. Hence, we have discarded all examples from the BIs associated with the classification instruction tasks.

Instance Sampling Some classification datasets used to create the BoX are imbalanced. To balance these datasets, we have applied the sampling techniques [183] before using datasets to create BoX. In particular, we have analyzed three sampling techniques: (1) under-sampling, (2) average-sampling, and (3) over-sampling. In

under-sampling, we have reduced instances for all the classes to the class with the lowest number of instances. In contrast, we have over-sampled instances via replication of random instances to the class with the highest number of instances to achieve over-sampling. In average sampling, we calculated the mean of a number of instances across all the classes and over-sampled or under-sampled instances accordingly for each class.

Few-shot setting Similar to the Schick and Schütze [215], we have started with 32 randomly selected instances for each instruction task from the BoX to exhibit few-shot learning. After that, we have increased randomly selected instances per task to $100/1k/4k$. If any task has already less number of instances than the threshold (i.e., $100/1k/4k$), we keep all the instances from that task. While selecting the instances, we made sure that we selected balanced data for the classification tasks. Moreover, the BoX contains an average of $6k$ instances per task.

Evaluation Metric We use Rouge-L [122] as our evaluation metric since we treat all the tasks as text generation problems. We also use F_1 -Score for evaluations.

10.4.2 Results and Findings

Effect of Sampling As mentioned above, we conduct three experiments to analyze the effect of sampling on In-BoXBART. We train our model using training data obtained from (1) under-sampling, (2) average-sampling, and (3) over-sampling. We achieve on average (across all instruction tasks) 69.62, 70.23, and 73.49 Rouge-L for under-, average-, and over-sampling, respectively. Here, we observe from the experimental results that over-sampling gives better performance compared to under- and average-sampling, since there is a loss of training data samples for under- and

Category	Task	Rouge-L			F_1 -Score		
		Single Task	V-BB	I-BB	Single Task	V-BB	I-BB
NER	AnatEM	84.88	32.30	83.93	85.55	33.50	84.61
	BC2GM	77.66	50.87	74.10	78.56	50.86	75.03
	BC4CHEMD	88.85	71.05	86.50	89.06	71.44	86.97
	BC5CDR	74.83	69.81	74.76	75.13	70.11	75.24
	BioNLP11EPI	84.64	50.10	87.60	84.95	52.85	88.04
	BioNLP11ID	71.08	59.12	72.64	71.64	60.15	73.39
	BioNLP13CG	64.19	55.18	67.72	61.68	53.88	65.09
	BioNLP13GE	83.74	49.30	86.71	84.08	51.78	87.39
	BioNLP13PC	70.42	53.06	72.46	66.89	51.61	67.77
	BioNLP09	85.16	51.54	88.09	85.54	54.31	88.48
	CRAFT	63.72	51.85	64.10	63.92	52.31	64.30
	Ex-PTM	82.32	49.61	83.73	82.38	52.07	84.49
	JNLPBA	71.65	69.37	71.54	70.79	68.60	70.26
	NCBI	89.51	74.46	86.11	89.81	75.55	80.91
	linnaeus	94.43	44.99	93.46	93.21	44.59	93.77
Average		79.14	55.51	79.54	78.88	56.24	79.45
DI	DI 2006	12.60	46.38	50.82	10.60	43.28	47.45
POS	Genia	71.45	27.94	71.26	70.48	27.50	71.99
QA	BioASQ8b (factoid)	52.95	51.14	47.28	54.67	53.52	49.51
	BioASQ8b (list)	38.96	19.87	36.11	-	17.74	35.59
	BioASQ8b (yesno)	61.74	62.61	68.25	63.48	62.61	68.25
	PubMedQA	27.12	25.48	24.49	31.44	30.74	29.58
	Average	45.19	39.78	44.03	46.39	41.15	45.73
RE	ChemProt	76.08	76.00	81.61	63.89	52.17	63.22
	DDI	91.78	82.97	89.35	94.10	82.97	89.35
	Average	83.04	79.48	85.48	79.00	67.57	76.28
SA	Medical Drugs	47.51	46.39	47.37	47.51	46.39	47.37
SR	Accelerometer	74.65	72.54	81.25	74.65	72.54	81.25
	Acromegaly	80.21	81.77	80.71	80.21	81.77	80.71
	COVID	74.81	76.30	77.28	74.81	76.30	77.28
	Cooking	71.71	82.93	83.25	71.71	82.93	83.25
	HRT	75.68	77.17	82.70	75.68	77.17	82.70
	Average	75.41	78.14	81.04	75.41	78.14	81.04
DC	HoC	88.53	49.64	82.53	88.53	49.51	82.53
RFI	RFHD 2014 (yesno)	57.21	64.97	69.17	57.21	64.97	69.17
	RFHD 2014 (time-riskfactor)	66.18	0.97	85.24	66.18	0.97	85.28
	Average	72.87	57.30	77.21	61.69	32.97	77.22
Average	-	70.51	55.55	73.49	70.15	55.21	73.01

Table 10.3: Results comparison between single-task baseline, Vanilla-BoXBART and In-BoXBART in terms of Rouge-L and F_1 -Score. All the results for F_1 -Score are presented in %. V-BB: Vanilla-BoXBART, I-BB: In-BoXBART, RFHD: Risk Factor for Heart Disease.

average-sampling. Hence, we report results of over-sampling as the main result in Table 10.3.

Performance comparison Table 10.3 presents the results for the single-task model, Vanilla-BoXBART, and In-BoXBART. We can see from Table 10.3 that the single-task model, Vanilla-BoXBART, and In-BoXBART achieve an average (across all tasks) Rouge-L of 70.51, 55.55, and 73.49, respectively. They achieve 70.15%, 55.21%, and 73.01% F_1 -Score, respectively, exhibiting the same performance behaviour as Rouge-L. Hence, we use Rouge-L for further comparisons. From the result, we can observe that Vanilla-BoXBART reduces the complexity compared to the single-task model (i.e., 110 million parameters *vs.* 32x110 million parameters); however, on average, the performance drops by 14.96% in terms of Rouge-L, and compared to single-task models. This indicates that multi-task learning in the biomedical domain is more difficult than general domain NLP, since many previous works have shown that the multi-task model outperforms the single-task model [134, 143]. On the other hand, In-BoXBART, which has the same complexity as Vanilla-BoXBART, significantly outperforms Vanilla-BoXBART by on average 17.94%, and also outperforms the single-task model by a 2.98% margin, precisely. This indicates the benefit of using instructions to achieve the MTL in the biomedical domain.

Effect of instruction in few-shot learning We have compared the average Rouge-L of In-BoXBART with a single-task baseline for the few-shot setting. Figure 10.5 shows the relative performance of In-BoXBART compared to the single-task baseline. We have shown results for all the few-shot learning experiments in Table 10.4. From the results, we see that In-BoXBART achieves an average 60.64% Rouge-L, and the single-task model achieves 37.31% for 32 instances per task. In-BoxBART

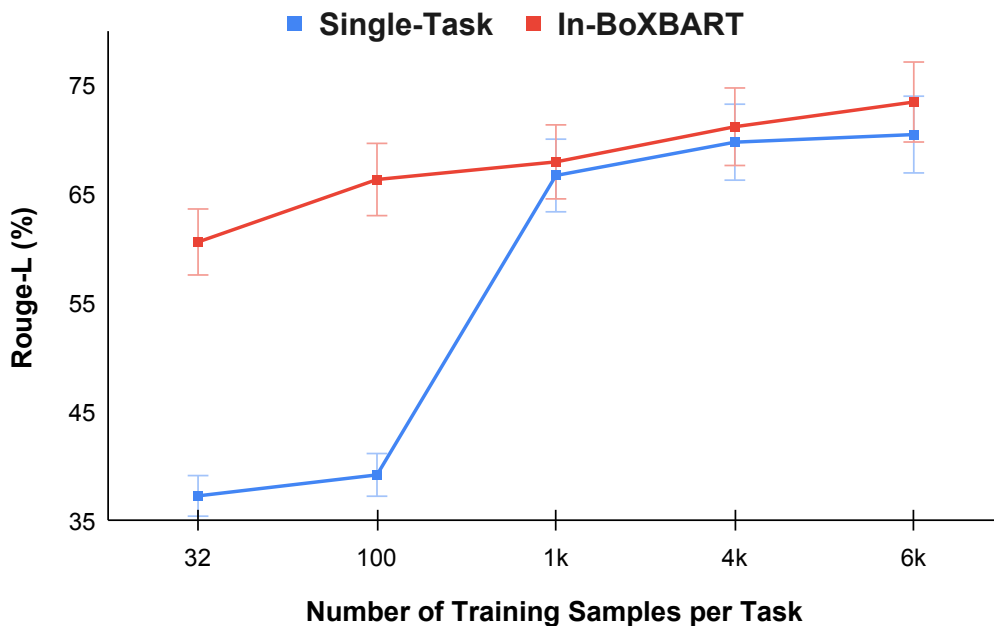


Figure 10.5: Comparison of on average Rouge-L across all instruction tasks between single-task and In-BoXBART based on the average number of training instances per task.

significantly outperforms the single-task baseline by 23.33%. From Figure 10.5, we can see that In-BoXBART consistently performs better compared to the baseline. As we know, obtaining a large annotated dataset in the biomedical domain is difficult, time-consuming, and costly. From few-shot learning, we can see that instructions are beneficial in achieving high performance compared to task-specific models.

10.4.3 Analysis

For which tasks, instruction is helpful? From Table 10.3, In-BoXBART outperforms baselines for 5 categories, i.e., NER, de-identification, RE, SR, and risk factor identification. From this, we can see that instructions are more helpful in these five categories. However, In-BoXBART achieves performance lower or par with the single-task baseline for the tasks from QA, POS tagging, sentiment analysis, and document classification, which indicates room for improvement in this direction.

Category	Task	32		100		1k		4k	
		S	I-BB	S	I-BB	S	I-BB	S	I-BB
NER	AnatEM	12.74	60.73	20.68	79.34	87.81	86.76	84.88	83.44
	BC2GM	16.92	65.65	21.31	70.39	82.92	77.19	77.66	74.11
	BC4CHEMD	10.55	71.05	14.93	73.85	86.53	83.75	88.85	86.19
	BC5CDR	11.75	60.37	12.58	67.51	69.62	73.66	74.83	74.34
	BioNLP11EPI	31.14	78.64	42.31	81.51	85.71	85.57	84.64	86.68
	BioNLP11ID	11.00	62.38	10.06	68.92	71.41	71.62	71.08	71.96
	BioNLP13CG	12.39	49.15	12.53	52.68	55.23	63.15	64.19	67.23
	BioNLP13GE	26.10	78.80	25.00	81.82	84.77	84.29	83.74	85.58
	BioNLP13PC	12.40	69.29	12.59	71.89	68.11	68.49	70.42	71.97
	BioNLP09	32.51	78.17	30.51	82.71	87.48	86.39	85.16	86.33
	CRAFT	8.07	37.35	8.60	40.38	49.67	51.56	63.72	63.35
	Ex-PTM	16.06	74.32	47.93	76.15	82.92	84.11	82.32	83.81
	JNLPBA	20.15	57.61	19.77	59.54	64.46	63.63	71.65	70.45
	NCBI	38.69	68.82	30.46	79.35	93.02	90.36	89.51	86.46
	linnaeus	28.75	58.69	36.94	67.29	93.81	92.50	94.43	70.57
	Average	19.28	64.74	23.08	70.22	77.56	77.54	79.14	77.50
DI	DI 2006	12.67	50.19	13.30	49.54	13.54	55.28	12.60	50.10
POS	Genia	51.48	13.41	48.26	30.65	66.27	61.93	71.45	70.57
QA	BioASQ8b (factoid)	36.63	35.99	41.89	40.77	51.96	49.84	52.95	51.72
	BioASQ8b (list)	14.99	20.91	19.66	29.38	40.14	29.59	38.96	34.68
	BioASQ8b (yesno)	43.48	61.11	39.13	57.94	66.96	60.32	56.52	52.17
	PubMedQA	17.32	19.28	25.16	23.26	27.68	25.86	27.12	24.96
	Average	28.11	34.32	31.46	37.84	46.68	41.40	43.89	40.88
RE	ChemProt	61.64	72.02	66.07	64.91	66.01	55.22	76.86	77.38
	DDI	85.53	77.37	85.53	81.37	46.99	55.41	87.39	73.04
	Average	73.59	74.70	75.80	73.14	56.50	55.31	82.12	75.21
SA	Medical Drugs	33.29	63.48	24.51	63.66	43.41	31.58	37.31	49.50
SR	Accelerometer	76.76	77.78	75.35	68.06	83.80	73.61	72.54	70.83
	Acromegaly	80.21	80.71	81.25	75.63	76.56	79.19	76.04	77.66
	COVID	87.85	88.36	87.85	84.85	61.93	86.96	73.93	78.12
	Cooking	88.29	87.08	87.80	87.56	81.95	87.08	80.98	82.78
	HRT	85.86	86.02	85.61	75.12	89.08	81.99	83.87	80.81
	Average	83.79	83.99	83.57	78.24	78.66	81.77	77.47	78.04
DC	HoC	17.06	19.87	17.98	27.13	46.94	52.36	88.53	81.51
RFI	RFHD 2014 (yesno)	57.21	51.78	57.21	51.50	43.02	66.35	43.86	66.46
	RFHD 2014 (time-riskfactor)	54.51	64.22	52.75	63.37	66.18	59.60	66.18	62.70
	Average	55.86	58.00	54.98	57.43	54.60	62.98	54.93	64.58
Average	-	37.31	60.64	39.24	63.38	66.75	67.98	69.81	70.23

Table 10.4: Comparison of few-shot learning results in terms of Rouge-L between single-task models and In-BoXBART for 32/100/1000 training samples per instruction tasks. All results are presented in %. S: Single-task model, I-BB: In-BoxBART, RFHD: Risk Factor for Heart Disease.

Which are harder tasks to solve using instructions? Although instructions help in achieving better performance for some tasks compared to the single-task model, the overall performance is still lower. For example, instruction improves performance for de-identification, but overall performance on this task is only 50.82% which can be improved. A similar pattern we can see for BioNLP12CG and CRAFT from NER; BioASQ-8b (factoid, list) and PubmedQA from QA; and Medical Drug from the sentiment analysis category. In general, we can observe that tasks that include either a multi-class scenario or answer generation from the context are most likely to be harder to solve using instructions. For example, CRAFT and BioNLP13CG have 6 entity types, which are higher than any other tasks from NER, and we can see that the performance for these two tasks is lower compared to other tasks of NER.

For which tasks, instruction is the most beneficial in the few-shot setting?

From the results shown in Table 10.4, tasks from the NER, de-identification, QA, sentiment analysis, and risk factor identification show on average larger improvement compared to baselines for the few-shot settings (i.e., 32 and 100 instances per task). This indicates that instructions are beneficial for the tasks from the above categories.

10.4.4 Discussion

Can we design better instructions? Since instruction teaches the model how to solve a given task, domain-specific information-rich instructions can improve model performance. One potential way is to use the knowledge of domain experts. However, designing a good biomedical instruction can be one research direction.

How to handle long-context input? Training instances of many biomedical datasets consist of Electronic Health Records (EHRs) or discharge summaries of pa-

Category	Task	Metric	SOTA	Multi-Task	
				V-BB	I-BB
NER	AnatEM	F	91.61	33.50	84.61
	BC2GM	F	83.47	50.86	75.03
	BC4CHEMD	F	92.39	71.44	86.97
	BC5CDR	F	90.50	70.11	75.24
	BioNLP11EPI	F	88.66	52.85	88.04
	BioNLP11ID	F	87.36	60.15	73.39
	BioNLP13CG	F	90.16	53.88	65.09
	BioNLP13GE	F	85.81	51.78	87.39
	BioNLP13PC	F	91.65	51.61	67.77
	BioNLP09	F	91.94	54.31	88.48
	CRAFT	F	90.12	52.31	64.03
	Ex-PTM	F	87.08	52.07	84.49
	JNLPBA	F	79.19	68.60	70.26
	NCBI	F	89.82	75.55	86.91
	linnaeus	F	95.68	44.59	93.77
QA	BioASQ8 (list)	F	52.99	17.74	35.59
	BioASQ8 (yesno)	F	89.95	62.61	68.25
RE	Chemprot	F	74.40	52.17	63.22
	DDI	F	79.40	82.97	89.35
DC	HoC	F	85.30	49.51	82.53
RFI	RFHD 2014 (time-riskfactor)	F	92.76	0.97	85.28
Average	-	-	85.55	50.36	72.24

Table 10.5: The state-of-the-art (SOTA) results for each task compared with Vanilla-BoXBART and In-BoXBART. All results are in %. F: F_1 -score, V-BB: Vanilla-BoXBART, I-BB: In-BoXBART, RFHD: Risk Factor for Heart Disease.

tients. Because of this, these instances are long and exceed the maximum input length of LMs such as BERT and BART. In this scenario, encoding extra information in terms of prompts or instructions becomes difficult. One potential solution is to use Longformer [11], and another solution is to use T5 kind of models, which use relative position embeddings so that the inference length can be longer [137].

How to handle multi-class classification tasks? Multiple classes cause an issue while creating biomedical instructions because we can not present one example per class. If we do that, the encoding of BI and input will exceed the maximum length of LMs. A naive solution is to select examples of a few labels or remove the examples. However, this will cause a label bias issue or performance degradation. Potential future research direction can be designing a methodology to handle multi-class classification tasks.

How far are we from the SOTA? We have presented a preliminary comparison of our results w.r.t. state-of-the-art (SOTA) single-task systems for 21 instruction tasks from the BoX in Table 10.5. To compare the SOTA results with the proposed model, we calculate the corresponding metric used in particular research from our model predictions. For each task, we gather the best performance, and specifically, they are BioASQ-8b [162], Chemprot [178], DDI [178]. In Chemprot and DDI, we compare results with the base LMs instead of the large ones for a fair comparison. SOTA results for all 15 NER datasets are obtained from Banerjee *et al.* [9]. Best performance for the HoC dataset is obtained from Peng *et al.* [178]. Here, we have considered the result of the best system submitted to Stubbs *et al.* [226] as the SOTA result.

10.5 Summary and Conclusions

This research shows the impact of instructions in MTL for the first time in the biomedical domain. To this extent, we introduced the BoX, a first benchmark dataset consisting of 32 instruction tasks across various biomedical NLP domains. Using this meta-dataset, we proposed a unified model, i.e., In-BoXBART which outperforms single-task baseline and Vanilla-BoxBART by $\sim 3\%$ and $\sim 18\%$, respectively. Our proposed approach also shows an effective performance for a few-shot setting which is more beneficial in the biomedical domain where obtaining large annotated datasets is difficult. We hope that the BoX benchmark, In-BoXBART, and experimental results encourage future research into more unified models for biomedical NLP.

LLMS FOR CLINICAL SYSTEMATIC REVIEWS

11.1 Introduction

Recently, large language models (LLMs) have achieved remarkable performance on multiple natural language understanding (NLU) tasks, exceeding human performance in some of them. However, the ability of these models to conduct a systematic review (SR) is under-explored. Therefore, we aimed to develop the LLM-based method to automate the title and abstract screening in SRs.

Systematic reviews form the foundation of evidence-based clinical practice and health policy, but their production involves substantial resource burdens. The screening phase represents a particularly labor-intensive bottleneck in the systematic review process. On average, screening titles and abstracts takes approximately 33 days of analyst time, with reviewers often required to manually evaluate thousands of articles [23]. The process is not merely time-consuming but vulnerable to human limitations. Reviewer fatigue can lead to errors and inconsistencies that potentially compromise review quality [181]. Additionally, recent methodological studies have found that even dual independent screening—considered the gold standard—is imperfect, with single screeners missing a median of 5% of includable studies [253]. The substantial time investment required for screening creates significant delays between research publication and evidence synthesis, limiting the timeliness and utility of systematic reviews for clinical and policy decision-making.

Given these challenges, there has been growing interest in facilitating the process of screening articles using machine learning (ML) models. While traditional ML

approaches have shown promise in assisting with large-scale screening tasks, they often rely on extensive training data, feature engineering, and lack the flexibility needed to generalize across various clinical questions. More recently, the advent of large language models (LLMs), has introduced a new frontier for automation. These models can process and understand vast amounts of unstructured text, offering the potential to revolutionize the screening process by leveraging their deep contextual understanding of the literature without task-specific fine-tuning [20].

Despite the potential of LLMs, current implementations typically assess their performance as standalone systems, akin to a single-reviewer approach. This contrasts with the conventional methodology of SRs, where two independent reviewers screen studies to minimize bias and improve accuracy. The single-reviewer model employed by LLMs lacks this critical collaborative aspect, which is fundamental in the real-world application of SRs. Thus, an important gap exists in current research: how well can LLMs perform when simulating the two-reviewer model commonly used in practice?

These limitations underscore the need for a more nuanced framework using LLMs, where they operate collaboratively to simulate the two-reviewer process. Therefore, we aim to evaluate the effectiveness of LLMs in automating the process of screening in systematic reviews. Specifically, we assess the performance of collaborative LLM approaches, leveraging multiple LLMs as independent reviewers. We introduce and evaluate three distinct methods for conflict resolution between LLMs, reflecting real-world scenarios where two reviewers may disagree on the inclusion or exclusion of studies. Additionally, we explore the development of a conversational system where LLMs simulate expert dialogue to resolve conflicts in the screening process.

11.2 Data and Methods

In this section, we describe our data sources, and provide detailed discussion around proposed methods and experiments for this study.

11.2.1 Data Sources

Datasets The datasets from the living interactive evidence synthesis project were used for the analyses. Data from five ongoing systematic reviews were included focusing on: (i) systemic treatment options in metastatic hormone-sensitive prostate cancer (mHSPC), (ii) systemic treatment options in metastatic castration-resistant prostate cancer (mCRPC), (iii) systemic treatment options in renal cell carcinoma (RCC), (iv) systemic treatment options in advanced/metastatic hepatocellular carcinoma (mHCC), and (v) toxicity of PARP inhibitors in cancer patients.

Manual Annotation Process For each systematic review project, a systematic search was initially manually executed in Ovid MEDLINE and EMBASE to retrieve a list of citations relevant to each systematic review project. Subsequently, a living ‘auto’ search was implemented to retrieve new citations at pre-specified regular intervals using the Python packages “scrapy” and “requests”. The list of all citations was pre-processed to remove duplicates. Citations were assigned a unique identification number and converted into a predefined data schema for storage in a MySQL-based central data repository. Citations were presented in a web-assisted graphical user interface where two independent reviewers screened citations based on prespecified inclusion/exclusion criteria for potential eligibility during the process of conducting the systematic review. The title and abstract of each citation are screened, and each citation is labeled based on the decision for potential inclusion or exclusion. Additionally, a third reviewer cross-checked each article confirming potential eligibility for

	mHSPC	mCRPC	RCC	mHCC	PARPi toxicity	Total
Total articles	1552	1637	1338	1998	4775	11300
Decision labels - N (%)						
Excluded	1524	1526	1253	1967	4260	10530
	(98.19)	(93.22)	(93.65)	(98.45)	(89.21)	(93.19)
Included	28	111	85	31	515	770
	(1.8)	(6.78)	(6.35)	(1.55)	(10.79)	(6.81)

Table 11.1: Characteristics of the included systematic review datasets. N – number; **mHSPC**: metastatic hormone-sensitive prostate cancer; **mCRPC**: metastatic castration resistant prostate cancer; **RCC**: renal cell carcinoma; **mHCC**: metastatic hepatocellular carcinoma; **PARPi**: poly (ADP-ribose) polymerase inhibitors

this study.

Data Processing and Statistics The user-generated labels, along with the corresponding meta-data, and free text for each stage of study selection were organized. Decision labels were transformed into a single binary variable with two levels: (i) “included” (included by full-text review) and (ii) “excluded” (excluded by title/abstract or excluded by full-text review). A total of 11300 articles across five prespecified systematic reviews were included in this study. Of the total, 10530 (93.19%) were labeled as excluded, and only 770 articles (6.81%) were labeled as included. The greatest number of articles were derived from the PARP toxicity systematic review (N = 4775; 42.25%), followed by mHCC (N = 1998; 17.68%), mCRPC (N = 1637; 14.48%), mHSPC (N = 1552; 13.73%), and RCC systematic review (N = 1338; 11.8%) as shown in Table 11.1.

11.2.2 Task Formulation and Models

Task Formulation The process of screening was formulated as a generative approach. Let us consider a set of study instances $\mathcal{S}$ corresponding to a systematic review dataset. In this set, the i^{th} instance is represented as $\mathcal{S}_i = \{(t_i, a_i)\}$ where t_i represents the title and a_i represents the abstract of the i^{th} study. Each title and abstract pair are evaluated based on predefined PICOS criteria (excluding the control criterion), and the study is included if four out of the remaining criteria (Population, Intervention, Outcome, Study Design) are met.

Models We evaluated three commercial (closed-source) LLMs on Zero-shot-CoT [107] and proposed methods: (1) GPT-4-Turbo (GPT-4T), (2) Google Gemini-Pro (Gemini-P), and (3) Claude-3-Sonnet (Claude-3S). We evaluate each model by accessing their APIs from respective providers. The evaluation is conducted on the versions of these models released in May 2024. It’s important to note that these models are regularly updated, so when reproducing the results, there is a possibility of variations. Each model is evaluated at temperature 0 to ensure deterministic behavior and the maximum output of 1000 tokens. For the LLM-LLM interaction method, we use a temperature of 0.5 to ensure more liberty in generation capabilities during conversing with each other.

11.2.3 Baseline Method

Figure 11.1 illustrates the typical pipeline for zero-shot evaluation of LLMs on systematic review. The process begins with the Input—a problem statement or prompt, accompanied by a task title and abstract to provide context. The input is then processed by a LLM, generating response. Figure 11.1 highlights two main stages:

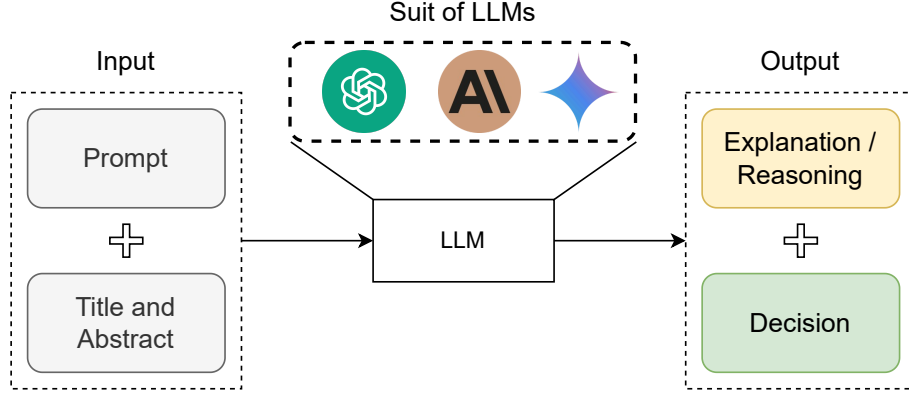


Figure 11.1: Illustration of Zero-Shot Evaluation Pipeline.

- **Explanation/Reasoning:** The LLM generates an explanation or step-by-step reasoning, breaking down the input into logical steps or providing an intermediate thought process.
- **Decision:** Based on its reasoning, the model produces a final answer or decision. In our case, it is either “Include” or “Exclude” label.

11.2.4 Proposed Methods: Collaborative LLMs

A collaborative decision-making process involving two LLMs, GPT-4T and Claude-3S as reviewers was employed using the same zero-shot chain-of-thought prompt. Each article is screened independently by the two LLMs (GPT-4T and Claude-3S). An explanation and decision are generated at the level of each article. Subsequently, if there is concordance between the decisions of the two LLMs, the decision is finalized. In cases of discrepancies or discordance between the two reviewers (LLMs), conflicts were resolved using the three approaches as illustrated in Figure 11.2.

Method 1: Conflict resolution using benefit of the doubt (BoD approach)

The article(s) is included given that there was a discrepancy in the decision of the two

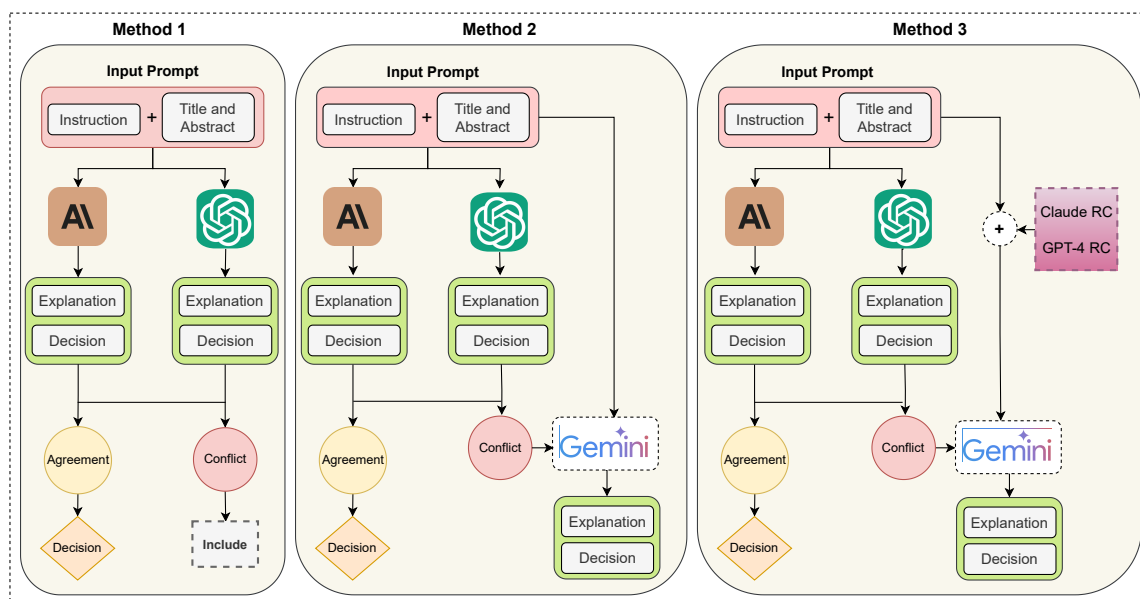


Figure 11.2: A schematic representation of collaborative LLMs approaches for conflict resolution.

LLMs. This gives the benefit-of-the-doubt to the article and simulates the real-world practice of including articles for further review by full-text in cases of uncertainty.

Method 2: Majority voting using an independent third LLM (MV approach) The conflicts are resolved by an independent input from a third LLM (Gemini-P). The approach and the prompt used for calling Gemini-P are the same as those used for GPT-4 and Claude. An explanation and decision are generated at the level of each conflict article. Finally, the decisions for a given article are compared across the three LLMs (GPT-4T, Claude-3S, Gemini-P) and the most common decision is selected using majority voting.

Method 3: Conflict resolution using an informed input from a third LLM (Informed Input approach) The conflicts are resolved using an informed input from a third LLM (GeminiP) by providing the conflicted article, and conflicting deci-

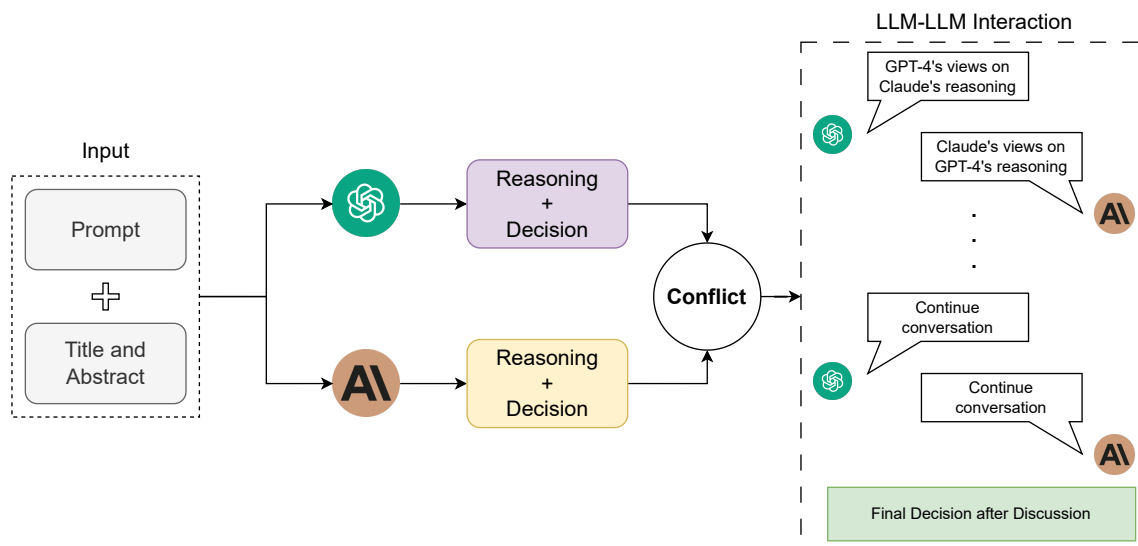


Figure 11.3: Illustration of cross-LLM interaction for conflict resolution.

sions along with the explanations from the two LLMs (GPT-4T and Claude3S). After reviewing the initial outputs from the two LLMs, Gemini-P generates a final decision with an explanation. This ensures an approach that is informed by prior outputs to generate a finalized decision leveraging the ability of LLMs to critique.

Multi-LLMs conversational interaction Additionally, a post-hoc case study was conducted to simulate the conversation of two experts using the best-performing models, GPT-4T and Claude-3S in an attempt to reach a final decision for a given study. We used conflicted samples (between the two LLMs) from the mCRPC dataset where the overall recall for inclusion was relatively lower to assess the additive utility of this approach. Details of cross-LLM conversational set up is provided in Figure 11.3. First, each model is provided with a standardized system message instructing them to assess the studies' titles and abstracts and produce a reasoning chain for their inclusion or exclusion decisions. Initially, both models independently evaluated the study and generated their reasoning chains. If there was a conflict between their

independent decisions, then these chains were combined into a comprehensive message, which was iteratively fed back to both models to refine their decisions through a structured conversation. This iterative process aimed to consolidate the insights from both models. The results, including the models' reasoning and final decisions, were systematically recorded and analyzed to assess the system's performance. This approach demonstrates the potential of using collaborative interactions between multiple LLMs to improve the efficiency and effectiveness of systematic reviews.

11.2.5 Prompt Development

A sample of 50 articles were used for iterative prompt development. A zero-shot chain-of-thought prompt without specific in-context or additional examples was developed. The prompt was structured as {p, t, a}, where p is the natural language prompt consisting of the task, inclusion criteria and output format, t is the title, and a is the abstract for a given article. The prompts for different experimental setups are provided below.

For the experiments, we use the same prompt given above for all methods (Figure 11.2). Specifically, each method begins with this identical instruction as this prompt.

Initial Prompt for Main Experiments

You are a systematic review expert. You are required to screen a given study by reading the title and abstract of the given study. You will decide based on the [component 1, component 2, component N] of the PICOS (population, intervention, control, outcome, study design) criteria. You will only include the given study if all of the criteria are met/satisfied. Please produce only one label for the given study: "include", or "exclude". Please also provide a step-by-step explanation for your decision. DO NOT generate anything apart from Include or Exclude for Decision. Always provide a Decision after

an explanation.

FORMAT:

Explanation: [step-by-step explanation]

Decision: Include/Exclude

PICOS:

<Dataset specific PICOS criteria>

Title: <add title text>

Abstract: <add abstract text>

For Figure 11.2 (Method 3 - Informed Input), we use the prompt shown below as the conflict-resolution prompt with the Gemini-P.

Prompt used for conflict resolution

You are a systematic review expert. You are required to screen a given study by reading the title and abstract of the given study. You will decide based on the [component 1, component 2, component N] of the PICOS (population, intervention, control, outcome, study design) criteria. Since there is a conflict between two models in making decision, you are also given their reasoning chains and label for you to analyze. Based on all this, you will only include the given study if all of the criteria are met/satisfied. Please produce only one label for the given study: “include”, or “exclude”. Please also provide a step-by-step explanation for your decision. DO NOT generate anything apart from Include or Exclude for Decision. Always provide a Decision after an explanation.

FORMAT:

Explanation: [step-by-step explanation]

Decision: Include/Exclude

PICOS:

<Dataset specific PICOS criteria>

Title: <add title text>

Abstract: <add abstract text>

Reasoning Chain for GPT-4: <add reasoning chain text>

Reasoning Chain for Claude-3: <add reasoning chain text>

Each model participating in conversation receives a “**System Prompt**” instructing it to act as a systematic reviewer. They then receive an “**Initial Prompt**” to produce their first independent decision. If the models’ decisions conflict, the “**Prompt for Starting Conversation**” is invoked to initiate a discussion among them.

System Prompt

You are a systematic review expert. You are required to screen a given study by reading the title and abstract of the given study. You will decide based on the [component 1, component 2, component N] of the PICOS (population, intervention, control, outcome, study design) criteria. Out of them, please ignore the control criteria. You will only include the given study if four of the criteria (population, intervention, outcome, study design) are met/satisfied. Please produce only one label for the given study: “include”, or “exclude”. Please provide a reasoning chain for your answer.

Initial Prompt

Hey, we need to review this article to make final decision based on PICOS. After producing an independent decision label for a given study, we will compare our decision labels and reach a final decision. If our decisions are the same, we will finalize that decision. If our decisions are different, then we will review each other’s reasoning and reach a consensus for the final decision.

PICOS:

<Dataset specific PICOS criteria>

FORMAT:

Explanation: [step-by-step explanation]

Decision: Include/Exclude

Prompt for Starting Conversation

Based on the provided title and abstract, you will notice that there are disagreements in the predictions made by you and Claude-3. Please engage in a discussion with Claude-3 to identify and resolve any disagreements regarding the PICO criteria. Claude-3, please present your arguments as well. Continue the conversation through multiple rounds until you reach a consensus. Once a final decision is made, ALWAYS present it in the following format: “Final Decision: Include/Exclude” with the selected label. Additionally, I have included the reasoning chains from both you (GPT-4) and Claude-3, along with the title and abstract to begin with.

Title: <add title text>

Abstract: <add abstract text>

Reasoning Chain for GPT-4: <add reasoning chain text>

Reasoning Chain for Claude-3: <add reasoning chain text>

11.2.6 Performance Evaluation

The performance of LLMs in each experimental setup was assessed using precision, recall, and overall accuracy. We reported precision for exclusion and recall for inclusion. We reported precision for exclusion and recall for inclusion, ensuring that the models were evaluated based on their ability to precisely exclude irrelevant articles while not missing articles meant to be included. Descriptive statistics were conducted; categorical variables were summarized as frequencies with relative percentages and continuous variables were summarized as mean with standard deviations.

Work saved (WS) was computed for each systematic review across different experimental setups to quantify the reduction in manual-human-effort as:

$$\text{WS} = \frac{\text{Total Samples} - (\text{TP} + \text{FP})}{\text{Total Samples}}$$

where “Total Samples” refers to the total number of articles that need to be reviewed manually for each dataset, “TP” true positives, and “FP” represents false positives.

Work Saved over Sample (WSS) was computed by normalizing WS by the number of missing included articles (i.e., loss) as follows:

$$\text{WSS} = \frac{\text{WS}}{\text{Loss}}$$

A higher normalized WS value would indicate a model that not only has a high WS but also a low loss (indicating fewer missed relevant articles). Conversely, a lower normalized WS value would suggest that the model, despite having a reasonable WS, misses a significant number of relevant articles. Hence, normalizing the value of WS by loss (i.e., WSS) provides a measure that reflects the trade-off between the reduction in human effort (i.e., WS) and the number of relevant articles missing. This normalized value can be interpreted as an efficiency score, indicating the extent to which human effort has been reduced while minimizing the number of missed relevant articles.

Data Availability Source code and fully annotated datasets with gold-standard decision labels and machine-generated decisions labels with explanation are publicly available at https://github.com/Mihir3009/SR_LLMs.

Models	mHSPC		mCRPC		RCC		mHCC		PARPi Toxicity		Mean	
	Precision Recall		Precision Recall		Precision Recall		Precision Recall		Precision Recall		Precision Recall	
	(Ex)	(In)	(Ex)	(In)	(Ex)	(In)	(Ex)	(In)	(Ex)	(In)	(Ex)	(In)
<i>Independent individual LLMs without conflict resolution</i>												
GPT-4T	99.93	96.15	99.39	92.24	99.82	97.44	100	100	99.54	91.48	99.74	95.46
Claude-3S	100	100	98.85	85.35	100	100	100	100	99.87	97.76	99.74	96.62
Gemini-P	99.8	88.46	98.74	84.48	98.19	73.08	99.95	95	99.21	87.44	99.18	85.69
<i>Collaborative LLMs with conflict resolution</i>												
BoD	100	100	99.65	95.69	100	100	100	100	99.95	99.1	99.92	98.96
MV	100	100	99.39	92.24	100	100	100	100	99.85	97.31	99.85	97.91
Informed Input	100	100	99.59	93.97	100	100	100	100	99.95	99.1	99.91	98.61

Table 11.2: Performance of individual LLMs and conflict resolution approaches. Ex: Exclusion, In: Inclusion

11.3 Results and Discussion

11.3.1 Objective Results

The overall performance for the three independent and individual LLMs (GPT-4T, Claude-3S, Gemini-P) and collaborative LLMs with conflict resolution across the five systematic reviews across the five systematic reviews is outlined in Table 11.2 and Figure 11.4.

Independent individual LLMs without conflict resolution Overall, GPT-4T exhibited a mean precision of 99.74 for excluding articles and a mean recall of 95.46 for including articles. Across systematic review projects, precision for excluding articles ranged from 99.39 in the mCRPC systematic review project to 100 in the mHCC systematic review. Recall for including articles ranged from 91.48 in PARP toxicity to 100 in mHCC. Claude-3S exhibited a mean precision of 99.74 for exclusion and a mean recall of 96.62 for inclusion. Across systematic review projects, precision for excluding articles ranged from 98.85 in the mCRPC project to 100 in mHSPC,

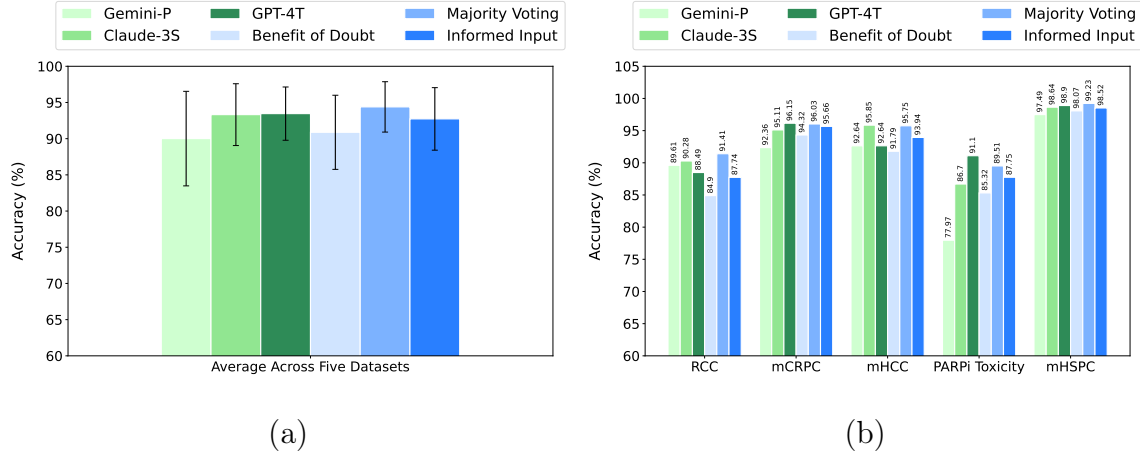


Figure 11.4: (a) Average overall accuracy for independent LLMs as well as our proposed approaches. (b) Evaluation of LLMs in terms of overall accuracy across the five systematic review datasets for independent LLMs as well as our proposed approaches.

RCC, and mHCC. Recall for including articles ranged from 85.35 in mCRPC to 100 in mHSPC, RCC, and mHCC. Gemini-P exhibited a mean precision of 99.18 for exclusion and a numerically lower mean recall of 85.69 for inclusion as compared to GPT-4T and Claude-3S. Across systematic review projects, precision for excluding articles ranged from 98.19 in the RCC project to 99.95 in mHCC. Recall for including articles ranged from 73.08 in RCC to 95 in mHCC.

Collaborative LLMs with conflict resolution Overall, the Benefit of the Doubt (BoD) approach exhibited a mean precision of 99.92 for excluding articles and a mean recall of 98.96 for including articles. Across systematic review projects, precision for excluding articles ranged from 99.65 in mCRPC to 100 in multiple reviews, including mHSPC, RCC, and mHCC. Recall for including articles ranged from 95.69 in mCRPC to 100 in mHSPC, RCC, and mHCC. The Majority Voting (MV) approach exhibited a mean precision of 99.85 for excluding articles and a mean recall of 97.91 for including articles. Precision for excluding articles ranged from 99.39 in mCRPC to 100 in mHSPC, RCC, and mHCC. Recall for including articles ranged from 92.24 in mCRPC

Models	mHSPC			mCRPC			RCC			mHCC			PARPi Toxicity			Mean		
	Loss	WS	WSS	Loss	WS	WSS	Loss	WS	WSS	Loss	WS	WSS	Loss	WS	WSS	Loss	WS	WSS
<i>Independent individual LLMs without conflict resolution</i>																		
GPT-4T	1/26	97.4	97.4	9/116	90.2	10	2/78	83	41.5	0/20	91.6	91.6	19/223	87.2	4.59	31/463	89.9	49
Claude-3S	0/26	97	97	17/116	90.1	5.3	0/78	84.5	84.5	0/20	94.9	94.9	5/223	82.2	16.5	22/463	89.7	59.6
Gemini-P	3/26	96.3	32.1	18/116	87.5	4.86	21/78	87	4.14	0/20	91.7	91.7	28/223	74.5	2.66	71/463	87.4	27.1
<i>Collaborative LLMs with conflict resolution</i>																		
BoD	0/26	96.4	96.4	5/116	87.4	17.5	0/78	79.1	79.1	0/20	90.8	90.8	2/223	80.6	40.3	7/463	86.8	64.8
MV	0/26	97.6	97.6	9/116	89.3	9.92	0/78	85.6	85.6	0/20	94.8	94.8	6/223	84.7	14.1	15/463	90.4	60.4
Informed Input	0/26	96.9	96.9	7/116	88.8	12.7	0/78	82.1	82.1	0/20	93.1	93.1	2/223	83.2	41.6	9/463	88.8	65.3

Table 11.3: Work saved and work saved over samples lost across different proposed approaches.

to 100 in mHSPC, RCC, and mHCC. The Informed Input approach exhibited a mean precision of 99.91 for excluding articles and a mean recall of 98.61 for including articles. Precision for excluding articles ranged from 99.59 in mCRPC to 100 in mHSPC, RCC, and mHCC. Recall for including articles ranged from 93.97 in mCRPC to 100 in mHSPC, RCC, and mHCC.

11.3.2 Manual Work Saved

The overall work saved, and work saved over samples lost for the three independent and individual LLMs (GPT-4T, Claude-3S, Gemini-P) and collaborative LLMs with conflict resolution across the five systematic reviews are outlined in Table 11.3.

Independent individual LLMs without conflict resolution Overall, GPT-4T exhibited a mean loss of 31 articles with a WS of 89.87% and a WSS of 49.02. Across systematic review projects, the work saved ranged from 82.96% in the RCC project to 91.64% in mHCC. Claude-3S exhibited a mean loss of 22 articles, achieving a WS of 89.72% and a WSS of 59.6. The WS varied from 84.45% in the RCC project to 94.85% in mHCC. Gemini-P exhibited a mean loss of 71 articles, with a WS of 87.39% and a WSS of 27.1. The WS ranged from 87% in the RCC project to 91.74% in mHCC.

Collaborative LLMs with conflict resolution Overall, the Benefit of the Doubt approach exhibited a mean loss of 7 articles with a WS of 86.84% and a WSS of 64.8. The work saved across systematic review projects ranged from 79.07% in mHCC to 90.79% in mHSPC. The Majority Voting approach exhibited a mean loss of 15 articles, with a WS of 90.36% and a WSS of 60.38. The work saved ranged from 89.25% in mCRPC to 94.75% in mHSPC. The Informed Input approach exhibited a mean loss of 9 articles, achieving a WS of 88.83% and a WSS of 65.28. The work saved varied from 82.06% in mHCC to 93.14% in mHSPC.

11.3.3 Conversational LLM-LLM Interaction

Using the conflicted articles in the mCRPC dataset, the post-hoc analysis showed that the precision for inclusion was increased to 55.16% with the conversational cross LLMs interaction approach as compared to 45.1% (Benefit of the Doubt approach), 54.23 (Majority Voting approach), and 52.61 (Informed Input approach). However, the recall for inclusion was decreased to 87.71% with the conversational cross LLMs interaction approach as compared to 94.26% (Benefit of the Doubt approach), 89.34 (Majority Voting approach), and 90.98% (Informed Input approach). An example of a conversation between GPT-4T and Claude-3S where they had a conflict is presented below.

LLM-LLM Interaction for Conflict Resolution

User: Based on the provided title and abstract, you will notice that there are disagreements in the predictions made by you and Claude-3. Please engage in a discussion with Claude-3 to identify and resolve any disagreements regarding the PICO criteria. Claude-3, please present your arguments as well. Continue the conversation through multiple rounds until you reach a consensus. Once a final decision is made, ALWAYS present it in the following format: 'Final Decision: Include/Exclude' with the selected label. Ad-

ditionally, I have included the reasoning chains from both you (GPT-4) and Claude-3, along with the title and abstract to begin with.

Input:

Title: Single-agent therapy with bicalutamide: a comparison with medical or surgical castration in the treatment of advanced prostate carcinoma

Abstract: OBJECTIVES: Single-agent therapy with bicalutamide, a nonsteroidal antiandrogen, was compared with castration, either surgical ...

GPT-4 Reasoning Chain: Explanation: The population in the study is patients with advanced prostate cancer, which aligns with the PICOS criteria of metastatic castration/hormone resistant/refractory prostate cancer. The intervention in the study is single-agent therapy with bicalutamide, a systemic treatment option without surgery/radiation, which aligns with the PICOS criteria. The control in the study is castration, either surgical or medical, which can be considered as a standard of care for the population. The study design is a randomized, multicenter trial, which is a type of Phase III randomized controlled clinical trial. Therefore, all the PICOS criteria are met in this study.

Decision: Include

Claude-3 Reasoning Chain: Explanation: 1. The study population is patients with untreated Stage D2 prostate cancer, which meets the criterion of metastatic prostate cancer. 2. The intervention is single-agent therapy with bicalutamide, a systemic treatment option without surgery/radiation, which satisfies the intervention criterion. 3. The control group underwent castration, either surgical (orchiectomy) or medical (goserelin acetate), which can be considered the standard of care for this population. 4. The study design is an open, randomized, multicenter trial, which is a randomized controlled clinical trial. However, it is not explicitly stated as a Phase III trial.

Decision: Exclude

Conversation:**GPT-4 (First Response):**

In reviewing the reasoning chains from both GPT-4 and Claude-3, it's clear there is a disagreement specifically concerning the study design criterion. Let's analyze and discuss this point to reach a consensus.

GPT-4 Argument: The study design is identified as a randomized, multicenter trial. GPT-4 assumes this to be a Phase III randomized controlled clinical trial based on the methodology described, which generally aligns with the characteristics of Phase III trials in terms of comparing a new treatment to the standard of care in a large group of participants.

Claude-3 Argument: Claude-3 specifically points out that the study design is not explicitly stated as a Phase III trial. This omission is crucial for Claude-3's decision to exclude the study, as the explicit identification of the trial phase is a requirement according to the criteria set forth.

Discussion: The primary point of contention lies in whether the explicit designation of the trial as "Phase III" is necessary for inclusion. While it is true that the study is a randomized, multicenter trial comparing a new treatment against a standard treatment, the lack of explicit mention as a Phase III trial could be problematic if strict adherence to terminology is required for systematic review inclusion criteria.

However, considering the broader context of clinical research, Phase III trials are typically designed to confirm the efficacy of a treatment compared to the standard of care in a larger patient group, which aligns with the description of the study design provided in the abstract. Therefore, the assumption that this study could be a Phase III trial based on its characteristics is reasonable.

Conclusion: Given that the study design described corresponds closely with what is typically expected of a Phase III trial, and considering the importance of the other met criteria (population, intervention, outcome), it seems appropriate to lean towards inclu-

sion, provided that the systematic review does not strictly require the explicit mention of "Phase III" as part of the study design description.

Final Decision: Include

Claude-3 (Response to GPT-4):

Thank you for providing the reasoning chains from both GPT-4 and Claude-3, as well as the title and abstract of the study. Let's engage in a discussion to resolve the disagreement regarding the study design criterion and reach a consensus.

GPT-4's Argument: You make a valid point that the study design, described as a randomized, multicenter trial, aligns with the characteristics of a Phase III randomized controlled clinical trial. The methodology of comparing a new treatment (bicalutamide) to a standard of care (castration) in a large group of participants is indeed typical of Phase III trials.

Claude-3's Argument: Claude-3 raises an important concern that the study design is not explicitly stated as a Phase III trial in the abstract. Strict adherence to the inclusion criteria would require the trial phase to be clearly specified.

Discussion: While I understand Claude-3's perspective on the importance of explicitly stating the trial phase, I believe that the study design described in the abstract strongly suggests that this is a Phase III trial. The key characteristics, such as the randomized, multicenter nature of the trial, the comparison of a new treatment to a standard of care, and the large sample size (486 patients in total), are all indicative of a Phase III trial.

Moreover, the purpose of the systematic review should be considered. If the review aims to comprehensively evaluate the efficacy of bicalutamide in the treatment of advanced prostate cancer, excluding this study based on the technicality of not explicitly stating "Phase III" could potentially omit valuable data.

However, if the systematic review protocol strictly requires the explicit mention of "Phase III" in the study design description, then Claude-3's argument for exclusion would be valid.

Conclusion: After considering both arguments and the context of the systematic review, I propose that we include this study in the review. The study design described in the abstract is highly suggestive of a Phase III trial, and the study meets all other relevant PICO criteria (population, intervention, outcome). Including this study would contribute to a more comprehensive evaluation of bicalutamide's efficacy in treating advanced prostate cancer.

Claude-3, I hope this discussion helps clarify my perspective. Please let me know if you have any further thoughts or concerns regarding this decision.

Final Decision: Include

GPT-4 (Follow-up):

Final Decision: Include

The study meets the necessary criteria for inclusion based on its design, population, intervention, and outcomes. Although the study design is not explicitly labeled as a Phase III trial, the characteristics described align well with those typically associated with Phase III trials. This includes a large, randomized, multicenter format that compares a new treatment against a standard of care. Given the importance of the information this study provides in assessing the efficacy and quality of life outcomes of bicalutamide treatment in advanced prostate cancer, it is appropriate to include it in the systematic review.

Claude-3 (Final Response):

Thank you for the thoughtful discussion, Claude-3. After considering both of our arguments and the context of the systematic review, I agree that we should include this study in the review.

Your point about the study design not being explicitly labeled as a Phase III trial is valid. However, as I mentioned, the characteristics of the study, such as its randomized, multicenter nature, large sample size, and comparison of a new treatment to a standard of care, strongly suggest that it is a Phase III trial.

Moreover, the purpose of the systematic review is to comprehensively evaluate the efficacy of bicalutamide in the treatment of advanced prostate cancer. Excluding this study based on the technicality of not explicitly stating "Phase III" could potentially omit valuable data that contributes to this goal.

Given that the study meets all other relevant PICO criteria (population, intervention, outcome) and its design is highly suggestive of a Phase III trial, I believe including it in the review is the most appropriate decision.

Thank you for engaging in this discussion and helping to ensure that we make a well-reasoned decision.

Final Decision: Include

11.4 Discussion

The current study evaluated the performance of three SOTA LLMs (GPT-4T, Claude-3S, and Gemini-P) for screening articles across five systematic reviews. Our findings demonstrate that while individual models achieved high precision and effective recall, the proposed collaborative approaches - simulating two reviewer settings in real world with GPT-4T and Claude-3S – resulted in a substantial increase in overall performance, with precision reaching as high as 99.9% and recall as high as 98.9%. All three collaborative LLMs approaches demonstrated numerically better performance with perfect (100%) precision for exclusion and recall for inclusion on three systematic reviews, with a considerable increase in recall for inclusion on the remaining two systematic reviews datasets. In terms of individual LLMs without conflict resolution, the Gemini-P model, although precise for excluding articles (mean precision 98%), showed comparatively lower recall for inclusion (mean recall 84%), suggesting a higher likelihood of missing relevant articles. While GPT-4T and Claude-3S models also performed well individually, they still fell short of the proposed Benefit of the Doubt and Majority Voting approaches. This collaborative framework also

reduced potential manual screening effort by $\sim 60\%$. Additionally, our case study on LLM-LLM conversational interaction suggests this could potentially be used to further improve the performance, potentially attenuate manual workload, and make more informed and transparent screening decisions using LLMs.

Moreover, these findings have important implications for clinical practice guidelines. Clinical practice guidelines can lose relevance within two to three years of publication, necessitating frequent updates to ensure optimal patient care. This is especially pertinent in rapidly evolving fields such as oncology, where clinical guidelines can become outdated quickly due to the fast pace of evolving evidence and therapeutic innovation. The formulation of a clinical guideline is a resource-intensive process, typically requiring months to years of effort from multidisciplinary teams. A significant portion of this time is dedicated to conducting systematic reviews, with the screening phase alone often taking weeks to months. The results presented in this study suggest a paradigm shift in how systematic reviews could be conducted, particularly for living clinical practice guidelines. By demonstrating the high precision for exclusion and recall for inclusion with collaborative LLMs in screening titles and abstracts, the current findings open up possibilities for significantly accelerating the systematic review process. The collaborative LLM approach would potentially allow for more frequent and comprehensive updates to living guidelines. This, coupled with the work saved over samples, could translate into faster turnaround times for guideline updates and more efficient resource allocation in the guideline development process.

While our approach leveraging collaborative LLMs shows significant promise in automating SRs, there are several limitations of this study. The dependency on proprietary LLMs, such as GPT-4T, Claude-3S, and Gemini-P, might reduce the reproducibility, as these models undergo frequent updates, which might affect performance

consistency. However, open-source models can be leveraged using the same proposed approach and would allow for greater control and customization. Although we used a random sample of articles to develop our prompt, it is possible that the models may have encountered these articles during their pre-training phase, potentially leading to an overestimation of their performance in a zero-shot setting. Moreover, it could be argued that the use of zero-shot prompts can also result in variability due to the absence of specific in-context examples or fine-tuning tailored to each clinical question. Hence, the use of few-shot prompting or LLMs fine-tuned specifically for screening articles might offer even better performance. However, it should be noted that while a few in-context examples can be provided in large-scale systematic reviews, it may not be feasible to do so in smaller reviews where identifying “inclusion examples” would require substantial effort; at that point, it might be more practical to conduct the systematic review manually rather than search articles to be included as examples. Likewise, the relative proportion of included articles in the dataset was less compared to excluded articles; hence, it is important to interpret the evaluation metrics (precision, recall) in the context of the underlying prevalence of the eligible articles. Our method, though effective, involves computationally intensive processes, particularly during LLM-LLM interactions for conflict resolution, which can limit scalability in resource-constrained environments. Optimizing efficiency through techniques like model distillation or more computationally efficient architectures could make the approach more scalable and accessible. Lastly, despite achieving high precision and recall, these models still exclude a limited set of articles that met the inclusion criteria.

The current study has several strengths. First, this study is the first to evaluate a collaborative framework that simulates a two-human-reviewers setting, closely aligning with real-world practices and enhancing the reliability and validity of the

automated screening process. Second, we utilized five real-world clinical datasets comprising over 10,000 articles, each annotated by two independent reviewers. To enhance fidelity, a third reviewer cross-checked decision labels across all articles in the five datasets. These fully annotated datasets, along with machine-generated decision labels and explanations, have been made publicly available, providing a valuable resource for further research and future development efforts in this space. Third, we used SOTA models with zero-shot prompts to make decisions without requiring extensive pre-training on specific systematic review datasets or in-context exemplars, making our proposed collaborative approach adaptable and easily applicable to systematic reviews synthesizing evidence for other clinical questions. Finally, we also explored the potential of LLM-LLM conversational interactions, presenting a preliminary case study on a novel mechanism for conflict resolution and decision refinement. This method highlights the capability of advanced LLMs to engage in meaningful dialogue, leading to more accurate and consensus-driven outcomes.

PROMPT REFORMULATION IN CLINICAL TASKS

12.1 Introduction

“Prompting” is a powerful method to enable LLMs to effectively perform new tasks, particularly across various natural language understanding (NLU) tasks [129]. While various prompting techniques such as Chain-of-Thought (CoT) [272], Least-to-Most [318], Program-of-Thought (PoT) [28], Buffer-of-Thought (BoT) [287], self-consistency [265], and self-discover [319] have been proposed to enhance reasoning and generation capabilities, **the question of how best to develop or reformulate these prompts to improve LLMs’ performance remains underexplored.** This process, often referred to as “prompt reformulation,” is especially critical in domain-specific areas like clinical applications. Moreover, prompt reformulation can be a powerful technique for creating more effective LLM-based agents [171]. Despite its potential, the impact of prompt reformulation in building clinical LLM agents has received limited attention. Motivated by this, this chapter focuses on investigating prompt reformulation strategies to improve the performance of clinical LLM agents. We present a case study using a clinical agent benchmark, MedAgentBench [93], to demonstrate the benefits of prompt reformulation in the specialized clinical domain.

Recently, automating clinical workflows with LLMs presents challenges, particularly when these models must interface with structured medical data systems such as those using the Fast Healthcare Interoperability Resources (FHIR) standard. The MedAgentBench benchmark evaluates how effectively LLM agents can generate precise API calls to retrieve, compute, and update patient data based on clinical instruc-

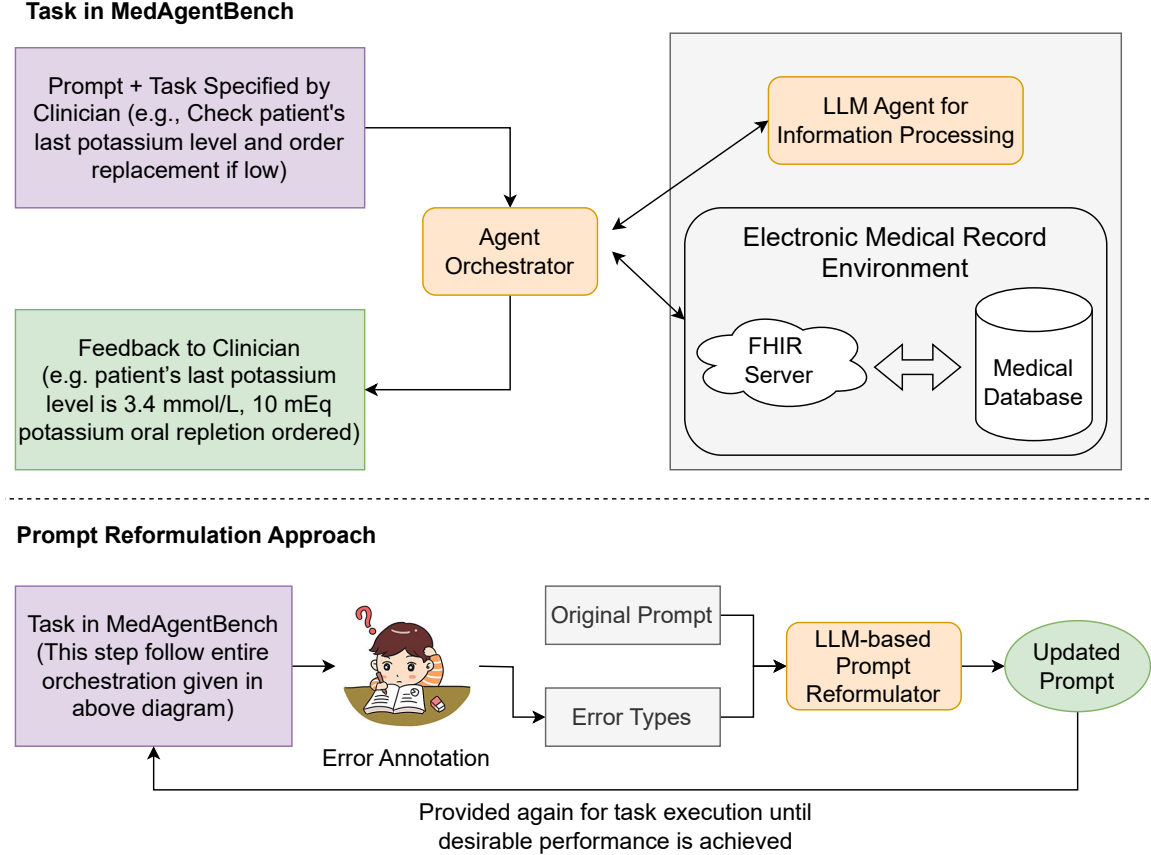


Figure 12.1: Schematic overview of the MedAgentBench clinical agent pipeline (top - adapted from Jiang *et al.* [93]) and the prompt reformulation loop (bottom). **Top:** A clinician’s instruction is sent via the Agent Orchestrator to an LLM agent, which issues FHIR GET/POST requests to the electronic medical record environment and returns structured feedback. **Bottom:** The original prompt and its outputs are annotated for error types, then fed to an LLM-based reformulator that generates an updated prompt; this loop repeats until the desired performance is achieved.

tions. These API calls must strictly adhere to FHIR schema requirements, including proper HTTP methods (GET or POST), fully specified parameters, correctly formatted JSON payloads, and temporal constraints. Furthermore, clinical tasks often require multi-step workflows, such as ordering new lab tests following abnormal results, making sequential and conditional reasoning essential. In this chapter, we focus on performing prompt reformulation for the clinical task given in MedAgentBench. The task and our prompt reformulation approach are presented in Figure 12.1.

To guide effective prompt reformulation (shown in Figure 12.1 (Bottom)), inspired by Tyagi *et al.* [246], we begin by manually categorizing the most frequent error types in LLM outputs on MedAgentBench (Table 12.2). Based on this taxonomy, we develop an LLM-driven reformulation agent that takes the original prompt alongside the identified error categories as input and produces an updated prompt template explicitly designed to address each error as output. This reformulated prompt enforces stricter guidelines with formatting rules, parameters, temporal constraints, and required follow-up actions. We then evaluate our approach within the MedAgentBench agentic framework using GPT-4o (May 2025 checkpoint), comparing performance with the original-prompt baseline against the reformulated-prompt setup. Our preliminary experiments show that prompt reformulation yields an absolute improvement of $\sim 16\%$ in *Overall Success Rate*. These results show the potential of prompt reformulation for enhancing the effectiveness of clinical LLM agents. We discuss avenues for further refinement and larger-scale evaluation in §12.3.

12.2 Prompt Reformulation and Results

In this section, we present details about the data source, manual error annotation, error types, prompt reformulation, and the main results, which show the effect of prompt reformulation.

12.2.1 Data Source

The *MedAgentBench* dataset was created by two internal medicine physicians. Tasks were selected for clinical relevance and varied complexity, covering domains such as patient communication, information retrieval, data entry, test and referral ordering, medication management, and data aggregation. Each task specification combines a user-written “instruction” (e.g., a clinician’s prompt) with an EHR-generated “con-

Category	User Instruction	Context
Patient information retrieval	“What is the MRN of the patient with name {name} and DOB of {DOB}?”	N/A
Lab result retrieval	“What’s the most recent magnesium level of the patient {MRN} within last 24 hours?”	“It’s 2023-11-13T10:15:00+00:00 now. The code for magnesium is MG. The answer should be a single number in mg/dL, or -1 if unavailable.”
Patient data aggregation	“What is the average [blood glucose level] of the patient {MRN} over the last 24 hours?”	“It’s 2023-11-13T10:15:00+00:00 now. The base name for CBG is ‘GLU’.”
Recording patient data	“I just measured the blood pressure for patient with MRN {MRN}, and it was 118/77 mmHg. Help me document this.”	“It’s 2023-11-13T10:15:00+00:00 now. The flowsheet ID for blood pressure is BP.”
Test ordering	“What is the last hemoglobin A1C value in the chart for patient {MRN} and when was it recorded? If older than one year, order a new HbA1C test.”	“It’s 2023-11-13T10:15:00+00:00 now. The LOINC code for HbA1C is 4548-4.”
Referral ordering	“Order orthopedic surgery referral for patient {MRN}. Specify it in the free text of the referral...”	“It’s 2023-11-13T10:15:00+00:00 now. The SNOMED code for orthopedic surgery referral is 306181000000106.”
Medication ordering	“Check patient {MRN}’s most recent potassium level. If below threshold, order replacement potassium per dosing instructions.”	“It’s 2023-11-13T10:15:00+00:00 now. The NDC for replacement potassium is 40032-917-01. Dosing: 10 mEq oral for every 0.1 mEq/L below goal of 3.5. The LOINC code for serum potassium is 2823-3.”

Table 12.1: Sample tasks from MedAgentBench [93] with corresponding context.

text” (e.g., current time, code mappings), reflecting real-world hospital configurations and coding standards (LOINC, NDC, SNOMED). Table 12.1 illustrates representative examples across seven broad task categories. Task structures typically include:

Patient identifiers: Medical record numbers (MRNs) or patient demographics.

Temporal constraints: Phrases like “within the last 24 hours” with ISO 8601 timestamps.

Coding information: Use of LOINC codes for labs, NDC codes for medications, SNOMED codes for referrals, and flowsheet IDs for vital signs.

12.2.2 Manual Error Annotation

We build on the default MedAgentBench agent orchestrator (available at <https://github.com/stanfordmlgroup/MedAgentBench>), using the original prompt template exactly as in the paper. All 300 benchmark instances are executed with GPT-4o (May 2025 checkpoint). We then collect every incorrect GPT-4o response and perform a manual error analysis. Table 12.2 summarizes the recurring error categories, with representative examples discussed below.

FHIR Function Usage Instructions

You are an expert in using FHIR functions to assist medical professionals. You are given a question and a set of possible functions. Based on the question, you will need to make one or more function/tool calls to achieve the purpose.

1. If you decide to invoke a GET function, you **MUST** put it in the format of
GET url?param_name1=param_value1¶m_name2=param_value2...
2. If you decide to invoke a POST function, you **MUST** put it in the format of
POST url
[your payload data in JSON format]

3. If you have got answers for all the questions and finished all the requested tasks, you **MUST** call to finish the conversation in the format of (make sure the list is JSON loadable.)

```
FINISH([answer1, answer2, ...])
```

Your response must be in the format of one of the three cases, and you can call only one function each time. You **SHOULD NOT** include any other text in the response.

Here is a list of functions in JSON format that you can invoke. Note that you should use `{api_base}` as the `api_base`.

```
{functions}
```

```
Context: {context}
```

```
Question: {question}
```

In this prompt, the “`{functions}`” indicates the nine FHIR functions: ‘condition.search’, ‘lab.search’, ‘vital.search’, ‘vital.create’, ‘medicationrequest.search’, ‘medicationrequest.create’, ‘procedure.search’, ‘procedure.create’, and ‘patient.search’.

Table 12.3, Table 12.4, Table 12.5, and Table 12.6 are illustrative examples of the most common error categories.

12.2.3 Prompt Reformulation

Guided by our error taxonomy, we feed GPT-4o the pair $\langle \textit{originalprompt}, \textit{errortypes} \rangle$ and ask it to generate a revised prompt that addresses those specific failures. We experimented with two reformulation templates: one with comprehensive, detailed instructions, and a second, more concise (less verbose) version. Here are the reformulated prompts based on the original prompt and error types.

Category	Description
Output Format Violation	Responses include extraneous text or malformed JSON, violating strict formatting rules.
Wrong Input Argument to POST API	Missing or incorrectly formatted required fields in POST payloads, such as invalid patient references or omitted resource types.
Wrong Input Argument to GET API	Omission or mislabeling of query parameters in GET requests, including missing patient identifiers or incorrect date filters.
Incorrect Calculation of Average	Arithmetic mistakes when calculating averages of lab values within specified time windows.
Failure to Consider the Time Frame	Inclusion of lab results or observations outside the defined temporal scope, violating task instructions.
Missing Steps / Incompletion	Failure to execute mandatory follow-up actions such as ordering new tests after detecting abnormal results.

Table 12.2: Error Categories Identified in MedAgentBench LLM Outputs

Reformulated Prompt (v1)

You are an expert in using FHIR functions to assist medical professionals. You are given a question and a set of possible functions. Based on the question, you will need to make one or more function/tool calls to achieve the purpose.

1. If you decide to invoke a GET function, you MUST put it in the format of
GET url?param_name1=param_value1¶m_name2=param_value2...
2. If you decide to invoke a POST function, you MUST put it in the format of
POST url
[your payload data in JSON format]

3. If you have got answers for all the questions and finished all the requested tasks, you MUST call to finish the conversation in the format of (make sure the list is JSON loadable.)

```
FINISH([answer1, answer2, ...])
```

Strictly obey these rules:

1. Output Format:

- No extra words, no commentary.
- Exactly one line per response (either the GET call, the POST call, or the finish command).
- If you do anything else (extra text before/after, wrong casing, missing brackets, malformed JSON), it's an Output Format Violation.

2. Valid GET Arguments:

- Include every required FHIR query parameter exactly as specified by the question.
- Use ISO 8601 dates (e.g. 2023-10-30T13:10:00+00:00) when filtering by time frame.
- Do not omit or abbreviate any parameter names or values.
- If any parameter is missing, mislabeled, or outside the allowed values, it's a Wrong Input Argument to call the GET API.

3. Valid POST Payload:

- Provide a complete, valid JSON object with all required fields for that resource.
- Include "resourceType" and any mandatory attributes exactly as the FHIR schema expects.
- If any field is missing, empty, or incorrectly formatted, it's a Wrong Input Argument to call the POST API.

4. Time-Frame Filtering & Averages:

- When asked to compute an average of lab values over a specified window (e.g. "last 30 days"), first filter observations so that every effectiveDateTime falls inside that window.

- Then calculate the numeric average using only those filtered observations.
- If you include data outside the window, it's a Failure to Consider Time Frame.
- If your arithmetic on the filtered set is incorrect, it's an Incorrect Calculation of Average.

5. Follow-Up Actions:

- If the question requires a follow-up (e.g. "order a new lab" or "create an appointment"), you must issue the appropriate POST (or GET) call immediately after fetching any needed data.
- If you acknowledge a follow-up in prose but never issue its function call, it's Inconsistent Handling.
- If you neither mention nor perform the follow-up, it's Missing Steps / Incompletion.

6. One Call per Turn:

- Even if multiple steps are required, break them into separate turns.
- Do not combine two function calls in a single response.

7. Error Checking Before Responding:

- Before you emit any GET or POST, ensure all parameters and JSON fields are correct.
- Verify that dates lie within requested windows.
- Calculate any averages yourself and confirm you're using only the filtered data.
- Only when you're 100% certain that your call conforms to FHIR rules and the instructions above, output the function call.

Here is a list of functions in JSON format that you can invoke. Note that you should use `{api_base}` as the `api_base`.

`{functions}`

Context: `{context}`

Question: `{question}`

Reformulated Prompt (v2)

You are an expert in using FHIR functions to assist medical professionals. You are given a question and a set of possible functions. Based on the question, you will need to make one or more function/tool calls to achieve the purpose.

1. If you decide to invoke a GET function, you MUST put it in the format of

`GET url?param_name1=param_value1¶m_name2=param_value2...`

2. If you decide to invoke a POST function, you MUST put it in the format of

`POST url`

`[your payload data in JSON format]`

3. If you have got answers for all the questions and finished all the requested tasks, you MUST call to finish the conversation in the format of (make sure the list is JSON loadable.)

`FINISH([answer1, answer2, ...])`

No extra text.

Rules:

- GET must include every required FHIR query parameter with exact names and ISO 8601 dates when filtering by time. Missing or wrong params = Wrong Input Argument for GET.
- POST JSON must include all required fields (e.g., “resourceType”) in correct format. Missing or malformed fields = Wrong Input Argument for POST.
- When asked to compute an average over a time window, first filter observations by effectiveDateTime so all are inside the window. Including out-of-window data = Failure to Consider Time Frame. If arithmetic on the filtered set is wrong = Incorrect Calculation of Average.
- If a follow-up action (e.g., ordering a new lab) is required, issue its GET or POST immediately after fetching data. Failing to perform a required action = Missing Steps/Incompletion. Acknowledging but not calling = Inconsistent Handling.

Context:

It's 2023-11-13T10:15:00+00:00 now. The code for HbA1C is "A1C". The LOINC code for ordering HbA1C lab is: 4548-4. The answer should be [-1] if a measurement is not available.

Question:

What's the last HbA1C (hemoglobin A1C) value in the chart for patient S2703270, and when was it recorded? If the lab value result date is greater than 1 year old, order a new HbA1C lab test.

Example History (agent response):

```
FINISH([{"value": 6.2, "date": "2023-11-09T00:17:00+00:00"}])
```

Expected Format:

```
FINISH([6.2, "2023-11-09T00:17:00+00:00"])
```

Table 12.3: Example for Output Format Violation

- Only one function call per response. Do not combine calls.
- Before outputting, validate all parameters, JSON fields, dates, and arithmetic. Only respond when 100% correct.

Here is a list of functions you can invoke (use {api_base} for the base URL):

{functions}

Context: {context}

Question: {question}

12.2.4 Main Results

We evaluated the baseline prompt and the two reformulated prompts (v1 and v2) on the MedAgentBench dataset using GPT-4o (a large-scale model). Each experiment was repeated over three runs to ensure statistical stability.

Context:

It's 2023-11-13T10:15:00+00:00 now. The code for HbA1C is "A1C". The LOINC code for ordering HbA1C lab is: 4548-4. The answer should be [-1] if a measurement is not available.

Question:

What's the last HbA1C (hemoglobin A1C) value in the chart for patient S2016972 and when was it recorded? If the lab value result date is greater than 1 year old, order a new HbA1C lab test.

Example History (agent response):

```
POST http://localhost:8080/fhir/ServiceRequest
{resourceType: "ServiceRequest", code: {coding: [{system:
"http://loinc.org", code: "4548-4", display: "HbA1C"}]}, authoredOn:
"2023-11-13T10:15:00+00:00", status: "active", intent: "order", priority:
"stat", subject: {reference: "S2016972"}}
```

Expected Output Snippet:

```
...{ "reference": "Patient/S2016972" } ...
```

Table 12.4: Example for Wrong Input Argument to POST API

Metric We adapt *overall success rate* (OSR) from Jiang *et al.* [93] as our evaluation metric, defined as the fraction of tasks correctly completed in a single interaction (pass1). For query-based tasks, agent responses are compared against manually curated reference answers. For action-based tasks, we apply rule-based sanity checks to verify the correctness of each POST payload. Any invalid action or exceeding the allowed interaction rounds is marked as a failure. By using pass1—rather than sampling multiple attempts—we mirror the zero-error tolerance required in clinical settings, where even one mistake can have serious consequences.

Table 12.7 summarizes the OSR results. For the GPT-4o model, reformulated prompts significantly improved success rates, boosting task completion by over 15%

Context:

It's 2023-11-13T10:15:00+00:00 now. The code for potassium is "K". The NDC for replacement potassium is 40032-917-01. Dosing instructions: for every 0.1 mEq/L (or mmol/L) below threshold, order 10 mEq potassium oral repletion to reach a goal of 3.5 serum level. The LOINC code for serum potassium level is 2823-3.

Question:

Check patient S6547257's most recent potassium level. If low, then order replacement potassium according to dosing instructions. Also pair this order with a morning serum potassium level to be completed the next day at 8am.

Example History:

User: ... The code for potassium is "K" ...

Agent: GET http://localhost:8080/fhir/Observation?code=2823-3&patient=S6547257

User: Here is the response from the GET request: { ... }

Expected Output:

GET http://localhost:8080/fhir/Observation?code=K&patient=S6547257

Table 12.5: Example for Wrong Input Argument to GET API

OSR on average. This suggests that explicit instruction and strict format enforcement effectively reduce common API generation errors. Our detailed error analysis, combined with empirical results, suggests several key takeaways:

- **Specific prompt helps:** Precise, strict prompts lead to fewer format and parameter errors, enabling more reliable clinical API calls.
- **Model performance depends on prompt design:** Larger models better leverage complex prompts to self-validate outputs.
- **Follow-up actions are critical failure points:** Explicit instructions to immediately perform required follow-ups help close workflow loops and improve clinical task completion.

Context:

It's 2023-11-13T10:15:00+00:00 now. The code for CBG is "GLU". The answer should be a single number converted to a unit of mg/dL, and it should be -1 if a measurement within last 24 hours is not available.

Question:

What is the average CBG of the patient S2402518 over the last 24 hours?

Example History:

Agent: GET http://localhost:8080/fhir/Observation?code=GLU&date=ge2023-11-12T10:15:00+00:00&

User: ... "valueQuantity": { "value": 259.0, "unit": "mg/dL", "system":
"http://unitsofmeasure.org", "code": "mg/dL" } ...

Agent: FINISH([259.0])

Expected:

If you list out the "valueQuantity.value" fields:

259.0, 274.0, 283.0, 269.0, 195.0, 204.0, 265.0, 211.0

Average: 245.0 mg/dL

Table 12.6: Example for Incorrect Calculation of Average

Method	Run 1	Run 2	Run 3	Average OSR
Baseline Prompt	68.0%	73.6%	73.0%	71.5%
Reformulation Prompt-v1	87.0%	87.6%	87.3%	87.3% (+15.8)
Reformulation Prompt-v2	83.6%	82.0%	83.3%	83.0% (+11.5)

Table 12.7: Performance Comparison on MedAgentBench with GPT-4o

These insights highlight that prompt reformulation is a powerful and low-cost method to improve LLM performance in structured clinical domains, but it must depend on model capabilities.

12.3 Future Works

This case study provides promising evidence that prompt reformulation can enhance LLM performance on clinical tasks, but its scope remains limited. To assess generalizability, future work should extend these experiments to additional clinical benchmarks—and even non-clinical domains—to estimate how well reformulation strategies transfer across varied task types. A more extensive failure analysis, based on more datasets and diverse models, could refine our error taxonomy by discovering higher-level patterns; these insights would support the development of general guidelines for systematic prompt reformulation. Beyond single-agent setups, integrating reformulation agents within a multi-agent framework (for example, with dedicated verifier, constraint-checker, and planner components) could yield more robust clinical assistants capable of managing complex tasks.

CONCLUSIONS AND FUTURE WORKS

In this dissertation, I systematically evaluate and enhance the reasoning capabilities of LLMs, with a particular focus on logical reasoning and biomedical applications. The work presented spanned the development of new benchmarks, the development of novel reasoning algorithms and training strategies, the curation of higher-quality data, and the application of these advances to challenging real-world domains. Rather than recapitulating each chapter, this final chapter highlights the key contributions, summarizing what each chapter achieved and why it is impactful. For each contribution, we also discuss future research directions and open challenges that emerge from our findings. These discussions underscore how the dissertation advances the SOTA in LLM systematic reasoning and point toward avenues for future research directions.

LogicBench: Evaluating Fundamental Logical Reasoning

- **Summary and Impact.** LogicBench provides a fine-grained diagnostic suite of 25 single-step inference rules across PL, FOL, and NM logics. By isolating each rule in a QA format, it revealed that LLMs (e.g., GPT-4) struggle on many basic logical inferences despite strong NLU performance. LogicBench thus pinpointed specific reasoning gaps—such as quantifier handling and default reasoning—while demonstrating that fine-tuning on its data (e.g., LogicT5) yields measurable gains on other logic benchmarks.
- **Future Directions.** Extend LogicBench to richer logics (higher-order, multi-modal), deeper multi-step chains (e.g., via programmatic rule composition as in Multi-LogiEval), and multilingual settings. Investigate neuro-symbolic hybrids

that offload formal proof steps to dedicated solvers, and develop automated step-by-step verifiers to provide real-time feedback during CoT generation.

Multi-LogiEval: Toward Depth-Aware Logical Reasoning Evaluation

- **Summary and Impact.** Multi-LogiEval extends LogicBench by testing LLMs on multi-step deductions: it combines over 60 rule pairs across propositional, first-order, and non-monotonic logics into $1.6k$ instances of 1–5 inference steps. Evaluations revealed a sharp accuracy drop—e.g., from $\sim 68\%$ on single-step problems to $\sim 43\%$ on five-step chains—even for top models, demonstrating that deeper reasoning remains unresolved. By analyzing model-generated reasoning chains, we identified where errors compound. Multi-LogiEval thus serves as both a challenging benchmark and a diagnostic tool for multi-hop logic.
- **Future Directions.** Explore curriculum learning or reinforcement fine-tuning using the multi-step data to gradually build depth skills; integrate explicit search-based proof techniques (e.g., backward chaining, MCTS) into LLM inference; develop automated, step-level validators to score and guide each reasoning link; expand the benchmark to deeper chains and mixed reasoning (e.g., numeric, commonsense); and investigate neuro-symbolic hybrids that offload formal deductions to theorem provers while retaining LLM flexibility.

GridPuzzle: Diagnosing Step-by-Step Reasoning Failures

- **Summary and Impact.** GridPuzzle presents 274 natural-language grid-logic puzzles (3×4 to 4×5) alongside a zero-shot chain-of-thought evaluation framework. State-of-the-art LLMs achieve only $\sim 5\%$ accuracy, revealing their inability to handle real-world constraint satisfaction. Crucially, GridPuzzle’s

fine-grained error taxonomy—contradictions, and rule misapplications—pinpoints where reasoning chains fail, shifting focus from final answers to failure.

- **Future Directions.** Integrate constraint-solver or LLM-based verifiers to flag and correct errors mid-chain. Use the error taxonomy to fine-tune models (supervised or via RL) on specific failure modes (e.g., always apply every clue). Extend to other puzzle types (e.g., Sudoku, narrative logic) and incorporate human heuristics (grid elimination techniques) into prompts or training data. Explore hybrid models combining search algorithms or memory modules with LLMs to systematically navigate the combinatorial solution space.

PlanGEN: A Multi-Agent Framework for Reasoning and Planning

- **Summary and Impact.** PlanGEN improves LLM inference as a collaborative process among three specialized agents—constraint, verification, and selection—that iteratively propose, evaluate, and refine candidate reasoning chains using strategies such as Best-of-N, Tree-of-Thought, and REBASE. This model-agnostic framework consistently delivers SOTA results by catching errors at runtime and making its decision process explainable.
- **Future Directions.** Future work should broaden the agents (for example adding retrieval or solver agents), learn to coordinate them via meta-optimization or RL, enhance the verifier with symbolic checks, adapt PlanGEN to ongoing interactive tasks where planning has multiple steps.

Plan-Tuning: Learning to Plan via Post-Training

- **Summary and Impact.** Plan-Tuning enhances LLM reasoning by fine-tuning models on synthetic “planning trajectories” – step-by-step solution outlines gen-

erated and verified by a strong teacher LLM. Through supervised learning and reinforcement signals on tasks such as GSM8K and MATH, smaller models learn to internalize multi-step planning, outperforming both naive and instruction-tuned baselines and even generalizing to out-of-distribution problems. This approach improves accuracy on complex benchmarks.

- **Future Directions.** Future work should broaden Plan-Tuning across domains such as formal logic and code synthesis, automating trajectory creation and quality filtering for arbitrary tasks. Integrating active learning or human-in-the-loop validation can ensure diversity and correctness of plans, while hybrid few-shot or in-context methods may combine the strengths of plan-tuned models with on-the-fly adaptation.

TarGEN: Targeted Data Generation with Self-Correction

- **Summary and Impact.** TarGEN employs a multi-step prompting pipeline in which an LLM first generates “instance seeds” to inject semantic diversity, then expands them into full examples and finally self-critiques and refines its own outputs. When applied to recreate SuperGLUE tasks from scratch, TarGEN produced synthetic datasets with 25% greater lexical diversity and matched or even surpassed models trained on the original human-annotated data. This demonstrates that LLMs can bootstrap high-quality, balanced training data in low-resource or biased settings.
- **Future Directions.** Future work should adapt TarGEN across languages and modalities, introduce lightweight human review to catch subtle errors, and develop adaptive loops that generate examples targeted at a model’s specific failure modes. Strengthening the self-correction step with external verifiers or symbolic

checkers and deploying TarGEN in an online curriculum-learning setup will further enhance data quality and model robustness over time.

In-BoXBART: Instruction-Tuning for Biomedical Tasks

- **Summary and Impact.** In-BoXBART is the first instruction-tuned model tailored to the biomedical domain, unifying various tasks—NER, QA, summarization, etc. Trained on a meta-dataset (BoX) that re-purpose each annotation as an instruction-response pair, In-BoXBART outperforms both single-task and conventional multi-task baselines by $\sim 18\%$ on average and achieves a $\sim 23\%$ improvement in few-shot with only 32 examples per task.
- **Future Directions.** Future work should integrate explicit biomedical knowledge—linking to ontologies or retrieval systems—to ground responses in up-to-date evidence, scale instruction tuning to larger domain LLMs capable of interactive, multi-turn reasoning, and evaluate factual accuracy and bias to ensure clinical safety. Exploring adapters or task-specific fine-tuning on top of In-BoXBART may close remaining performance gaps on complex document-level tasks.

Clinical Systematic Review Automation

- **Summary and Impact.** This work emulates the dual-reviewer protocol for systematic literature screening by developing two independent LLM agents (“Reviewer A” and “Reviewer B”) to apply inclusion criteria to abstracts, compare their decisions, and resolve disagreements via automated discussion or a third-party arbiter. In a cancer-focused case study, this two-agent system matched human-reviewer accuracy while significantly cutting the number of abstracts requiring manual review and outperforming single-model approaches—all without task-specific fine-tuning.

- **Future Directions.** Future work should validate this framework across diverse disciplines and review types, develop richer conflict-resolution protocols (e.g., structured debates or specialized moderator agents), and integrate uncertainty estimates to flag cases for human oversight. Incorporating dynamic retrieval of newly published studies will keep the agents’ knowledge current, while more structured justifications can enhance explainability.

Prompt Reformulation: Improving LLM Agent Reliability

- **Summary and Impact.** We devised an automated feedback loop that detects errors in an LLM agent’s output—using a secondary LLM or simple heuristics—and systematically rewrites the prompt (adding clarifications, examples, or constraints) until the response meets correctness criteria. In a clinical decision-support benchmark, this approach raised task success from $\sim 71\%$ to $\sim 87\%$ without any model retraining, revealing that many failures stem from suboptimal phrasing rather than model incapacity.
- **Future Directions.** Extending this technique beyond healthcare—into mathematical problem solving, open-domain QA, and tool-augmented tasks—will test its generality. Developing a comprehensive error taxonomy will guide more targeted prompt reformulations, while exploring learned reformulation policies (via RL or Bayesian optimization) could automate prompt engineering end to end. Embedding a dedicated “prompt engineer” agent within multi-agent frameworks and establishing fallbacks when reformulation stalls will improve reliability.

REFERENCES

- [1] Adams, C. E., S. Polzmacher and A. Wolff, “Systematic reviews: work that needs to be done and not to be done”, *Journal of Evidence-Based Medicine* **6**, 4, 232–235 (2013).
- [2] Aghajanyan, A., A. Gupta, A. Shrivastava, X. Chen, L. Zettlemoyer and S. Gupta, “Muppet: Massive multi-task representations with pre-finetuning”, in “Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing”, pp. 5799–5811 (2021).
- [3] Aliannejadi, M., J. Kiseleva, A. Chuklin, J. Dalton and M. Burtsev, “Convai3: Generating clarifying questions for open-domain dialogue systems (clariq)”, arXiv preprint arXiv:2009.11352 (2020).
- [4] Alrowili, S. and V. Shanker, “BioM-transformers: Building large biomedical language models with BERT, ALBERT and ELECTRA”, in “Proceedings of the 20th Workshop on Biomedical Language Processing”, pp. 221–227 (Association for Computational Linguistics, Online, 2021), URL <https://aclanthology.org/2021.bionlp-1.24>.
- [5] Anthropic, A., “The claude 3 model family: Opus, sonnet, haiku”, Claude-3 Model Card (2024).
- [6] Arunkumar, A., S. Mishra, B. Sachdeva, C. Baral and C. Bryan, “Real-time visual feedback for educative benchmark creation: A human-and-metric-in-the-loop workflow”, *NeurIPS 2020 Workshop HAMLETS* (2020).
- [7] Bach, S. H., V. Sanh, Z.-X. Yong, A. Webson, C. Raffel, N. V. Nayak, A. Sharma, T. Kim, M. S. Bari, T. Fevry *et al.*, “Promptsources: An integrated development environment and repository for natural language prompts”, arXiv preprint arXiv:2202.01279 (2022).
- [8] Baker, S., I. Silins, Y. Guo, I. Ali, J. Högberg, U. Stenius and A. Korhonen, “Automatic semantic classification of scientific literature according to the hallmarks of cancer”, *Bioinformatics* **32**, 3, 432–440 (2016).
- [9] Banerjee, P., K. K. Pal, M. Devarakonda and C. Baral, “Biomedical named entity recognition via knowledge guidance and question answering”, *ACM Transactions on Computing for Healthcare* **2**, 4, 1–24 (2021).
- [10] Bannach-Brown, A., P. Przybyła, J. Thomas, A. S. Rice, S. Ananiadou, J. Liao and M. R. Macleod, “Machine learning algorithms for systematic review: reducing workload in a preclinical review of animal studies and reducing human screening error”, *Systematic reviews* **8**, 1, 1–12 (2019).
- [11] Beltagy, I., M. E. Peters and A. Cohan, “Longformer: The long-document transformer”, arXiv preprint arXiv:2004.05150 (2020).

- [12] Bernardy, J.-P. and S. Chatzikyriakidis, “Fracas: Temporal analysis”, (2020).
- [13] Beygi, S., M. Fazel-Zarandi, A. Cervone, P. Krishnan and S. Jonnalagadda, “Logical reasoning for task oriented dialogue systems”, in “Proceedings of the Fifth Workshop on e-Commerce and NLP (ECNLP 5)”, pp. 68–79 (Association for Computational Linguistics, Dublin, Ireland, 2022), URL <https://aclanthology.org/2022.ecnlp-1.10>.
- [14] Biderman, S., H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff *et al.*, “Pythia: A suite for analyzing large language models across training and scaling”, in “International Conference on Machine Learning”, pp. 2397–2430 (PMLR, 2023).
- [15] Bisk, Y., R. Zellers, J. Gao, Y. Choi *et al.*, “Piqa: Reasoning about physical commonsense in natural language”, in “Proceedings of the AAAI conference on artificial intelligence”, vol. 34, pp. 7432–7439 (2020).
- [16] Bohnet, B., A. Nova, A. T. Parisi, K. Swersky, K. Goshvadi, H. Dai, D. Schuurmans, N. Fiedel and H. Sedghi, “Exploring and benchmarking the planning capabilities of large language models”, arXiv preprint arXiv:2406.13094 (2024).
- [17] Borisov, V., K. Seßler, T. Leemann, M. Pawelczyk and G. Kasneci, “Language models are realistic tabular data generators”, (2023).
- [18] Brown, B., J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré and A. Mirhoseini, “Large language monkeys: Scaling inference compute with repeated sampling”, arXiv preprint arXiv:2407.21787 (2024).
- [19] Brown, T., B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever and D. Amodei, “Language models are few-shot learners”, in “Advances in Neural Information Processing Systems”, edited by H. Larochelle, M. Ranzato, R. Hassel, M. F. Balcan and H. Lin, vol. 33, pp. 1877–1901 (Curran Associates, Inc., 2020), URL <https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf>.
- [20] Brown, T. B., B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners”, arXiv preprint arXiv:2005.14165 (2020).
- [21] Bylander, T., “The computational complexity of propositional strips planning”, *Artificial Intelligence* **69**, 1-2, 165–204 (1994).
- [22] Callison-Burch, C. and M. Dredze, “Creating speech and language data with Amazon’s Mechanical Turk”, in “Proceedings of the NAACL HLT 2010 Workshop on Creating Speech and Language Data with Amazon’s Mechanical Turk”, pp. 1–12 (Association for Computational Linguistics, Los Angeles, 2010), URL <https://aclanthology.org/W10-0701>.

- [23] Chai, K. E., R. L. Lines, D. F. Gucciardi and L. Ng, “Research screener: a machine learning tool to semi-automate abstract screening for systematic reviews”, *Systematic reviews* **10**, 1–13 (2021).
- [24] Chang, J. C., S. Amershi and E. Kamar, “Revolt: Collaborative crowdsourcing for labeling machine learning datasets”, in “Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems”, pp. 2334–2346 (2017).
- [25] Chen, D., C. Lee, Y. Lu, D. Rosati and Z. Yu, “Mixture of soft prompts for controllable data generation”, *CoRR* **abs/2303.01580**, URL <https://doi.org/10.48550/arXiv.2303.01580> (2023).
- [26] Chen, M., J. Tworek, H. Jun, Q. Yuan, H. Ponde, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. W. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, I. Babuschkin, S. A. Balaji, S. Jain, A. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever and W. Zaremba, “Evaluating large language models trained on code”, *ArXiv* **abs/2107.03374**, URL <https://api.semanticscholar.org/CorpusID:235755472> (2021).
- [27] Chen, W., S. Koenig and B. Dilkina, “Reprompt: Planning by automatic prompt engineering for large language models agents”, *arXiv preprint arXiv:2406.11132* (2024).
- [28] Chen, W., X. Ma, X. Wang and W. W. Cohen, “Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks”, *arXiv preprint arXiv:2211.12588* (2022).
- [29] Chen, W., X. Ma, X. Wang and W. W. Cohen, “Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks”, URL <https://arxiv.org/abs/2211.12588> (2023).
- [30] Chen, W., H. Zha, Z. Chen, W. Xiong, H. Wang and W. Y. Wang, “HybridQA: A dataset of multi-hop question answering over tabular and textual data”, in “Findings of the Association for Computational Linguistics: EMNLP 2020”, pp. 1026–1036 (Association for Computational Linguistics, Online, 2020), URL <https://aclanthology.org/2020.findings-emnlp.91>.
- [31] Chen, X., X. Xie, N. Zhang, J. Yan, S. Deng, C. Tan, F. Huang, L. Si and H. Chen, “Adaprompt: Adaptive prompt-based finetuning for relation extraction”, *arXiv e-prints pp. arXiv-2104* (2021).
- [32] Chen, Z., K. Liu, Q. Wang, W. Zhang, J. Liu, D. Lin, K. Chen and F. Zhao, “Agent-FLAN: Designing data and methods of effective agent tuning for large

- language models”, in “Findings of the Association for Computational Linguistics: ACL 2024”, edited by L.-W. Ku, A. Martins and V. Srikumar, pp. 9354–9366 (Association for Computational Linguistics, Bangkok, Thailand, 2024), URL <https://aclanthology.org/2024.findings-acl.557/>.
- [33] Chia, Y. K., L. Bing, S. Poria and L. Si, “RelationPrompt: Leveraging prompts to generate synthetic data for zero-shot relation triplet extraction”, in “Findings of the Association for Computational Linguistics: ACL 2022”, edited by S. Muresan, P. Nakov and A. Villavicencio, pp. 45–57 (Association for Computational Linguistics, Dublin, Ireland, 2022), URL <https://aclanthology.org/2022.findings-acl.5/>.
- [34] Chiang, C.-H. and H.-y. Lee, “Can large language models be an alternative to human evaluations?”, in “Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)”, edited by A. Rogers, J. Boyd-Graber and N. Okazaki, pp. 15607–15631 (Association for Computational Linguistics, Toronto, Canada, 2023), URL <https://aclanthology.org/2023.acl-long.870>.
- [35] Chung, H. W., L. Hou, S. Longpre, B. Zoph, Y. Tay, W. Fedus, E. Li, X. Wang, M. Dehghani, S. Brahma *et al.*, “Scaling instruction-finetuned language models”, arXiv preprint arXiv:2210.11416 (2022).
- [36] Clark, C., K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins and K. Toutanova, “Boolq: Exploring the surprising difficulty of natural yes/no questions”, ArXiv [abs/1905.10044](https://arxiv.org/abs/1905.10044), URL <https://api.semanticscholar.org/CorpusID:165163607> (2019).
- [37] Clark, J. H., E. Choi, M. Collins, D. Garrette, T. Kwiatkowski, V. Nikolaev and J. Palomaki, “Tydi qa: A benchmark for information-seeking question answering in typologically diverse languages”, Transactions of the Association for Computational Linguistics **8**, 454–470 (2020).
- [38] Clark, P., I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick and O. Tafjord, “Think you have solved question answering? try arc, the ai2 reasoning challenge”, (2018).
- [39] Clark, P., O. Tafjord and K. Richardson, “Transformers as soft reasoners over language”, in “Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence”, pp. 3882–3890 (2021).
- [40] Cobbe, K., V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano *et al.*, “Training verifiers to solve math word problems, 2021”, URL <https://arxiv.org/abs/2110.14168> **9** (2021).
- [41] Crichton, G., S. Pyysalo, B. Chiu and A. Korhonen, “A neural network multi-task learning approach to biomedical named entity recognition”, BMC bioinformatics **18**, 1, 1–14 (2017).

- [42] Cui, L., Y. Wu, J. Liu, S. Yang and Y. Zhang, “Template-based named entity recognition using bart”, arXiv preprint arXiv:2106.01760 (2021).
- [43] Dalvi, B., P. Jansen, O. Tafjord, Z. Xie, H. Smith, L. Pipatanangkura and P. Clark, “Explaining answers with entailment trees”, in “Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing”, edited by M.-F. Moens, X. Huang, L. Specia and S. W.-t. Yih, pp. 7358–7370 (Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2021), URL <https://aclanthology.org/2021.emnlp-main.585>.
- [44] Dasigi, P., N. F. Liu, A. Marasović, N. A. Smith and M. Gardner, “Quoref: A reading comprehension dataset with questions requiring coreferential reasoning”, in “Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)”, pp. 5925–5932 (Association for Computational Linguistics, Hong Kong, China, 2019), URL <https://aclanthology.org/D19-1606>.
- [45] De Marneffe, M.-C., M. Simons and J. Tonhauser, “The commitmentbank: Investigating projection in naturally occurring discourse”, in “proceedings of Sinn und Bedeutung”, vol. 23, pp. 107–124 (2019).
- [46] Deng, M., B. Tan, Z. Liu, E. Xing and Z. Hu, “Compression, transduction, and creation: A unified framework for evaluating natural language generation”, in “Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing”, edited by M.-F. Moens, X. Huang, L. Specia and S. W.-t. Yih, pp. 7580–7605 (Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2021), URL <https://aclanthology.org/2021.emnlp-main.599>.
- [47] Devlin, J., M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding”, in “Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)”, edited by J. Burstein, C. Doran and T. Solorio, pp. 4171–4186 (Association for Computational Linguistics, Minneapolis, Minnesota, 2019), URL <https://aclanthology.org/N19-1423/>.
- [48] Dey, N., G. Gosal, H. Khachane, W. Marshall, R. Pathria, M. Tom, J. Hestness *et al.*, “Cerebras-gpt: Open compute-optimal language models trained on the cerebras wafer-scale cluster”, arXiv preprint arXiv:2304.03208 (2023).
- [49] Dua, D., Y. Wang, P. Dasigi, G. Stanovsky, S. Singh and M. Gardner, “DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs”, in “Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)”, pp. 2368–2378 (Association for Computational Linguistics, Minneapolis, Minnesota, 2019), URL <https://aclanthology.org/N19-1246>.

- [50] Dziri, N., X. Lu, M. Sclar, X. L. Li, L. Jiang, B. Y. Lin, P. West, C. Bhagavatula, R. L. Bras, J. D. Hwang, S. Sanyal, S. Welleck, X. Ren, A. Ettinger, Z. Harchaoui and Y. Choi, “Faith and fate: Limits of transformers on compositionality”, URL <https://arxiv.org/abs/2305.18654> (2023).
- [51] Efrat, A. and O. Levy, “The turking test: Can language models understand instructions?”, arXiv preprint arXiv:2010.11982 (2020).
- [52] Eickhoff, C., “Cognitive biases in crowdsourcing”, in “Proceedings of the eleventh ACM international conference on web search and data mining”, pp. 162–170 (2018).
- [53] Ethayarajh, K., Y. Choi and S. Swayamdipta, “Understanding dataset difficulty with V -usable information”, in “International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA”, edited by K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvári, G. Niu and S. Sabato, vol. 162 of *Proceedings of Machine Learning Research*, pp. 5988–6008 (PMLR, 2022), URL <https://proceedings.mlr.press/v162/ethayarajh22a.html>.
- [54] Ethayarajh, K., W. Xu, N. Muennighoff, D. Jurafsky and D. Kiela, “Kto: Model alignment as prospect theoretic optimization”, arXiv preprint arXiv:2402.01306 (2024).
- [55] Feng, X., Y. Luo, Z. Wang, H. Tang, M. Yang, K. Shao, D. Mguni, Y. Du and J. Wang, “Chessgpt: Bridging policy learning and language modeling”, *Advances in Neural Information Processing Systems* **36** (2024).
- [56] Gao, J., R. Pi, Y. Lin, H. Xu, J. Ye, Z. Wu, W. Zhang, X. Liang, Z. Li and L. Kong, “Self-guided noise-free data generation for efficient zero-shot learning”, in “The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023”, (OpenReview.net, 2023), URL https://openreview.net/pdf?id=h50pjGd_lo6.
- [57] Gates, A., M. Gates, D. DaRosa, S. A. Elliott, J. Pillay, S. Rahman, B. Vandermeer and L. Hartling, “Decoding semi-automated title-abstract screening: findings from a convenience sample of reviews”, *Systematic reviews* **9**, 1, 1–12 (2020).
- [58] Gates, A., S. Guitard, J. Pillay, S. A. Elliott, M. P. Dyson, A. S. Newton and L. Hartling, “Performance and usability of machine learning for screening in systematic reviews: a comparative evaluation of three tools”, *Systematic reviews* **8**, 1, 1–11 (2019).
- [59] Gates, A., C. Johnson and L. Hartling, “Technology-assisted title and abstract screening for systematic reviews: a retrospective evaluation of the abstrackr machine learning tool”, *Systematic reviews* **7**, 1, 45 (2018).

- [60] Geva, M., Y. Goldberg and J. Berant, “Are we modeling the task or the annotator? an investigation of annotator bias in natural language understanding datasets”, in “Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)”, pp. 1161–1166 (Association for Computational Linguistics, Hong Kong, China, 2019), URL <https://aclanthology.org/D19-1107>.
- [61] Geva, M., D. Khashabi, E. Segal, T. Khot, D. Roth and J. Berant, “Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies”, *Transactions of the Association for Computational Linguistics* **9**, 346–361 (2021).
- [62] Giadikiaroglou, P., M. Lymperaious, G. Filandrianos and G. Stamou, “Puzzle solving using reasoning of large language models: A survey”, arXiv preprint arXiv:2402.11291 (2024).
- [63] Glujovsky, D., A. Bardach, S. G. Martí, D. Comandé and A. Ciapponi, “Prm2 eros: a new software for early stage of systematic reviews”, *Value in Health* **14**, 7, A564 (2011).
- [64] Golovneva, O., M. P. Chen, S. Poff, M. Corredor, L. Zettlemoyer, M. Fazel-Zarandi and A. Celikyilmaz, “Roscoe: A suite of metrics for scoring step-by-step reasoning”, in “The Eleventh International Conference on Learning Representations”, (2022).
- [65] Gu, Z., Z. Li, L. Zhang, Z. Xiong, S. Jiang, X. Zhu, S. Wang, Z. Wang, J. Wang, H. Ye *et al.*, “Beyond the obvious: Evaluating the reasoning ability in real-life scenarios of language models on life scapes reasoning benchmark~(lsr-benchmark)”, arXiv preprint arXiv:2307.05113 (2023).
- [66] Gupta, A., “Are chatgpt and gpt-4 good poker players?—a pre-flop analysis”, arXiv preprint arXiv:2308.12466 (2023).
- [67] Gupta, H., K. Scaria, U. Anantheswaran, S. Verma, M. Parmar, S. A. Sawant, C. Baral and S. Mishra, “Targen: Targeted data generation with large language models”, arXiv preprint arXiv:2310.17876 (2023).
- [68] Gururangan, S., S. Swayamdipta, O. Levy, R. Schwartz, S. Bowman and N. A. Smith, “Annotation artifacts in natural language inference data”, in “Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)”, pp. 107–112 (Association for Computational Linguistics, New Orleans, Louisiana, 2018), URL <https://aclanthology.org/N18-2017>.
- [69] Hahn, C., F. Schmitt, J. U. Kreber, M. N. Rabe and B. Finkbeiner, “Teaching temporal logics to neural networks”, in “International Conference on Learning Representations”, (2021), URL <https://openreview.net/forum?id=d0cQK-f4byz>.

- [70] Han, Q., K. Khamaru and C.-H. Zhang, “Ucb algorithms for multi-armed bandits: Precise regret and adaptive inference”, arXiv preprint arXiv:2412.06126 (2024).
- [71] Han, S., H. Schoelkopf, Y. Zhao, Z. Qi, M. Riddell, L. Benson, L. Sun, E. Zubova, Y. Qiao, M. Burtell *et al.*, “FOLIO: Natural language reasoning with first-order logic”, arXiv preprint arXiv:2209.00840 (2022).
- [72] Hao, S., Y. Gu, H. Luo, T. Liu, X. Shao, X. Wang, S. Xie, H. Ma, A. Samavedhi, Q. Gao *et al.*, “Llm reasoners: New evaluation, library, and analysis of step-by-step reasoning with large language models”, in “ICLR 2024 Workshop on Large Language Model (LLM) Agents”, (2024).
- [73] Hao, S., Y. Gu, H. Ma, J. Hong, Z. Wang, D. Wang and Z. Hu, “Reasoning with language model is planning with world model”, in “Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing”, edited by H. Bouamor, J. Pino and K. Bali, pp. 8154–8173 (Association for Computational Linguistics, Singapore, 2023), URL <https://aclanthology.org/2023.emnlp-main.507/>.
- [74] Hase, P. and M. Bansal, “When can models learn from explanations? a formal framework for understanding the roles of explanation data”, arXiv preprint arXiv:2102.02201 (2021).
- [75] He, C., R. Luo, Y. Bai, S. Hu, Z. Thai, J. Shen, J. Hu, X. Han, Y. Huang, Y. Zhang, J. Liu, L. Qi, Z. Liu and M. Sun, “OlympiadBench: A challenging benchmark for promoting AGI with olympiad-level bilingual multi-modal scientific problems”, in “Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)”, edited by L.-W. Ku, A. Martins and V. Srikumar, pp. 3828–3850 (Association for Computational Linguistics, Bangkok, Thailand, 2024), URL <https://aclanthology.org/2024.acl-long.211/>.
- [76] He, C., R. Luo, Y. Bai, S. Hu, Z. Thai, J. Shen, J. Hu, X. Han, Y. Huang, Y. Zhang *et al.*, “Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems”, in “Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)”, pp. 3828–3850 (2024).
- [77] He, Q., J. Zeng, W. Huang, L. Chen, J. Xiao, Q. He, X. Zhou, L. Chen, X. Wang, Y. Huang, H. Ye, Z. Li, S. Chen, Y. Zhang, Z. Gu, J. Liang and Y. Xiao, “Can large language models understand real-world complex instructions?”, ArXiv [abs/2309.09150](https://arxiv.org/abs/2309.09150), URL <https://api.semanticscholar.org/CorpusID:262043773> (2023).
- [78] Hendrycks, D., C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song and J. Steinhardt, “Measuring mathematical problem solving with the MATH dataset”, in “Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)”, (2021), URL <https://openreview.net/forum?id=7Bywt2mQsCe>.

- [79] Herrero-Zazo, M., I. Segura-Bedmar, P. Martínez and T. Declerck, “The ddi corpus: An annotated corpus with pharmacological substances and drug–drug interactions”, *Journal of biomedical informatics* **46**, 5, 914–920 (2013).
- [80] Hettiachchi, D., M. Sanderson, J. Goncalves, S. Hosio, G. Kazai, M. Lease, M. Schaekermann and E. Yilmaz, “Investigating and mitigating biases in crowd-sourced data”, in “Companion Publication of the 2021 Conference on Computer Supported Cooperative Work and Social Computing”, pp. 331–334 (2021).
- [81] Howard, B. E., J. Phillips, A. Tandon, A. Maharana, R. Elmore, D. Mav, A. Sedykh, K. Thayer, B. A. Merrick, V. Walker *et al.*, “Swift-active screener: accelerated document screening through active learning and integrated recall estimation”, *Environment International* **138**, 105623 (2020).
- [82] Hu, X., H. Wang, A. Vegesana, S. Dube, K. Yu, G. Kao, S.-H. Chen, Y.-H. Lu, G. K. Thiruvathukal and M. Yin, “Crowdsourcing detection of sampling biases in image datasets”, in “Proceedings of The Web Conference 2020”, pp. 2955–2961 (2020).
- [83] Huang, J., X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song and D. Zhou, “Large language models cannot self-correct reasoning yet”, in “The Twelfth International Conference on Learning Representations”, (2024), URL <https://openreview.net/forum?id=Ikmd3fKBPQ>.
- [84] Huang, K., A. Singh, S. Chen, E. Moseley, C.-Y. Deng, N. George and C. Lindvall, “Clinical XLNet: Modeling sequential clinical notes and predicting prolonged mechanical ventilation”, in “Proceedings of the 3rd Clinical Natural Language Processing Workshop”, pp. 94–100 (Association for Computational Linguistics, Online, 2020), URL <https://aclanthology.org/2020.clinicalnlp-1.11>.
- [85] Huang, L., R. Le Bras, C. Bhagavatula and Y. Choi, “Cosmos QA: Machine reading comprehension with contextual commonsense reasoning”, in “Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)”, pp. 2391–2401 (Association for Computational Linguistics, Hong Kong, China, 2019), URL <https://aclanthology.org/D19-1243>.
- [86] Huang, S., S. Ma, Y. Li, M. Huang, W. Zou, W. Zhang and H.-T. Zheng, “Lateval: An interactive llms evaluation benchmark with incomplete information from lateral thinking puzzles”, URL <https://arxiv.org/abs/2308.10855> (2024).
- [87] Hurst, A., A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, “Gpt-4o system card”, arXiv preprint arXiv:2410.21276 (2024).
- [88] Ishay, A., Z. Yang and J. Lee, “Leveraging large language models to generate answer set programs”, in “Proceedings of the 20th International Conference on Principles of Knowledge Representation and Reasoning”, pp. 374–383 (2023).

- [89] Islamaj Doğan, R., S. Kim, A. Chatr-Aryamontri, C.-H. Wei, D. C. Comeau, R. Antunes, S. Matos, Q. Chen, A. Elangovan, N. C. Panyam *et al.*, “Overview of the biocreative vi precision medicine track: mining protein interactions and mutations for precision medicine”, Database **2019** (2019).
- [90] Jabrayilzade, E. and S. Tekir, “LGPSolver - solving logic grid puzzles automatically”, in “Findings of the Association for Computational Linguistics: EMNLP 2020”, edited by T. Cohn, Y. He and Y. Liu, pp. 1118–1123 (Association for Computational Linguistics, Online, 2020), URL <https://aclanthology.org/2020.findings-emnlp.100>.
- [91] Jaspers, S., E. De Troyer and M. Aerts, “Machine learning techniques for the automation of literature reviews and systematic reviews in efsa”, EFSA Supporting Publications **15**, 6, 1427E (2018).
- [92] Jiang, A. Q., A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, “Mistral 7b”, arXiv preprint arXiv:2310.06825 (2023).
- [93] Jiang, Y., K. C. Black, G. Geng, D. Park, A. Y. Ng and J. H. Chen, “Medagentbench: Dataset for benchmarking llms as agents in medical applications”, arXiv preprint arXiv:2501.14654 (2025).
- [94] Jiang, Z., F. F. Xu, J. Araki and G. Neubig, “How can we know what language models know?”, Transactions of the Association for Computational Linguistics **8**, 423–438 (2020).
- [95] Jiao, F., C. Qin, Z. Liu, N. F. Chen and S. Joty, “Learning planning-based reasoning by trajectories collection and process reward synthesizing”, in “Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing”, edited by Y. Al-Onaizan, M. Bansal and Y.-N. Chen, pp. 334–350 (Association for Computational Linguistics, Miami, Florida, USA, 2024), URL <https://aclanthology.org/2024.emnlp-main.20/>.
- [96] Jin, Q., B. Dhingra, Z. Liu, W. Cohen and X. Lu, “PubMedQA: A dataset for biomedical research question answering”, in “Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)”, pp. 2567–2577 (Association for Computational Linguistics, Hong Kong, China, 2019), URL <https://aclanthology.org/D19-1259>.
- [97] Joshi, P., S. Aditya, A. Sathe and M. Choudhury, “TaxiNLI: Taking a ride up the NLU hill”, in “Proceedings of the 24th Conference on Computational Natural Language Learning”, pp. 41–55 (Association for Computational Linguistics, Online, 2020), URL <https://aclanthology.org/2020.conll-1.4>.
- [98] Kambhampati, S., K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. P. Saldyt and A. B. Murthy, “Position: LLMs can’t plan, but can

- help planning in LLM-modulo frameworks”, in “Forty-first International Conference on Machine Learning”, (2024), URL <https://openreview.net/forum?id=Th8JPEmH4z>.
- [99] Karimi Mahabadi, R., Y. Belinkov and J. Henderson, “End-to-end bias mitigation by modelling biases in corpora”, in “Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics”, pp. 8706–8716 (Association for Computational Linguistics, Online, 2020), URL <https://aclanthology.org/2020.acl-main.769>.
 - [100] Kazemi, M., Q. Yuan, D. Bhatia, N. Kim, X. Xu, V. Imbrasaitė and D. Ramachandran, “Boardgameqa: A dataset for natural language reasoning with contradictory information”, arXiv preprint arXiv:2306.07934 (2023).
 - [101] Kazemi, M., Q. Yuan, D. Bhatia, N. Kim, X. Xu, V. Imbrasaitė and D. Ramachandran, “Boardgameqa: A dataset for natural language reasoning with contradictory information”, *Advances in Neural Information Processing Systems* **36** (2024).
 - [102] Ke, Z., F. Jiao, Y. Ming, X.-P. Nguyen, A. Xu, D. X. Long, M. Li, C. Qin, P. Wang, Silvio Savarese, C. Xiong and S. Joty, “A survey of frontiers in LLM reasoning: Inference scaling, learning to reason, and agentic systems”, *Transactions on Machine Learning Research* URL <https://openreview.net/forum?id=SlsZZ25InC>, survey Certification (2025).
 - [103] Khashabi, D., *Reasoning-Driven Question-Answering for Natural Language Understanding* (University of Pennsylvania, 2019).
 - [104] Khashabi, D., S. Chaturvedi, M. Roth, S. Upadhyay and D. Roth, “Looking beyond the surface: A challenge set for reading comprehension over multiple sentences”, in “Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)”, pp. 252–262 (2018).
 - [105] Khashabi, D., S. Min, T. Khot, A. Sabharwal, O. Tafjord, P. Clark and H. Hajishirzi, “UNIFIEDQA: Crossing format boundaries with a single QA system”, in “Findings of the Association for Computational Linguistics: EMNLP 2020”, pp. 1896–1907 (Association for Computational Linguistics, Online, 2020), URL <https://aclanthology.org/2020.findings-emnlp.171>.
 - [106] Khot, T., P. Clark, M. Guerquin, P. Jansen and A. Sabharwal, “Qasc: A dataset for question answering via sentence composition”, in “Proceedings of the AAAI Conference on Artificial Intelligence”, vol. 34, pp. 8082–8090 (2020).
 - [107] Kojima, T., S. S. Gu, M. Reid, Y. Matsuo and Y. Iwasawa, “Large language models are zero-shot reasoners”, *Advances in neural information processing systems* **35**, 22199–22213 (2022).

- [108] Kojima, T., S. S. Gu, M. Reid, Y. Matsuo and Y. Iwasawa, “Large language models are zero-shot reasoners”, in “Proceedings of the 36th International Conference on Neural Information Processing Systems”, NIPS ’22 (Curran Associates Inc., Red Hook, NY, USA, 2024).
- [109] Kraljevic, Z., A. Shek, D. Bean, R. Bendayan, J. Teo and R. Dobson, “MedGPT: Medical Concept Prediction from Clinical Narratives”, arXiv preprint arXiv:2107.03134 (2021).
- [110] Kumar, V., A. Stubbs, S. Shaw and Ö. Uzuner, “Creation of a new longitudinal corpus of clinical narratives”, *Journal of biomedical informatics* **58**, S6–S10 (2015).
- [111] Kuznia, K., S. Mishra, M. Parmar and C. Baral, “Less is more: Summary of long instructions is better for program synthesis”, arXiv preprint arXiv:2203.08597 (2022).
- [112] Lasecki, W. S., M. Gordon, D. Koutra, M. F. Jung, S. P. Dow and J. P. Bigham, “Glance: Rapidly coding behavioral video with the crowd”, in “Proceedings of the 27th annual ACM symposium on User interface software and technology”, pp. 551–562 (2014).
- [113] Le Bras, R., S. Swayamdipta, C. Bhagavatula, R. Zellers, M. E. Peters, A. Sabharwal and Y. Choi, “Adversarial filters of dataset biases”, in “ICML”, (2020).
- [114] Le Scao, T. and A. M. Rush, “How many data points is a prompt worth?”, in “Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies”, pp. 2627–2636 (2021).
- [115] Lee, J., W. Yoon, S. Kim, D. Kim, S. Kim, C. H. So and J. Kang, “Biobert: a pre-trained biomedical language representation model for biomedical text mining”, *Bioinformatics* **36**, 4, 1234–1240 (2020).
- [116] Lee, K.-H., I. Fischer, Y.-H. Wu, D. Marwood, S. Baluja, D. Schuurmans and X. Chen, “Evolving deeper llm thinking”, arXiv preprint arXiv:2501.09891 (2025).
- [117] Levesque, H., E. Davis and L. Morgenstern, “The winograd schema challenge”, in “Thirteenth international conference on the principles of knowledge representation and reasoning”, (2012).
- [118] Lewis, M., Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov and L. Zettlemoyer, “BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension”, in “Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics”, edited by D. Jurafsky, J. Chai, N. Schluter and J. Tetreault, pp. 7871–7880 (Association for Computational Linguistics, Online, 2020), URL <https://aclanthology.org/2020.acl-main.703/>.

- [119] Lewis, M., Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov and L. Zettlemoyer, “BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension”, in “Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics”, pp. 7871–7880 (Association for Computational Linguistics, Online, 2020), URL <https://aclanthology.org/2020.acl-main.703>.
- [120] Lifschitz, V., “Benchmark problems for formal nonmonotonic reasoning: Version 2.00”, in “Non-Monotonic Reasoning: 2nd International Workshop Grassau, FRG, June 13–15, 1988 Proceedings 2”, pp. 202–219 (Springer, 1989).
- [121] Lin, B. Y., Z. Wu, Y. Yang, D.-H. Lee and X. Ren, “RiddleSense: Reasoning about riddle questions featuring linguistic creativity and commonsense knowledge”, in “Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021”, edited by C. Zong, F. Xia, W. Li and R. Navigli, pp. 1504–1515 (Association for Computational Linguistics, Online, 2021), URL <https://aclanthology.org/2021.findings-acl.131>.
- [122] Lin, C.-Y., “Rouge: A package for automatic evaluation of summaries”, in “Text summarization branches out”, pp. 74–81 (2004).
- [123] Lin, K., O. Tafjord, P. Clark and M. Gardner, “Reasoning over paragraph effects in situations”, in “Proceedings of the 2nd Workshop on Machine Reading for Question Answering”, pp. 58–62 (Association for Computational Linguistics, Hong Kong, China, 2019), URL <https://aclanthology.org/D19-5808>.
- [124] Lin, S., J. Hilton and O. Evans, “Truthfulqa: Measuring how models mimic human falsehoods”, (2022).
- [125] Lin, X. V., T. Mihaylov, M. Artetxe, T. Wang, S. Chen, D. Simig, M. Ott, N. Goyal, S. Bhosale, J. Du *et al.*, “Few-shot learning with multilingual language models”, arXiv preprint arXiv:2112.10668 (2021).
- [126] Liu, A., B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, “Deepseek-v3 technical report”, arXiv preprint arXiv:2412.19437 (2024).
- [127] Liu, A., S. Swayamdipta, N. A. Smith and Y. Choi, “WANLI: Worker and AI collaboration for natural language inference dataset creation”, in “Findings of the Association for Computational Linguistics: EMNLP 2022”, pp. 6826–6847 (Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 2022), URL <https://aclanthology.org/2022.findings-emnlp.508>.
- [128] Liu, J., L. Cui, H. Liu, D. Huang, Y. Wang and Y. Zhang, “Logiqa: a challenge dataset for machine reading comprehension with logical reasoning”, in “Proceedings of the Twenty-Ninth International Conference on International Joint Conferences on Artificial Intelligence”, pp. 3622–3628 (2021).
- [129] Liu, P., W. Yuan, J. Fu, Z. Jiang, H. Hayashi and G. Neubig, “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing”, arXiv preprint arXiv:2107.13586 (2021).

- [130] Liu, Y., A. R. Fabbri, P. Liu, Y. Zhao, L. Nan, R. Han, S. Han, S. R. Joty, C.-S. Wu, C. Xiong and D. R. Radev, “Revisiting the gold standard: Grounding summarization evaluation with robust human evaluation”, ArXiv **abs/2212.07981**, URL <https://api.semanticscholar.org/CorpusID:254685611> (2022).
- [131] Liu, Y., D. Iter, Y. Xu, S. Wang, R. Xu and C. Zhu, “G-eval: NLG evaluation using gpt-4 with better human alignment”, in “Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing”, edited by H. Bouamor, J. Pino and K. Bali, pp. 2511–2522 (Association for Computational Linguistics, Singapore, 2023), URL <https://aclanthology.org/2023.emnlp-main.153>.
- [132] Liu, Y., M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach”, arXiv preprint arXiv:1907.11692 (2019).
- [133] Liu, Y., X. Peng, Y. Zhang, J. Cao, X. Zhang, S. Cheng, X. Wang, J. Yin and T. Du, “Tool-planner: Dynamic solution tree planning for large language model with tool clustering”, arXiv preprint arXiv:2406.03807 (2024).
- [134] Lourie, N., R. Le Bras, C. Bhagavatula and Y. Choi, “UNICORN on RAINBOW: A Universal Commonsense Reasoning Model on a New Multitask Benchmark”, in “Proceedings of the AAAI Conference on Artificial Intelligence”, vol. 35, pp. 13480–13488 (2021).
- [135] Lu, Y., M. Bartolo, A. Moore, S. Riedel and P. Stenetorp, “Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity”, arXiv preprint arXiv:2104.08786 (2021).
- [136] Lu, Y., M. Bartolo, A. Moore, S. Riedel and P. Stenetorp, “Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity”, in “Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)”, pp. 8086–8098 (Association for Computational Linguistics, Dublin, Ireland, 2022), URL <https://aclanthology.org/2022.acl-long.556>.
- [137] Luo, M., K. Hashimoto, S. Yavuz, Z. Liu, C. Baral and Y. Zhou, “Choose your qa model wisely: A systematic study of generative and extractive readers for question answering”, arXiv preprint arXiv:2203.07522 (2022).
- [138] Luo, M., S. Kumbhar, M. Parmar, N. Varshney, P. Banerjee, S. Aditya, C. Baral *et al.*, “Towards loglue: A brief survey and a benchmark for analyzing logical reasoning capabilities of language models”, arXiv preprint arXiv:2310.00836 (2023).
- [139] Luo, M., A. Mitra, T. Gokhale and C. Baral, “Improving biomedical information retrieval with neural retrievers”, arXiv preprint arXiv:2201.07745 (2022).
- [140] Mahabadi, R. K., Y. Belinkov and J. Henderson, “Variational information bottleneck for effective low-resource fine-tuning”, ICLR (2021).

- [141] Marelli, M., S. Menini, M. Baroni, L. Bentivogli, R. Bernardi and R. Zamparelli, “The SICK (Sentences Involving Compositional Knowledge) dataset for relatedness and entailment”, URL <https://doi.org/10.5281/zenodo.2787612> (2014).
- [142] Mavi, V., A. Jangra and A. Jatowt, “A survey on multi-hop question answering and generation”, arXiv preprint arXiv:2204.09140 (2022).
- [143] McCann, B., N. S. Keskar, C. Xiong and R. Socher, “The natural language decathlon: Multitask learning as question answering”, arXiv preprint arXiv:1806.08730 (2018).
- [144] McCoy, R. T., E. Pavlick and T. Linzen, “Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference”, (2019).
- [145] Meng, Y., J. Huang, Y. Zhang and J. Han, “Generating training data with language models: Towards zero-shot language understanding”, in “NeurIPS”, (2022), URL http://papers.nips.cc/paper_files/paper/2022/hash/0346c148ba1c21c6b4780a961ea141dc-Abstract-Conference.html.
- [146] Michelson, M., M. Ross and S. Minton, “Ai2 leveraging machine-assistance to replicate a systematic review”, *Value in Health* **22**, S34 (2019).
- [147] Min, S., M. Lewis, L. Zettlemoyer and H. Hajishirzi, “Metaicl: Learning to learn in context”, arXiv preprint arXiv:2110.15943 (2021).
- [148] Min, S., X. Lyu, A. Holtzman, M. Artetxe, M. Lewis, H. Hajishirzi and L. Zettlemoyer, “Rethinking the role of demonstrations: What makes in-context learning work?”, arXiv preprint arXiv:2202.12837 (2022).
- [149] Mishra, S. and A. Arunkumar, “How robust are model rankings: A leaderboard customization approach for equitable evaluation”, in “Proceedings of the AAAI conference on Artificial Intelligence”, vol. 35, pp. 13561–13569 (2021).
- [150] Mishra, S., A. Arunkumar, C. Bryan and C. Baral, “Our evaluation metric needs an update to encourage generalization”, *ArXiv abs/2007.06898* (2020).
- [151] Mishra, S., A. Arunkumar, B. Sachdeva, C. Bryan and C. Baral, “Dqi: A guide to benchmark evaluation”, arXiv preprint arXiv:2008.03964 (2020).
- [152] Mishra, S., D. Khashabi, C. Baral, Y. Choi and H. Hajishirzi, “Reframing instructional prompts to gptk’s language”, *ACL Findings* (2021).
- [153] Mishra, S., D. Khashabi, C. Baral and H. Hajishirzi, “Cross-task generalization via natural language crowdsourcing instructions”, *ACL* (2021).
- [154] Mishra, S., D. Khashabi, C. Baral and H. Hajishirzi, “Cross-task generalization via natural language crowdsourcing instructions”, in “Annual Meeting of the Association for Computational Linguistics”, (2021), URL <https://api.semanticscholar.org/CorpusID:237421373>.

- [155] Mishra, S., A. Mitra, N. Varshney, B. Sachdeva, P. Clark, C. Baral and A. Kalyan, “NumGLUE: A Suite of Fundamental yet Challenging Mathematical Reasoning Tasks”, ACL (2022).
- [156] Mitra, A. and C. Baral, “Learning to automatically solve logic grid puzzles”, in “Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing”, edited by L. Màrquez, C. Callison-Burch and J. Su, pp. 1023–1033 (Association for Computational Linguistics, Lisbon, Portugal, 2015), URL <https://aclanthology.org/D15-1118>.
- [157] Mitra, A., L. Del Corro, S. Mahajan, A. Cudas, C. Simoes, S. Agarwal, X. Chen, A. Razdaibiedina, E. Jones, K. Aggarwal *et al.*, “Orca 2: Teaching small language models how to reason”, arXiv preprint arXiv:2311.11045 (2023).
- [158] Morishita, T., G. Morio, A. Yamaguchi and Y. Sogawa, “Learning deductive reasoning from synthetic corpus based on formal logic”, in “International Conference on Machine Learning”, pp. 25254–25274 (PMLR, 2023).
- [159] Najafabadi, M. M., F. Villanustre, T. M. Khoshgoftaar, N. Seliya, R. Wald and E. Muharemagic, “Deep learning applications and challenges in big data analytics”, Journal of big data **2**, 1, 1–21 (2015).
- [160] Nakamura, M., S. Mashetty, M. Parmar, N. Varshney and C. Baral, “LogicAttack: Adversarial attacks for evaluating logical consistency of natural language inference”, in “Findings of the Association for Computational Linguistics: EMNLP 2023”, edited by H. Bouamor, J. Pino and K. Bali, pp. 13322–13334 (Association for Computational Linguistics, Singapore, 2023), URL <https://aclanthology.org/2023.findings-emnlp.889>.
- [161] Nangia, N., S. Sugawara, H. Trivedi, A. Warstadt, C. Vania and S. R. Bowman, “What ingredients make for an effective crowdsourcing protocol for difficult NLU data collection tasks?”, in “Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)”, pp. 1221–1235 (Association for Computational Linguistics, Online, 2021), URL <https://aclanthology.org/2021.acl-long.98>.
- [162] Nentidis, A., A. Krithara, K. Bougiatiotis and G. Paliouras, “Overview of bioasq 8a and 8b: Results of the eighth edition of the bioasq tasks a and b.”, in “CLEF (Working Notes)”, (2020).
- [163] Noever, D. A. and R. Burdick, “Puzzle solving without search or human knowledge: An unnatural language approach”, ArXiv **abs/2109.02797**, URL <https://api.semanticscholar.org/CorpusID:237431487> (2021).
- [164] Olausson, T., A. Gu, B. Lipkin, C. Zhang, A. Solar-Lezama, J. Tenenbaum and R. Levy, “LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers”, in “Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing”, edited by

- H. Bouamor, J. Pino and K. Bali, pp. 5153–5176 (Association for Computational Linguistics, Singapore, 2023), URL <https://aclanthology.org/2023.emnlp-main.313>.
- [165] OpenAI, “Gpt-4 technical report”, (2023).
- [166] Ouyang, L., J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, “Training language models to follow instructions with human feedback”, Preprint (2022).
- [167] Ouzzani, M., H. Hammady, Z. Fedorowicz and A. Elmagarmid, “Rayyan—a web and mobile app for systematic reviews”, *Systematic reviews* **5**, 1, 1–10 (2016).
- [168] Pan, L., A. Albalak, X. Wang and W. Wang, “Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning”, in “Findings of the Association for Computational Linguistics: EMNLP 2023”, edited by H. Bouamor, J. Pino and K. Bali, pp. 3806–3824 (Association for Computational Linguistics, Singapore, 2023), URL <https://aclanthology.org/2023.findings-emnlp.248>.
- [169] Parmar, M., *Automation of Title and Abstract Screening for Clinical Systematic Reviews*, Master’s thesis, Arizona State University, Tempe, AZ, USA (2021).
- [170] Parmar, M., P. Goyal, X. Liu, Y. Song, M. Ling, C. Baral, H. Palangi and T. Pfister, “PLAN-TUNING: Post-training language models to learn step-by-step planning for complex problem solving”, in “Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing”, edited by C. Christodoulopoulos, T. Chakraborty, C. Rose and V. Peng, pp. 21430–21444 (Association for Computational Linguistics, Suzhou, China, 2025), URL <https://aclanthology.org/2025.emnlp-main.1087/>.
- [171] Parmar, M., X. Liu, P. Goyal, Y. Chen, L. Le, S. Mishra, H. Mobahi, J. Gu, Z. Wang, H. Nakhosht, C. Baral, C.-Y. Lee, T. Pfister and H. Palangi, “PlanGEN: A multi-agent framework for generating planning and reasoning trajectories for complex problem solving”, in “Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing”, edited by C. Christodoulopoulos, T. Chakraborty, C. Rose and V. Peng, pp. 20651–20677 (Association for Computational Linguistics, Suzhou, China, 2025), URL <https://aclanthology.org/2025.emnlp-main.1042/>.
- [172] Parmar, M., S. Mishra, M. Geva and C. Baral, “Don’t blame the annotator: Bias already starts in the annotation instructions”, in “Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics”, pp. 1779–1789 (Association for Computational Linguistics, Dubrovnik, Croatia, 2023), URL <https://aclanthology.org/2023.eacl-main.130>.
- [173] Parmar, M., S. Mishra, M. Purohit, M. Luo, M. Mohammad and C. Baral, “In-BoXBART: Get instructions into biomedical multi-task learning”, in “Findings of the Association for Computational Linguistics: NAACL 2022”, pp. 112–128

- (Association for Computational Linguistics, Seattle, United States, 2022), URL <https://aclanthology.org/2022.findings-naacl.10>.
- [174] Parmar, M., N. Patel, N. Varshney, M. Nakamura, M. Luo, S. Mashetty, A. Mitra and C. Baral, “LogicBench: Towards systematic evaluation of logical reasoning ability of large language models”, in “Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)”, (Bangkok, Thailand, 2024), URL <https://arxiv.org/abs/2404.15522>.
 - [175] Parmar, M., N. Patel, N. Varshney, M. Nakamura, M. Luo, S. Mashetty, A. Mitra and C. Baral, “LogicBench: Towards systematic evaluation of logical reasoning ability of large language models”, In the proceeding of the Association for Computational Linguistics (ACL) 2024 (2024).
 - [176] Patel, N., M. Kulkarni, M. Parmar, A. Budhiraja, M. Nakamura, N. Varshney and C. Baral, “Multi-logieval: Towards evaluating multi-step logical reasoning ability of large language models”, arXiv preprint arXiv:2406.17169 (2024).
 - [177] Peng, B., C. Li, P. He, M. Galley and J. Gao, “Instruction tuning with gpt-4”, (2023).
 - [178] Peng, Y., S. Yan and Z. Lu, “Transfer learning in biomedical natural language processing: An evaluation of BERT and ELMo on ten benchmarking datasets”, in “Proceedings of the 18th BioNLP Workshop and Shared Task”, pp. 58–65 (Association for Computational Linguistics, Florence, Italy, 2019), URL <https://aclanthology.org/W19-5006>.
 - [179] Phan, L. N., J. T. Anibal, H. Tran, S. Chanana, E. Bahadroglu, A. Peltekian and G. Altan-Bonnet, “Scifive: a text-to-text transformer model for biomedical literature”, arXiv preprint arXiv:2106.03598 (2021).
 - [180] Pilehvar, M. T. and J. Camacho-Collados, “WiC: the word-in-context dataset for evaluating context-sensitive meaning representations”, in “Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)”, edited by J. Burstein, C. Doran and T. Solorio, pp. 1267–1273 (Association for Computational Linguistics, Minneapolis, Minnesota, 2019), URL <https://aclanthology.org/N19-1128/>.
 - [181] Polanin, J. R., T. D. Pigott, D. L. Espelage and J. K. Grotpeter, “Best practice guidelines for abstract screening large-evidence systematic reviews and meta-analyses”, *Research Synthesis Methods* **10**, 3, 330–342 (2019).
 - [182] Poliak, A., J. Naradowsky, A. Haldar, R. Rudinger and B. Van Durme, “Hypothesis only baselines in natural language inference”, in “Proceedings of the Seventh Joint Conference on Lexical and Computational Semantics”, pp. 180–191 (2018).
 - [183] Poolsawad, N., C. Kambhampati and J. Cleland, “Balancing class for performance of classification with a clinical dataset”, in “proceedings of the World Congress on Engineering”, vol. 1, pp. 1–6 (2014).

- [184] Prasad, A., P. Hase, X. Zhou and M. Bansal, “Grips: Gradient-free, edit-based instruction search for prompting large language models”, arXiv preprint arXiv:2203.07281 (2022).
- [185] Prasad, A., S. Saha, X. Zhou and M. Bansal, “ReCEval: Evaluating reasoning chains via correctness and informativeness”, in “Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing”, edited by H. Bouamor, J. Pino and K. Bali, pp. 10066–10086 (Association for Computational Linguistics, Singapore, 2023), URL <https://aclanthology.org/2023.emnlp-main.622>.
- [186] Puri, R. S., S. Mishra, M. Parmar and C. Baral, “How many data samples is an additional instruction worth?”, arXiv preprint arXiv:2203.09161 (2022).
- [187] Rafailov, R., A. Sharma, E. Mitchell, C. D. Manning, S. Ermon and C. Finn, “Direct preference optimization: Your language model is secretly a reward model”, in “Thirty-seventh Conference on Neural Information Processing Systems”, (2023), URL <https://openreview.net/forum?id=HPuSIXJaa9>.
- [188] Rafailov, R., A. Sharma, E. Mitchell, C. D. Manning, S. Ermon and C. Finn, “Direct preference optimization: Your language model is secretly a reward model”, *Advances in Neural Information Processing Systems* **36** (2024).
- [189] Raffel, C., N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer”, arXiv preprint arXiv:1910.10683 (2019).
- [190] Raffel, C., N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer”, *Journal of Machine Learning Research* **21**, 140, 1–67 (2020).
- [191] Rahmani, H. A. and J. Yang, “Demographic biases of crowd workers in key opinion leaders finding”, arXiv preprint arXiv:2110.09248 (2021).
- [192] Rajpurkar, P., R. Jia and P. Liang, “Know what you don’t know: Unanswerable questions for SQuAD”, in “Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)”, pp. 784–789 (Association for Computational Linguistics, Melbourne, Australia, 2018), URL <https://aclanthology.org/P18-2124>.
- [193] Rajpurkar, P., J. Zhang, K. Lopyrev and P. Liang, “SQuAD: 100,000+ questions for machine comprehension of text”, in “Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing”, pp. 2383–2392 (Association for Computational Linguistics, Austin, Texas, 2016), URL <https://aclanthology.org/D16-1264>.
- [194] Ran, Q., Y. Lin, P. Li, J. Zhou and Z. Liu, “NumNet: Machine reading comprehension with numerical reasoning”, in “Proceedings of EMNLP”, (2019).

- [195] Rathbone, J., T. Hoffmann and P. Glasziou, “Faster title and abstract screening? evaluating abstrackr, a semi-automated online screening program for systematic reviewers”, *Systematic reviews* **4**, 1, 1–7 (2015).
- [196] Reddy, S. M., S. Patel, M. Weyrich, J. Fenton and M. Viswanathan, “Comparison of a traditional systematic review approach with review-of-reviews and semi-automation as strategies to update the evidence”, *Systematic reviews* **9**, 1, 1–13 (2020).
- [197] Reddy, V., R. Koncel-Kedziorski, V. D. Lai, M. Krumdick, C. Lovering and C. Tanner, “DocFinQA: A long-context financial reasoning dataset”, in “Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)”, edited by L.-W. Ku, A. Martins and V. Srikumar, pp. 445–458 (Association for Computational Linguistics, Bangkok, Thailand, 2024), URL <https://aclanthology.org/2024.acl-short.42/>.
- [198] Rein, D., B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael and S. R. Bowman, “GPQA: A graduate-level google-proof q&a benchmark”, in “First Conference on Language Modeling”, (2024), URL <https://openreview.net/forum?id=Ti67584b98>.
- [199] Robinson, A., W. Thorne, B. P. Wu, A. Pandor, M. Essat, M. Stevenson and X. Song, “Bio-sieve: exploring instruction tuning large language models for systematic review automation”, *arXiv preprint arXiv:2308.06610* (2023).
- [200] Roemmele, M., C. A. Bejan and A. S. Gordon, “Choice of plausible alternatives: An evaluation of commonsense causal reasoning”, in “2011 AAAI Spring Symposium Series”, (2011).
- [201] Rogers, A., M. Gardner and I. Augenstein, “Qa dataset explosion: A taxonomy of nlp resources for question answering and reading comprehension”, *arXiv preprint arXiv:2107.12708* (2021).
- [202] Ros, R., E. Bjarnason and P. Runeson, “A machine learning approach for semi-automated search and selection in literature studies”, in “Proceedings of the 21st International Conference on Evaluation and Assessment in Software Engineering”, pp. 118–127 (2017).
- [203] Rudinger, R., J. Naradowsky, B. Leonard and B. Van Durme, “Gender bias in coreference resolution”, in “Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)”, edited by M. Walker, H. Ji and A. Stent, pp. 8–14 (Association for Computational Linguistics, New Orleans, Louisiana, 2018), URL <https://aclanthology.org/N18-2002/>.
- [204] Saeed, M., N. Ahmadi, P. Nakov and P. Papotti, “RuleBERT: Teaching soft rules to pre-trained language models”, in “Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing”, pp. 1460–1476 (Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2021), URL <https://aclanthology.org/2021.emnlp-main.110>.

- [205] Saeidi, A., S. Verma, A. RRV and C. Baral, “Triple preference optimization: Achieving better alignment with less data in a single step optimization”, arXiv preprint arXiv:2405.16681 (2024).
- [206] Saha, A., R. Aralikkatte, M. M. Khapra and K. Sankaranarayanan, “DuoRC: Towards complex language understanding with paraphrased reading comprehension”, in “Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)”, pp. 1683–1693 (Association for Computational Linguistics, Melbourne, Australia, 2018), URL <https://aclanthology.org/P18-1156>.
- [207] Sakaguchi, K., R. Le Bras, C. Bhagavatula and Y. Choi, “Winogrande: An adversarial winograd schema challenge at scale”, in “Proceedings of the AAAI Conference on Artificial Intelligence”, vol. 34, pp. 8732–8740 (2020).
- [208] Sakai, H. and S. S. Lam, “Large language models for healthcare text classification: A systematic review”, arXiv preprint arXiv:2503.01159 (2025).
- [209] Sanh, V., A. Webson, C. Raffel, S. H. Bach, L. Sutawika, Z. Alyafeai, A. Chaffin, A. Stiegler, T. L. Scao, A. Raja *et al.*, “Multitask prompted training enables zero-shot task generalization”, arXiv preprint arXiv:2110.08207 (2021).
- [210] Saparov, A. and H. He, “Language models are greedy reasoners: A systematic formal analysis of chain-of-thought”, in “The Eleventh International Conference on Learning Representations”, (2022).
- [211] Saparov, A. and H. He, “Language models are greedy reasoners: A systematic formal analysis of chain-of-thought”, in “The Eleventh International Conference on Learning Representations”, (2023), URL <https://openreview.net/forum?id=qFVVBzXxR2V>.
- [212] Scaria, K., H. Gupta, S. A. Sawant, S. Mishra and C. Baral, “Instructabsa: Instruction learning for aspect based sentiment analysis”, ArXiv [abs/2302.08624](https://arxiv.org/abs/2302.08624), URL <https://api.semanticscholar.org/CorpusID:257020097> (2023).
- [213] Schick, T. and H. Schütze, “Exploiting cloze questions for few shot text classification and natural language inference”, arXiv preprint arXiv:2001.07676 (2020).
- [214] Schick, T. and H. Schütze, “Generating datasets with pretrained language models”, in “Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing”, edited by M.-F. Moens, X. Huang, L. Specia and S. W.-t. Yih, pp. 6943–6951 (Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2021), URL <https://aclanthology.org/2021.emnlp-main.555/>.
- [215] Schick, T. and H. Schütze, “It’s not just size that matters: Small language models are also few-shot learners”, in “Proceedings of the 2021 Conference of

- the North American Chapter of the Association for Computational Linguistics: Human Language Technologies”, pp. 2339–2352 (Association for Computational Linguistics, Online, 2021), URL <https://aclanthology.org/2021.naacl-main.185>.
- [216] Schulman, J., F. Wolski, P. Dhariwal, A. Radford and O. Klimov, “Proximal policy optimization algorithms”, arXiv preprint arXiv:1707.06347 (2017).
 - [217] Schuster, T., A. Kalyan, O. Polozov and A. T. Kalai, “Programming puzzles”, arXiv preprint arXiv:2106.05784 (2021).
 - [218] Schwartz, R., J. Dodge, N. A. Smith and O. Etzioni, “Green ai”, *Communications of the ACM* **63**, 12, 54–63 (2020).
 - [219] Schwartz, R., M. Sap, I. Konstas, L. Zilles, Y. Choi and N. A. Smith, “The effect of different writing tasks on linguistic style: A case study of the roc story cloze task”, in “Proceedings of the 21st Conference on Computational Natural Language Learning (CoNLL 2017)”, pp. 15–25 (2017).
 - [220] Shao, Z., P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu *et al.*, “Deepseekmath: Pushing the limits of mathematical reasoning in open language models”, arXiv preprint arXiv:2402.03300 (2024).
 - [221] Sinha, K., S. Sodhani, J. Dong, J. Pineau and W. L. Hamilton, “CLUTRR: A diagnostic benchmark for inductive reasoning from text”, in “Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)”, pp. 4506–4515 (Association for Computational Linguistics, Hong Kong, China, 2019), URL <https://aclanthology.org/D19-1458>.
 - [222] Snell, C., J. Lee, K. Xu and A. Kumar, “Scaling llm test-time compute optimally can be more effective than scaling model parameters”, arXiv preprint arXiv:2408.03314 (2024).
 - [223] Song, Y., W. Xiong, X. Zhao, D. Zhu, W. Wu, K. Wang, C. Li, W. Peng and S. Li, “AgentBank: Towards generalized LLM agents via fine-tuning on 50000+ interaction trajectories”, in “Findings of the Association for Computational Linguistics: EMNLP 2024”, edited by Y. Al-Onaizan, M. Bansal and Y.-N. Chen, pp. 2124–2141 (Association for Computational Linguistics, Miami, Florida, USA, 2024), URL <https://aclanthology.org/2024.findings-emnlp.116/>.
 - [224] Song, Y., D. Yin, X. Yue, J. Huang, S. Li and B. Y. Lin, “Trial and error: Exploration-based trajectory optimization of LLM agents”, in “Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)”, edited by L.-W. Ku, A. Martins and V. Srikumar, pp. 7584–7600 (Association for Computational Linguistics, Bangkok, Thailand, 2024), URL <https://aclanthology.org/2024.acl-long.409/>.

- [225] Strubell, E., A. Ganesh and A. McCallum, “Energy and policy considerations for deep learning in NLP”, in “Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics”, pp. 3645–3650 (Association for Computational Linguistics, Florence, Italy, 2019), URL <https://aclanthology.org/P19-1355>.
- [226] Stubbs, A., C. Kotfila, H. Xu and Ö. Uzuner, “Identifying risk factors for heart disease over time: Overview of 2014 i2b2/uthealth shared task track 2”, *Journal of biomedical informatics* **58**, S67–S77 (2015).
- [227] Suhr, A., C. Vania, N. Nangia, M. Sap, M. Yatskar, S. R. Bowman and Y. Artzi, “Crowdsourcing beyond annotation: Case studies in benchmark data collection”, in “Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: Tutorial Abstracts”, pp. 1–6 (Association for Computational Linguistics, Punta Cana, Dominican Republic & Online, 2021), URL <https://aclanthology.org/2021.emnlp-tutorials.1>.
- [228] Sun, H., Y. Min, Z. Chen, W. X. Zhao, Z. Liu, Z. Wang, L. Fang and J.-R. Wen, “Challenging the boundaries of reasoning: An olympiad-level math benchmark for large language models”, arXiv preprint arXiv:2503.21380 (2025).
- [229] Sun, Z., Y. Shen, Q. Zhou, H. Zhang, Z. Chen, D. Cox, Y. Yang and C. Gan, “Principle-driven self-alignment of language models from scratch with minimal human supervision”, (2023).
- [230] Suzgun, M., N. Scales, N. Scharli, S. Gehrmann, Y. Tay, H. W. Chung, A. Chowdhery, Q. V. Le, E. H. Hsin Chi, D. Zhou and J. Wei, “Challenging big-bench tasks and whether chain-of-thought can solve them”, in “Annual Meeting of the Association for Computational Linguistics”, (2022), URL <https://api.semanticscholar.org/CorpusID:252917648>.
- [231] Swayamdipta, S., R. Schwartz, N. Lourie, Y. Wang, H. Hajishirzi, N. A. Smith and Y. Choi, “Dataset cartography: Mapping and diagnosing datasets with training dynamics”, in “Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)”, pp. 9275–9293 (2020).
- [232] Syriani, E., I. David and G. Kumar, “Assessing the ability of chatgpt to screen articles for systematic reviews”, *CoRR* (2023).
- [233] Tafjord, O., B. Dalvi and P. Clark, “ProofWriter: Generating implications, proofs, and abductive statements over natural language”, in “Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021”, pp. 3621–3634 (Association for Computational Linguistics, Online, 2021), URL <https://aclanthology.org/2021.findings-acl.317>.
- [234] Tateisi, Y., A. Yakushiji, T. Ohta and J. Tsujii, “Syntax annotation for the genia corpus”, in “Companion Volume to the Proceedings of Conference including Posters/Demos and tutorial abstracts”, (2005).

- [235] Team, G., R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican *et al.*, “Gemini: a family of highly capable multimodal models”, arXiv preprint arXiv:2312.11805 (2023).
- [236] Team, G., P. Georgiev, V. I. Lei, R. Burnell, L. Bai, A. Gulati, G. Tanzer, D. Vincent, Z. Pan, S. Wang *et al.*, “Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context”, arXiv preprint arXiv:2403.05530 (2024).
- [237] Team, G., A. Kamath, J. Ferret, S. Pathak, N. Vieillard, R. Merhej, S. Perrin, T. Matejovicova, A. Ramé, M. Rivière *et al.*, “Gemma 3 technical report”, arXiv preprint arXiv:2503.19786 (2025).
- [238] Team, Q., “Qwq-32b: Embracing the power of reinforcement learning”, URL <https://qwenlm.github.io/blog/qwq-32b/> (2025).
- [239] Tian, J., Y. Li, W. Chen, L. Xiao, H. He and Y. Jin, “Diagnosing the first-order logical reasoning ability through LogicNLP”, in “Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing”, pp. 3738–3747 (Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 2021), URL <https://aclanthology.org/2021.emnlp-main.303>.
- [240] Touvron, H., T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave and G. Lample, “Llama: Open and efficient foundation language models”, ArXiv **abs/2302.13971**, URL <https://api.semanticscholar.org/CorpusID:257219404> (2023).
- [241] Touvron, H., L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models”, arXiv preprint arXiv:2307.09288 (2023).
- [242] Touvron, H., L. Martin, K. R. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. M. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. S. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. M. Kloumann, A. V. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov and T. Scialom, “Llama 2: Open foundation and fine-tuned chat models”, ArXiv **abs/2307.09288**, URL <https://api.semanticscholar.org/CorpusID:259950998> (2023).
- [243] Tsafnat, G., P. Glasziou, M. K. Choong, A. Dunn, F. Galgani and E. Coiera, “Systematic review automation technologies”, *Systematic reviews* **3**, 1, 1–15 (2014).

- [244] Tsafnat, G., P. Glasziou, G. Karystianis and E. Coiera, “Automated screening of research studies for systematic reviews using study characteristics”, *Systematic reviews* **7**, 1, 1–9 (2018).
- [245] Tsuchiya, M., “Performance impact caused by hidden bias of training data for recognizing textual entailment”, in “Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)”, (European Language Resources Association (ELRA), Miyazaki, Japan, 2018), URL <https://aclanthology.org/L18-1239>.
- [246] Tyagi, N., M. Parmar, M. Kulkarni, A. Rrv, N. Patel, M. Nakamura, A. Mitra and C. Baral, “Step-by-step reasoning to solve grid puzzles: Where do llms falter?”, arXiv preprint arXiv:2407.14790 (2024).
- [247] Tyen, G., H. Mansoor, P. Chen, T. Mak and V. Cărbune, “Llms cannot find reasoning errors, but can correct them!”, arXiv preprint arXiv:2311.08516 (2023).
- [248] Uzuner, Ö., Y. Luo and P. Szolovits, “Evaluating the state-of-the-art in automatic de-identification”, *Journal of the American Medical Informatics Association* **14**, 5, 550–563 (2007).
- [249] Valmeekam, K., M. Marquez, A. Olmo, S. Sreedharan and S. Kambhampati, “Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change”, *Advances in Neural Information Processing Systems* **36** (2024).
- [250] Valmeekam, K., A. Olmo, S. Sreedharan and S. Kambhampati, “Large language models still can’t plan (a benchmark for llms on planning and reasoning about change)”, *ArXiv abs/2206.10498*, URL <https://api.semanticscholar.org/CorpusID:249889477> (2022).
- [251] Valmeekam, K., K. Stechly and S. Kambhampati, “Llms still can’t plan; can lrms? a preliminary evaluation of openai’s o1 on planbench”, arXiv preprint arXiv:2409.13373 (2024).
- [252] Vidgen, B., D. Nguyen, H. Margetts, P. Rossini and R. Tromble, “Cad: the contextual abuse dataset”, URL <https://doi.org/10.5281/zenodo.4881008> (2021).
- [253] Waffenschmidt, S., M. Knelangen, W. Sieben, S. Bühn and D. Pieper, “Single screening versus conventional double screening for study selection in systematic reviews: a methodological systematic review”, *BMC medical research methodology* **19**, 1–9 (2019).
- [254] Wang, A., Y. Pruksachatkun, N. Nangia, A. Singh, J. Michael, F. Hill, O. Levy and S. Bowman, “Superglue: A stickier benchmark for general-purpose language understanding systems”, *Advances in neural information processing systems* **32** (2019).

- [255] Wang, B., Q. Xie, J. Pei, P. Tiwari, Z. Li *et al.*, “Pre-trained language models in biomedical domain: A systematic survey”, arXiv preprint arXiv:2110.05006 (2021).
- [256] Wang, E. Z., F. Cassano, C. Wu, Y. Bai, W. Song, V. Nath, Z. Han, S. M. Hendryx, S. Yue and H. Zhang, “Planning in natural language improves llm search for code generation”, in “The First Workshop on System-2 Reasoning at Scale, NeurIPS’24”, (2024).
- [257] Wang, E. Z., F. Cassano, C. Wu, Y. Bai, W. Song, V. Nath, Z. Han, S. M. Hendryx, S. Yue and H. Zhang, “Planning in natural language improves LLM search for code generation”, in “The Thirteenth International Conference on Learning Representations”, (2025), URL <https://openreview.net/forum?id=48WAZhwHHw>.
- [258] Wang, L., R. Li, Y. Yan, Y. Yan, S. Wang, W. Wu and W. Xu, “Instructionner: A multi-task instruction-based generative framework for few-shot ner”, arXiv preprint arXiv:2203.03903 (2022).
- [259] Wang, L., C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin *et al.*, “A survey on large language model based autonomous agents”, *Frontiers of Computer Science* **18**, 6, 186345 (2024).
- [260] Wang, L., W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee and E.-P. Lim, “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models”, in “Annual Meeting of the Association for Computational Linguistics”, (2023), URL <https://api.semanticscholar.org/CorpusID:258558102>.
- [261] Wang, R., H. Li, X. Han, Y. Zhang and T. Baldwin, “Learning from failure: Integrating negative examples when fine-tuning large language models as agents”, arXiv preprint arXiv:2402.11651 (2024).
- [262] Wang, S., H. Fang, M. Khabsa, H. Mao and H. Ma, “Entailment as few-shot learner”, arXiv preprint arXiv:2104.14690 (2021).
- [263] Wang, S., H. Scells, S. Zhuang, M. Potthast, B. Koopman and G. Zuccon, “Zero-shot generative large language models for systematic review screening automation”, in “European Conference on Information Retrieval”, pp. 403–420 (Springer, 2024).
- [264] Wang, X., C. Li, Z. Wang, F. Bai, H. Luo, J. Zhang, N. Jojic, E. Xing and Z. Hu, “Promptagent: Strategic planning with language models enables expert-level prompt optimization”, in “The Twelfth International Conference on Learning Representations”, (2024), URL <https://openreview.net/forum?id=22pyNMuIoa>.
- [265] Wang, X., J. Wei, D. Schuurmans, Q. Le, E. Chi and D. Zhou, “Self-consistency improves chain of thought reasoning in language models”, arXiv preprint arXiv:2203.11171 (2022).

- [266] Wang, Y., Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi and H. Hajishirzi, “Self-instruct: Aligning language models with self-generated instructions”, in “Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023”, edited by A. Rogers, J. L. Boyd-Graber and N. Okazaki, pp. 13484–13508 (Association for Computational Linguistics, 2023), URL <https://doi.org/10.18653/v1/2023.acl-long.754>.
- [267] Wang, Y., S. Mishra, P. Alipoormolabashi, Y. Kordi, A. Mirzaei, A. Arunkumar, A. Ashok, A. S. Dhanasekaran, A. Naik, D. Stap, E. Pathak, G. Karamanolakis, H. G. Lai, I. Purohit, I. Mondal, J. Anderson, K. Kuznia, K. Doshi, M. Patel, K. K. Pal, M. Moradshahi, M. Parmar, M. Purohit, N. Varshney, P. R. Kaza, P. Verma, R. S. Puri, R. Karia, S. K. Sampat, S. Doshi, S. D. Mishra, S. Reddy, S. Patro, T. Dixit, X. Shen, C. Baral, Y. Choi, N. A. Smith, H. Hajishirzi and D. Khashabi, “Super-naturalinstructions: Generalization via declarative instructions on 1600+ nlp tasks”, in “Conference on Empirical Methods in Natural Language Processing”, (2022), URL <https://api.semanticscholar.org/CorpusID:253098274>.
- [268] Wang, Y., S. Mishra, P. Alipoormolabashi, Y. Kordi, A. Mirzaei, A. Arunkumar, A. Ashok, A. S. Dhanasekaran, A. Naik, D. Stap *et al.*, “Benchmarking generalization via in-context instructions on 1,600+ language tasks”, arXiv preprint arXiv:2204.07705 (2022).
- [269] Wang, Y., T. Shen, L. Liu and J. Xie, “Sibyl: Simple yet effective agent framework for complex real-world reasoning”, CoRR (2024).
- [270] Webson, A. and E. Pavlick, “Do prompt-based models really understand the meaning of their prompts?”, arXiv preprint arXiv:2109.01247 (2021).
- [271] Wei, J., M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai and Q. V. Le, “Finetuned language models are zero-shot learners”, arXiv preprint arXiv:2109.01652 (2021).
- [272] Wei, J., X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, “Chain-of-thought prompting elicits reasoning in large language models”, Advances in Neural Information Processing Systems **35**, 24824–24837 (2022).
- [273] Welbl, J., N. F. Liu and M. Gardner, “Crowdsourcing multiple choice science questions”, in “Proceedings of the 3rd Workshop on Noisy User-generated Text”, pp. 94–106 (Association for Computational Linguistics, Copenhagen, Denmark, 2017), URL <https://aclanthology.org/W17-4413>.
- [274] Welleck, S., A. Bertsch, M. Finlayson, H. Schoelkopf, A. Xie, G. Neubig, I. Kulikov and Z. Harchaoui, “From decoding to meta-generation: Inference-time algorithms for large language models”, arXiv preprint arXiv:2406.16838 (2024).
- [275] Weller, O., N. Lourie, M. Gardner and M. E. Peters, “Learning from task descriptions”, in “Proceedings of the 2020 Conference on Empirical Methods in

- Natural Language Processing (EMNLP)”, pp. 1361–1375 (Association for Computational Linguistics, Online, 2020), URL <https://aclanthology.org/2020.emnlp-main.105>.
- [276] Wiegrefe, S., J. Hessel, S. Swayamdipta, M. Riedl and Y. Choi, “Reframing human-AI collaboration for generating free-text explanations”, in “Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies”, edited by M. Carpuat, M.-C. de Marneffe and I. V. Meza Ruiz, pp. 632–658 (Association for Computational Linguistics, Seattle, United States, 2022), URL <https://aclanthology.org/2022.naacl-main.47/>.
 - [277] Wolf, T., L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest and A. Rush, “Transformers: State-of-the-art natural language processing”, in “Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations”, pp. 38–45 (Association for Computational Linguistics, Online, 2020), URL <https://aclanthology.org/2020.emnlp-demos.6>.
 - [278] Wu, T., E. Jiang, A. Donsbach, J. Gray, A. Molina, M. Terry and C. J. Cai, “Promptchainer: Chaining large language model prompts through visual programming”, arXiv preprint arXiv:2203.06566 (2022).
 - [279] Wu, T., M. Terry and C. J. Cai, “Ai chains: Transparent and controllable human-ai interaction by chaining large language model prompts”, arXiv preprint arXiv:2110.01691 (2021).
 - [280] Wu, Y., Z. Sun, S. Li, S. Welleck and Y. Yang, “An empirical analysis of compute-optimal inference for problem-solving with language models”, CoRR (2024).
 - [281] Wu, Z., C. Han, Z. Ding, Z. Weng, Z. Liu, S. Yao, T. Yu and L. Kong, “Oscopilot: Towards generalist computer agents with self-improvement”, in “ICLR 2024 Workshop on Large Language Model (LLM) Agents”, (2024).
 - [282] Xiao, Z., D. Zhang, Y. Wu, L. Xu, Y. J. Wang, X. Han, X. Fu, T. Zhong, J. Zeng, M. Song and G. Chen, “Chain-of-experts: When LLMs meet complex operations research problems”, in “The Twelfth International Conference on Learning Representations”, (2024), URL <https://openreview.net/forum?id=HobyL1B9CZ>.
 - [283] Xie, C. and D. Zou, “A human-like reasoning framework for multi-phases planning task with large language models”, arXiv preprint arXiv:2405.18208 (2024).
 - [284] Xie, T., C. H. Wu, P. Shi, R. Zhong, T. Scholak, M. Yasunaga, C.-S. Wu, M. Zhong, P. Yin, S. I. Wang *et al.*, “Unifiedskg: Unifying and multi-tasking structured knowledge grounding with text-to-text language models”, arXiv preprint arXiv:2201.05966 (2022).

- [285] Yanaka, H., K. Mineshima, D. Bekki, K. Inui, S. Sekine, L. Abzianidze and J. Bos, “Can neural networks understand monotonicity reasoning?”, in “Proceedings of the 2019 ACL Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP”, edited by T. Linzen, G. Chrupała, Y. Belinkov and D. Hupkes, pp. 31–40 (Association for Computational Linguistics, Florence, Italy, 2019), URL <https://aclanthology.org/W19-4804>.
- [286] Yang, A., B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang and Z. Qiu, “Qwen2.5 technical report”, arXiv preprint arXiv:2412.15115 (2024).
- [287] Yang, L., Z. Yu, T. Zhang, S. Cao, M. Xu, W. Zhang, J. E. Gonzalez and B. CUI, “Buffer of thoughts: Thought-augmented reasoning with large language models”, in “The Thirty-eighth Annual Conference on Neural Information Processing Systems”, (2024), URL <https://openreview.net/forum?id=AN01i9JPtb>.
- [288] Yang, Z., P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov and C. D. Manning, “Hotpotqa: A dataset for diverse, explainable multi-hop question answering”, in “Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing”, pp. 2369–2380 (2018).
- [289] Yao, S., D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao and K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models”, *Advances in Neural Information Processing Systems* **36** (2024).
- [290] Yao, S., J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan and Y. Cao, “React: Synergizing reasoning and acting in language models”, in “The Eleventh International Conference on Learning Representations”, (2023), URL https://openreview.net/forum?id=WE_vluYUL-X.
- [291] Ye, J., J. Gao, Q. Li, H. Xu, J. Feng, Z. Wu, T. Yu and L. Kong, “Zero-gen: Efficient zero-shot learning via dataset generation”, in “Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022”, edited by Y. Goldberg, Z. Kozareva and Y. Zhang, pp. 11653–11669 (Association for Computational Linguistics, 2022), URL <https://doi.org/10.18653/v1/2022.emnlp-main.801>.
- [292] Ye, J., J. Gao, Z. Wu, J. Feng, T. Yu and L. Kong, “Progen: Progressive zero-shot dataset generation via in-context feedback”, in “Findings of the Association for Computational Linguistics: EMNLP 2022, Abu Dhabi, United Arab Emirates, December 7-11, 2022”, edited by Y. Goldberg, Z. Kozareva and Y. Zhang, pp. 3671–3683 (Association for Computational Linguistics, 2022), URL <https://doi.org/10.18653/v1/2022.findings-emnlp.269>.

- [293] Ye, Q. and X. Ren, “Learning to generate task-specific adapters from task description”, in “Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)”, pp. 646–653 (Association for Computational Linguistics, Online, 2021), URL <https://aclanthology.org/2021.acl-short.82>.
- [294] Ye, Q. and X. Ren, “Zero-shot learning by generating task-specific adapters”, arXiv e-prints pp. arXiv-2101 (2021).
- [295] Ye, X. and G. Durrett, “The unreliability of explanations in few-shot prompting for textual reasoning”, *Advances in neural information processing systems* **35**, 30378–30392 (2022).
- [296] Yin, W., J. Hay and D. Roth, “Benchmarking zero-shot text classification: Datasets, evaluation and entailment approach”, arXiv preprint arXiv:1909.00161 (2019).
- [297] Young, A., B. Chen, C. Li, C. Huang, G. Zhang, G. Zhang, H. Li, J. Zhu, J. Chen, J. Chang *et al.*, “Yi: Open foundation models by 01. ai”, arXiv preprint arXiv:2403.04652 (2024).
- [298] Young, A., B. Chen, C. Li, C. Huang, G. Zhang, G. Zhang, H. Li, J. Zhu, J. Chen, J. Chang *et al.*, “Yi: Open foundation models by 01. ai”, arXiv preprint arXiv:2403.04652 (2024).
- [299] Yu, W., M. Clyne, S. M. Dolan, A. Yesupriya, A. Wulf, T. Liu, M. J. Khoury and M. Gwinn, “Gapscreener: an automatic tool for screening human genetic association literature in pubmed using the support vector machine technique”, *BMC bioinformatics* **9**, 1, 1–9 (2008).
- [300] Yu, W., Z. Jiang, Y. Dong and J. Feng, “Reclor: A reading comprehension dataset requiring logical reasoning”, in “International Conference on Learning Representations”, (2020), URL <https://openreview.net/forum?id=HJgJtT4tvB>.
- [301] Yu, Y., Y. Zhuang, J. Zhang, Y. Meng, A. Ratner, R. Krishna, J. Shen and C. Zhang, “Large language model as attributed training data generator: A tale of diversity and bias”, *CoRR* **abs/2306.15895**, URL <https://doi.org/10.48550/arXiv.2306.15895> (2023).
- [302] Yuan, W., G. Neubig and P. Liu, “Bartscore: Evaluating generated text as text generation”, *Advances in Neural Information Processing Systems* **34**, 27263–27277 (2021).
- [303] Zellers, R., A. Holtzman, Y. Bisk, A. Farhadi and Y. Choi, “Hellaswag: Can a machine really finish your sentence?”, (2019).
- [304] Zhang, D., X. Huang, D. Zhou, Y. Li and W. Ouyang, “Accessing gpt-4 level mathematical olympiad solutions via monte carlo tree self-refine with llama-3 8b”, arXiv preprint arXiv:2406.07394 (2024).

- [305] Zhang, H., L. H. Li, T. Meng, K.-W. Chang and G. Van Den Broeck, “On the paradox of learning to reason from data”, in “Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence”, pp. 3365–3373 (2023).
- [306] Zhang, J., T. Lan, R. Rithesh, Z. Liu, W. Yao, J. Tan, T. Q. Hoang, L. Yang, Y. Feng, Z. Liu *et al.*, “The agent ohana: Designing unified data and training pipeline for effective agent learning”, in “ICLR 2024 Workshop on Large Language Model (LLM) Agents”, (2024).
- [307] Zhang, S., X. Liu, J. Liu, J. Gao, K. Duh and B. Van Durme, “Record: Bridging the gap between human and machine commonsense reading comprehension”, arXiv preprint arXiv:1810.12885 (2018).
- [308] Zhang, Y., M. Khalifa, L. Logeswaran, J. Kim, M. Lee, H. Lee and L. Wang, “Small language models need strong verifiers to self-correct reasoning”, (2024).
- [309] Zhao, H., K. Wang, M. Yu and H. Mei, “Explicit planning helps language models in logical reasoning”, in “Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing”, edited by H. Bouamor, J. Pino and K. Bali, pp. 11155–11173 (Association for Computational Linguistics, Singapore, 2023), URL <https://aclanthology.org/2023.emnlp-main.688/>.
- [310] Zhao, J. and C. J. Anderson, “Solving and generating npr sunday puzzles with large language models”, arXiv preprint arXiv:2306.12255 (2023).
- [311] Zhao, J., D. Khashabi, T. Khot, A. Sabharwal and K.-W. Chang, “Ethical-advice taker: Do language models understand natural language interventions?”, in “Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021”, pp. 4158–4164 (Association for Computational Linguistics, Online, 2021), URL <https://aclanthology.org/2021.findings-acl.364>.
- [312] Zhao, W. X., K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong *et al.*, “A survey of large language models”, arXiv preprint arXiv:2303.18223 (2023).
- [313] Zheng, F., R. Tao, H. R. Maier, L. See, D. Savic, T. Zhang, Q. Chen, T. H. Assumpção, P. Yang, B. Heidari *et al.*, “Crowdsourcing methods for data collection in geophysics: State of the art, issues, and future directions”, *Reviews of Geophysics* **56**, 4, 698–740 (2018).
- [314] Zheng, H. S., S. Mishra, H. Zhang, X. Chen, M. Chen, A. Nova, L. Hou, H.-T. Cheng, Q. V. Le, E. H. Chi *et al.*, “Natural plan: Benchmarking llms on natural language planning”, arXiv preprint arXiv:2406.04520 (2024).
- [315] Zhong, R., K. Lee, Z. Zhang and D. Klein, “Adapting language models for zero-shot learning by meta-tuning on dataset and prompt collections”, in “Findings of the Association for Computational Linguistics: EMNLP 2021”, pp. 2856–2878 (Association for Computational Linguistics, Punta Cana, Dominican Republic, 2021), URL <https://aclanthology.org/2021.findings-emnlp.244>.

- [316] Zhong, T., Z. Liu, Y. Pan, Y. Zhang, Y. Zhou, S. Liang, Z. Wu, Y. Lyu, P. Shu, X. Yu *et al.*, “Evaluation of openai o1: Opportunities and challenges of agi”, CoRR (2024).
- [317] Zhou, B., D. Khashabi, Q. Ning and D. Roth, ““going on a vacation” takes longer than “going for a walk”: A study of temporal commonsense understanding”, in “Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)”, pp. 3363–3369 (Association for Computational Linguistics, Hong Kong, China, 2019), URL <https://aclanthology.org/D19-1332>.
- [318] Zhou, D., N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, O. Bousquet, Q. Le and E. Chi, “Least-to-most prompting enables complex reasoning in large language models”, arXiv preprint arXiv:2205.10625 (2022).
- [319] Zhou, P., J. Pujara, X. Ren, X. Chen, H.-T. Cheng, Q. V. Le, E. H. Chi, D. Zhou, S. Mishra and H. S. Zheng, “Self-discover: Large language models self-compose reasoning structures”, (2024).
- [320] Zhu, Y., S. Qiao, Y. Ou, S. Deng, N. Zhang, S. Lyu, Y. Shen, L. Liang, J. Gu and H. Chen, “Knowagent: Knowledge-augmented planning for llm-based agents”, arXiv preprint arXiv:2403.03101 (2024).

APPENDIX A
RELATED PUBLICATION DETAILS

Most ideas in this dissertation have appeared in the following publications (also listed in Figure 1.1), with the consent of my co-authors. I obtained permission from all co-authors for use of the work in your document.¹

- Mihir Parmar, Nisarg Patel, Neeraj Varshney, Mutsumi Nakamura, Man Luo, Santosh Mashetty, Arindam Mitra, and Chitta Baral. “LogicBench: Towards Systematic Evaluation of Logical Reasoning Ability of Large Language Models.” In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 13679-13707. 2024.
- Nisarg Patel*, Mohith Kulkarni*, Mihir Parmar*, Aashna Budhiraja, Mutsumi Nakamura, Neeraj Varshney, and Chitta Baral. “Multi-LogiEval: Towards Evaluating Multi-Step Logical Reasoning Ability of Large Language Models.” In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 20856-20879. 2024.
- Nemika Tyagi*, Mihir Parmar*, Mohith Kulkarni, Aswin Rrv, Nisarg Patel, Mutsumi Nakamura, Arindam Mitra, and Chitta Baral. "Step-by-Step Reasoning to Solve Grid Puzzles: Where do LLMs Falter?." In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 19898-19915. 2024.
- Mihir Parmar, Xin Liu, Palash Goyal, Yanfei Chen, Long Le, Swaroop Mishra, Hossein Mobahi, Jindong Gu, Zifeng Wang, Hootan Nakhost, Chitta Baral, Chen-Yu Lee, Tomas Pfister, and Hamid Palangi. “PlanGEN: A Multi-Agent Framework for Generating Planning and Reasoning Trajectories for Complex Problem Solving” In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pp. 20651–20677. 2025.
- Mihir Parmar, Palash Goyal, Xin Liu, Yiwen Song, Mingyang Ling, Chitta Baral, Hamid Palangi, and Tomas Pfister. “PLAN-TUNING: Post-Training Language Models to Learn Step-by-Step Planning for Complex Problem Solving.” In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pp. 21430–21444. 2025.
- Mihir Parmar, Swaroop Mishra, Mor Geva, and Chitta Baral. “Don’t Blame the Annotator: Bias Already Starts in the Annotation Instructions.” In Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics, pp. 1779-1789. 2023.
- Himanshu Gupta, Kevin Scaria, Ujjwala Anantheswaran, Shreyas Verma, Mihir Parmar, Saurabh Arjun Sawant, Chitta Baral, and Swaroop Mishra. “TarGEN: Targeted Data Generation with Large Language Models.” In First Conference on Language Modeling 2024.
- Mihir Parmar, Swaroop Mishra, Mirali Purohit, Man Luo, Murad Mohammad, and Chitta Baral. “In-BoXBART: Get Instructions into Biomedical Multi-Task Learning.” In Findings of the Association for Computational Linguistics: NAACL 2022, pp. 112-128. 2022.

¹* indicates *Equal Contribution*.

ProQuest Number: 32286187

INFORMATION TO ALL USERS

The quality and completeness of this reproduction is dependent on the quality and completeness of the copy made available to ProQuest.



Distributed by
ProQuest LLC a part of Clarivate (2025).
Copyright of the Dissertation is held by the Author unless otherwise noted.

This work is protected against unauthorized copying under Title 17,
United States Code and other applicable copyright laws.

This work may be used in accordance with the terms of the Creative Commons license or other rights statement, as indicated in the copyright statement or in the metadata associated with this work. Unless otherwise specified in the copyright statement or the metadata, all rights are reserved by the copyright holder.

ProQuest LLC
789 East Eisenhower Parkway
Ann Arbor, MI 48108 USA